Penguin Documentation

Release 0.0.0.dev0+g9aa9036

Penguin Team

Nov 12, 2026

DOCUMENTATION

CHAPTER

ONE

PENGUIN COMPOSE — MULTI-DEVICE FIRMWARE REHOSTING

`penguin compose` runs multiple firmware guests in parallel and wires them together over a shared L2 broadcast domain. Each device runs as its own PANDA-QEMU guest with its own project config; the only thing compose adds is the virtual network glue.

The networking backend is QEMU's `socket,mcast=...` netdev. No tap devices, no Linux bridge, no `CAP_NET_ADMIN` — everything works inside an ordinary unprivileged container.

1.1 Invocation

1.1.1 1. Create a compose project

Pass two or more device project directories. Compose generates a fresh compose project:

```
./penguin compose init ./projects/fw1 ./projects/fw2
./penguin compose init ./projects/fw1 ./projects/fw2 ./projects/fw3
./penguin compose init --name three_node ./projects/fw1 ./projects/fw2 ./projects/fw3
```

The scaffolded directory is placed alongside the device projects' parent — if the device projects live in `./projects/`, compose writes the new project under `./compose_projects/<name>/`. The default `<name>` is the device basenames joined with `_` (so `fw1` + `fw2` → `fw1_fw2`); pass `--name` to override. Names must match `[A-Za-z0-9._-]+` and be at most 64 characters. If the default join would exceed the cap (many devices or long basenames), `init` refuses and asks for an explicit `--name`.

Each device is attached to a single `lan` network (`192.168.1.0/24`) on `eth0`, with IPs assigned in argument order: the first project gets `192.168.1.1`, the second `192.168.1.2`, and so on.

If the target directory already exists, `compose init` refuses rather than overwriting. To re-run an existing setup, pass it to `compose run`; to scaffold a separate copy with the same device set, use `--name`.

After `init`, inspect or edit the generated `compose.yaml` if you need different IPs, interfaces, networks, or config overrides.

1.1.2 2. Run a compose project

Pass a directory containing a `compose.yaml`, or the YAML file directly:

```
./penguin compose run ./compose_projects/my_setup
./penguin compose run ./compose_projects/my_setup/compose.yaml
```

Results land in `<compose-dir>/results/<N>/`, with `results/latest` updated to point at the most recent numbered run.

1.1.3 Shortcut: scaffold and run

For quick smoke runs, pass two or more device project directories directly to `compose`. This is equivalent to `compose init` followed immediately by `compose run`:

```
./penguin compose ./projects/fw1 ./projects/fw2
./penguin compose ./projects/fw1 ./projects/fw2 ./projects/fw3
```

Re-running the shortcut with the same arguments produces a different timestamp, so the second invocation creates a new scaffolded directory rather than overwriting the first. To re-run an already-scaffolded setup (and keep its IP assignments and numbered results history), use `penguin compose run` against that directory.

1.2 YAML schema

```yaml
version: 1

networks:
  lan:
    subnet: 192.168.1.0/24       # optional; informational only

devices:
  server:
    project: ./projects/fw1      # path to a penguin project (required)
    networks:
      lan:
        iface: eth0              # guest interface name (required)
        ip: 192.168.1.1/24       # optional: static IP via startup script
        mac: "52:54:00:aa:01:01" # optional: auto-generated if omitted
    config_overrides:            # optional: merged as a final patch layer
      core:
        guest_cmd_timeout: 120

  client:
    project: ./projects/fw2
    networks:
      lan:
        iface: eth0
        ip: 192.168.1.2/24

output:
  base_dir: ./results            # optional; default ./results next to compose.yaml
```

Field notes:

- **`project`** — path to a device project directory containing `config.yaml`, resolved relative to `compose.yaml`.
- **`iface`** — the Linux interface name the guest is expected to use for this attachment. The compose-generated NIC's MAC is set so the firmware can pin the interface; verify your firmware actually names it as declared (typically `eth0` for a single attachment).
- **`ip`** — if set, compose writes `ip link set <iface> up; ip addr add <ip> dev <iface>` into `/igloo/init.d/zz_compose_net` so traffic flows at boot.
- **`mac`** — if omitted, a stable locally-administered MAC is derived from `(compose path, device name, network name)`.

- **config_overrides** — merged as the final patch layer on top of the device project's own `config.yaml` and any `patch_*.yaml` files.
- **output.base_dir** — base directory for numbered runs, resolved relative to `compose.yaml`.

The authoritative schema lives in `src/penguin/compose.py` (`load_compose`, `DeviceConfig`, `NetworkSpec`, `DeviceNetAttachment`).

1.3 Output layout

```
<compose-project>/
  compose.yaml
  results/
    latest -> ./0
    0/
      compose.yaml                  # copy of the input for reproducibility
      compose_summary.yaml          # per-device scores and errors
      server/                       # runner's output dir for this device
        console.log
        netbinds.csv
        summary.json                # per-device machine-readable run digest
        ...
      client/
        console.log
        ...
      .compose/
        server/
          derived_config.yaml       # device config + injected compose patch
          patch_compose_net.yaml    # auto-generated net patch
          instance.yaml             # planned metadata (endpoints, networks)
          runtime.yaml              # actual runtime metadata from the runner
          qemu_stdout.txt
          qemu_stderr.txt
        client/
          ...
```

Per-device analysis output (`console.log`, `netbinds.csv`, `health_final.yaml`, etc.) sits directly under `<device_name>/`, exactly as a single-device `penguin run` would write it. Compose-specific bookkeeping (derived configs, generated patches, QEMU stdio, runtime metadata) is tucked under `.compose/<device_name>/` so it doesn't clutter the analysis tree.

`compose_summary.yaml` aggregates each device's score dict and any top-level error from its run.

1.4 Inspecting a running compose session

`penguin utils list` prints a table of the devices in a compose run. With no `--dir`, it searches the current directory for a `results/latest` (or numbered run) layout.

```
$ penguin utils list
DEVICE    STATUS   PID    SHELL  CID  NETWORKS         OUTPUT
device_a  running  12345  20000  16   lan(192.168.1.2) results/latest/device_a
device_b  running  12346  20100  17   lan(192.168.1.3) results/latest/device_b
```

Columns are: device name, runner status, runner PID, per-device telnet port on the docker container, vsock CID for

guest_cmd, attached networks with the assigned guest IP, and the device's output directory. When devices are running, the command also prints ready-to-paste telnet and penguin utils guest-cmd lines.

If core.core_shell is enabled, each device exposes a root shell on its telnet port. From the docker host:

```
telnet 192.168.0.2 20000
```

or, from inside the penguin container:

```
docker exec -it <container-name> telnet localhost 20000
```

Endpoint allocation is controlled by three env vars (read at compose-start time):

Var	Default	Effect
PENGUIN_COMPOSE_TELNET_BASE	20000	Telnet port for the first device.
PENGUIN_COMPOSE_TELNET_BLOCK_SIZE	100	Port stride between devices.
PENGUIN_COMPOSE_VSOCK_CID_BASE	16	vsock CID for the first device.

Bump these when running multiple compose sessions on the same host (so their port/CID blocks don't collide) or when the default block runs into the 65535 cap with very large device counts.

The source-of-truth files behind penguin utils list are .compose/<device>/instance.yaml (planned endpoints + networks, written before the runner starts) and .compose/<device>/runtime.yaml (actual PID, status, and endpoints from the runner). If a session crashes, those two files plus qemu_stdout.txt / qemu_stderr.txt next to them are the right place to start.

1.5 How it works

Each entry under networks: is assigned a UDP multicast port, and every device attached to that network gets matching QEMU args injected via core.extra_qemu_args:

```
-netdev socket,id=compose.0,mcast=230.0.0.1:<port>
-device virtio-net-pci,netdev=compose.0,mac=<mac>
```

All QEMU processes that join the same mcast=<group>:<port> share one L2 broadcast domain — ARP, DHCP, and link-local discovery all work because QEMU just wraps raw Ethernet frames in UDP. The kernel inside each guest enumerates the injected NIC normally (typically as eth0 for a single attachment), and the compose-generated /igloo/init.d/zz_compose_net script brings it up and assigns the static IP. The multicast plumbing is internal to the QEMU processes; the host does not need any privileges or network configuration.

1.6 Caveats

- **No boot ordering.** All devices start in parallel. If a client probes a server's port before the server has bound it, the probe will fail — application-level retry is on you.
- **L2 only.** Compose provides a shared broadcast domain and (optionally) static IPs. There is no DHCP server, no default gateway, no NAT, and no routing between compose networks. Devices that need DHCP must serve it themselves from one of the guests.
- **Single container, shared host ports.** All compose devices run as parallel QEMU subprocesses inside one Penguin container. Plugins that expose host-facing ports (e.g. the VPN plugin) share that namespace, so fixed host-port mappings must not collide across devices.

- **Scaffolded directories refuse to overwrite.** `compose init` and the scaffold-and-run shortcut both fail if the target `compose_projects/<name>/` already exists. Use `compose run` against it to re-run, delete it to start fresh, or pass `--name` to scaffold a separate copy.

CHAPTER

TWO

PENGUIN DEVELOPMENT GUIDE

2.1 Installation and `./penguin` usage

After cloning this repository you can use the local `./penguin` script to build your container and to run Penguin.

To build the image you can run:

```
./penguin --build
```

`--build` builds the image from the Nix flake (`nix build .#dockerImage`), loads it into your container engine, and runs it. It requires `nix` on your host (with flakes enabled); see *nix.md* for the full Nix build reference. With no `--image`, `--build` builds and runs `localhost/penguin:dev` so it never shadows the pinned `rehosting/penguin:latest`.

You can view all the wrapper arguments that may assist with development by running

```
./penguin --wrapper-help
```

The penguin wrapper arguments go before penguin commands (e.g. `run`, `init`), but can be combined, for example the following command will rebuild your container and then run the configuration in `projects/myfw/config.yaml`

```
./penguin --build run projects/myfw
```

Note that the standard `penguin` wrapper command that we recommend users install supports these development options, so you can use these wrapper flags even if the `penguin` command is installed for a user or system.

2.2 Iterating on Penguin's Python code

There are two loops, fastest first. Neither rebuilds the image.

2.2.1 Host-only logic + unit tests — `nix develop` (no container at all)

For plugin *host-side* logic, config/schema work, `gen_docs`, linting, and the `tests/unit/` suite, drop into the dev shell: it's the exact interpreter the image runs (`pythonEnv`, all runtime deps) with the live `src/`, `pengutils/`, and `pyplugins/` trees on `PYTHONPATH`, so `import penguin` resolves to your worktree — edits take effect immediately, no build, no boot.

```
nix develop            # then, inside the shell:
python3 -m pytest tests/unit/ pengutils/    # host-side tests in seconds
python3 -m penguin ...                       # host tooling against live sources
```

This can't run emulation (PANDA-QEMU, `/igloo_static`, and the guest tools only exist inside the image) — use it for everything that doesn't need a guest boot.

2.2.2 Plugin behavior in a running guest — `--dev` (no rebuild)

To iterate against a *real boot* without rebuilding, use `--dev`: it bind-mounts the live Python trees (`src/ -> /pkg`, `pengutils/ -> /pengutils`, `pyplugins/ -> /pyplugins`) and prepends them to `PYTHONPATH` (the image is an immutable `python3.withPackages` env with no pip, so this overlays the live tree rather than reinstalling). Edits apply on the next `run` — no rebuild, no root/pip step. (`--pydev` still works as a deprecated alias.)

```
./penguin --dev run projects/myfw     # edit src/, pengutils/, or pyplugins/ -> re-run
./penguin --build --dev run ...        # rebuild the image too (only if a baked-in dep␣
↪changed)
```

By default `--dev` finds the worktree to overlay by walking up from the current directory for a `src/penguin/` dir (falling back to the cwd), so it works from a subdirectory. To overlay a checkout elsewhere — e.g. when the installed `penguin` command runs from outside the repo — point it explicitly:

```
./penguin --dev --dev-root /abs/path/to/penguin run projects/myfw
PENGUIN_DEV_ROOT=/abs/path/to/penguin penguin --dev run projects/myfw
```

Precedence: `--dev-root` > `PENGUIN_DEV_ROOT` > auto-detect > cwd. A root that isn't a penguin worktree (no `src/penguin/`) fails loudly instead of silently mounting empty directories.

What's live vs. not (`--dev` prints this on startup):

- **Live** — `src/` (the `penguin` package *and* `src/resources/`: init scripts, `source.d/`, `static_keys`, … via `PENGUIN_RESOURCES=/pkg/resources`), `pengutils/`, and `pyplugins/`.
- **Live with `--dev-static`** — anything under `/igloo_static` (the guest tools busybox/console/guesthopper/vpnguin, kernels, `igloo.ko`, libnvram sources, guest-utils): overlay a built fragment at run time, no image rebuild (next section).
- **Not live — needs `--build`** — the qemu fork and the debug-tool closures. For a local sibling-repo checkout, combine with `--override-input` (see below).

2.2.3 Guest binaries in a running guest — `--dev-static` (no rebuild)

Everything binary that penguin uses at runtime lives under one directory in the image, `/igloo_static`, and the tool flakes' `dist` outputs are exactly fragments of that tree. `--dev-static <dir>` bind-mounts each **file** in a fragment read-only over its path in the image's `/igloo_static` — so a guest tool (or kernel) iteration loop needs no image rebuild at all:

```
cd ../vpnguin && nix build .#dist       # seconds once deps are cached
cd ../penguin
./penguin --dev --dev-static ../vpnguin/result run projects/myfw
```

Notes:

- The flag is **repeatable**; on overlapping paths the **later fragment wins**.
- Mounts are per-file, so a fragment only replaces what it ships — the rest of `/igloo_static` (other tools in the same `<arch>/` dir, sysroots, …) is untouched. The image's flat `vpn/vpn.<arch>` / `utils.bin/*` symlinks resolve through the mounts automatically.
- A **new** file (one the image doesn't ship) is mounted in fine, but the flat `utils.bin/<bin>.<arch>` links are generated at image build, so a brand-new tool won't get one — reference it by its full `/igloo_static/<arch>/<bin>` path, or do a real `--build` when adding a tool for keeps.
- For kernels / the driver: unpack the built tarball so its contents sit under `kernels/` inside your fragment dir (mirroring `/igloo_static/kernels/`).

- Doesn't cover the qemu fork (lives at `/usr/local`, not `/igloo_static`) — use `--override-input penguin-qemu` with a rebuild for that.

2.3 Dependency development

Penguin's dependencies (the PANDA-QEMU fork, kernels, igloo-driver, penguin-tools, the firmware-extraction stack, …) are pinned as **Nix flake inputs** in `flake.nix`, not downloaded at build time. To prototype a change to one of them, point its input at your local checkout for a build:

```
./penguin --image penguin:dev --build ...   # uses flake.nix as-is
# or build the image directly against a local sibling-repo checkout:
nix build .#dockerImage \
  --override-input penguin-qemu path:/abs/path/to/qemu \
  --accept-flake-config
docker load < result
```

To make the change permanent, edit that input's `url` in `flake.nix` and run `nix flake update <input>`. See *nix.md* for the input list and more detail.

CHAPTER

THREE

EXAMPLE: LOCAL KERNEL DEVELOPMENT

Clone the kernel builder and make your changes:

```
$ git clone --recurse-submodules git@github.com:rehosting/linux_builder.git && cd linux_
↪builder
$ # Edit some files in linux/ (not shown)
$ # Build kernels-latest.tar.gz, e.g. for just 4.10 + armel
$ ./build.sh --versions 4.10 --targets armel
```

Then build the penguin image against that local artifact by overriding the `kernels` input:

```
$ cd ../penguin
$ nix build .#dockerImage \
    --override-input kernels path:/abs/path/to/linux_builder/kernels-latest.tar.gz \
    --accept-flake-config
$ docker load < result
```

This builds an image with your kernel without needing to tag a new release of the dependency on GitHub. When you are finished, build without the `--override-input` to return to the pinned version.

CHAPTER

FOUR

HOOKLOGGER: FORMAT-STRING INSTRUMENTATION

HookLogger is the "easy mode" for dynamic binary instrumentation. You describe what to capture with a printf-style **action string** and it builds the uprobe or syscall hook for you — handling the parts that are tedious and architecture-specific:

- per-arch argument retrieval (calling conventions),
- memory dereferencing (strings, pointers, buffers),
- endianness conversion (detected automatically),
- pairing entry and exit probes so a hook can report a return value.

4.1 The action string

```
[bp|break] [print] func_name(arg_fmt, ...) [= ret_fmt, ...]
```

- `func_name` — the function or syscall to hook (`malloc`, `sys_read`).
- `(...)` — formats for the arguments, captured at **entry**.
- `= ...` — optional; formats for the return value(s), captured at **exit**.
- `bp` / `break` — drop into a Python debugger (PDB) when the hook fires.

4.1.1 Format specifiers

Specifier	Meaning	Source
`%d` / `%i`	signed integer	register/stack value
`%u`	unsigned integer	register/stack value
`%x` / `%X`	hexadecimal	register/stack value
`%p`	pointer (`0x...`)	register/stack value
`%s`	string (`char *`)	dereferenced at entry, read to the NUL
`%c`	character	low 8 bits
`%b`	boolean	True/False
`%fd`	file descriptor	resolved to a filename via OSI
`%proc`	process name	current process, via OSI
`%x64`, `%u32`, …	memory dump	reads that bit width (8/16/32/64) from the pointer

4.1.2 Deferred resolution: `:out`

Pointer formats such as `%s` are dereferenced at function *entry*. For output parameters — a buffer the callee fills in — you want the pointer captured at entry but the memory read at exit. Append `:out` to defer it:

- `%s:out` — capture the `char *` at entry, read the string at exit.
- `%x64:out` — capture the pointer at entry, read 64 bits at exit.

So to see the data a `read()` actually returned:

```
sys_read(%fd, %s:out, %d) = %d
```

4.2 Filters and output

Every registration takes the same optional scoping arguments:

Argument	CLI flag	Effect
`pid_filter`	`--pid`	only fire for this PID
`process_filter`	`--proc`	only fire for this process name
`logfile`	`--output`	append to this file in the results directory; otherwise the hook logs through penguin's logger

4.3 Driving it from the command line

The `breakpoint` CLI talks to a running guest over the RemoteCtrl plugin's Unix socket (`results/latest/remotectrl.sock` by default, overridable with `--sock`). RemoteCtrl loads HookLogger itself if it is not already loaded.

```
# trace every strlen argument in libc
breakpoint uprobe --path /lib/libc.so.6 --symbol strlen --action "print %s"

# what did this read() actually return?
breakpoint syscall --name sys_read --action "print %fd, %s:out, %d = %d"

# scope to one process, log to a file in the results dir
breakpoint uprobe --path /bin/mybin --symbol do_work \
    --action "print %x, %d" --proc mybin --output work.log

breakpoint list             # what is registered
breakpoint disable 3        # remove one hook by id
```

4.4 Driving it from Python

From a pyplugin, or any code with the plugin manager in scope:

```
plugins.load_plugin('hooklogger')

hook_id = plugins.hooklogger.register_uprobe(
    path="/usr/bin/wget",
    symbol="connect",
```

(continues on next page)

(continued from previous page)

```
    action_str="print(%fd, %p) = %d",
    process_filter="wget",
    logfile="connections.log",
)

plugins.hooklogger.register_syscall(
    "sys_read", "print %fd, %s:out, %d = %d", logfile="reads.log",
)

plugins.hooklogger.list_hooks()        # [{'id': .., 'type': .., 'action': ..}]
plugins.hooklogger.disable_hook(hook_id)
plugins.hooklogger.disable_all()
```

4.5 See also

- *uprobes.md* and *syscalls.md* — the lower-level interfaces HookLogger is built on, for when you need a real callback rather than a format string.
- *plugins.md* — the plugin catalogue, including RemoteCtrl.

CHAPTER

FIVE

KPROBES PLUGIN SYSTEM

The Kprobes plugin provides a flexible interface for monitoring, filtering, and intercepting kernel-space function calls in the guest via the IGLOO hypervisor and portal interface. This enables dynamic kernel tracing, security monitoring, and program analysis at the function level inside the guest kernel.

5.1 Overview

The kprobes system operates through a multi-layer architecture:

1. **Kernel Layer**: The IGLOO kernel module registers kprobes on kernel functions identified by symbol name. The guest kernel resolves the symbol via `kallsyms`, applies an optional byte offset, and supports process and PID filtering.
2. **Hypervisor Layer**: The portal system manages communication between guest and host, relaying probe events.
3. **Plugin Layer**: Python plugins register kprobes and process events via callback functions.

Unlike uprobes, which hook user-space code by file path and offset, kprobes hook **kernel** functions by symbol name. There is no path or address resolution on the host side; the symbol string is sent to the guest kernel, which resolves it.

5.2 Quick Start

```
from penguin import plugins, Plugin

class MyKprobeMonitor(Plugin):
    def __init__(self):
        # Register a kprobe on the kernel function 'do_filp_open'
        @plugins.kprobes.kprobe(symbol="do_filp_open")
        def on_open_entry(pt_regs):
            print(f"do_filp_open() called! PC={pt_regs.pc:#x}")

        # Register a return probe on a kernel function
        @plugins.kprobes.kretprobe(symbol="do_filp_open")
        def on_open_return(pt_regs):
            print(f"do_filp_open returned! Return value: {pt_regs.retval}")
```

5.3 Core Concepts

5.3.1 Kprobe Events

A kprobe event is generated when a monitored kernel function is entered or returned from. Each event provides access to the CPU register state (`pt_regs`) at the probe point.

5.3.2 Probe Registration

Probes are registered using the `@kprobes.kprobe()` or `@kprobes.kretprobe()` decorators, supporting symbol-based selection, an optional byte offset, and process / PID filtering.

```
@kprobes.kprobe(
    symbol="do_filp_open",
    process_filter="target_app",
    on_enter=True,
    on_return=False,
    pid_filter=1234
)
def my_handler(pt_regs):
    # Handler code here
    pass
```

5.4 API Reference

5.4.1 Decorator Parameters

`symbol` (str)

Kernel symbol name to probe. The guest kernel resolves it via `kallsyms`. Some kernel functions carry compiler-generated suffixes (e.g. `do_execveat_common.isra.0`); register on the concrete name. You can discover the real name with:

```
for name in plugins.kffi.ffi.search_symbols("do_execveat_common*"):
    ...
```

`offset` (int)

Byte offset from the symbol at which to place the probe (default `0`). Return probes (`kretprobe`) force `offset=0`.

> **MIPS caveat — use `offset=0` (function entry).** MIPS has no hardware single-step, so the kernel single-steps a probed instruction *out of line* (it copies the instruction to a scratch slot, executes it there, then resumes). For instructions in the middle of a function this can resume with corrupted CPU state and crash the probed thread. In testing, an entry probe (`offset=0`) and `kretprobe` on a function are rock-solid on all MIPS variants, but a non-zero offset into some functions (e.g. `vfs_read+4`) reliably Oopses the guest kernel (`get_signal` NULL-deref → SIGSEGV → panic), while the identical probe on ARM/x86 runs fine. This is an inherent MIPS kprobe (single-step-out-of-line) limitation, not specific to Penguin. Prefer entry probes / kretprobes on MIPS; treat non-zero offsets as unsupported there.

`process_filter` (Optional[str])

Filter by process name (comm):

```
@kprobes.kprobe(symbol="do_filp_open", process_filter="target")
def monitor_open(pt_regs):
    pass
```

`pid_filter` (Optional[int])

Filter by specific process ID:

```
@kprobes.kprobe(symbol="do_filp_open", pid_filter=1234)
def monitor_pid(pt_regs):
    pass
```

`on_enter` / `on_return` (bool)

Control when the probe triggers:

```
# Entry only (default)
@kprobes.kprobe(symbol="do_filp_open", on_enter=True)

# Return only
@kprobes.kretprobe(symbol="do_filp_open")

# Both entry and return
@kprobes.kprobe(symbol="do_filp_open", on_enter=True, on_return=True)
```

When a probe is registered for both entry and return, the handler is invoked twice. Use the injected `is_enter` argument (see below) to distinguish the two.

`read_only` (bool)

If True, modifications to `pt_regs` made in the handler are not written back to the guest.

`fail_register_ok` (bool)

If True, do not raise an error if registration fails.

5.4.2 Handler Function Signature

The handler always receives `pt_regs` as its first argument. Additional context can be requested using "signature sugar": declaring extra positional or keyword parameters injects context values.

```
def handler(pt_regs):
    """pt_regs: register state at the probe point."""

def handler(pt_regs, is_enter):
    """is_enter: True on function entry, False on return."""

def handler(pt_regs, is_enter, tgid_pid):
    """tgid_pid: (tgid << 32) | tid for the triggering task."""

def handler(pt_regs, **ctx):
    """ctx contains is_enter and tgid_pid."""
```

The injection rules:

- 1 extra positional arg -> `is_enter`
- 2 extra positional args -> `is_enter`, `tgid_pid`
- Parameters explicitly named `is_enter` / `tgid_pid` are bound by name.
- A `**kwargs` parameter receives `is_enter` and `tgid_pid`.

Handlers may be plain functions or generators (`yield from`) so they can perform portal memory reads.

5.5 pt_regs Object

The `pt_regs` parameter provides architecture-agnostic access to the CPU register state at the probe point:

```
def my_handler(pt_regs):
    print(f"PC: {pt_regs.pc:#x}")
    print(f"SP: {pt_regs.sp:#x}")
    print(f"Return value: {pt_regs.get_return_value()}")  # On return probes
    a, b = pt_regs.get_args(2)                              # First two args
```

Note that kernel functions follow the kernel calling convention; `get_args` returns register-passed arguments for the architecture.

5.6 Practical Examples

5.6.1 Inspect a kernel path open

```
@kprobes.kprobe(symbol="do_filp_open")
def on_open(pt_regs):
    args = pt_regs.get_args(5)
    struct_filename = yield from plugins.kffi.read_type(args[1], "struct filename")
    path = yield from plugins.mem.read_str(struct_filename.name.address)
    print(f"opening {path}")
```

5.6.2 Check a kernel return value

```
@kprobes.kprobe(symbol="do_filp_open", on_enter=False, on_return=True)
def on_open_return(pt_regs):
    retval = int(plugins.kprobes.panda.ffi.cast("target_long", pt_regs.get_return_
↪value()))
    if retval == -2:  # -ENOENT
        print("open failed: no such file")
    if False:
        yield
```

5.6.3 Probe a symbol with a suffix

```
for name in plugins.kffi.ffi.search_symbols("do_execveat_common*"):
    plugins.kprobes.kprobe(symbol=name)(on_exec)
    break
```

5.7 Hook Management

5.7.1 Unregistering

Probes can be unregistered by handle (the decorator return value), by function, or by name:

```
handle = kprobes.kprobe(symbol="do_filp_open")(on_open)
...
kprobes.unregister(handle)          # by handle
kprobes.unregister(on_open)         # by function
kprobes.unregister("on_open")       # by name
```

5.8 Integration with Other Plugins

5.8.1 With KFFI

Use `plugins.kffi.read_type` / `read_type_panda` to decode kernel structs referenced by probe arguments, and `plugins.kffi.ffi.search_symbols` to resolve symbol names.

5.8.2 With OSI

Combine with OS introspection for process metadata:

```
def handler(pt_regs):
    proc = yield from plugins.osi.get_proc()
    print(f"kprobe hit in process {proc.name} (PID {proc.pid})")
```

5.9 Best Practices

1. **Be specific**: Probe only the kernel functions you need to minimize overhead.
2. **Filter early**: Use process and PID filters to reduce event volume.
3. **Mind symbol suffixes**: Inlined/cloned kernel functions may carry `.isra.N`, `.constprop.N`, etc. — resolve the concrete name.
4. **Handle errors**: Memory reads and symbol resolution can fail; guard accordingly.
5. **Use `is_enter`**: For combined entry+return probes, branch on `is_enter`.

The kprobes system provides a powerful foundation for dynamic kernel analysis, security monitoring, and program understanding in virtualized environments.

CHAPTER

SIX

LIBRARY INJECTION (`LIB_INJECT`)

Penguin can replace library functions in the guest at runtime by preloading a small shared object (`/igloo/lib_inject_<abi>.so`, wired in via `/etc/ld.so.preload`). This is how NVRAM shims work, and it is the mechanism behind three user-facing config knobs, all under `lib_inject:`.

Because injection is a real `LD_PRELOAD`, it only affects **dynamically linked** guest binaries; statically linked programs are unaffected.

6.1 `lib_inject.aliases` — point a symbol at an existing shim

```
lib_inject:
  aliases:
    nvram_load: nvram_init      # call nvram_load -> run the nvram_init shim
    fputs: "false"              # fputs always "fails"
```

Maps a symbol name to the name of a function already defined in the injected library (the NVRAM shims, or the generic `libinject_ret_0` / `libinject_ret_1` constant returns). Wired with the linker's `--defsym`.

6.2 `lib_inject.extra` — inline custom C

```
lib_inject:
  extra: |
    int my_check(void) { return 1; }
```

Raw C compiled straight into the injected library. Pair with `aliases` to route a real symbol at your function.

6.3 `lib_inject.d/` — hand-authored shim files

Any `.c`/`.h` file dropped in `<project>/lib_inject.d/` is compiled into the injected library and the directory is added to the include path. This is the file-based counterpart to `extra:` — better when you have more than a snippet. Dotfiles and subdirectories are ignored, so `.generated/` (below) is never double-compiled.

6.4 `lib_inject.stubs` — declarative "just return a constant"

Forcing a function to "just return 0/1" is a constant rehosting move. `stubs:` makes it declarative: it **compiles down to the machinery above** — one generated C shim per stub, plus an `aliases` entry — rather than making you hand-write both.

```yaml
lib_inject:
  stubs:
    libX.so:                        # library key: a label + the glob target
      get_flag: { return: 0 }       # get_flag() -> 0
      set_mode: { return: 1, type: int }
      "nvram_*": { return: 0 }      # glob: every exported nvram_* symbol -> 0
    /lib/libc.so:                   # absolute guest path also accepted
      memcpy: { guard_null_args: [0, 1], return: 0 }
```

A stub takes one of two mutually exclusive forms.

Symbol return — replace a symbol globally (LD_PRELOAD), touching no binary on disk:

- **`return`** — the constant to return. Also the value returned on the NULL path of a guarded stub (defaults to `0` if omitted).
- **`type`** — the C return type of the generated shim. Defaults to `long` (register width), which is right for most integer/pointer returns; override for e.g. `int` or `void *`.
- **`guard_null_args`** — zero-based argument positions to NULL-check. If any listed argument is NULL the shim returns `return`; otherwise it **calls through to the real function** (via `dlsym(RTLD_NEXT, ...)`).

Assembly body — overwrite the instructions at one specific location in one specific binary:

- **`body`** — assembly assembled with keystone and written over the symbol's code. The stub key is `symbol` (patch at the symbol's start) or `symbol@offset` (`offset` is hex `0x..` or decimal, added to the symbol address). This form compiles down to a `static_files` `binary_patch` action — `binary_patch` stays the single owner of on-disk patching.
- **`mode`** — assembly mode passed to `binary_patch` (e.g. `arm`/`thumb`); defaults to the target arch's natural mode.
- **`expect`** — optional hex of the bytes expected at the patch site, checked before the overwrite.

```yaml
lib_inject:
  stubs:
    libvendor.so:
      hw_probe:        { body: "movs r0, #0\nbx lr", mode: thumb }  # return 0 in Thumb
      init_dsp@0x10:   { body: "nop" }                              # patch 16 bytes in
```

Unlike the symbol-return forms, an assembly-body stub edits exactly one binary (the library key), not every importer of the symbol.

6.4.1 Notes and limits

- **Library key.** It is an organizational label *and* the target for glob expansion. Aliasing itself is global (a stubbed symbol is replaced everywhere it is imported), matching how `lib_inject` works. An absolute path (`/lib/libc.so`) is matched exactly; a bare basename (`libX.so`) is searched for in the rootfs and must be unambiguous.
- **Globs** are expanded against the library's exported (`.dynsym`) symbols at build time. A glob that resolves to no library, an ambiguous basename, a stripped object, or zero matching symbols is a hard error — a silently-empty stub is worse than a loud failure.
- **`guard_null_args` call-through** preserves the first 8 integer/pointer register arguments only; floating-point, struct-by-value, and stack arguments are not carried through. It also imports `dlsym`, so use it only where a real dynamic linker is present (the normal LD_PRELOAD case).
- **Precedence.** A symbol may not appear in both `stubs` and `aliases`; doing so is a hard error, so there is exactly one owner per symbol.

- **Generated files.** The symbol-return shims are written to `<project>/lib_inject.d/.generated/` (regenerated on every build) so you can inspect the exact C that was compiled. They live under a dotfile subdir so the `lib_inject.d/` scanner ignores them.
- **Assembly bodies delegate to `binary_patch`.** The `body/symbol@offset` form does not invent a new patching mechanism — it resolves the symbol to a file offset (statically, from the rootfs) and emits a `static_files` `binary_patch`, which applies the edit at boot. Multiple body stubs on the same binary are coalesced into one action. Symbol resolution needs a non-stripped symbol table in the target object.

CHAPTER

SEVEN

LOCAL PLUGINS

Pengiun supports local Plugins in the project directory. Add the filename of the plugin without .py as you would a standard plugin in config.yaml.

7.1 Example

Here is a test plugin that prints test on load and a message on unload. testplugin.py :

```
from penguin import Plugin
import itertools
import string
import yaml


class testplugin(Plugin):
  def __init__(self):
      self.outdir = self.get_arg("outdir")
      print("Put your code here")
  def uninit(self):
      print("testplugin uninit")
```

Here is the changes to the config needed to enable said test plugin.

```
plugins:
  testplugin:
    description: 'test plugin'
    version: 1.0.0
```

CHAPTER

EIGHT

BUILDING PENGUIN WITH NIX

The `rehosting/penguin` image is built from a **Nix flake** (`flake.nix` at the repo root). There is no Dockerfile — the flake is the sole build path.

The flake produces a reproducible image: every dependency (the Python environment, the PANDA-QEMU fork, `/igloo_static`, the firmware-extraction stack, clang-20) is pinned by `flake.lock`, so two builds of the same commit yield the same closure. It replaces what the old multi-stage Dockerfile did with `apt` + `pip` + release downloads.

8.1 Prerequisites

- **Nix** with flakes enabled (`experimental-features = nix-command flakes`).
- The flake declares the shared **Cachix** substituter (`rehosting-tools.cachix.org`) in its `nixConfig`, so a first build pulls the heavy artifacts (cross toolchains, the qemu fork) from the cache instead of compiling them. Accept it when prompted, or pass `--accept-flake-config`.

A cold build with no cache hits is expensive (it cross-compiles guest helpers for ~14 arches and pulls the qemu closure); with Cachix it is mostly downloads.

Check your setup in one shot — verifies nix + flakes, cache reachability (with the exact fix if not), a container engine, disk headroom, and `/dev/kvm`:

```
./nix-dev.sh doctor
```

8.2 Build and run the image

The quickest path — `nix run` streams the image straight into your local Docker/Podman daemon (no multi-GB tarball realised in the store), tagged by the **nix build** rather than a shared `:latest`:

```
# Build + load into docker/podman, tagged rehosting/penguin:<nix-build-hash>
nix run --accept-flake-config .          # (or `nix run .#load`)
# ...
# Loaded rehosting/penguin:zpfmbzp06f6pwf5yw9l4sfknyxjbwzdn
# Run it with, e.g.:  ./penguin --image rehosting/penguin:<hash> run projects/your_fw
```

The tag is the image's layer-closure hash, so each distinct build loads under its own unique, reproducible reference: same inputs → same tag, any change → a new tag, and builds never clobber each other on `:latest`. The command prints the exact `--image` reference to run.

Alternatively, build a tarball first (e.g. to inspect it or push it), then load:

```
# Build the image (result is a docker-loadable tarball, tagged :latest)
nix build .#dockerImage --accept-flake-config

# Load it into the local Docker daemon as rehosting/penguin:latest
docker load < result

# Run with the normal wrapper, pointing at the loaded tag
./penguin --image rehosting/penguin:latest run projects/your_fw
```

`nix build .#dockerImage` emits a `dockerTools.buildLayeredImage` tarball; the image is named `rehosting/penguin:latest`, layered on `ubuntu:22.04` (so it keeps a normal FHS userland + `apt`), with the Nix-built components on top. `nix run/.#load` instead uses the streaming variant (`dockerImageStreamHashed`, `streamLayeredImage` with `tag = null`) — same layers, but piped directly into the engine and content-hash-tagged.

The `./penguin` wrapper's `--build` flag does the above for you — it runs `nix build .#dockerImage`, loads the result into your container engine, and retags it to the run image (`localhost/penguin:dev` by default, or whatever `--image` selects). It uses `buildLayeredImage` rather than the streaming variant on purpose: the per-layer tarballs are cached in the store, so a warm rebuild only re-tars the layers that actually changed (fast dev inner loop), whereas `nix run/.#load` re-streams the whole image each time:

```
./penguin --build run projects/your_fw   # rebuild from the flake, then run
```

8.3 Flake outputs

Built for both `x86_64-linux` and `aarch64-linux`. Build any with `nix build .#<name>`:

Output	What it is
`dockerImage`	the full `rehosting/penguin:latest` image tarball
`dockerImageStrea`	streaming variant of `dockerImage` (`nix run .#dockerImageStream | docker load`); no store tarball, still `:latest`
`dockerImageStrea`	streaming image tagged by the nix build hash (`:` <hash>), backing `nix run` / `.#load`
`docsImage`	`rehosting/penguin:docs` — the runtime image plus the sphinx toolchain + texlive, used by the docs release job
`portableImage`	`rehosting/penguin:portable` — same image with the Nix closure relocated out of `/nix`, for hosts that own `/nix` themselves (*details*)
`pythonEnv` *(default)*	the interpreter the image runs: penguin + all runtime deps
`penguin`	the penguin Python package alone
`pengutils`	the `pengutils` helper package
`penguinQemu`	the PANDA-QEMU fork staged as the `/usr/local` overlay
`iglooStatic`	the assembled `/igloo_static` tree (kernels, driver, guest utils)
`nativeHelpersTre`	the cross-built guest native helpers, `out/`<arch>`/`<bin>
`muslHeaders`	per-arch musl headers for the drop-in sysroots
`vhostDeviceVsock`	the static `vhost-device-vsock` host backend

Inspect without building the whole image, e.g. `nix build .#iglooStatic` then `find result/`.

8.4 How the pieces fit together

8.4.1 The qemu seam

The PANDA-QEMU fork is the one input most likely to change. It is consumed as **its own flake**, pinned by release tag:

```
inputs.penguin-qemu.url = "github:rehosting/qemu/v0.0.12";
```

We use the flake's `penguin-qemu` *package* output (not a `fetchurl` of the release tarball): the Nix-built `libqemu-system-*.so` / `qemu-img` carry `/nix/store` rpaths, so consuming the package drags its runtime closure (glibc, pixman, libfdt, glib, slirp) into the image — a plain tarball would leave those dangling. It also ships CFFI env modules built against this flake's CPython (3.13), so they match penguin's interpreter.

To move to a new qemu release (after it is tagged and released):

```
# edit flake.nix: inputs.penguin-qemu.url = "github:rehosting/qemu/vX.Y.Z";
nix flake update penguin-qemu --accept-flake-config
nix build .#dockerImage --accept-flake-config   # re-validate
```

The qemu fork's `build.sh` controls which system targets ship via `PENGUIN_SYSTEM_ARCHES`. The library name must match what penguin's `arch_registry` resolves — e.g. the x86_64 guest needs `libqemu-system-x86_64.so` (shipped from qemu **v0.0.12**; earlier releases only had the `intel64` alias).

8.4.2 Guest native helpers — dynamic vs. static

`nix/mk-native-helpers.nix` cross-compiles `guest-utils/native/*.c` per arch, mirroring that directory's `Makefile` **per-binary** link rules. This distinction is load-bearing, not cosmetic:

- `test_nvram`, `uprobes_test`, `proc_mtd_dynamic` are built **dynamic**, with an explicit `--dynamic-linker=/igloo/dylibs/ld-musl-<arch>.so.1` (`proc_mtd_dynamic` also links `-ldl`). These rely on `LD_PRELOAD` / `dlsym` at runtime. **A statically linked binary silently ignores `LD_PRELOAD`**, so e.g. `lib_inject.so`'s constructor never fires — the guest boots but the injected behavior is missing, with no error.
- `test_executable` is static but **unstripped**; everything else is `-s -static`.

If a helper "builds fine" but behaves wrong at runtime, diff its ELF against the Docker image's copy (`readelf -h/-l/-d`): a wrong static-vs-dynamic choice or a missing `INTERP` is invisible until the guest runs.

8.4.3 `/igloo_static`

`nix/mk-igloo-static.nix` assembles `/igloo_static` from the pinned kernel, igloo-driver, and penguin-tools release tarballs plus the `guest-utils` source, reproducing the Dockerfile's per-arch symlink staging. The result is golden-diffed against the Docker image's tree.

8.4.4 The portable image: running where something else owns `/nix`

Everything in the normal image is a symlink into `/nix/store` — `/bin/bash`, `/bin/python3`, `/usr/local/bin/penguin`, `/igloo_static`, `/pyplugins`. That is fine until a host mounts *its own* `/nix` over the container's, which shadows the store and takes all of penguin with it. The image doesn't degrade; it vanishes. `docker run` fails before exec with `stat /usr/local/bin/penguin: no such file or directory`, leaving only Ubuntu's real `/usr/bin`.

The concrete case is the **pwn.college dojo platform**, which starts every challenge container with a read-only bind of its own workspace store at `/nix` because its userland lives there. You can reproduce the failure against any normal image in one command:

```
mkdir /tmp/empty
docker run --rm -v /tmp/empty:/nix:ro rehosting/penguin:latest penguin --version
```

There is no runtime workaround. Re-mounting our store over theirs needs `mount`, and moby's default seccomp profile — which that platform uses — gates `mount/umount2/unshare/setns/pivot_root` behind `CAP_SYS_ADMIN`; unprivileged challenge containers get only `SYS_PTRACE`.

`portableImage` solves it by moving the closure somewhere nobody else claims:

```
nix build .#portableImage --accept-flake-config
docker load < result                        # rehosting/penguin:portable

# It still works with /nix shadowed:
docker run --rm -v /tmp/empty:/nix:ro rehosting/penguin:portable penguin --version
```

The closure lands at **`/opt/store`** instead of `/nix/store`, and every reference to it is rewritten. The prefix is deliberately the **same byte length** as `/nix/store`, which is what makes the rewrite safe: it happens in place, so ELF `PT_INTERP`/`DT_RUNPATH` strings, compiled-in path constants, shebangs and `.pyc` paths all stay structurally valid — no `patchelf`, no per-format handling. `nix/relocate-store.py` does the rewrite and fails the build if any reference to the old store survives. `portablePrefix` in `nix/mk-image.nix` can change it, but only to another 10-byte path.

Two things to know before reaching for it:

- **It is one large layer, not 115.** `contents` can't be used, because `streamLayeredImage` assembles its customisation layer with `symlinkJoin` — which would leave the image a farm of symlinks back into `/nix/store`. So the relocated tree is copied in as real files with `includeStorePaths = false`, which forgoes per-store-path layer dedup between releases.
- **Prefer the normal image** wherever `/nix` is yours. This variant exists for hosts that take it away.

8.4.5 Writable paths

The image is otherwise immutable, but qemu writes its `snapshot=on` drive overlay under `/var/tmp`, so `nix/mk-image.nix` materializes a writable `/var/tmp` (and `/tmp`, `/root`). Without it the guest fails to launch with "Could not open temporary file '/var/tmp/vl.XXXXXX'".

8.5 Architecture coverage

The flake builds every bootable **system** target — **12** distinct arches:

```
armel  aarch64  mipsel  mipseb  mips64el  mips64eb
powerpc  powerpc64  powerpc64le  riscv64  loongarch64  x86_64
```

`powerpc64el` is an accepted *alias* of `powerpc64le` (same `ppc64-softmmu` library), so it is not a separate build. `x86_64` and `intel64` likewise share one compiled module.

8.6 CI integration

`.github/workflows/build.yaml` validates the flake on every PR **alongside** the Docker build:

- **`build_container_nix`** — runs `nix build .#dockerImage` (via `rehosting/ci/actions/nix-setup`, pulling from the `rehosting-tools` Cachix cache), loads it, and pushes the image under a `<sha>-nix` tag. The Docker `build_container` job is untouched and still pushes `<sha>`.

- **run_tests_nix** — runs the full `basic_target` + `test_target` suites against the `<sha>-nix` image across all 12 system arches. This includes `powerpc` (32-bit BE) and `powerpc64le`, which the flake ships but the legacy qemu `.deb` does not — the Docker matrix stays at its original 10.

8.7 Updating pinned inputs

All external artifacts are flake inputs pinned in `flake.lock`. The easy path is `./nix-dev.sh`, which knows each input's URL shape (github tag, release-tarball URL, busybox's git ref+rev) and rewrites `flake.nix` for you:

```
./nix-dev.sh pins                 # every versioned input: pinned vs latest upstream tag
./nix-dev.sh bump vpnguin         # bump one input to its latest tag + relock
./nix-dev.sh bump kernels v3.6.4  # or to a specific tag
```

The raw mechanism underneath (also what you use for the rev-pinned inputs like `fw2tar` and `nixpkgs`, which `bump` doesn't cover):

```
nix flake metadata                          # show every input + its pin
nix flake update <input> --accept-flake-config   # relock one after editing its url
nix flake update --accept-flake-config            # relock all
```

Inputs include `penguin-qemu`, `kernels`, `igloo-driver`, `penguin-tools`, the guest tools (`console`, `busybox`, `guesthopper`, `vpnguin`, `libnvram`), `musl-src`, `ltrace-src`, `vhost-device`, and `fw2tar` (whose closure supplies the firmware-extraction stack). Re-run `nix build .#dockerImage` after any bump.

8.8 Dependency overrides (replacing `local_packages/`)

The old Dockerfile let you drop a prebuilt artifact into `local_packages/` to override a pinned dependency. For anything that lands under `/igloo_static` (guest tools, kernels, the driver), the successor is **`./penguin --dev-static <dist>`** — a run-time overlay, no image rebuild (see the Development loop below). For inputs baked elsewhere in the image (the qemu fork, the debug-tool closures), point the flake input at your checkout and rebuild:

```
# Build + load the image with a local checkout substituted in:
./nix-dev.sh override vpnguin ../vpnguin

# Or the raw form, for a plain build:
nix build .#dockerImage --override-input penguin-qemu path:/abs/path/to/qemu --accept-
↪flake-config
```

For a permanent change, edit the input's `url` in `flake.nix` and `nix flake update <input>` (see *Updating pinned inputs*).

8.9 Development loop

The image is an immutable `python3.withPackages` env (no pip, read-only `/nix/store`), so iteration doesn't reinstall — it overlays live sources:

- **`nix develop`** — the host-side fast lane. Drops into `pythonEnv` with `src/`, `pengutils/`, and `pyplugins/` on `PYTHONPATH`, so `import penguin` resolves to the worktree. For host-side plugin logic, `tests/unit/`, `gen_docs`, and linting — no container, no boot. (Not emulation: PANDA-QEMU and `/igloo_static` only exist in the image.)
- **`./penguin --dev run ...`** — iterate on `src//pengutils//pyplugins/` against a real guest boot without a rebuild: bind-mounts `src/`→`/pkg`, `pengutils/`→`/pengutils`, `pyplugins/`→`/pyplugins` and prepends them to `PYTHONPATH`. (`--pydev` is a deprecated alias.)

- **`./penguin --dev-static <dist> run ...`** — iterate on anything under `/igloo_static` (guest tools, kernels, `igloo.ko`, guest-utils) without a rebuild: bind-mounts each file of a built fragment (e.g. a tool flake's `dist` output) over the image's copy, read-only. Repeatable; later wins. See *dev.md* for details.
- **`./penguin --build`** — a full image rebuild (`nix build .#dockerImage`), needed only when something *baked into the image* changes (guest tools, qemu, kernel, native helpers). For a local dep checkout, add `--override-input <input> path:/abs/checkout`. `buildLayeredImage` re-tars only the changed layers, but Nix evaluation of the flake (~tens of seconds) is a fixed per-build floor.

See *dev.md* for the full walkthrough.

CHAPTER

NINE

PENGUIN PLAYBOOK

As you go through your rehosting loop, editing configs and seeing what happens when they're run, you're trying to mitigate observed failures and improve system health. There are three key choices you'll want to focus on at the start:

- init program selection
- pseudofile modeling
- kernel environment variables

9.1 Init program selection

In a Linux-based system, the **init program** is the first program (script or binary) run by the kernel. This program is responsible for starting all other programs on the system. If the wrong program is selected, it might crash, error, or even run successfully and exit. An init program should never do any of these things: you want your init program to run until the system shuts down.

Penguin configs set the init program in the `env` section as the `igloo_init` field.

Your initial rehosting configuration is automatically populated with a potentially-correct init binary. However this may be incorrect and you might want to change it.

9.1.1 When to change init

If you firmware kernel panics when init exits with a code of 0, you probably have the wrong init binary. In this case towards the end of your `console.log` you'll see a line like this:

```
[   63.581671] Kernel panic - not syncing: Attempted to kill init! exitcode=0x00000000
```

If you instead see a similar line with a non-zero exit code, this could indicate that you have the wrong init selected or that something else is going wrong causing the correct init program to crash. You could either try another init or try tracking down other failures and seeing what changes.

9.1.2 Potential init programs to choose from

Penguin's initial static analysis writes its results to `<project_dir>/static/`. The init candidates are the list in `static/InitFinder.yaml`, shortest path first. `static/EnvFinder.yaml` carries the same list under its `igloo_init` key, alongside the other environment variables the analysis suggests. Note that this list isn't comprehensive (it's just finding executables that contain `start` or `init`), but it will usually find the right binary.

9.1.3 How to change init:

Set `env.igloo_init` **in a patch**, not in `config.yaml`:

```
# patch_init.yaml
env:
  igloo_init: /your/desired/init
```

Then either drop that file into `<project_dir>/patches/` — every `*.yaml` there is applied automatically, no `config.yaml` edit needed — or append it to the `patches:` list in `config.yaml`.

Editing `env.igloo_init` in `config.yaml` **looks like it works and silently does nothing**. `penguin init` records the init it picked in `static_patches/base.yaml`, and patches are applied *on top of* `config.yaml`, so the generated value wins and the guest goes on dispatching the original init. Patches later in the list override earlier ones, which is why a user patch has to come after `static_patches/base.yaml`.

9.2 Pseudofile modeling

Unlike regular files stored on disk, files in `/proc` (procfs) and `/dev` (devtmpfs) aren't really a part of your filesystem. Instead these files just a way for user space applications and the kernel to communicate. Through this interface applications can learn about the hardware state of a system and interact with attached peripherals.

In the rehosting context, many changes will be visible here. Many hardware peripherals a system expects to run with will no longer be present or, if they are present, behave differently than expected. While many of these changes are acceptable, some will be fatal and must be handled in your rehosting config.

For the full set of models you can attach to a pseudofile — including stateful registers, sequenced responses, ioctl struct-fills, and the seek/open/release ops — see the *pseudofile model catalog*.

9.2.1 When to create pseudofile models

If you see errors in your `console.log` about missing devices or it seems like services are crashing or failing to start, pseudofiles are a good place to begin making changes.

9.2.2 Potential pseudofiles to add

After each run, the `pseudofiles` plugin will populate the `pseudofiles_failures.yaml` file in the output directory. The first layer of keys show the filenames that programs tried and failed to access during execution. Within each of those values, you'll get the name of the interaction as well as a count of the number of times it was attempted.

Note that you don't generally need to model every pseudofile you see here. These models are generally low quality shims just trying to get a program to stop crashing. But if applications are behaving correctly when a pseudofile is missing, you may be better off by leaving that alone instead of creating a model and trying to make it correct.

First, you'll see failed accesses to a pseudofile and add this file into your config. Then, you can run with the new config and see if and how applications try to interact with the newly added pseudofile.

9.2.3 Modeling pseudofiles

Beyond allowing you to add pseudofiles into a system, penguin also allows you to specify how `read`s, `write`s, and `ioctl`s of these files should be modeled.

After adding a pseudofile to a config and running it, you might see guest applications try to interact with this newly created psueodfile. The `pseudofiles` plugin will collect the details of these accesses in the `pseudofiles_modeled.yaml`.

In this file, you'll see keys of device paths with a list of interactions that were modeled on that device. You'll see details for `read`s, `write`s, and `ioctl`s.

Each of these three behaviors can be modeled in various ways. By default, the `default` model is used. The *docs/schema_doc.md* contains auto-generated documentation for these fields which will always be the most up to date. However, the following descriptions may be of value.

Read modeling

default: Return an empty string with return value `-EINVAL`.

const_buf: Given some string in `val` model the file as containing that data.

const_map: Given a list of constants, model the file as a large buffer with just the specified values set. Arguments: `size` for total size, `pad` for what to place between values (byte value as int or a character). Provide a dictionary in `vals` with keys as an integer offset into the buffer and values as strings or lists of byte values.

const_map_file: like const_map, but adds a `filename` field. Will create this file on your host (container) if it doesn't exist with contents based on the const_map details and then read from the host file.

from_file: Given a host (container) file path in `filename` read from that file

Write modeling

default: Return value `-EINVAL`

discard: Do nothing with the value and return as if the write was successful.

to_file: Given a host (container) file path in `filename` write to that file

IOCTL modeling

IOCTLs have a command number and each command can be modeled distinctly. A wildcard `*` can be used as a command number to indicate that all other ioctls should be modeled in a given way.

default Return `-ENOTTY`

return_const: Return the specified `val`

return_symex: Coming soon.

9.2.4 How to model pseudofiles:

In your config file, you'll insert new keys udner `pseudofiles` for each file you want to model. By specifying a key (which must start with `/dev/` or `/proc/`), you'll change the system so that a pseudofile is present at the specified location. If this is all you wish to do, you'll specify the key as having a value of `{}`. Otherwise, if you'd like to model the behavior of the pseudofile, you'll add one or more subkeys of `read`, `write,` and `ioctl` and specify the model details.

To just add `/dev/missing` into the filesystem:

```
pseudofiles:
  /dev/missing: {}
```

To actually model what happens once this device is accessed we could expand this so

- reads return the string hello world,
- writes appear to work but actually do nothing, and
- IOCTLs all return 0

```
pseudofiles:
  /dev/missing:
    read:
        model: const_buf
        val: "hello world"
    write:
        model: discard
    ioctls:
        '*':
            model: return_const
            val: 0
```

9.3 Kernel environment variables

Before the Linux kernel is launched, a bootloader typically sets up a system with some initial state. There are two key sets of environment variables we may care about, the first is much more common than the second.

Linux kernel boot arguments: these are arguments passed to the Linux kernel at boot time. These control things such as where the root filesystem is and there serial console configuration. A system's bootloader may be configured to pass nonstandard arguments through these arguments.

The init program will be given many of these values in its environment while regular applications may examine these arguemnts by reading from `/proc/cmdline`.

U-Boot Environment: If the `U-Boot` bootloader is used, it may have its own set of environment variables that can be passed through to the Linux kernel through one of the `/dev/mtdX` devices (where X is a number). These are typically stored with a hash followed by null-terminated key=value pairs: `[crc32][key=value]\0[key=value]...`.

A mapping of MTD device names to the corresponding `/dev/mtdX` is available at `/proc/mtd`. Applications may hardcode which MTD devices they try to read from or dynamically search `/proc/mtd` to find which device a given name corresponds to.

Typically a custom binary is used to access U-Boot environments with support for getting and setting keys. These are often based off the open source fw_env program.

9.3.1 When to set boot arguments

After running a configuration, examine `env_missing.yaml` to find a list of environment variables that were searched for in `/proc/cmdline`.

If you're unsure of what values you might want to set a key to, try the magic value `DYNVALDYNVALDYNVAL` in your config and run again. Then examine the generated `env_cmp.txt` output file which will report strings that this magic value was compared against.

Alteratively, you might want to search through the filesystem to identify which binaries or scripts are parsing this environment variable and reverse engineer them to determine a good value. For identifying such binaries, extract the `<project_dir>/base/fs.tar` file and then `grep` through all files. For example, to find programs that reference `myvar` you could do:

```
# Extract filesystem
mkdir /tmp/fs
tar xvf base/fs.tar -C /tmp/fs

# Find binaries and scripts that reference `myvar`
grep --binary 'myvar' /tmp/fs'
```

Matching binaries can be analyzed with a tool like Ghidra while scripts can be analyzed with any text editor.

9.3.2 How to set boot arguments:

In your config file add new values into the `env` section as key-value pairs.

```
env:
  my_env_name: my_value
```

9.3.3 When to set U-Boot Environment variables

If you see output in `env_mtd.txt`, penguin detected an application searching for an MTD device with a specified name; `env_uboot.txt` logs the U-Boot variable names it went looking for. When you see this, give the guest a flash device of that name.

Flash has its own subsystem, configured under `plugins.mtd.devices`. Each entry is named, and that name is what the guest matches on in `/proc/mtd`:

```
plugins:
  mtd:
    devices:
      flash:
        model: backing_file          # persists guest writes to the host file
        backing_file: ./flash.bin    # relative paths resolve inside the project
        personality:
          type: nor
```

Then, from the root shell, `/proc/mtd` lists the name-to-device mapping.

Match on the name, not the index. The kernel assigns `/dev/mtdN` numbers as devices register, so a device named `flash` is not necessarily `/dev/mtd0`. Look its node up by name:

```
MTD=$(grep flash /proc/mtd | cut -d: -f1)   # -> mtd3, say
cat /dev/$MTD
```

The available models (`zeros`, `const_buf`, `backing_file`), the `personality` geometry fields and their defaults, and how to implement custom flash physics in Python are documented in *pseudofile_models.md*.

Legacy `pseudofiles:` entries for `/dev/mtdN` are migrated into this subsystem automatically, so older configs keep working — but new configs should use `plugins.mtd.devices` directly.

9.4 Stubbing library functions

When a program calls a library function that misbehaves under emulation (a hardware probe, a licensing check, a NVRAM accessor), forcing that function to return a constant is often enough to get past it. Declare it under `lib_inject.stubs`:

```
lib_inject:
  stubs:
    libvendor.so:
      hw_present: { return: 1 }
      "check_*":  { return: 0 }        # glob over exported symbols
    /lib/libc.so:
      memcpy: { guard_null_args: [0, 1], return: 0 }  # skip on NULL, else real
```

This compiles down to the injected library (a generated shim + a `--defsym` alias), so it reuses the same build path as `lib_inject.aliases/extra`. See *lib_inject.md* for the full reference, including globs, return-type overrides, the `guard_null_args` call-through, and its limits.

9.5 Advanced debugging

If you've tried selecting the correct init program, modeling psueodfiles, and adding environment variables but things are still failing, you'll need to try some more involved debugging.

First examine the console output for error messages that relate to processes being killed, missing files, bad arguments, and so on. Examine scripts and binaries as necessary.

Next examine output from other plugins. The `shell_cov_trace.csv` script may be particularly useful as it shows each line from shell scripts executed in the order they were run with concrete values listed along with each variable. For example if a script `foo.sh` has a 10th line of `if [ -e $myfile ]; then` and the `myfile` variable was set to `/root/myfile`, the log would show this as:

```
foo.sh:10,if [ -e $(myfile=>/root/myfile)]
```

Next enable the root shell by changing your config's `base` section's `root_shell` value to be `true`. Then run your target and connect with `telnet as described in the Penguin output to get a root shell.

After launching this, press enter a few times and perhaps wait ~10s. You should then get a root prompt and be able to run shell commands.

From this shell you can try running `strace` on various guest applications to see how they behave dynamically. You can examine running processes with `ps aux` and then connect strace to a running process with `strace -p [PID]`

If your guest is kernel panicking and shutting down or you'd just like more explicit control of what's being run, you can change your config to skip running the right init program and instead just launch a shell that doesn't exit by setting:

```
env:
    igloo_init: /igloo/utils/sh
```

CHAPTER

TEN

PENGUIN PLUGINS

The following penguin plugins are currently supported. Each is documented below.

- *Coverage*: Track block-level coverage of binaries
- *Env*: Track usage of boot arguments, environment variable accesses, and environment variable comparisons.
- *Health*: Track system health metrics including processes run.
- *Interfaces*: Track network interfaces referenced
- *KernelVersion*: Change output of uname syscall
- *Lifeguard*: Track and block signals
- *Mounts*: Track attempts to mount file systems
- *NVRAM2*: Tracks accesses to NVRAM
- *Netbinds*: Track network listening guest processes
- *Nmap*: Network scanning for guest applications that bind to TCP ports
- *Pseudofiles*: Model and monitor interactions to devices in `/dev` `/proc` and `/sys`
- *Shell*: Track behavior of shell scripts including lines executed
- *VPNguin*: Bridge network connections to networked guest processes
- *Zap*: **Currently disabled** Network scanning of guest web applications

10.1 Coverage

This plugin tracks the module and offset block level coverage of all binaries in the system. These results are reported in `coverage.csv`. The file `coverage_tree.csv` stores this information with parent/child relationships to visualize as a tree. The file `coverage_transitions.csv` records all context switches between processes.

10.2 Crashes

The `crashes` plugin records fatal signal deliveries to guest processes (userland crashes) in `crashes.yaml`. Each record includes the process name, pid, signal number and name, the faulting program counter, the time (host wall-clock seconds since emulation start) of the first occurrence, and a `count` de-duplicating identical `(proc, signal, pc)` deliveries. By default it records SIGSEGV, SIGBUS, SIGILL, SIGABRT, SIGFPE, and SIGSYS; the set is configurable via the `signals` argument (use signal names — numbering is resolved per guest architecture). The number of unique crash records is also reported (negated) as the `crashes` score category.

Note that records are signal *deliveries*, not confirmed terminations: the driver hook fires when a signal is dequeued for delivery, before the kernel applies its disposition, and the event does not indicate whether the process has a handler

installed. A process that catches a fatal-class signal and survives is still recorded. Deliveries that another plugin has already dropped (`event.drop`, e.g. a SIGILL-bypass consumer) are not recorded.

10.3 Env

The `env` plugin dynamically tracks linux environment variables accessed through `/proc/cmdline` and calls to `getenv`. It also tracks accesses to `/proc/mtd` as well as `/dev/mtdX` to identify accesses to u-boot environment variables.

If an env value is set to the magic string `DYNVALDYNVALDYNVAL` a dynamic analysis to detect comparisons between this magic string and any other string will be enabled. The results of this analysis will be stored in `env_cmp.txt`

In a config file, a user may add key-value pairs into the `env` filed to set new values into the linux environment. Note that a number of required internal variables (e.g., `root=/dev/vda`) will added to the system's arguments *after* any arguments you specify here.

10.4 Health

The `health` plugin tracks the system health over time. `health.csv` tracks counts of various behaviors of interest over time while `health_final.yaml` just reports these values at the end of execution.

The plugin also creates `health_procs.txt` as a sorted list of processes run and `health_procs_with_args.txt` as a sorted list of processes with their arguments.

10.5 HookLogger

"Easy mode" instrumentation: register uprobes and syscall hooks from printf-style action strings (`sys_read(%fd, %s:out, %d) = %d`) instead of writing callbacks. Drive it from the `breakpoint` CLI (over the RemoteCtrl plugin's socket) or the Python API. The grammar, format specifiers and filters are in *hooklogger.md*.

10.6 Interfaces

Track network interfaces referenced in executed commands. Results are reported in `iface.log`.

10.7 KernelVersion

Change output of uname syscall. Config Options:

- `sysname`: changes operating system name
- `nodename`: changtes network node hostname
- `release`: changes the kernel release
- `kversion`: changes the kernel version
- `machine`: changes the machine hardware name
- `domainname`: changes NIS or YP domain name

10.8 Lifeguard

Track and block signals sent between processes. Results are stored in `lifeguard.csv`.

Lifeguard suppresses configured signals sent through supported signal-send syscalls by skipping the syscall before the kernel sends the signal. Supported syscalls are defined in one table in the plugin and resolved through the syscall prototype argument helpers supplied by the driver, falling back to fixed indexes only if the prototype names are unavailable. It also consumes `signal_monitor` for configured signals other than `SIGKILL` and `SIGSTOP`, allowing delivery paths to be observed and dropped when the driver hook is effective.

Delivery-time drops are not equivalent to preventing a signal from being sent. The driver hooks the kernel dequeue path, so it can drop catchable signal deliveries after the kernel has selected a target. Default-fatal signals that do not have a userspace handler may start process or thread-group exit before that dequeue path is reached. For process-preserving behavior, the preferred path is still to block the sending syscall before the kernel applies signal semantics. `SIGKILL` and `SIGSTOP` cannot be caught or ignored by Linux processes, so Lifeguard treats them as syscall-only and only suppresses instances generated by supported signal-send syscalls.

The syscall path covers the supported signal-send syscalls currently listed by Lifeguard: `kill`, `tkill`, `tgkill`, `rt_sigqueueinfo`, `rt_tgsigqueueinfo`, and `pidfd_send_signal`. Signals sent by other kernel paths or unsupported syscalls may be visible through delivery monitoring when the driver hooks them, but the syscall fallback will not prevent them before send time.

Some signals also require guest state repair when dropped. For example, blindly dropping synchronous fault signals such as `SIGILL`, `SIGSEGV`, `SIGBUS`, `SIGFPE`, `SIGTRAP`, or `SIGSYS` can immediately re-enter the same faulting instruction. Use a purpose-built `signal_monitor` consumer, such as `sigill_bypass.py`, when the handler needs to advance the PC, emulate an instruction, or otherwise repair guest state.

10.9 Mounts

Track which file systems are mounted (or attempted to be mounted) at which paths. Results stored in `mounts.csv`. Note this plugin will track some penguin-internal initialization logic with mounts in the `/igloo` directory.

10.10 NVRAM2

This plugin tracks accesses to keys and values stored in NVRAM. Results are stored in `nvram.csv`

10.11 Netbinds

This plugin detects and logs network binds by guest processes. The results are logged into `netbinds.csv` and include a `time` column indicating how many seconds after boot until the bind occurred, plus `state` and `closed_time` columns describing each socket's final lifecycle state:

- `listening` — announced and still bound.
- `transient` — released within the announce window; not a working service.
- `closed` — listened, then released for good (`closed_time` set).
- `pending` — bound when the run ended but younger than the announce window; never confirmed, so not a working service either.

New binds are announced to downstream plugins (VPN, Nmap, FetchWeb) only after an announce debounce (`announce_debounce_s`, default 2s), so short-lived binds never trigger VPN bridging or scans. Both the announce and the close/flap debounce windows use host wall-clock time, so the classification of a very short-lived bind can vary with emulation speed. Every open/flap/ transient/close transition is appended to `netbind_events.csv`, and a per-socket lifecycle summary is written to `netbinds_lifecycle.csv` at run end.

The socket lifecycle survives a VM snapshot/restore: the record is bundled with the snapshot and rebuilt on restore (a restored guest is already past its binds and would otherwise vanish from `netbinds.csv`). Already-announced services are rebuilt silently — downstream consumers replay their own state — while a service still within its announce window at snapshot time is announced on restore.

10.12 Nmap

This plugin runs nmap scans on all network-listening services. It depends on the VPN plugin to establish network connections to guest services. Logs are written to `nmap_{protocol}_{port}.log`

10.13 Pseudofiles

This plugin tracks accesses and interactions with files in `/dev/` and `/proc/`. In `pseudofiles_failures.yaml` details of failed interactions are reported, ranked by impact (`crashing_callers` >> `distinct_callers` > `hits`), with the observed calling processes and a suggested starting model per path that can be pasted directly into the `pseudofiles` config section.

Users can add pseudofiles and configure models for reads, writes, and IOCTLs on these files by adding entries into the `pseudofiles` config section. See the *pseudofile model catalog* for every available model (read/write/ioctl/poll/seek/mmap/open/release), copy-paste snippets, and how to add custom models in Python via `@register_model`.

10.14 Shell

This plugin tracks the behavior of shell scripts, capturing coverage in `shell_cov.csv`, environment variable values in `shell_env.csv` and a combined trace in `shell_cov_trace.csv`.

10.15 VPNguin

This plugin detects network binds and configures a custom VPN to bridge network connections to guest services. The mappings between guest network services and what port the VPN exposes them on are listed in `vpn_bridges.csv` For example, if the file contains:

```
procname,ipvn,domain,guest_ip,guest_port,host_port
lighttpd,ipv4,tcp,127.0.0.1,80,80
lighttpd,ipv4,tcp,192.168.0.1,80,48823
```

This means `lighttpd` started listening on port 80 on the loopback interface as well as another IP address. To talk to the service as if you were connecting via loopback, you'd connect to the relevant `host_port`, here 80. To talk to the service as if you were connecting via the other IP address, you'd connect to the other `host_port`, here 48823. Note these are ports within your container, not on your host, so you must connect to the appropriate IP address to reach the container.

The plugin also writes two convenience files in the results directory, regenerated as bridges are established: `guest_ip` (the bare reachable IP, same content as `ipaddr.txt`) and `endpoints.txt` (one `proto host_ip:host_port -> guest_ip:guest_port (proc)` line per bridge).

10.15.1 The published port is not always the guest's port

`host_port` matches `guest_port` when it can, but the plugin has to move it when that port is unavailable — already bound in the container, already claimed by an earlier bind in this same run, or privileged (below 1024) when the process cannot bind privileged ports. It then searches deterministically in strides of 1000: guest port 80 becomes 1080, or 2080 if 1080 is taken too, and so on.

So the same guest port can be published on different host ports in different runs of the same container, without anything having changed in the config. That is working as intended, but it means **nothing may hardcode a host port**. Read it from `endpoints.txt` (or the `host_port` column of `netbinds.csv`), which is the only source of truth for a given run. The run log also states the mapping and the reason for it as each bind happens:

```
      lighttpd binds tcp 0.0.0.0:80    reach it at container foo:2080   80 is already␣
↪in use
```

An explicit mapping is available when you need a fixed port: set `plugins.vpn.IGLOO_VPN_PORT_MAPS` (or the `IGLOO_VPN_PORT_MAPS` environment variable) to comma-separated `<proto>:<host_port>:<guest_ip>:<guest_port>` rules, e.g. `tcp:8080:192.168.0.1:80`. A fixed mapping is never relocated: if the host port is busy the run fails loudly instead.

10.15.2 connect.sh

Every run also gets an executable `results/<n>/connect.sh`, generated by penguin (not the VPN plugin) next to `runtime.yaml`. It is the one-glance "get me into this device" helper and talks to the guest's **serial root shell** through the container's localhost (`docker exec <container> telnet localhost <port>`), so it does **not** depend on the container's per-run IP address:

- `./connect.sh` — attach to the serial console.
- `./connect.sh <cmd...>` — run a command on the serial console, print its output, and exit with the command's own status. Waits up to `PENGUIN_CONNECT_TIMEOUT` seconds (default 15) for the guest to answer, and exits 1 with a diagnostic if it does not — so it is safe in a script:

```
if ./connect.sh "test -f /var/run/httpd.pid"; then echo "httpd is up"; fi
```

It requires `core.root_shell: true` (there is no serial console otherwise) and prints how to enable it when it is off.

10.16 ZAP

Currently disabled This plugin runs the zap web application scanner to crawl and interact with guest web applications listening on TCP port 80. Logs are written to `zap.log` and `zap_tcp_80.log`.

CHAPTER

ELEVEN

PORTAL

11.1 What is Portal?

Portal is a powerful memory introspection and manipulation tool that provides a cooperative, hypercall-based protocol for reading from and writing to guest memory. Unlike traditional memory introspection approaches that directly access the physical memory, Portal uses a cooperative approach between the guest and hypervisor, providing robust memory access and manipulation capabilities across different architectures and OS versions.

11.2 How Hypermem Works

1. **Guest-Hypervisor Communication:** The guest OS registers a memory region with the hypervisor via a hypercall. This region serves as a communication channel between the guest and hypervisor.
2. **Memory Operations:** The hypervisor can request various operations (read, write, etc.) by writing commands to this shared memory region. The guest kernel then processes these requests and writes results back to the shared region.
3. **Python API:** Hypermem exposes a convenient Python API for PANDA plugins to read and write guest memory, access process information, inspect file descriptors, and more.

11.3 Memory Region Structure

The communication between the guest kernel and hypervisor happens through a shared memory region with the following structure:

```
struct mem_region {
    __le64 op;       // Operation code (read, write, etc.)
    __le64 addr;     // Target address or parameter
    __le64 size;     // Size of data or parameter
    char data[CHUNK_SIZE]; // Data buffer (4072 bytes)
};
```

11.4 Example: Using Hypermem in a Python Plugin

11.4.1 Basic Usage Pattern

```
from penguin import getColoredLogger, plugins, Plugin

class MemoryAccess(Plugin):
```

(continues on next page)

(continued from previous page)

```
    def __init__(self):
        self.logger = getColoredLogger("plugins.memory_access")

        self.panda.hsyscall("on_sys_read_return")(self.on_read)

    @plugins.portal.wrap
    def on_read(self, cpu, proto, syscall, hook, fd, buf_addr, count):
        # Read a string from the buffer address
        buffer_content = yield from plugins.portal.read_str(buf_addr)
        self.logger.info(f"Read buffer content: {buffer_content}")

        # Write a string to memory
        yield from plugins.portal.write_str(buf_addr, "Modified content")

        # Read an integer value from memory
        value = yield from plugins.portal.read_int(buf_addr + 16)
        self.logger.info(f"Integer value: {value:#x}")
```

11.4.2 Advanced Examples

Worked Example: IOCTL Interaction with RALink Ethernet

This example demonstrates how Hypermem can be used to monitor and modify IOCTL calls for hardware interaction, specifically for the RALink Ethernet device used in TP-Link routers.

11.4.3 The Problem

When rehosting TP-Link Archer C20 devices, the boot process stalls with a process repeatedly printing "swRegRead" to stderr. This function is responsible for reading registers from the ethernet switch hardware through IOCTL calls. Since the actual hardware isn't present in our emulation environment, we need to intercept these calls and provide appropriate responses.

11.4.4 The Original Function (from GPL code)

```
// From ArcherC20_V4_GPL/mtk_ApSoC_4320/apps/mtk7530_switch/switch.c
int reg_read(int offset, int *value)
{
    struct ifreq ifr;
    esw_reg reg;

    if (value == NULL)
        return -1;
    reg.off = offset;
    strncpy(ifr.ifr_name, "eth0", 5);
    ifr.ifr_data = &reg;
    if (-1 == ioctl(esw_fd, RAETH_ESW_REG_READ, &ifr)) {
        perror("ioctl");
        close(esw_fd);
        exit(0);
    }
    *value = reg.val;
```

(continues on next page)

(continued from previous page)

```
    return 0;
}
```

This function is called by code that polls for specific bit patterns in the register values:

```
// From ArcherC20_V4_GPL/mtk_ApSoC_4320/apps/mtk7530_switch/switch.c
void table_del(int argc, char *argv[])
{
    // ...
    for (i = 0; i < 20; i++) {
        reg_read(REG_ESW_WT_MAC_AD0, &value);
        if (value & 0x2) { //w_mac_done
            if (argv[1] != NULL)
                printf("done.\n");
            return;
        }
        usleep(1000);
    }
    if (i == 20)
        printf("timeout.\n");
}
```

11.4.5 The Solution with Hypermem and Syscall Filtering

Using Hypermem combined with hypersyscalls filtering capabilities, we can create a plugin that specifically targets IOCTL system calls for the RALink Ethernet device and provides appropriate register values to allow the device to boot:

```
from penguin import getColoredLogger, plugins, Plugin

# RALink Ethernet register definitions
RAETH_ESW_REG_READ = 0x89F1
REG_ESW_WT_MAC_AD0 = 0x34
REG_ESW_WT_MAC_ATC = 0x80
REG_ESW_TABLE_STATUS0 = 0x90

class RAEthPlugin(Plugin):
    def __init__(self):
        self.logger = getColoredLogger("plugins.raeth")

        # Register our callback for IOCTL syscalls with specific filtering
        # Only intercept ioctl calls with cmd=RAETH_ESW_REG_READ (0x89F1)
        self.panda.hsyscall(
            "on_sys_ioctl_return",
            arg_filter=[None, RAETH_ESW_REG_READ, None]
        )(self.handle_raeth_ioctl)

        # Register definitions for logging
        self.registers = {
            REG_ESW_WT_MAC_AD0: "REG_ESW_WT_MAC_AD0",
            REG_ESW_WT_MAC_ATC: "REG_ESW_WT_MAC_ATC",
            REG_ESW_TABLE_STATUS0: "REG_ESW_TABLE_STATUS0"
```

(continues on next page)

(continued from previous page)

```
        }

    @plugins.portal.wrap
    def handle_raeth_ioctl(self, cpu, proto, syscall, hook, fd, cmd, arg):
        """Handler specifically for RALink Ethernet register read operations"""
        # Read the interface name
        interface = yield from plugins.portal.read_str(arg)

        # Only handle eth0 ioctls
        if interface != "eth0":
            return

        # Read the esw_reg structure pointer from ifr_data
        esw_reg_ptr = yield from plugins.portal.read_ptr(arg + 16)

        # Read the register code (offset)
        code = yield from plugins.portal.read_int(esw_reg_ptr)

        # Decide what value to provide based on the register
        if code == REG_ESW_WT_MAC_AD0:
            # Set bit 0x2 to indicate w_mac_done
            val = 0x2
        elif code == REG_ESW_WT_MAC_ATC:
            val = 0x8234
        elif code == REG_ESW_TABLE_STATUS0:
            # This value needs to meet 0x1 & 0x2
            val = 0x73
        else:
            val = 0x10173

        # Log the operation
        self.logger.info(f"RAEth ioctl: reg={self.registers.get(code, hex(code))},␣
↪returning val={hex(val)}")

        # Write the value back to the esw_reg structure (at offset +4 for val)
        yield from plugins.portal.write_int(esw_reg_ptr + 4, val)

        # Set the syscall return value to 0 (success)
        syscall.retval = 0
```

11.4.6 Using Advanced Filtering to Target Specific Code Paths

We can further refine our intervention by combining various filters to precisely target specific code paths:

```
class AdvancedRAEthPlugin(PyPlugin):
    def __init__(self, panda):
        self.panda = panda
        self.logger = getColoredLogger("plugins.advanced_raeth")

        # Target only table_del's register read operations
        # This uses both command filtering and process name filtering
        self.panda.hsyscall(
```

(continues on next page)

(continued from previous page)

```
            "on_sys_ioctl_enter",
            comm_filter="mtk7530_switch",  # Process name filter
            arg_filter=[None, RAETH_ESW_REG_READ, None]  # Argument filter
        )(self.handle_table_del_ioctl)

    @plugins.portal.wrap
    def handle_table_del_ioctl(self, cpu, proto, syscall, hook, fd, cmd, arg):
        """
        This function specifically targets the table_del function's ioctl calls
        by checking the register being read
        """
        # Read the interface name
        interface = yield from plugins.portal.read_str(arg)
        if interface != "eth0":
            return

        # Read the esw_reg structure pointer from ifr_data
        esw_reg_ptr = yield from plugins.portal.read_ptr(arg + 16)

        # Read the register code (offset)
        code = yield from plugins.portal.read_int(esw_reg_ptr)

        # Check if this is the specific register read from table_del
        if code == REG_ESW_WT_MAC_AD0:
            self.logger.info("Detected table_del function polling for w_mac_done bit")

            # Instead of modifying the value, we can skip the syscall entirely
            # and provide our own return value directly
            syscall.skip_syscall = True

            # Write the w_mac_done bit directly to the return value pointer
            # First, get the value pointer from the calling code
            caller_frame_ptr = yield from plugins.portal.read_ptr(self.panda.arch.get_
↪reg(cpu, "sp"))
            value_ptr = yield from plugins.portal.read_ptr(caller_frame_ptr + 8)  #␣
↪Assuming x86_64 calling convention

            # Set the value with the w_mac_done bit set
            yield from plugins.portal.write_int(value_ptr, 0x2)

            # Make the ioctl call itself return success
            syscall.retval = 0

            self.logger.info("Bypassed ioctl call and directly set w_mac_done bit")
```

11.4.7 Benefits of Using Hypermem with Syscall Filtering

1. **Precision Targeting**: By using arg_filter and comm_filter, we can precisely target only the specific ioctl calls we need to handle.
2. **Reduced Overhead**: We avoid intercepting irrelevant syscalls, improving performance.
3. **Contextual Awareness**: We can make decisions based on process name, syscall arguments, and other context.

4. **Flexible Intervention**: We can choose to modify arguments, skip syscalls, or change return values as needed.
5. **Clean Memory Access**: Using Hypermem's higher-level API (`read_int`, `write_int`, etc.) is much cleaner than direct memory manipulation.

11.4.8 Result

By intercepting only the specific IOCTL calls needed and providing appropriate register values, the TP-Link device bootup process continues successfully beyond the hardware check. The filtering capabilities ensure we only intervene where necessary, maintaining performance while still providing the emulation needed for the missing hardware.

This example demonstrates how combining Hypermem with hypersyscalls' filtering capabilities provides a powerful and precise approach to firmware rehosting.

11.5 API Reference

11.5.1 Reading Memory

Reading Raw Bytes

```
data = yield from plugins.portal.read_bytes(addr, size)
```

Reads `size` bytes from memory starting at `addr`.

Reading Strings

```
string = yield from plugins.portal.read_str(addr)
```

Reads a null-terminated string from memory starting at `addr`.

Reading Integers/Pointers

```
# 32-bit integer
value = yield from plugins.portal.read_int(addr)

# 64-bit integer
value = yield from plugins.portal.read_long(addr)

# Architecture-appropriate pointer
ptr = yield from plugins.portal.read_ptr(addr)
```

11.5.2 Writing Memory

Writing Raw Bytes

```
yield from portal.write_bytes(addr, data)
```

Writes the bytes in `data` to memory starting at `addr`.

Writing Strings

```
yield from portal.write_str(addr, string)
```

Writes `string` to memory at `addr`

Writing Integers/Pointers

```
# 32-bit integer
yield from portal.write_int(addr, value)

# 64-bit integer
yield from portal.write_long(addr, value)

# Architecture-appropriate pointer
yield from portal.write_ptr(addr, value)
```

11.5.3 Process Information

```
# Get command line arguments
args = yield from portal.get_proc_args()

# Get environment variables
env = yield from portal.get_proc_env()

# Get process ID
pid = yield from portal.get_proc_pid()
```

11.5.4 File Descriptor Information

```
# Get file name associated with a file descriptor
fd_name = yield from portal.read_fd_name(fd)
```

11.6 Real-world Example: IOCTL Interaction Test

Here's an example from the `ioctl_interaction_test.py` that demonstrates advanced memory manipulation using Hypermem:

```
def syscall_test(self, cpu, proto, syscall, hook, fd, op, arg):
    if op == SIOCDEVPRIVATE:
        # Read a string from the arg address
        interface = yield from plugins.portal.read_str(arg)
        self.logger.info(f"Interface: {interface}")

        # Read an integer from memory at offset IFNAMSIZ from arg
        data = yield from plugins.portal.read_int(arg + IFNAMSIZ)
        self.logger.info(f"Data: {data:#x}")

        # Write a string to memory and read it back to verify
        to_write = "test"
        yield from plugins.portal.write_str(arg, to_write)
        interface = yield from plugins.portal.read_str(arg)
        assert interface == to_write, f"Expected {to_write}, got {interface}, r/w failed"

        # Write an integer to memory and read it back to verify
        to_write_int = 0x12345678
        yield from plugins.portal.write_int(arg + IFNAMSIZ, to_write_int)
```

(continues on next page)

(continued from previous page)

```
        data = yield from plugins.portal.read_int(arg + IFNAMSIZ)
        assert data == to_write_int, f"Expected {to_write_int:#x}, got {data:#x}, r/w␣
↪failed"

        # Get file descriptor name
        fd_name = yield from plugins.portal.read_fd_name(fd) or "[???]"
        self.logger.info(f"FD: {fd_name}")

        # Get process arguments
        args = yield from plugins.portal.get_proc_args()
        self.logger.info(f"Found process: {args}")

        # Get environment variables
        env = yield from plugins.portal.get_proc_env()
        self.logger.info(f"Found env: {env}")

        # Get process ID
        pid = yield from plugins.portal.get_proc_pid()
        self.logger.info(f"Found pid: {pid}")
```

11.7 Understanding the Kernel Implementation

The Hypermem system works by registering a shared memory region between the guest kernel and hypervisor. The kernel module `hypermemcall.c` handles the guest-side implementation:

1. **Memory Region Registration:**

```
// Called when the hypervisor registers a memory region
static void _register_cpu_memregion(cpu) {
    self.cpu_memregions[cpu] = self.panda.arch.get_arg(cpu, 1, convention="syscall")
}
```

2. **Operation Handling:** The kernel module defines a set of operations and handlers:

```
enum HYPER_OP {
    HYPER_OP_NONE = 0,
    HYPER_OP_READ,
    HYPER_RESP_READ_OK,
    // ...and more
};
```

3. **Handler Functions:** The kernel implements handlers for each operation:

```
static void handle_op_read(struct mem_region *mem_region) {
    // Copy data from user memory to the shared buffer
    resp = copy_from_user(
        (void*)mem_region->data,
        (const void __user *)(uintptr_t)le64_to_cpu(mem_region->addr),
        le64_to_cpu(mem_region->size));
    // Set response code
    mem_region->op = cpu_to_le64(HYPER_RESP_READ_OK);
}
```

11.8 Operation Flow

1. The Python plugin requests an operation through the portal API
2. The portal plugin writes the operation code and parameters to the shared memory region
3. The hypervisor signals the guest kernel via a hypercall
4. The guest kernel processes the operation, accessing memory as needed
5. Results are written back to the shared memory region
6. The hypervisor reads the results and provides them to the Python plugin

11.9 Differences from Traditional Memory Access

Feature	Portal	Traditional Memory Access
Access Method	Cooperative via guest kernel	Direct physical memory access
Reliability	High - Uses proper kernel APIs	Medium - May break with kernel changes
Error Handling	Guest provides explicit error codes	Silent failures possible
Virtual Memory	Access respects virtual memory mappings	Requires manual translation
Higher-level Functions	Read strings, FDs, process info, etc.	Basic memory reads only
Performance	Slight overhead due to guest involvement	Generally faster
Use Case	Deep introspection, advanced manipulation	Simple memory access

11.10 References

- See *pyplugins/analysis/portal.py* for the full implementation
- See `pyplugins/testing/ioctl_interaction_test.py` for usage examples

CHAPTER

TWELVE

PSEUDOFILE MODEL CATALOG

Penguin emulates device files (`/dev`, `/proc`, `/sys`, `/proc/sys`) by attaching **models** to them in the `pseudofiles` config section. Each node configures one or more **domains** — `read`, `write`, `ioctl`, `poll`, and the lifecycle/seek ops `lseek`, `mmap`, `open`, `release` — and each domain selects a named model with `model:` plus that model's fields.

This page is the catalog: every built-in model, what it does, when to use it, and a copy-paste snippet. The goal is that most devices can be modeled in YAML alone — no Python. When you do need Python, see *Custom models* below; the rest of the plugin authoring surface is in *pyplugin_architecture.md*.

The full field reference is auto-generated in *schema_doc.md*; this page is the human/AI-facing how-to. Everything here is backwards compatible: omitting a domain keeps the default behavior.

```yaml
pseudofiles:
  /dev/example:
    read:  {model: const_buf, val: "hello\n"}
    write: {model: discard}
    ioctl:
      "*": {model: return_const, val: 0}
```

12.1 Read models

`model:`	Behavior	Use when
`zero`	Reads return the byte `0`.	Stub a node that just needs to be non-empty.
`one`	Reads return the byte `1`.	A flag/gpio that should read high.
`empty`	Immediate EOF.	A readable node that should look empty.
`const_buf`	Return a fixed buffer.	A node with fixed contents (version string, status).
`cycle`	Repeat a buffer forever (never EOF).	An endless stream (entropy-like, repeating pattern).
`const_map`	Sparse offset→value buffer of a given size.	A register/struct image with values at known offsets.
`const_map_file`	`const_map` persisted to a host file.	Same, but you want the rendered image on disk.
`from_file`	Serve bytes from a host file.	Back the node with a real file you provide.
`stateful`	Read back whatever was last **written** to this node.	A read/write register or scratch node.
`sequence`	Return each entry of a list on successive reads.	"busy… busy… ready" status polling.
`from_plugin`	Call a plugin function.	Logic that can't be expressed declaratively.
`custom`	A model registered via `@register_model`.	Reusable Python model, selected by name.
`default`	Return `-EINVAL`.	Explicitly reject reads.

```
# Fixed contents
/proc/version_stub:
  read: {model: const_buf, val: "Linux 4.10\n"}

# Read-after-write register (pair stateful read + recording write)
/dev/scratch:
  read:  {model: stateful, initial: "0"}
  write: {model: default}

# Status that becomes ready after two polls, then stays ready
/proc/device_status:
  read: {model: sequence, vals: ["busy\n", "busy\n", "ready\n"]}
```

12.2 Write models

model:	Behavior	Use when
discard	Accept and drop writes (returns count).	Writes you can safely ignore.
default	Record writes into an in-memory buffer.	Pair with read: stateful for a register.
to_file	Append/write to a host file.	Capture what the guest writes for inspection.
return_const	Return a fixed value/errno from write().	Force a specific write result (e.g. an errno).
unhandled	Return -EINVAL.	Explicitly reject writes.
from_plugin	Call a plugin function.	Custom write logic.
custom	A model registered via @register_model.	Reusable Python model.

```
/dev/logsink:
  write: {model: to_file, filename: "./results/latest/guest_writes.bin"}
```

12.3 ioctl models

ioctl is a map from command number (or "*" for the wildcard) to a model. A bare model (no command map) applies to every command.

model:	Behavior	Use when
return_const	Return a fixed value.	The driver just checks the return code.
zero	Return 0 (success).	Acknowledge any ioctl.
write_data	Write a constant buffer to the arg pointer, then return val.	An ioctl that fills a caller-supplied struct.
unhandled	Return -ENOTTY.	Explicitly reject an ioctl.
from_plugin	Call a plugin function.	Command-dependent logic.

```
/dev/widget:
  ioctl:
    0x1234: {model: write_data, data: "\x01\x00\x00\x00", val: 0}
    "*":    {model: return_const, val: 0}
```

12.4 poll models

model:	Behavior	Use when
always_ready	Report readable+writable (legacy default for /dev).	Node is always ready.
from_plugin	Plugin returns a data-aware poll mask.	Readiness depends on state.

12.5 Other VFS operations

Beyond read/write/ioctl/poll you can model the rest of the file_operations surface. Omit a domain to keep the kernel default. Most are plugin-driven; the plugin function takes that op's native VFS arguments.

domain	model:	Behavior
lseek	default	Standard offset arithmetic against the node's size.
lseek	unsupported	Return -ESPIPE (pipe/stream-like node).
lseek	from_plugin	Plugin lseek(ptregs, file, offset, whence).
mmap	from_plugin	Plugin mmap(ptregs, file, vm_area_struct).
open	from_plugin	Fire a plugin function when the node is opened.
release	from_plugin	Fire a plugin function when the node is closed.
compat_ioctl	same_as_ioctl	Route 32-bit compat ioctls through the ioctl model.
compat_ioctl	from_plugin	Plugin compat_ioctl(ptregs, file, cmd, arg).
flush	from_plugin	/dev-only. Plugin flush(ptregs, file, owner).
fsync	from_plugin	/dev-only. Plugin fsync(ptregs, file, start, end, datasync).
fasync	from_plugin	/dev-only. Plugin fasync(ptregs, fd, file, on).
lock	from_plugin	/dev-only. Plugin lock(ptregs, file, cmd, file_lock).
read_iter	from_plugin	Plugin read_iter(ptregs, kiocb, iov_iter).
write_iter	from_plugin	Plugin write_iter(ptregs, kiocb, iov_iter).
get_unmapped_area	from_plugin	Plugin get_unmapped_area(ptregs, file, addr, len, pgoff, flags).

flush/fsync/fasync/lock/write_iter are character-device fops: valid only on /dev nodes. lseek/mmap/open/release/compat_ioctl/read_iter/ get_unmapped_area are valid on /dev and /proc but not /proc/sys (sysctl) or /sys (sysfs), which only service reads/writes. Config validation **rejects** an op attached to a node whose filesystem can't service it.

```
/dev/stream:
  read:  {model: sequence, vals: ["a", "b", "c"]}
  lseek: {model: unsupported}
  open:  {model: from_plugin, plugin: my_plugin, function: on_open}

/dev/widget64:
  ioctl: {"*": {model: return_const, val: 0}}
  compat_ioctl: {model: same_as_ioctl}   # 32-bit callers get the same handlers
```

12.6 MTD flash devices

MTD nodes (/dev/mtdN, /proc/mtd) are **not** modeled through pseudofiles — they have a dedicated native subsystem with its own plugins.mtd.devices config block (geometry, personality, backing). Legacy pseudofiles: /dev/mtdN entries are auto-migrated into it.

```yaml
plugins:
  mtd:
    devices:
      flash0:
        id: 0
        model: backing_file        # or 'zeros' (blank flash) / 'const_buf'
        backing_file: ./flash0.bin
        mode: rw                   # 'ro' for read-only
        personality:
          type: nor                # 'nand' or 'nor' (sets erase/write/oob defaults)
          erase_size: 64k
```

Models: `zeros` (RAM-backed blank flash), `const_buf` (in-RAM initial image), `backing_file` (host file). For fully custom flash physics (read/write/erase), register an `MtdDevice` subclass in Python via `plugins.mtd.register_mtd(dev)`.

`id` is optional: devices without one are numbered alphabetically by name into the ids left free by those that declare one. The kernel then assigns `/dev/mtdN` numbers as devices register, which need not match `id` — the guest should find its node by name out of `/proc/mtd` rather than assuming an index.

12.6.1 Personality defaults

`personality.type` selects a set of geometry defaults, and defaults to `nand` when the block is absent. Individual fields override what the type implies:

type	erase_size	write_size	oob_size	default size (model zeros)
nand (default)	128 KiB	2048	64	256 MiB
nor	64 KiB	1	0	16 MiB

For `const_buf`, the size is the payload rounded up to a 128 KiB minimum. For `backing_file` it is the size of the file.

> Note: the `devices:` block is currently a free-form mapping in the schema (not yet a typed/validated structure), so consult this section for field names. Geometry units accept suffixes (`k`/`m`/`g`).

12.7 Provenance (discovered defaults)

Every model takes an optional `provenance:` tag. During `init`, the `pseudofiles.dynamic` patch auto-generates **stub** models (`read: zero`, `write: discard`, `ioctl "*": return_const 0`) for every discovered pseudofile and tags them `provenance: default`. A tagged-default model reports its own activity into `pseudofiles_failures.yaml` (events `default_read`, `default_write`, `default_ioctl`, with captured write payloads and ioctl commands), so a stub that is actively shaping guest behavior shows up as "needs a real model" instead of silently succeeding with garbage.

Leave `provenance` unset on models you author intentionally — only tagged defaults self-report.

```yaml
/dev/discovered:
  read:  {model: zero,    provenance: default}
  write: {model: discard, provenance: default}
```

12.8 Custom models (`@register_model`)

When declarative models aren't enough, register a Python mixin under a name and select it from YAML with `model: custom`. This is the low-friction "expand with Python" path — far less boilerplate than a full backing class.

Drop a file in the project's `plugins.d/` (auto-loaded — see *local_plugins.md*):

```
# plugins.d/my_models.py
from hyperfile.models.registry import register_model
from hyperfile.models.read import ReadBufWrapper

@register_model("read", "my_sensor")
class MySensorRead(ReadBufWrapper):
    def __init__(self, *, scale=1, **kwargs):
        super().__init__(buffer=str(42 * scale).encode(), **kwargs)
```

Then in `config.yaml` — `model_name` selects the registered model and any extra keys are forwarded to its constructor:

```
pseudofiles:
  /dev/sensor:
    read: {model: custom, model_name: my_sensor, scale: 10}
```

`register_model(domain, name)` supports the `read`, `write`, `poll`, `lseek`, `mmap`, `open`, and `release` domains. The mixin follows the same VFS contract as the built-ins (generator methods, `ptregs.retval`, `super().__init__(**kwargs)`). For a model that owns the *entire* file-operations surface in one class, use a backing class instead (`plugin: file:ClassName`) — see *plugins.md*.

CHAPTER

THIRTEEN

PLUGINS

Plugins are a way to extend the functionality of penguin.

13.1 The Plugin Class

All plugins should inherit from the `Plugin` base class provided by `penguin`. This class provides useful methods and properties for plugin development:

- `self.get_arg(arg_name)`: Retrieve a plugin argument by name, or None if not set.
- `self.get_arg_bool(arg_name)`: Retrieve a plugin argument as a boolean (interprets common true/false values).
- `self.logger`: A logger instance for the plugin, automatically named for the plugin class.
- `self.plugins`: Reference to the plugin manager, allowing access to other plugins (e.g., `self.plugins.other_plugin`).
- `self.name`: The name of the plugin (class name).

The plugin manager will automatically instantiate your plugin and call its `__init__` method. You do not need to handle argument parsing or plugin registration manually.

Example:

```
from penguin import Plugin

class MyPlugin(Plugin):
    def __init__(self):
        self.logger.info(f"Initializing {self.name}")
        foo = self.get_arg("foo")
        if self.get_arg_bool("debug"):
            self.logger.setLevel("DEBUG")
```

13.2 Plugin Arguments

In your config.yaml file, you can specify arguments for your plugin. These arguments are passed to the plugin and can be accessed using the `get_arg` or `get_arg_bool` methods. For example, if you have a plugin that takes a `foo` argument, you can specify it in your config.yaml file like this:

```
plugins:
    pluginA:
        argumentA: valueA
```

Then, in your plugin, you can access the argument like this:

```
from penguin import Plugin

class PluginA(Plugin):
    def __init__(self):
        self.argumentA = self.get_arg("argumentA")
```

13.3 Declaring arguments with an `Args` schema (recommended)

A plugin can declare the arguments it accepts by nesting an `Args` class that inherits from `PluginArgs` (a Pydantic model). This is **optional and fully backwards compatible** — plugins without an `Args` class keep working exactly as before (any arg accepted, `get_arg` returns `None` when unset). Declaring `Args` adds three things:

1. **Validation** — args are checked at config-load time. Unknown keys, wrong types, etc. produce a clear, located error before the run starts.
2. **Defaults** — declared defaults are applied automatically, so `get_arg` returns the default instead of `None` when an arg is omitted.
3. **Schema & docs** — `penguin schema <plugin>` renders the declared arguments (types, defaults, descriptions), so users can discover what a plugin accepts without reading its source. Every declaring plugin is also collected into the generated *Plugin arguments reference*.

Configure a plugin's arguments under the top-level `plugins:` section, keyed by the plugin's name:

```
plugins:
  kmods:
    allowlist: [wireguard]
```

```
from typing import List
from pydantic import Field
from penguin import Plugin, PluginArgs

class Kmods(Plugin):
    class Args(PluginArgs):
        allowlist: List[str] = Field(default=[], description="Modules allowed to load")
        denylist:  List[str] = Field(default=[], description="Modules to block")
        quiet:     bool      = Field(default=False, description="Only log errors")

    def __init__(self):
        # get_arg now returns declared defaults when an arg is omitted
        self.allowlist = self.get_arg("allowlist")
        self.quiet = self.get_arg_bool("quiet")
```

13.4 First-class plugin syntax (deprecated)

> **Deprecated.** A plugin that declares an `Args` schema may also be configured at the **top level** of the config (e.g. `kmods:` at the root instead of under `plugins:`). This still works but **logs a warning at load time and may be removed in a future release** — prefer the `plugins:` form above.

```
# Deprecated - equivalent to the plugins: form above, but warns.
kmods:
  allowlist: [wireguard]
```

The top-level form is only honored for plugins that declare `Args`; any other unknown top-level key is still rejected as a typo. Configuring the same plugin both at the top level and under `plugins:` is an error.

Why it's deprecated: it makes the set of valid top-level keys depend on which plugins declare `Args`, so a reader can't tell at a glance whether a top-level key is a core config section, a plugin, or a typo — and a plugin name could one day shadow (or be shadowed by) a reserved section. Keeping plugin args under `plugins:` keeps that boundary explicit.

13.5 Meta-variable templating

Config and patch files support Jinja2 templating so values that depend on the architecture (or other settings) don't have to be hardcoded in multiple places. Available variables: `{{ arch }}`, the arch-derived static-layout subdirs `{{ arch_dir }}` and `{{ dylib_dir }}`, `{{ core.<field> }}` (e.g. `{{ core.mem }}`), `{{ kernel_version }}`, and anything you define in a top-level `vars:` section.

Penguin's own auto-generated base patch uses this: instead of baking the arch subdir into host paths, it emits `{{ arch_dir }}` / `{{ dylib_dir }}` so changing `core.arch` reconfigures those paths automatically.

```
core:
  arch: mipsel
vars:
  webroot: /www
static_files:
  "{{ webroot }}/index.html":
    type: inline_file
    contents: "running on {{ arch }} (kernel {{ kernel_version }})"
```

`vars:` is consumed at load time and does not appear in the realized config. Files containing no `{{ }}`/`{% %}` are left untouched. `{{ kernel_version }}` is resolved after the kernel path is selected.

13.6 Plugin File Locations and Discovery

Plugins can be placed in several locations, and the plugin manager will search for them using common naming conventions. When you reference a plugin by name, the manager will look for files matching the plugin name (in CamelCase, snake_case, or lowercase) in the following locations:

- The directory specified by your `plugin_path` (recursively, including subdirectories)
- The project directory (`proj_dir`)
- A `plugins/` subdirectory inside your project directory

Supported file extensions are `.py` (Python source files). For example, a plugin named `MyPlugin` can be found as any of the following:

- `myplugin.py` or `MyPlugin.py` in your plugin path or project directory
- `plugins/myplugin.py` or `plugins/MyPlugin.py` in your project directory

This flexible search allows you to organize plugins as needed for your project. You do not need to specify the full path—just the plugin name.

13.7 The Plugin Manager

The Plugin Manager manages plugin lifecycle and inter-plugin interaction. It is accessible through the `penguin.plugins` object.

13.7.1 Plugin Interaction

Plugins can interact with each other through the `penguin.plugins` object. This object is a namespace that exposes plugin instances as attributes. For example, if you have a plugin named `pluginA` and a plugin named `pluginB`, you can access `pluginB` from `pluginA` like this:

```
from penguin import Plugin, plugins

class PluginA(Plugin):
    def __init__(self):
        plugins.pluginB.do_something()
```

13.7.2 Auto-Loading Plugins

If a plugin references another plugin via the Plugin Manager and that plugin is not specified in the config, the Plugin Manager will automatically load it. This allows for a more flexible plugin architecture where plugins can be added and removed without changing the config file.

```
from penguin import Plugin, plugins

class PluginA(Plugin):
    def __init__(self):
        plugins.pluginB.do_something()
```

13.7.3 Publish/Subscribe Model

Plugins can also interact with each other through a publish/subscribe model. This allows plugins to subscribe to events and publish events. This is useful for plugins that need to interact with each other but don't have a direct dependency. For example, if you have a plugin that needs to know when a new process is created, you can subscribe to the `process_created` event like this:

```
from penguin import Plugin, plugins

class ProcessCreatedDetector(Plugin):
    def __init__(self):
        plugins.register(self, "process_created", register_notify=self.notify_subscribed)

    def notify_subscribed(self, event, cb):
        print(f"New subscriber to event {event} with {cb}")

    def some_analysis(self):
        print(f"New process created: {process.name}")
        args = (arg1, arg2, arg3)
        plugins.publish(self, "process_created", *args)
```

Here the `ProcessCreatedDetector` plugin lets the plugin manager know that it will publish events called "process_created" and optionally chooses to be notified every time a plugin subscribes to this event. The notification here can be important as effort is wasted publishing events if no subscribers exist.

After creating the terms of its publication it does some analysis in `some_analysis` and determines that a new process has been created. It then publishes the event "process_created" with the arguments `arg1`, `arg2`, and `arg3`. Any plugin that has subscribed to this event will be notified and can take action based on the arguments passed.

Next, we have a plugin that subscribes to the event "process_created":

```
from penguin import Plugin, plugins

class ProcessSubscriber(Plugin):
    def __init__(self):
        plugins.subscribe(plugins.ProcessCreatedDetector, "process_created", self.
↪process_created)

    def process_created(self, *args):
        print(f"New process created with args: {args}")
```

13.8 Init plugins

A second family of pyplugins runs at `penguin init` time (and not during emulation): **init plugins**, subclasses of `penguin.init_plugin.InitPlugin`. They analyze the extracted root filesystem and generate the project's initial config and `static_patches/`. They use the same plugin manager and inter-plugin access as runtime plugins, but run with `panda = None`, are discovered automatically (built-ins from `<plugin_path>/init/`, project-local plugins from `<project>/plugins.d/`, plus `penguin init --init-plugin-path` directories), and execute concurrently. Shared analyses are exposed as `@cached_analysis` attributes that compute once on first access.

See `pyplugins/init/README.md` for the full authoring guide.

13.9 Testing a plugin

A plugin's host-side logic — the files it writes, the events it emits, the portal commands it issues — can be unit-tested with **no PANDA and no guest** using the `penguin.testing` harness, which stands up a null backend and drives your plugin where it lives. When you add or change a plugin, add a host test. See *testing.md* for the harness API (`load_pyplugin`, `drive`, `load_module`, the `real_isf=` FFI-enum path) and the in/out-of-scope rule.

CHAPTER

FOURTEEN

PENGUIN CONFIGURATION

Configuration file for config-file-based rehosting with IGLOO

14.1 `core` Core configuration options

Core configuration options for this rehosting

14.1.1 `core.arch` Architecture of guest

Type	"armel" or "arm" or "armle" or "aarch64" or "arm64" or "mipsel" or "mipseb" or "mipsbe" or "mips64el" or "mips64eb" or "mips64be" or "powerpc" or "ppc" or "powerpc64" or "ppc64" or "powerpc64le" or "ppc64le" or "powerpc64el" or "ppc64el" or "riscv64" or "riscv" or "rv64" or "loongarch64" or "loongarch" or "la64" or "x86_64" or "intel64" or "amd64" or "x86-64" or "x64" or null
Default	null

Canonical name or an accepted alias (normalized at load, e.g. intel64 -> x86_64).

```
x86_64
```

```
armel
```

```
aarch64
```

```
mipsel
```

```
mipseb
```

```
mips64el
```

```
powerpc64le
```

14.1.2 `core.kernel` Path to kernel image

Type	string or null
Default	`null`

```
/igloo_static/kernels/zImage.armel
```

```
/igloo_static/kernels/zImage.arm64
```

```
/igloo_static/kernels/vmlinux.mipsel
```

```
/igloo_static/kernels/vmlinux.mipseb
```

```
/igloo_static/kernels/vmlinux.mips64el
```

```
/igloo_static/kernels/vmlinux.mips64eb
```

14.1.3 `core.fs` Project-relative path to filesystem tarball

Type	string or null
Default	`./base/fs.tar.gz`

```
base/fs.tar.gz
```

14.1.4 `core.plugin_path` Path to search for PyPlugins

Type	string
Default	`/pyplugins`

```
/pyplugins
```

14.1.5 `core.root_shell` Enable root shell

Type	boolean
Default	`false`

Whether to enable a root shell into the guest

```
false
```

```
true
```

14.1.6 `core.root_shell_backend` Root shell backend

Type	`"telnet"` or `"vsock"`
Default	`vsock`

Which console backend serves core.root_shell. 'vsock' (default) runs an on-demand pty over the guest command channel (guesthopper), freeing the ttyS1 serial line; it requires the vsock transport, so core.guest_cmd is enabled automatically and the VPN transport must be available (with vpn disabled, this falls back to 'telnet'). 'telnet' uses the legacy QEMU -serial telnet console on ttyS1.

```
vsock
```

```
telnet
```

14.1.7 `core.analysis_scope` Scope of per-process analysis

Type	boolean or string
Default	`firmware`

Which processes the per-process analysis loggers capture. Affects the syscall/exec-derived loggers (syscalls, exec, read/write, ficd, interfaces) and busybox shell coverage; netbinds always reports for every process regardless of this setting. Recognized values: 'firmware' (default) captures only the firmware-under-analysis process subtree, excluding Penguin's own infrastructure (boot machinery and the vpnguin/console/guesthopper helpers); 'none' captures every process, including Penguin infrastructure; 'infra' inverts the firmware filter to capture only Penguin's own tools. Booleans are accepted for backward compatibility: true == 'firmware', false == 'none'. The field is a string so further interpretations can be added without a schema change.

```
firmware
```

```
none
```

```
infra
```

14.1.8 `core.strace` Enable strace

Type	boolean or list of string
Default	`false`

If true, run strace for entire system starting from init. If names of programs, enable strace only for those programs.

```
false
```

```
true
```

```
- lighttpd
```

14.1.9 `core.ltrace` Enable ltrace

Type	boolean or list of string
Default	`false`

If true, run ltrace for entire system starting from init. If names of programs, enable ltrace only for those programs.

```
false
```

```
true
```

```
- lighttpd
```

14.1.10 `core.gdbserver` Programs to run through gdbserver

Default	`{}`

Mapping between names of programs and ports for gdbserver. When a program in this mapping is run, it will start paused with gdbserver attached, listening on the specified port.

```
{}
```

```
lighttpd: 9999
```

`core.gdbserver.<string>` Port

Type	integer
Default	`null`

14.1.11 `core.snapshot` Snapshot configuration

Default	`null`

VM snapshot (savevm/loadvm) configuration.

Snapshotting is *active* whenever `save_at` or `boot_from` is set — there is no separate enable flag. When active, the guest runs on a persistent qcow2 overlay (rather than the throwaway immutable overlay) so an internal VM snapshot can be saved and later restored. Saving a snapshot at a chosen point lets a later run boot directly from that state instead of re-booting the firmware.

`core.snapshot.backend` Snapshot backend

Type	`"internal"` or `"file"`
Default	`internal`

'internal' stores the snapshot inside the qcow2 overlay (savevm/loadvm). 'file' (not yet implemented) writes a standalone migration file bundle.

```
internal
```

```
file
```

`core.snapshot.tag` Snapshot tag

Type	string
Default	`boot`

Name of the internal VM snapshot to save and/or restore.

```
boot
```

```
post_init
```

`core.snapshot.save_at` When to save the snapshot

Type	`"readiness"` or `"manual"` or null
Default	`null`

'readiness' saves once the guest reaches steady state; 'manual' arms the Snapshot plugin to save on request (via guest_cmd / hypercall). None disables saving.

```
readiness
```

```
manual
```

`core.snapshot.boot_from` Snapshot tag to boot from

Type	string or null
Default	`null`

If set, restore this internal snapshot at startup (-loadvm).

```
boot
```

`core.snapshot.stop_after_save` End the run after saving

Type	boolean
Default	`false`

Shut the guest down immediately after the snapshot is saved.

```
false
```

```
true
```

14.1.12 `core.force_www` Try to force webserver start

Type	boolean
Default	`false`

Whether to try forcing webserver start

```
false
```

```
true
```

14.1.13 `core.cpu` CPU model

Type	string or null
Default	`null`

Specify non-default QEMU CPU model

14.1.14 `core.show_output` Write serial to stdout

Type	boolean
Default	`false`

Whether to print QEMU serial output to stdout instead of writing to a log file

```
false
```

```
true
```

14.1.15 `core.log_file` Penguin log file

Type	string or null
Default	`null`

If set, write penguin/plugin logger output to this file (relative paths resolve under the results dir).

```
penguin.log
```

14.1.16 `core.immutable` Enable immutable mode

Type	boolean
Default	`true`

Whether to run the guest filesystem in immutable mode

```
false
```

```
true
```

14.1.17 `core.network` Connect guest to network

Type	boolean
Default	`false`

Whether to connect the guest to the network

```
false
```

```
true
```

14.1.18 `core.shared_dir` Shared directory configuration

Default	`null`

Host<->guest shared directory (9p) configuration.

Accepted in `core.shared_dir` as `true` (enable with defaults), a string (shorthand for `path`), or this object. Core dumps ride the same single mount (see `core.core_dumps`); they do not need this feature enabled.

```
true
```

```
my_shared_directory
```

```
msize: 131072
path: shared
```

`core.shared_dir.path` Results-relative share directory

Type	string
Default	`shared`

Directory shared into the guest at /igloo/shared. Resolved under the run's results dir unless core.shared_dir.host_path is set.

```
shared
```

```
my_shared_directory
```

`core.shared_dir.host_path` Absolute host directory to share

Type	string or null
Default	`null`

If set, share this absolute host directory instead of a results-relative one (path is ignored).

```
/data/fixtures
```

`core.shared_dir.msize` 9p msize override

Type	integer or null
Default	`null`

Override the 9p transport buffer size. Unset uses the default 8MB with an automatic fallback to 128KB on memory-tight guests.

```
8192000
```

```
131072
```

14.1.19 `core.core_dumps` Core dump configuration

Default	`null`

Guest core-dump capture configuration.

Accepted in `core.core_dumps` as `true` (enable with defaults), a string (shorthand for `pattern`), or this object. When enabled, penguin points core_pattern at /igloo/core_dumps (a symlink into the shared mount) and brings that mount up even when core.shared_dir is unset.

```
true
```

```
/igloo/core_dumps/core_%e.%p
```

```
lock: false
```

`core.core_dumps.lock` Lock core_pattern

Type	boolean
Default	`true`

Install a sysctl pseudofile that eats guest writes to core_pattern so dumps can't be redirected. Set false to let the guest firmware keep its own core_pattern.

```
true
```

```
false
```

`core.core_dumps.pattern` core_pattern override

Type	string or null
Default	`null`

Override the core_pattern string. Unset uses /igloo/core_dumps/core_%e.%p.

```
/igloo/core_dumps/core_%e.%p.%t
```

14.1.20 `core.version` Config format version

Type	`"1.0.0"` or `2`

Version of the config file format

14.1.21 `core.auto_patching` Enable automatic patching

Type	boolean
Default	`true`

Whether to automatically apply patches named patch_*.yaml or from patches/*.yaml in the project directory

```
false
```

```
true
```

14.1.22 `core.guest_cmd` Enable running commands in the guest

Type	boolean
Default	`false`

When enabled, starts the guesthopper daemon in the guest that the host can use to run commands over vsock

```
false
```

```
true
```

14.1.23 `core.execution_mode` Execution Mode

Type	"qemu" or "kvm"
Default	qemu

The execution backend to use for the guest (qemu for TCG, kvm for hardware acceleration)

```
qemu
```

```
kvm
```

14.1.24 `core.extra_qemu_args` Extra QEMU arguments

Type	string
Patch merge behavior	Concatenate strings separated by ' '
Default	null

A list of additional QEMU command-line arguments to use when booting the guest

```
-vnc :0 -vga std -device usb-kbd -device usb-tablet
```

14.1.25 `core.kernel_cmdline_append` Extra kernel command-line tokens

Type	string
Patch merge behavior	Concatenate strings separated by ' '
Default	null

Tokens appended verbatim to the kernel command line (-append). Use this to set real kernel parameters or anything you need on /proc/cmdline. Unlike `env`, these are never diverted to the early-boot env blob, so they always reach the kernel cmdline. They count against the per-arch COMMAND_LINE_SIZE budget (256 bytes on MIPS), so penguin warns/errors if they overflow it.

```
nokaslr mem=256M
```

```
igloo_debug=1
```

14.1.26 `core.mem` Panda Memory Value

Type	string or null
Default	2G

Allows users to customize memory allocation for guest

```
16K
```

```
512M
```

```
1G
```

```
2G
```

14.1.27 `core.kernel_quiet` Whether to include quiet flag in kernel command line

Type	boolean
Default	`true`

If true, the kernel command line will include the quiet flag, otherwise all kernel boot messages will be printed to the console

```
false
```

```
true
```

14.1.28 `core.smp` Number of CPUs

Type	integer or null
Default	1

Number of CPUs to emulate in the guest (Warning: This can break things)

```
1
```

```
2
```

```
4
```

14.1.29 `core.timeout` Run timeout (seconds)

Type	integer or null
Default	`null`

If set, automatically shut the guest down after this many seconds. Overridden by the –timeout CLI flag. No timeout when unset.

```
60
```

```
300
```

14.1.30 `core.graphics` Enable graphics

Type	boolean
Default	`false`

Whether to enable graphics in the guest

```
false
```

```
true
```

14.1.31 `core.allow_reboot` Allow the guest to reboot

Type	boolean
Default	`false`

If False (default), QEMU is launched with '-no-reboot' so a guest-initiated reboot terminates the emulation (one boot per run). If True, '-no-reboot' is omitted so QEMU resets the machine in place and the guest reboots within the same run. Persistent drives and host-file-backed MTD devices survive the reset; stateful plugins must tolerate a second guest init.

```
false
```

```
true
```

14.1.32 `core.startup_script` Inline guest startup script

Type	string or null
Default	`null`

Shell script body dropped into /igloo/init.d to run during guest boot. Installed under a name that sorts after other init.d entries so it runs last. A '#!/igloo/utils/sh' shebang is prepended automatically.

```
'ip link set eth0 up

  udhcpc -i eth0

  '
```

14.2 patches Patches

Type	list of string
Default	null

List of paths to patch files

14.3 vars Template variables

Type	mapping from string to any or null
Default	null

User-defined variables usable elsewhere via Jinja2 templating, e.g. {{ myvar }}. Alongside these, {{ arch }}, {{ core.<field> }}, and {{ kernel_version }} are available. This section is consumed at load time and does not appear in the realized config.

```
libdir: /lib/{{ arch }}
webroot: /www
```

14.4 env Environment

Default	null

Environment variables to set in the guest

```
VAR1: VAL1
VAR2: VAL2
```

```
FOO: DYNVALDYNVALDYNVAL
PATH: /bin:/sbin
TMPDIR: /tmp
```

14.4.1 env.<string> Value

Type	string
Default	null

Value of the environment variable

```
DYNVALDYNVALDYNVAL
```

14.5 pseudofiles Pseudo-files

Default	{}

Device files to emulate in the guest

14.5.1 pseudofiles.<string> File emulation spec

How to emulate a device file

pseudofiles.<string>.name MTD name

Type	string or null
Default	null

Name of an MTD device (ignored for non-mtd)

```
flash
```

```
uboot
```

pseudofiles.<string>.size File size

Type	integer or null
Default	null

Size of the pseudofile to be reported by stat(). This must be specified for mmap() on the pseudofile to work.

```
1
```

```
4096
```

pseudofiles.<string>.plugin Single backing class

Type	string or null
Default	null

Name of a single backing class that owns the whole file_operations surface (read/write/ioctl/poll) for this node. Reference a built-in backing by name, or a user class as 'file:ClassName' (the file is found via the normal pyplugin search path). When set, the per-domain read/write/ioctl/poll keys are ignored — the class owns them all.

```
my_backing:SerialBacking
```

`pseudofiles.<string>.read` Read

Default	`null`

How to handle reads from the file

`pseudofiles.<string>.read.<model=zero>` Read a zero

`pseudofiles.<string>.read.<model=zero>.model` Read modelling method (read a zero)

Type	`"zero"`

`pseudofiles.<string>.read.<model=zero>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=one>` Read a one

`pseudofiles.<string>.read.<model=one>.model` Read modelling method (read a one)

Type	`"one"`

`pseudofiles.<string>.read.<model=one>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=empty>` Read empty file

`pseudofiles.<string>.read.<model=empty>.model` Read modelling method (read empty file)

Type	`"empty"`

`pseudofiles.<string>.read.<model=empty>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=const_buf>` Read a constant buffer

`pseudofiles.<string>.read.<model=const_buf>.model` Read modelling method (read a constant buffer)

Type	`"const_buf"`

`pseudofiles.<string>.read.<model=const_buf>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=const_buf>.val` Pseudofile contents

Type	string

`pseudofiles.<string>.read.<model=const_buf>.null_terminate` Append a NUL byte to the configured contents

Type	boolean
Default	`false`

pseudofiles.<string>.read.<model=const_buf>.nul_terminate Alias for null_terminate

Type	boolean
Default	`false`

pseudofiles.<string>.read.<model=const_map> Read a constant map

pseudofiles.<string>.read.<model=const_map>.model Read modelling method (read a constant map)

Type	`"const_map"`

pseudofiles.<string>.read.<model=const_map>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.read.<model=const_map>.pad Byte for padding file

Type	string or integer
Default	`0`

pseudofiles.<string>.read.<model=const_map>.size File size

Type	integer
Default	`65536`

pseudofiles.<string>.read.<model=const_map>.vals Mapping from file offsets to data

pseudofiles.<string>.read.<model=const_map>.vals.<integer> Data to place in the file at an offset

Type	string or list of integer or list of string
Default	`null`

When this is a list of integers, it treated as a byte array. When this is a list of strings, the strings are separated by null bytes.

`pseudofiles.<string>.read.<model=const_map_file>` Read a constant map with host file

`pseudofiles.<string>.read.<model=const_map_file>.model` Read modelling method (read a constant map with host file)

Type	`"const_map_file"`

`pseudofiles.<string>.read.<model=const_map_file>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=const_map_file>.filename` Path to host file to store constant map

Type	string

`pseudofiles.<string>.read.<model=const_map_file>.pad` Byte for padding file

Type	string or integer
Default	`0`

`pseudofiles.<string>.read.<model=const_map_file>.size` File size

Type	integer
Default	`65536`

`pseudofiles.<string>.read.<model=const_map_file>.vals` Mapping from file offsets to data

`pseudofiles.<string>.read.<model=const_map_file>.vals.<integer>` Data to place in the file at an offset

Type	string or list of integer or list of string
Default	`null`

When this is a list of integers, it treated as a byte array. When this is a list of strings, the strings are separated by null bytes.

`pseudofiles.<string>.read.<model=cycle>` Read a repeating buffer

Repeat the configured buffer forever (never reports EOF).

`pseudofiles.<string>.read.<model=cycle>.model` Read modelling method (read a repeating buffer)

Type	`"cycle"`

`pseudofiles.<string>.read.<model=cycle>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=cycle>.val` Buffer to repeat

Type	string

`pseudofiles.<string>.read.<model=from_file>` Read from a host file

`pseudofiles.<string>.read.<model=from_file>.model` Read modelling method (read from a host file)

Type	`"from_file"`

`pseudofiles.<string>.read.<model=from_file>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=from_file>.filename` Path to host file

Type	string

`pseudofiles.<string>.read.<model=stateful>` Read back what was written

Serve bytes from this node's write buffer, giving a read-after-write register. Pair with write model 'default' (or 'discard'/'record', which all record) so writes are stored.

`pseudofiles.<string>.read.<model=stateful>.model` Read modelling method (read back what was written)

Type	`"stateful"`

`pseudofiles.<string>.read.<model=stateful>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=stateful>.initial` Initial buffer contents

Type	string or null
Default	`null`

pseudofiles.<string>.read.<model=sequence> Read successive values

Return each entry of 'vals' on successive reads; the common 'busy… busy… ready' status pattern. Holds the last entry when exhausted unless 'cycle' wraps around.

pseudofiles.<string>.read.<model=sequence>.model Read modelling method (read successive values)

Type	`"sequence"`

pseudofiles.<string>.read.<model=sequence>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.read.<model=sequence>.vals Ordered values to return

Type	list

pseudofiles.<string>.read.<model=sequence>.cycle Wrap around when exhausted

Type	boolean
Default	`false`

pseudofiles.<string>.read.<model=from_plugin> Read from a custom PyPlugin

pseudofiles.<string>.read.<model=from_plugin>.model Read modelling method (read from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.read.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.read.<model=from_plugin>.function` Function to call

Type	string or null
Default	`read`

`pseudofiles.<string>.read.<model=default>` Default

`pseudofiles.<string>.read.<model=default>.model` Read modelling method (default)

Type	`"default"`

`pseudofiles.<string>.read.<model=default>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.read.<model=custom>` Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

`pseudofiles.<string>.read.<model=custom>.model` Read modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.read.<model=custom>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.read.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.write Write

Default	null

How to handle writes to the file

pseudofiles.<string>.write.<model=to_file> Write to host file

pseudofiles.<string>.write.<model=to_file>.model Write modelling method (write to host file)

Type	"to_file"

pseudofiles.<string>.write.<model=to_file>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=to_file>.filename Path to host file

Type	string

pseudofiles.<string>.write.<model=from_plugin> Read from a custom PyPlugin

pseudofiles.<string>.write.<model=from_plugin>.model Write modelling method (read from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.write.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.write.<model=from_plugin>.function Function to call

Type	string or null
Default	`write`

pseudofiles.<string>.write.<model=discard> Discard write

pseudofiles.<string>.write.<model=discard>.model Write modelling method (discard write)

Type	`"discard"`

pseudofiles.<string>.write.<model=discard>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=return_const> Return a constant on write

Return a fixed value (e.g. a byte count or a negative errno) without storing data.

pseudofiles.<string>.write.<model=return_const>.model Write modelling method (return a constant on write)

Type	`"return_const"`

pseudofiles.<string>.write.<model=return_const>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=return_const>.const Value to return from write()

Type	integer

pseudofiles.<string>.write.<model=unhandled> Reject writes

Return -EINVAL for every write.

pseudofiles.<string>.write.<model=unhandled>.model Write modelling method (reject writes)

Type	`"unhandled"`

pseudofiles.<string>.write.<model=unhandled>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=default> Default

pseudofiles.<string>.write.<model=default>.model Write modelling method (default)

Type	`"default"`

pseudofiles.<string>.write.<model=default>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.write.<model=custom>.model Write modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.write.<model=custom>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.ioctl ioctl

Default	`null`

How to handle ioctl() calls

```
'*':
  model: return_const
  val: 0
'1000':
  model: return_const
  val: 5
```

```
'*':
  model: return_const
```

```
function: ioctl_handler
model: from_plugin
plugin: my_plugin
```

pseudofiles.<string>.ioctl.<model=return_const> Return a constant

pseudofiles.<string>.ioctl.<model=return_const>.model ioctl modelling method (return a constant)

Type	`"return_const"`

pseudofiles.<string>.ioctl.<model=return_const>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.ioctl.<model=return_const>.val Constant to return

Type	integer
Default	`0`

pseudofiles.<string>.ioctl.<model=zero> Return zero

pseudofiles.<string>.ioctl.<model=zero>.model ioctl modelling method (return zero)

Type	`"zero"`

pseudofiles.<string>.ioctl.<model=zero>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.ioctl.<model=unhandled> Reject ioctl

Return -ENOTTY (inappropriate ioctl for device).

pseudofiles.<string>.ioctl.<model=unhandled>.model ioctl modelling method (reject ioctl)

Type	`"unhandled"`

pseudofiles.<string>.ioctl.<model=unhandled>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.ioctl.<model=write_data> Write a buffer to the arg pointer

Write a constant buffer to the user pointer in 'arg' (the common shape of an ioctl that fills a struct), then return 'val'.

pseudofiles.<string>.ioctl.<model=write_data>.model ioctl modelling method (write a buffer to the arg pointer)

Type	`"write_data"`

pseudofiles.<string>.ioctl.<model=write_data>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.ioctl.<model=write_data>.data Bytes to write to *arg

Type	string

pseudofiles.<string>.ioctl.<model=write_data>.val Value to return from ioctl()

Type	integer
Default	0

pseudofiles.<string>.ioctl.<model=from_plugin> ioctl from a custom PyPlugin

pseudofiles.<string>.ioctl.<model=from_plugin>.model ioctl modelling method (ioctl from a custom pyplugin)

Type	"from_plugin"

pseudofiles.<string>.ioctl.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.ioctl.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.ioctl.<model=from_plugin>.function Function to call

Type	string or null
Default	`ioctl`

pseudofiles.<string>.poll Poll

Default	`null`

How to answer poll()/select() on the file

pseudofiles.<string>.poll.<model=always_ready> Always report ready

Constant POLLIN|POLLRDNORM|POLLOUT|POLLWRNORM mask (legacy behavior).

pseudofiles.<string>.poll.<model=always_ready>.model Poll modelling method (always report ready)

Type	`"always_ready"`

pseudofiles.<string>.poll.<model=always_ready>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.poll.<model=blocking> Never report ready (block the waiter)

Return a zero mask so poll()/select()/epoll parks the caller on the node's wait queue instead of spinning. Models an event-source device whose read() blocks until a hardware event that never occurs under emulation (e.g. a watchdog). A write to the node wakes any parked waiter.

pseudofiles.<string>.poll.<model=blocking>.model Poll modelling method (never report ready (block the waiter))

Type	`"blocking"`

pseudofiles.<string>.poll.<model=blocking>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.poll.<model=periodic> Report ready on a fixed cadence (heartbeat)

Report the node readable once every 'interval_ms', parking the waiter on the node's wait queue in between. An igloo_driver kernel timer drives the cadence, so a poll()/epoll(timeout=-1) main loop advances at a fixed rate instead of spinning (always_ready) or deadlocking (blocking). Models a device that delivers a periodic hardware event (e.g. a watchdog heartbeat). Note the interval is in guest time, which runs slower than wall-clock under emulation.

pseudofiles.<string>.poll.<model=periodic>.model Poll modelling method (report ready on a fixed cadence (heartbeat))

Type	"periodic"

pseudofiles.<string>.poll.<model=periodic>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.poll.<model=periodic>.interval_ms Heartbeat interval (guest milliseconds)

Type	integer
Default	1000

pseudofiles.<string>.poll.<model=from_plugin> Poll from a custom PyPlugin

Data-aware poll: the plugin returns a poll mask reflecting actual readiness.

pseudofiles.<string>.poll.<model=from_plugin>.model Poll modelling method (poll from a custom pyplugin)

Type	"from_plugin"

`pseudofiles.<string>.poll.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.poll.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.poll.<model=from_plugin>.function` Function to call

Type	string or null
Default	`poll`

`pseudofiles.<string>.poll.<model=custom>` Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

`pseudofiles.<string>.poll.<model=custom>.model` Poll modelling method (custom registered model)

Type	`"custom"`

`pseudofiles.<string>.poll.<model=custom>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.poll.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.lseek Seek

Default	`null`

How to handle lseek() on the file

pseudofiles.<string>.lseek.<model=default> Standard offset arithmetic

SEEK_SET/CUR/END against the node's reported size.

pseudofiles.<string>.lseek.<model=default>.model Seek modelling method (standard offset arithmetic)

Type	`"default"`

pseudofiles.<string>.lseek.<model=default>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.lseek.<model=unsupported> Reject seeks

Return -ESPIPE (for pipe/stream-like nodes).

pseudofiles.<string>.lseek.<model=unsupported>.model Seek modelling method (reject seeks)

Type	`"unsupported"`

pseudofiles.<string>.lseek.<model=unsupported>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.lseek.<model=from_plugin> lseek from a custom PyPlugin

pseudofiles.<string>.lseek.<model=from_plugin>.model Seek modelling method (lseek from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.lseek.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.lseek.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.lseek.<model=from_plugin>.function Function to call

Type	string or null
Default	`lseek`

pseudofiles.<string>.lseek.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.lseek.<model=custom>.model Seek modelling method (custom registered model)

Type	"custom"

pseudofiles.<string>.lseek.<model=custom>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.lseek.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.mmap Mmap

Default	null

How to handle mmap() on the file

pseudofiles.<string>.mmap.<model=from_plugin> mmap from a custom PyPlugin

pseudofiles.<string>.mmap.<model=from_plugin>.model Mmap modelling method (mmap from a custom pyplugin)

Type	"from_plugin"

pseudofiles.<string>.mmap.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.mmap.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.mmap.<model=from_plugin>.function Function to call

Type	string or null
Default	`mmap`

pseudofiles.<string>.mmap.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.mmap.<model=custom>.model Mmap modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.mmap.<model=custom>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.mmap.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.open Open

Default	`null`

How to handle open() on the file

pseudofiles.<string>.open.<model=from_plugin> open from a custom PyPlugin

Fire a plugin function when the guest opens this node.

pseudofiles.<string>.open.<model=from_plugin>.model Open modelling method (open from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.open.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.open.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.open.<model=from_plugin>.function Function to call

Type	string or null
Default	open

pseudofiles.<string>.open.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.open.<model=custom>.model Open modelling method (custom registered model)

Type	`"custom"`

`pseudofiles.<string>.open.<model=custom>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.open.<model=custom>.model_name` Registered model name

Type	string

`pseudofiles.<string>.release` Release

Default	`null`

How to handle release()/close() on the file

`pseudofiles.<string>.release.<model=from_plugin>` release from a custom PyPlugin

Fire a plugin function when the guest closes this node.

`pseudofiles.<string>.release.<model=from_plugin>.model` Release modelling method (release from a custom pyplugin)

Type	`"from_plugin"`

`pseudofiles.<string>.release.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.release.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.release.<model=from_plugin>.function` Function to call

Type	string or null
Default	`release`

`pseudofiles.<string>.release.<model=custom>` Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

`pseudofiles.<string>.release.<model=custom>.model` Release modelling method (custom registered model)

Type	`"custom"`

`pseudofiles.<string>.release.<model=custom>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.release.<model=custom>.model_name` Registered model name

Type	string

`pseudofiles.<string>.compat_ioctl` Compat ioctl

Default	`null`

How to handle 32-bit compat_ioctl() on the file

`pseudofiles.<string>.compat_ioctl.<model=same_as_ioctl>` Reuse the ioctl model

Route compat_ioctl through the same handlers as ioctl (the common driver pattern).

pseudofiles.<string>.compat_ioctl.<model=same_as_ioctl>.model compat_ioctl modelling method (reuse the ioctl model)

Type	`"same_as_ioctl"`

pseudofiles.<string>.compat_ioctl.<model=same_as_ioctl>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.compat_ioctl.<model=from_plugin> compat_ioctl from a custom PyPlugin

pseudofiles.<string>.compat_ioctl.<model=from_plugin>.model compat_ioctl modelling method (compat_ioctl from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.compat_ioctl.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.compat_ioctl.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.compat_ioctl.<model=from_plugin>.function Function to call

Type	string or null
Default	`compat_ioctl`

`pseudofiles.<string>.flush` Flush

Default	`null`

How to handle flush() on the file

`pseudofiles.<string>.flush.<model=from_plugin>` flush from a custom PyPlugin

`pseudofiles.<string>.flush.<model=from_plugin>.model` Flush modelling method (flush from a custom pyplugin)

Type	`"from_plugin"`

`pseudofiles.<string>.flush.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.flush.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.flush.<model=from_plugin>.function` Function to call

Type	string or null
Default	`flush`

`pseudofiles.<string>.flush.<model=custom>` Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

`pseudofiles.<string>.flush.<model=custom>.model` Flush modelling method (custom registered model)

Type	`"custom"`

`pseudofiles.<string>.flush.<model=custom>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.flush.<model=custom>.model_name` Registered model name

Type	string

`pseudofiles.<string>.fsync` Fsync

Default	`null`

How to handle fsync() on the file

`pseudofiles.<string>.fsync.<model=from_plugin>` fsync from a custom PyPlugin

`pseudofiles.<string>.fsync.<model=from_plugin>.model` Fsync modelling method (fsync from a custom pyplugin)

Type	`"from_plugin"`

`pseudofiles.<string>.fsync.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.fsync.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.fsync.<model=from_plugin>.function Function to call

Type	string or null
Default	`fsync`

pseudofiles.<string>.fsync.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.fsync.<model=custom>.model Fsync modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.fsync.<model=custom>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.fsync.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.fasync Fasync

Default	`null`

How to handle fasync() on the file

`pseudofiles.<string>.fasync.<model=from_plugin>` fasync from a custom PyPlugin

`pseudofiles.<string>.fasync.<model=from_plugin>.model` Fasync modelling method (fasync from a custom pyplugin)

Type	`"from_plugin"`

`pseudofiles.<string>.fasync.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.fasync.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.fasync.<model=from_plugin>.function` Function to call

Type	string or null
Default	`fasync`

`pseudofiles.<string>.fasync.<model=custom>` Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

`pseudofiles.<string>.fasync.<model=custom>.model` Fasync modelling method (custom registered model)

Type	`"custom"`

`pseudofiles.<string>.fasync.<model=custom>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.fasync.<model=custom>.model_name` Registered model name

Type	string

`pseudofiles.<string>.lock` Lock

Default	`null`

How to handle lock() on the file

`pseudofiles.<string>.lock.<model=from_plugin>` lock from a custom PyPlugin

`pseudofiles.<string>.lock.<model=from_plugin>.model` Lock modelling method (lock from a custom pyplugin)

Type	`"from_plugin"`

`pseudofiles.<string>.lock.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.lock.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.lock.<model=from_plugin>.function` Function to call

Type	string or null
Default	`lock`

pseudofiles.<string>.lock.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.lock.<model=custom>.model Lock modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.lock.<model=custom>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.lock.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.read_iter Read iterator

Default	`null`

How to handle read_iter() on the file

pseudofiles.<string>.read_iter.<model=from_plugin> read_iter from a custom PyPlugin

pseudofiles.<string>.read_iter.<model=from_plugin>.model Read iterator modelling method (read_iter from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.read_iter.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.read_iter.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.read_iter.<model=from_plugin>.function Function to call

Type	string or null
Default	`read_iter`

pseudofiles.<string>.read_iter.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.read_iter.<model=custom>.model Read iterator modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.read_iter.<model=custom>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.read_iter.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.write_iter Write iterator

Default	`null`

How to handle write_iter() on the file

pseudofiles.<string>.write_iter.<model=from_plugin> write_iter from a custom PyPlugin

pseudofiles.<string>.write_iter.<model=from_plugin>.model Write iterator modelling method (write_iter from a custom pyplugin)

Type	`"from_plugin"`

pseudofiles.<string>.write_iter.<model=from_plugin>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write_iter.<model=from_plugin>.plugin Name of the loaded PyPlugin

Type	string

pseudofiles.<string>.write_iter.<model=from_plugin>.function Function to call

Type	string or null
Default	`write_iter`

pseudofiles.<string>.write_iter.<model=custom> Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

pseudofiles.<string>.write_iter.<model=custom>.model Write iterator modelling method (custom registered model)

Type	`"custom"`

pseudofiles.<string>.write_iter.<model=custom>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.write_iter.<model=custom>.model_name Registered model name

Type	string

pseudofiles.<string>.get_unmapped_area Get unmapped area

Default	`null`

How to handle get_unmapped_area() on the file

pseudofiles.<string>.get_unmapped_area.<model=from_plugin> get_unmapped_area from a custom PyPlugin

pseudofiles.<string>.get_unmapped_area.<model=from_plugin>.model Get unmapped area modelling method (get_unmapped_area from a custom pyplugin)

Type	`"from_plugin"`

`pseudofiles.<string>.get_unmapped_area.<model=from_plugin>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`pseudofiles.<string>.get_unmapped_area.<model=from_plugin>.plugin` Name of the loaded PyPlugin

Type	string

`pseudofiles.<string>.get_unmapped_area.<model=from_plugin>.function` Function to call

Type	string or null
Default	`get_unmapped_area`

`pseudofiles.<string>.get_unmapped_area.<model=custom>` Custom registered model

Use a model registered via @register_model in a loaded plugin. 'model_name' selects it; any extra keys are forwarded to the model.

`pseudofiles.<string>.get_unmapped_area.<model=custom>.model` Get unmapped area modelling method (custom registered model)

Type	`"custom"`

`pseudofiles.<string>.get_unmapped_area.<model=custom>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

pseudofiles.<string>.get_unmapped_area.<model=custom>.model_name Registered model name

Type	string

14.6 nvram NVRAM

Default	{}

NVRAM values to add to the guest

14.6.1 nvram.<string> NVRAM value

Type	string or integer
Default	null

14.7 netdevs Network devices

Type	list of string
Default	[]

Names for guest network interfaces

```
- eth0
- eth1
```

```
- ens33
- wlp3s0
```

14.8 uboot_env U-Boot environment

Default	{}

U-Boot environment variables to set in the guest

14.8.1 `uboot_env.<string>` Value

Type	string
Default	`null`

Value of the U-Boot environment variable

14.9 `blocked_signals` List of blocked signals

Type	list of integer
Default	`[]`

Signals numbers to block within the guest. Supported values are 6 (SIGABRT), 9 (SIGKILL), 15 (SIGTERM), and 17 (SIGCHLD).

14.10 `lib_inject` Injected library configuration

Library functions to be intercepted

14.10.1 `lib_inject.enabled` Enables LD_PRELOAD in init script

Type	boolean or null
Default	`true`

Whether to enable LD_PRELOAD variable in init script

```
false
```

```
true
```

14.10.2 `lib_inject.aliases` Injected library aliases

Default	`{}`

Mapping between names of external library functions and names of functions defined in the injected library. This allows replacing arbitrary library functions with your own code.

```
fputs: 'false'
nvram_load: nvram_init
```

`lib_inject.aliases.<string>` Injected library alias target

Type	string
Default	`null`

This is the name of the target function that the alias points to.

```
nvram_init
```

```
'true'
```

```
'false'
```

14.10.3 `lib_inject.extra` Extra injected library code

Type	string
Patch merge behavior	Concatenate strings separated by `'\n'`
Default	`null`

Custom source code for library functions to intercept and model

14.10.4 `lib_inject.stubs` Declarative symbol stubs

Default	`null`

Declarative symbol stubs, grouped by library/object.

Each key is a library/object (an absolute guest path like `/lib/libc.so` or a bare basename like `libX.so` searched for in the rootfs). Each value maps a symbol key to a stub action. A symbol key is either a symbol name, a glob such as `nvram_*` (expanded against the library's exported symbols at build time; symbol-return stubs only), or `symbol@offset` (assembly-body stubs only, where `offset` is added to the symbol address).

```
{}
```

```
/lib/libc.so:
  memcpy:
    guard_null_args:
    - 0
    - 1
    return: 0
libX.so:
  get_flag:
    return: 0
  hw_probe:
    body: 'movs r0, #0
```

(continues on next page)

(continued from previous page)

```
    bx lr'
  mode: thumb
nvram_*:
  return: 0
```

`lib_inject.stubs.<string>` Per-library symbol stubs

Symbols (or globs) to stub within one library/object. The library key is an organizational label and the glob-resolution target; aliasing itself is global (`--defsym`).

`lib_inject.stubs.<string>.<string>` Symbol stub

A single declarative symbol stub. Two mutually exclusive forms: symbol-return ('return'/'type'/'guard_null_args'), which compiles to a generated C shim plus a lib_inject '–defsym' alias; and assembly-body ('body'/'mode'/'expect') at a 'symbol' or 'symbol@offset' key, which compiles to a static_files 'binary_patch'.

`lib_inject.stubs.<string>.<string>.return` Constant value to return

Type	integer or null
Default	`null`

Value the stubbed symbol returns. For a plain stub this is the return value; for a 'guard_null_args' stub it is the value returned on the NULL path (defaults to 0).

```
0
```

```
1
```

```
42
```

`lib_inject.stubs.<string>.<string>.type` Return C type

Type	string or null
Default	`long`

C return type of the generated shim. Defaults to register-width 'long', correct for most integer/pointer returns.

```
long
```

```
int
```

```
unsigned int
```

```
void *
```

`lib_inject.stubs.<string>.<string>.guard_null_args` NULL-guarded argument positions

Type	list of integer or null
Default	`null`

Zero-based argument positions to NULL-check. If any listed argument is NULL the shim returns the 'return' constant; otherwise it calls through to the real symbol. Limited to the first 8 integer/pointer register arguments (indices 0-7); floating-point, struct-by-value, and stack arguments are not preserved on the call-through path.

```
- 0
```

```
- 0
- 1
```

`lib_inject.stubs.<string>.<string>.body` Assembly body to write at the symbol

Type	string
Patch merge behavior	Concatenate strings separated by `'\n'`
Default	`null`

Assembly to assemble (via keystone) and write over the instructions at the stub's symbol/offset. Use with a 'symbol' or 'symbol@offset' key. Compiles down to a static_files 'binary_patch'. Mutually exclusive with 'return'/'guard_null_args'.

```
'mov v0, $zero

  jr $ra'
```

```
'movs r0, #0

  bx lr'
```

`lib_inject.stubs.<string>.<string>.mode` Assembly mode for 'body'

Type	string or null
Default	`null`

Assembly mode passed through to binary_patch (e.g. 'arm' or 'thumb' on ARM). Defaults to the target arch's natural mode.

```
arm
```

```
thumb
```

lib_inject.stubs.<string>.<string>.expect Expected original bytes

Type	string or null
Default	null

Optional hex string of the bytes expected at the patch site; passed through to binary_patch as a safety check before the 'body' overwrite.

```
'00000000'
```

```
1eff2fe1
```

14.11 static_files Static files

Files to create in the guest filesystem

```
{}
```

```
/path/to/file:
  contents: Hello world!
  type: file
```

```
/path/to/symlink/source:
  target: /path/to/symlink/target
  type: symlink
```

```
/dev/some_device:
  devtype: char
  major: 1
  minor: 2
  mode: 438
  type: dev
```

14.11.1 static_files.<string> Static filesystem action

Default	null

static_files.<string>.<type=inline_file> Add inline file

Add a file with contents specified inline in this config

static_files.<string>.<type=inline_file>.type Type of file action (add inline file)

Type	`"inline_file"`

static_files.<string>.<type=inline_file>.provenance Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

static_files.<string>.<type=inline_file>.mode Permissions of file

Type	integer
Default	`420`

static_files.<string>.<type=inline_file>.contents Contents of file

Type	string

static_files.<string>.<type=inline_file>.patches Binary patches to apply after this file is placed

Type	list of Binary patch entry
Default	`null`

A list of binary edits applied to this file after it is staged into the guest, in a single host-side pass (same semantics as the standalone 'binary_patch' action). Each edit may verify the bytes currently at its offset (expect/on_mismatch) and record rationale (why/tag); every outcome is written to binary_patches.yaml in the run output. Overlapping write ranges are rejected. Cannot be combined with a glob source or destination (the patch target would be ambiguous).

static_files.<string>.<type=inline_file>.patches.<item> Binary patch entry

A single edit within a `binary_patch` action: bytes to write at one file offset, optionally guarded by an `expect` check. Multiple entries can target one file via the action's `patches` list; they are applied host-side to one buffer in a single pass, and overlapping write ranges are rejected.

static_files.<string>.<type=inline_file>.patches.<item>.file_offset File offset (integer)

Type	integer

static_files.<string>.<type=inline_file>.patches.<item>.hex_bytes Bytes to write at offset (hex string)

Type	string or null
Default	null

```
DEADBEEF
```

```
90 90
```

static_files.<string>.<type=inline_file>.patches.<item>.asm Assembly code to write at offset (runs through keystone)

Type	string or null
Default	null

```
nop
```

```
'mov r0, #0xdeadbeef'
```

static_files.<string>.<type=inline_file>.patches.<item>.mode Assembly mode

Type	string or null
Default	null

```
arm
```

```
thumb
```

static_files.<string>.<type=inline_file>.patches.<item>.expect Expected bytes at offset before patching (hex string)

Type	string or null
Default	null

If set, the current bytes at file_offset are compared against this hex string (over its own length, which may differ from the patch length) before the patch is written. If the bytes at the offset already equal the patch bytes, the patch is skipped (idempotent re-run); otherwise the on_mismatch policy applies.

```
0102 0304
```

```
DEADBEEF
```

`static_files.<string>.<type=inline_file>.patches.<item>.on_mismatch` Policy when 'expect' does not match

Type	`"fail"` or `"skip"` or `"warn"`
Default	`fail`

fail: abort the run (default, safest). skip: leave the file unpatched and continue. warn: log a warning and write the patch anyway. Only meaningful when 'expect' is set.

`static_files.<string>.<type=inline_file>.patches.<item>.why` Rationale for this patch, recorded in the run's binary_patches.yaml

Type	string or null
Default	`null`

```
NOP out the secure-boot check
```

`static_files.<string>.<type=inline_file>.patches.<item>.tag` Label grouping related patches, recorded in the run's binary_patches.yaml

Type	string or null
Default	`null`

```
secureboot
```

`static_files.<string>.<type=host_file>` Copy host file

Copy a file from the host into the guest

`static_files.<string>.<type=host_file>.type` Type of file action (copy host file)

Type	`"host_file"`

`static_files.<string>.<type=host_file>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`static_files.<string>.<type=host_file>.mode` Permissions of file

Type	integer
Default	`493`

`static_files.<string>.<type=host_file>.host_path` Host path

Type	string

`static_files.<string>.<type=host_file>.patches` Binary patches to apply after this file is placed

Type	list of Binary patch entry
Default	`null`

A list of binary edits applied to this file after it is staged into the guest, in a single host-side pass (same semantics as the standalone 'binary_patch' action). Each edit may verify the bytes currently at its offset (expect/on_mismatch) and record rationale (why/tag); every outcome is written to binary_patches.yaml in the run output. Overlapping write ranges are rejected. Cannot be combined with a glob source or destination (the patch target would be ambiguous).

`static_files.<string>.<type=host_file>.patches.<item>` Binary patch entry

A single edit within a `binary_patch` action: bytes to write at one file offset, optionally guarded by an `expect` check. Multiple entries can target one file via the action's `patches` list; they are applied host-side to one buffer in a single pass, and overlapping write ranges are rejected.

`static_files.<string>.<type=host_file>.patches.<item>.file_offset` File offset (integer)

Type	integer

`static_files.<string>.<type=host_file>.patches.<item>.hex_bytes` Bytes to write at offset (hex string)

Type	string or null
Default	`null`

```
DEADBEEF
```

```
90 90
```

`static_files.<string>.<type=host_file>.patches.<item>.asm` Assembly code to write at offset (runs through keystone)

Type	string or null
Default	`null`

```
nop
```

```
'mov r0, #0xdeadbeef'
```

`static_files.<string>.<type=host_file>.patches.<item>.mode` Assembly mode

Type	string or null
Default	`null`

```
arm
```

```
thumb
```

`static_files.<string>.<type=host_file>.patches.<item>.expect` Expected bytes at offset before patching (hex string)

Type	string or null
Default	`null`

If set, the current bytes at file_offset are compared against this hex string (over its own length, which may differ from the patch length) before the patch is written. If the bytes at the offset already equal the patch bytes, the patch is skipped (idempotent re-run); otherwise the on_mismatch policy applies.

```
0102 0304
```

```
DEADBEEF
```

static_files.<string>.<type=host_file>.patches.<item>.on_mismatch Policy when 'expect' does not match

Type	"fail" or "skip" or "warn"
Default	fail

fail: abort the run (default, safest). skip: leave the file unpatched and continue. warn: log a warning and write the patch anyway. Only meaningful when 'expect' is set.

static_files.<string>.<type=host_file>.patches.<item>.why Rationale for this patch, recorded in the run's binary_patches.yaml

Type	string or null
Default	null

```
NOP out the secure-boot check
```

static_files.<string>.<type=host_file>.patches.<item>.tag Label grouping related patches, recorded in the run's binary_patches.yaml

Type	string or null
Default	null

```
secureboot
```

static_files.<string>.<type=dir> Add directory

static_files.<string>.<type=dir>.type Type of file action (add directory)

Type	"dir"

static_files.<string>.<type=dir>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

static_files.<string>.<type=dir>.mode Permissions of directory

Type	integer
Default	493

static_files.<string>.<type=symlink> Add symbolic link

static_files.<string>.<type=symlink>.type Type of file action (add symbolic link)

Type	"symlink"

static_files.<string>.<type=symlink>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

static_files.<string>.<type=symlink>.target Target linked path

Type	string

static_files.<string>.<type=dev> Add device file

static_files.<string>.<type=dev>.type Type of file action (add device file)

Type	"dev"

static_files.<string>.<type=dev>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

static_files.<string>.<type=dev>.devtype Type of device file

Type	"char" or "block"

static_files.<string>.<type=dev>.major Major device number

Type	integer

static_files.<string>.<type=dev>.minor Minor device number

Type	integer

static_files.<string>.<type=dev>.mode Permissions of device file

Type	integer
Default	438

static_files.<string>.<type=delete> Delete file

static_files.<string>.<type=delete>.type Type of file action (delete file)

Type	"delete"

static_files.<string>.<type=delete>.provenance Model provenance

Type	string or null
Default	null

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`static_files.<string>.<type=move>` Move file

`static_files.<string>.<type=move>.type` Type of file action (move file)

Type	`"move"`

`static_files.<string>.<type=move>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`static_files.<string>.<type=move>.from` File to be moved to the specified location

Type	string

`static_files.<string>.<type=move>.mode` Permissions of target file

Type	integer or null
Default	`null`

`static_files.<string>.<type=shim>` Shim file

`static_files.<string>.<type=shim>.type` Type of file action (shim file)

Type	`"shim"`

`static_files.<string>.<type=shim>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`static_files.<string>.<type=shim>.target` Target file we want the shim to be symlinked to

Type	string

`static_files.<string>.<type=binary_patch>` Patch binary file

Patch a binary file at one or more offsets. A single edit is given inline (file_offset + one of hex_bytes/asm); multiple edits to the same file go in the 'patches' list (applied together in one host-side pass, with overlapping write ranges rejected). Each edit may verify the bytes currently at its offset first (expect/on_mismatch) so the patch is idempotent and safe across firmware variants, and record rationale (why/tag); every edit's outcome is written to binary_patches.yaml in the run output.

`static_files.<string>.<type=binary_patch>.type` Type of file action (patch binary file)

Type	`"binary_patch"`

`static_files.<string>.<type=binary_patch>.provenance` Model provenance

Type	string or null
Default	`null`

Origin tag. Set 'default' for a synthesized stub (it reports its hits into pseudofiles_failures.yaml); leave unset for author-intentional models.

`static_files.<string>.<type=binary_patch>.file_offset` File offset (integer) — for a single inline edit; omit when using 'patches'

Type	integer or null
Default	`null`

`static_files.<string>.<type=binary_patch>.hex_bytes` Bytes to write at offset (hex string)

Type	string or null
Default	`null`

```
DEADBEEF
```

```
90 90
```

static_files.<string>.<type=binary_patch>.asm Assembly code to write at offset (runs through keystone)

Type	string or null
Default	null

```
nop
```

```
'mov r0, #0xdeadbeef'
```

static_files.<string>.<type=binary_patch>.mode Assembly mode

Type	string or null
Default	null

```
arm
```

```
thumb
```

static_files.<string>.<type=binary_patch>.expect Expected bytes at offset before patching (hex string)

Type	string or null
Default	null

If set, the current bytes at file_offset are compared against this hex string (over its own length, which may differ from the patch length) before the patch is written. If the bytes at the offset already equal the patch bytes, the patch is skipped (idempotent re-run); otherwise the on_mismatch policy applies.

```
0102 0304
```

```
DEADBEEF
```

static_files.<string>.<type=binary_patch>.on_mismatch Policy when 'expect' does not match

Type	"fail" or "skip" or "warn"
Default	fail

fail: abort the run (default, safest). skip: leave the file unpatched and continue. warn: log a warning and write the patch anyway. Only meaningful when 'expect' is set.

static_files.<string>.<type=binary_patch>.why Rationale for this patch, recorded in the run's binary_patches.yaml

Type	string or null
Default	null

```
NOP out the secure-boot check
```

static_files.<string>.<type=binary_patch>.tag Label grouping related patches, recorded in the run's binary_patches.yaml

Type	string or null
Default	null

```
secureboot
```

static_files.<string>.<type=binary_patch>.patches Multiple edits to this file

Type	list of Binary patch entry
Default	null

A list of edits applied to this one file in a single host-side pass. Use this instead of the inline file_offset/hex_bytes/asm fields when patching a binary at more than one offset. Overlapping write ranges are rejected.

static_files.<string>.<type=binary_patch>.patches.<item> Binary patch entry

A single edit within a `binary_patch` action: bytes to write at one file offset, optionally guarded by an `expect` check. Multiple entries can target one file via the action's `patches` list; they are applied host-side to one buffer in a single pass, and overlapping write ranges are rejected.

static_files.<string>.<type=binary_patch>.patches.<item>.file_offset File offset (integer)

Type	integer

static_files.<string>.<type=binary_patch>.patches.<item>.hex_bytes Bytes to write at offset (hex string)

Type	string or null
Default	null

```
DEADBEEF
```

```
90 90
```

static_files.<string>.<type=binary_patch>.patches.<item>.asm Assembly code to write at offset (runs through keystone)

Type	string or null
Default	null

```
nop
```

```
'mov r0, #0xdeadbeef'
```

static_files.<string>.<type=binary_patch>.patches.<item>.mode Assembly mode

Type	string or null
Default	null

```
arm
```

```
thumb
```

static_files.<string>.<type=binary_patch>.patches.<item>.expect Expected bytes at offset before patching (hex string)

Type	string or null
Default	null

If set, the current bytes at file_offset are compared against this hex string (over its own length, which may differ from the patch length) before the patch is written. If the bytes at the offset already equal the patch bytes, the patch is skipped (idempotent re-run); otherwise the on_mismatch policy applies.

```
0102 0304
```

```
DEADBEEF
```

`static_files.<string>.<type=binary_patch>.patches.<item>.on_mismatch` Policy when 'expect' does not match

Type	`"fail"` or `"skip"` or `"warn"`
Default	`fail`

fail: abort the run (default, safest). skip: leave the file unpatched and continue. warn: log a warning and write the patch anyway. Only meaningful when 'expect' is set.

`static_files.<string>.<type=binary_patch>.patches.<item>.why` Rationale for this patch, recorded in the run's binary_patches.yaml

Type	string or null
Default	`null`

```
NOP out the secure-boot check
```

`static_files.<string>.<type=binary_patch>.patches.<item>.tag` Label grouping related patches, recorded in the run's binary_patches.yaml

Type	string or null
Default	`null`

```
secureboot
```

14.12 `plugins` Plugins

14.12.1 `plugins.<string>` Plugin

Field	Type	Default	Title
`description`	string or null	`null`	Plugin description
`depends_on`	string or null	`null`	Plugin dependency
`enabled`	boolean	`true`	Enable this plugin (default depends on plugin)
`version`	string or null	`null`	Plugin version

14.13 `init_plugins` Init plugins

Default	`null`

The init plugins that generated this project (recorded by `penguin init`). Drives which plugins re-run on `penguin refresh`; newly available plugins are appended when they run.

14.13.1 `init_plugins.<string>` Init plugin

One init plugin's record/settings in the init_plugins section.

`init_plugins.<string>.enabled` Run this init plugin during penguin refresh

Type	boolean
Default	`true`

Set to false to skip this plugin entirely when re-running init analyses with `penguin refresh`.

14.14 `network` Network Configuration

Default	`null`

Configuration for networks to attach to guest

14.14.1 `network.external` Set up NAT for outgoing connections

Configuration for NAT for external connections

`network.external.mac` MAC Address for external interface

Type	string or null
Default	`'52:54:00:12:34:56'`

MAC Address for external network interface

`network.external.pcap` pcap file name

Type	boolean or null
Default	`null`

Whether to capture traffic over the external net in a pcap file. The file will be called 'ext.pcap' in the output directory. Capture disabled if unset.

CHAPTER

FIFTEEN

SCRIPTING PLUGINS IN PENGUIN

Penguin supports both class-based plugins and scripting plugins. Scripting plugins allow you to write simple, top-level Python scripts that interact with the plugin manager and the emulation environment, without needing to define a class. This is useful for quick automation, prototyping, or when you want to use decorators and plugin APIs without boilerplate.

> **How scripting plugins work:** When you load a script as a plugin, the script's top-level code is executed as if it were inside the `__init__` method of a special `ScriptingPlugin` class. This means all global code runs at plugin load time, decorators are registered, and any setup logic is performed immediately. The script is managed by the plugin manager just like a class-based plugin.

15.1 Key Differences: Plugins vs. Scripts

Feature	Class-based Plugin	Scripting Plugin (Script)
Structure	Subclass `Plugin`	Plain Python script
Entry Point	`__init__` method	Top-level code (runs in `__init__` of ScriptingPlugin)
Arguments	`self.get_arg("foo")`	`args` global
Decorators	Use as `@self.plugins.syscalls...`	Use as `@plugins.syscalls...`
Unload/cleanup	`uninit()` method	Define `uninit()` function
Access to manager	`self.plugins` or `import`	`plugins` global or `import`
Access to logger	`self.logger`	`logger` global

- Scripting plugins are loaded and managed just like class-based plugins. If a `.py` file does not define a `Plugin` subclass, it is loaded as a scripting plugin.
- The script's top-level code is executed at load time, and the `plugins` object is available for decorators and API calls.
- The `logger` global is automatically injected and named for the script file, so you can use `logger.info(...)` directly in your script.
- You can define an `uninit()` function in your script for cleanup, which will be called when the plugin is unloaded.

15.2 Example: Scripting Plugin

```
# Example scripting plugin for Penguin
# This script demonstrates how to use scripting plugins with the Penguin plugin manager.
```

(continues on next page)

(continued from previous page)

```
# It uses the global 'plugins' object for API access and the injected 'logger' for logging.
# No class definition is needed; top-level code is executed at load time.
# The 'uninit' function will be called automatically when the plugin is unloaded.

logger.info("Initializing scripting_test.py")
outdir = args.outdir
getpid_ran = False

@plugins.syscalls.syscall("on_sys_getpid_enter")
def getpid_enter(*args):
    """
    This function is registered as a syscall callback for 'on_sys_getpid_enter'.
    It writes a message to a file the first time it is called.
    """
    global getpid_ran
    if getpid_ran:
        return
    logger.info("Received getpid_enter syscall")

    with open(f"{outdir}/scripting_test.txt", "w") as f:
        f.write("Hello from scripting_test.py\n")
    getpid_ran = True

def uninit():
    """
    This function is called when the scripting plugin is unloaded.
    It appends a message to the output file, or writes a failure message if the syscall␣
↪was never triggered.
    """
    logger.info("Got uninit() call")
    if getpid_ran:
        with open(f"{outdir}/scripting_test.txt", "a") as f:
            f.write("Unloading scripting_test.py\n")
    else:
        with open(f"{outdir}/scripting_test.txt", "w") as f:
            f.write("FAIL: scripting_test.py was never run\n")
```

15.3 Example: Class-based Plugin

```
from penguin import Plugin, plugins

class ClassExample(Plugin):
    def __init__(self):
        self.getpid_ran = False
        self.logger.info(f"Initializing {self.name}")
        self.outdir = self.get_arg("outdir")
        plugins.syscalls.syscall("on_sys_getpid_enter")(self.getpid_enter)

    # Alternatively: you can also use the decorator directly
    # instead of plugins.syscalls.syscall in __init__
    # @plugins.syscalls.syscall("on_sys_getpid_enter")
```

(continues on next page)

(continued from previous page)

```
    def getpid_enter(self, *args):
        if self.getpid_ran:
            return
        self.logger.info("Received getpid_enter syscall")
        with open(f"{self.outdir}/scripting_test.txt", "w") as f:
            f.write("Hello from scripting_test.py\n")
        self.getpid_ran = True

    def uninit(self):
        self.logger.info("Got uninit() call")
        if self.getpid_ran:
            with open(f"{self.outdir}/class_example.txt", "a") as f:
                f.write("Unloading class_example.py\n")
        else:
            with open(f"{self.outdir}/class_example.txt", "w") as f:
                f.write("FAIL: class_example.py was never run\n")
```

15.4 When to Use Scripting Plugins

- For quick experiments, automation, or glue code.
- When you want to use decorators or plugin APIs without class boilerplate.
- When you don't need advanced features like a logger or inheritance.

15.5 When to Use Class-based Plugins

- For reusable, modular, or complex plugins.
- When you need a logger, inheritance, or advanced plugin lifecycle management.

Both plugin types are fully supported and can be mixed in the same project.

CHAPTER

SIXTEEN

SINGULARITY

Penguin can be built as a singularity container for use in HPC environments where limited permissions are available.

16.1 Building Singularity container

On a machine where you have docker and root, clone this repo and `cd` into it. Then build a docker container and convert it to singularity with

```
./penguin --build-singularity
```

This will produce a singularity image named `penguin.sif`.

Alternatively, you can build the container directly and convert to singularity by running the following within the root directory of the project.

```
# Build container
DOCKER_BUILDKIT=1 docker build -t rehosting/pegnuin .

# Convert to singularity
docker run -v /var/run/docker.sock:/var/run/docker.sock \
    -v $(pwd):/output \
    --privileged  -t \
    --rm quay.io/singularity/docker2singularity:v3.9.0 rehosting/penguin

# Rename to penguin.sif
mv rehosting_penguin*.sif penguin.sif
```

Now copy your `penguin.sif` file to your target machine with singularity.

16.2 Using your Singularity container

Finally, run your container under singularity with a host directory and projects directory mapped into the container in your home directory. Note the following explanation of the arguments:

- `-e` unsets host environment variables so a host `$TMPDIR` (which isn't mapped) isn't used
- `-c` contains the container filesystem to be isolated from the host (i.e., only explicitly shared directories are shared)
- `-B` binds two directories from the host into the container: the current directory and ./projects. Both are mapped into the home directory in the container (which is the same as it is on your host).

```
mkdir projects
singularity exec -e -c -B $(pwd):$HOME/host,$(pwd)/projects:$HOME/projects,$TMPDIR:/tmp␣
↪penguin.sif bash
Singularity> penguin init host/fws/my_fw.rootfs.tar.gz
Singularity> penguin run projects/my_fw
```

Alternatively, you can run these commands from your host directly:

```
mkdir projects
singularity exec -e -c -B $(pwd):$HOME/host,$(pwd)/projects:$HOME/projects,$TMPDIR:/tmp␣
↪penguin.sif penguin init host/fws/my_fw.rootfs.tar.gz
singularity exec -e -c -B $(pwd):$HOME/host,$(pwd)/projects:$HOME/projects,$TMPDIR:/tmp␣
↪penguin.sif penguin run projects/my_fw
```

If the VPN informs you of a network-reachable service, you'll connect to localhost on the specified port.

CHAPTER

SEVENTEEN

SYSCALLS PLUGIN SYSTEM

The Syscalls plugin provides a comprehensive interface for monitoring, filtering, and intercepting system calls in the guest via the IGLOO hypervisor interface. This system enables real-time analysis of guest behavior, security monitoring, and dynamic program analysis.

17.1 Overview

The syscalls system operates through a multi-layer architecture:

1. **Kernel Layer**: IGLOO kernel module hooks into syscall entry/return points
2. **Hypervisor Layer**: Portal system manages communication between guest and host
3. **Plugin Layer**: Python plugins register callbacks and process syscall events

17.2 Quick Start

```
from penguin import plugins, Plugin

class MySyscallMonitor(Plugin):
    def __init__(self):
        # Register syscall hooks in __init__
        plugins.syscalls.syscall("on_sys_execve_enter")(self.monitor_execve)
        plugins.syscalls.syscall("on_all_sys_enter", comm_filter="target_process")(self.
↪monitor_all)

    def monitor_execve(self, pt_regs, proto, syscall, filename, argv, envp):
        """Monitor process execution"""
        filename_str = yield from plugins.mem.read_str(filename)
        print(f"Process executing: {filename_str}")

    def monitor_all(self, pt_regs, proto, syscall):
        """Monitor all syscalls for a specific process"""
        print(f"Syscall: {proto.name} by {syscall.task.comm}")
```

17.3 Core Concepts

17.3.1 Syscall Events

Every syscall generates events that can be intercepted at two points:

- **Entry (on_enter)**: Before the syscall executes in the kernel

- **Return (on_return)**: After the syscall completes

17.3.2 Hook Registration

Hooks are registered using the `@syscalls.syscall()` decorator with flexible filtering options:

```
@syscalls.syscall(
    name_or_pattern="sys_openat",
    on_enter=True,
    on_return=True,
    comm_filter="target_app",
    pid_filter=1234,
    arg_filters=[None, ValueFilter.exact(0x42)],
    retval_filter=ValueFilter.success()
)
def my_handler(pt_regs, proto, syscall, *args):
    # Handler code here
    pass
```

17.4 API Reference

17.4.1 Decorator Parameters

`name_or_pattern` (Optional[str])

Specifies which syscalls to hook. Supports multiple formats:

```
# Specific syscall
@syscalls.syscall("sys_open")

# Pattern-based (hsyscall format)
@syscalls.syscall("on_sys_execve_enter")
@syscalls.syscall("on_sys_openat_return")

# All syscalls
@syscalls.syscall("on_all_sys_enter")

# Unknown syscalls (those without metadata)
@syscalls.syscall("on_unknown_sys_enter")
```

`on_enter` / `on_return` (Optional[bool])

Control when the hook triggers:

```
# Entry only (default if neither specified)
@syscalls.syscall("sys_open", on_enter=True)

# Return only
@syscalls.syscall("sys_open", on_return=True)

# Both entry and return
@syscalls.syscall("sys_open", on_enter=True, on_return=True)
```

comm_filter (Optional[str])

Filter by process name:

```
@syscalls.syscall("sys_write", comm_filter="nginx")
def monitor_nginx_writes(pt_regs, proto, syscall, fd, buf, count):
    pass
```

pid_filter (Optional[int])

Filter by specific process ID:

```
@syscalls.syscall("sys_read", pid_filter=1234)
def monitor_specific_process(pt_regs, proto, syscall, fd, buf, count):
    pass
```

17.4.2 Advanced Filtering with ValueFilter

The `ValueFilter` class provides sophisticated argument and return value filtering:

```
from apis.syscalls import ValueFilter

# Exact match
ValueFilter.exact(42)

# Numeric comparisons
ValueFilter.greater(100)
ValueFilter.less_equal(1000)
ValueFilter.range(10, 20)

# Success/error filtering
ValueFilter.success()  # >= 0
ValueFilter.error()    # < 0

# Bitwise operations
ValueFilter.bitmask_set(0x0001)    # All specified bits set
ValueFilter.bitmask_clear(0x0002)  # All specified bits clear
```

arg_filters (Optional[List[Any]])

Filter by syscall arguments:

```
@syscalls.syscall(
    "sys_ioctl",
    arg_filters=[
        None,                          # arg0: no filter
        ValueFilter.exact(0xabcd),     # arg1: exact match
        ValueFilter.range(100, 200)    # arg2: range filter
    ]
)
def filtered_ioctl(pt_regs, proto, syscall, fd, cmd, arg):
    pass
```

retval_filter (Optional[Any])

Filter by return value:

```
# Only successful calls
@syscalls.syscall("sys_open", on_return=True, retval_filter=ValueFilter.success())

# Specific error codes
@syscalls.syscall("sys_open", on_return=True, retval_filter=ValueFilter.exact(-2))  #␣
↪ENOENT

# Backward compatibility with simple values
@syscalls.syscall("sys_getpid", on_return=True, retval_filter=1234)
```

17.5 Handler Function Signatures

17.5.1 Standard Handlers

```
def handler(regs, proto, syscall, *args):
    """
    regs: Register/context object (formerly pt_regs, now typically a PtRegsWrapper)
        - Contains the CPU register state at the time of the syscall.
        - Provides access to general-purpose registers, PC, SP, etc.
        - Used for advanced introspection or argument extraction.
    proto: SyscallPrototype with metadata (name, types, arg names)
    syscall: SyscallEvent object with runtime data
    *args: Unpacked syscall arguments (for known syscalls)
    """
```

17.5.2 All-Syscall Handlers

```
def all_handler(regs, proto, syscall):
    """
    For on_all_sys_* handlers, arguments are not unpacked
    Access via syscall.args array instead
    """
```

17.6 SyscallEvent Object

The `syscall` parameter provides access to runtime information:

```
def my_handler(regs, proto, syscall, *args):
    # Basic information
    print(f"Syscall: {syscall.name}")
    print(f"Process: {syscall.task.comm}")
    print(f"PC: {syscall.pc:#x}")

    # Arguments (raw values)
    for i in range(syscall.argc):
        print(f"arg[{i}]: {syscall.args[i]:#x}")
```

(continues on next page)

(continued from previous page)

```
    # Return value (for return handlers)
    print(f"Return value: {syscall.retval}")

    # Modify behavior
    syscall.skip_syscall = True    # Skip syscall execution
    syscall.retval = 42           # Set return value
```

17.7 SyscallPrototype Object

The `proto` parameter contains syscall metadata:

```
def my_handler(regs, proto, syscall, *args):
    print(f"Syscall name: {proto.name}")
    print(f"Argument count: {proto.nargs}")

    # Argument metadata
    for i in range(proto.nargs):
        print(f"arg[{i}]: {proto.types[i]} {proto.names[i]}")
```

17.8 Practical Examples

17.8.1 Process Monitoring

```
@syscalls.syscall("on_sys_execve_enter")
def track_process_creation(regs, proto, syscall, filename, argv, envp):
    """Monitor process creation"""
    args = []
    try:
        # Read string arguments from guest memory
        filename_str = yield from plugins.mem.read_str(filename)
        args_list = yield from plugins.mem.read_ptrlist(argv)

        print(f"New process: {filename_str}")
        print(f"Arguments: {args_list}")

        # Log to file or database
        with open("process_log.txt", "a") as f:
            f.write(f"{filename_str} {' '.join(args_list)}\n")

    except Exception as e:
        print(f"Error reading process arguments: {e}")
```

17.8.2 File Access Monitoring

```
@syscalls.syscall("on_sys_openat_enter")
def monitor_file_access(regs, proto, syscall, dirfd, pathname, flags, mode):
    """Monitor file access attempts"""
    try:
        path_str = yield from plugins.mem.read_str(pathname)
```

(continues on next page)

(continued from previous page)

```
        # Check for sensitive files
        sensitive_files = ["/etc/passwd", "/etc/shadow", "/etc/ssh/"]
        if any(sens in path_str for sens in sensitive_files):
            print(f"ALERT: Access to sensitive file: {path_str}")

            # Optionally block the access
            if "/etc/shadow" in path_str:
                syscall.skip_syscall = True
                syscall.retval = -13  # EACCES

    except Exception as e:
        print(f"Error reading pathname: {e}")
```

17.8.3 Network Monitoring

```
@syscalls.syscall("on_sys_sendto_enter", comm_filter="target_app")
def monitor_network_send(regs, proto, syscall, sockfd, buf, length, flags, dest_addr,␣
↪addrlen):
    """Monitor network sends from specific application"""
    try:
        # Read the data being sent
        data = yield from plugins.mem.read_bytes(buf, min(length, 100))  # First 100␣
↪bytes

        print(f"Network send: {len(data)} bytes")
        print(f"Data preview: {data[:50]}")

        # Check for suspicious patterns
        if b"password" in data.lower():
            print("WARNING: Potential password transmission detected")

    except Exception as e:
        print(f"Error reading network data: {e}")
```

17.8.4 Return Value Analysis

```
@syscalls.syscall("on_sys_open_return", retval_filter=ValueFilter.error())
def track_failed_opens(regs, proto, syscall):
    """Track failed file open attempts"""
    # Get the original arguments from saved state
    filename = syscall.args[0]  # pathname argument

    try:
        path_str = yield from plugins.mem.read_str(filename)
        error_code = -syscall.retval

        print(f"Failed to open: {path_str} (error: {error_code})")

        # Log failed access attempts
        with open("failed_access.log", "a") as f:
```

(continues on next page)

(continued from previous page)

```
            f.write(f"{path_str} {error_code}\n")

    except Exception as e:
        print(f"Error processing failed open: {e}")
```

17.8.5 Complex Filtering Example

```
@syscalls.syscall(
    "sys_ioctl",
    comm_filter="target_driver",
    arg_filters=[
        None,                                   # fd: any
        ValueFilter.range(0x1000, 0x2000),     # cmd: in range
        ValueFilter.bitmask_set(0x0001)         # arg: has specific bit set
    ],
    retval_filter=ValueFilter.success()
)
def monitor_specific_ioctls(regs, proto, syscall, fd, cmd, arg):
    """Monitor specific ioctl patterns"""
    print(f"Targeted ioctl: fd={fd}, cmd={cmd:#x}, arg={arg:#x}")
```

17.9 Performance Considerations

17.9.1 Hook Granularity

- **Specific syscalls**: Lowest overhead, only hook what you need
- **Process filtering**: Reduces events but still processes all syscalls
- **All syscalls**: Highest overhead, use sparingly

```
# Good: Specific and filtered
@syscalls.syscall("sys_write", comm_filter="target_app")

# Moderate: Broader but still filtered
@syscalls.syscall("on_all_sys_enter", comm_filter="target_app")

# Heavy: No filtering
@syscalls.syscall("on_all_sys_enter")
```

17.9.2 Memory Access

Reading guest memory is expensive. Cache results when possible:

```
# Cache frequently accessed strings
_path_cache = {}

def cached_read_string(addr):
    if addr in _path_cache:
        return _path_cache[addr]
```

(continues on next page)

(continued from previous page)

```
    path = yield from plugins.mem.read_str(addr)
    _path_cache[addr] = path
    return path
```

17.9.3 Filtering Efficiency

Use the most restrictive filters first:

```
# Good: Process filter eliminates most events early
@syscalls.syscall("sys_write", comm_filter="target", retval_filter=ValueFilter.success())

# Less efficient: Complex argument filters on all processes
@syscalls.syscall("sys_write", arg_filters=[None, None, ValueFilter.greater(1000)])
```

17.10 Error Handling

17.10.1 Memory Access Failures

Always handle memory read failures gracefully:

```
def safe_handler(regs, proto, syscall, pathname):
    try:
        path_str = yield from plugins.mem.read_str(pathname)
        # Process path_str
    except Exception as e:
        # Log error but don't crash
        print(f"Failed to read pathname: {e}")
        return
```

17.10.2 Portal Communication

The portal system handles most communication errors, but be aware of timeouts:

```
def robust_handler(regs, proto, syscall, buf, length):
    if length > 1024 * 1024:  # 1MB
        print("Buffer too large, skipping read")
        return

    try:
        data = yield from plugins.mem.read_bytes(buf, length)
        # Process data
    except Exception as e:
        print(f"Memory read failed: {e}")
```

17.11 Hook Management

17.11.1 Dynamic Control

Enable/disable hooks at runtime:

```
# Disable a hook
yield from syscalls.disable_syscall(my_handler_function)

# Re-enable it
yield from syscalls.enable_syscall(my_handler_function)

# Or by function name
yield from syscalls.disable_syscall("my_handler_function")
```

17.11.2 Hook Information

Access hook configuration:

```
# Get syscall prototype information
proto = syscalls.get_syscall_info_by_name("sys_open")
if proto:
    print(f"Syscall: {proto.name}")
    for i in range(proto.nargs):
        print(f"  {proto.types[i]} {proto.names[i]}")
```

17.12 Integration with Other Plugins

17.12.1 With Portal Plugin

The syscalls plugin integrates seamlessly with the portal system:

```
def combined_handler(pt_regs, proto, syscall, fd):
    # Use portal for additional context
    proc = yield from plugins.portal.get_proc()
    mappings = yield from plugins.portal.get_mappings()

    print(f"Syscall by PID {proc.pid}: {proto.name}")
```

17.12.2 With OSI Plugin

Combine with OS introspection:

```
def osi_enhanced_handler(pt_regs, proto, syscall, *args):
    # Get process information
    current_proc = yield from plugins.OSI.get_proc()
    if current_proc:
        print(f"Process: {current_proc.name} (PID: {current_proc.pid})")
```

17.13 Testing

The system includes comprehensive tests demonstrating usage patterns:

```
# From syscall_test.py
@syscalls.syscall("on_sys_ioctl_enter",
                  comm_filter="send_syscall",
                  arg_filters=[None, 0xabcd])
```

(continues on next page)

(continued from previous page)

```
def test_skip_retval(pt_regs, proto, syscall, fd, op, arg):
    """Test syscall skipping and return value modification"""
    assert fd == 9, f"Expected fd 9, got {fd:#x}"
    assert op == 0xabcd, f"Expected op 0xabcd, got {op:#x}"

    # Skip the syscall and set custom return value
    syscall.skip_syscall = True
    syscall.retval = 43
```

17.14 Troubleshooting

17.14.1 Common Issues

1. **Hook not triggering**: Check syscall name normalization

   ```
   # These are equivalent:
   "sys_openat"
   "openat"
   "_sys_openat"
   ```

2. **Argument mismatch**: Verify syscall prototype

   ```
   proto = syscalls.get_syscall_info_by_name("openat")
   print(f"Expected args: {proto.nargs}")
   ```

3. **Memory read failures**: Handle invalid pointers

   ```
   if addr and addr != 0:
       try:
           data = yield from plugins.mem.read_str(addr)
       except:
           print("Invalid memory address")
   ```

17.14.2 Debugging

Enable debug logging:

```
import logging
logging.getLogger("syscalls").setLevel(logging.DEBUG)
```

Use the syscall event information for debugging:

```
def debug_handler(regs, proto, syscall, *args):
    print(f"Syscall: {proto.name}")
    print(f"Args: {[hex(arg) for arg in syscall.args[:syscall.argc]]}")
    print(f"Process: {syscall.task.comm}")
    print(f"PC: {syscall.pc:#x}")
```

17.15 Best Practices

1. **Filter early and often**: Use the most specific filters possible
2. **Handle errors gracefully**: Always expect memory access to fail
3. **Cache expensive operations**: Don't re-read the same memory
4. **Use appropriate hook points**: Entry for argument analysis, return for result analysis
5. **Test thoroughly**: Use the provided test framework to verify behavior
6. **Monitor performance**: Syscall hooks can impact guest performance significantly

The syscalls system provides a powerful foundation for dynamic analysis, security monitoring, and program understanding in virtualized environments.

CHAPTER

EIGHTEEN

TESTING PYPLUGINS HOST-SIDE

Most Penguin behaviour that matters — the files a plugin writes, the events it emits, the portal commands it issues — is **host-side Python logic**. The guest only produces the raw event stream. So that logic can be tested by a fast, host-only unit test in `tests/unit/`, with **no PANDA, no guest, and no per-arch boot**, using the `penguin.testing` harness.

When you add or change a pyplugin, add a host test for its host-side logic. These tests run in the `unit_tests` CI job (a bare runner, no emulator, no matrix), so they gate every PR and stay fast.

18.1 The one rule: what is testable host-side

Draw the line per plugin:

- **In scope — test it here.** A plugin (or model) that *consumes events and writes host files*, or *decides which portal command to emit*. The `analysis/` writers, the `hyperfile/models/*` read/write/seek/ioctl models, the pseudofile registrars (`devfs`/`procfs`/`sysfs`/`sysctl`), the loggers.
- **Out of scope — leave it a `tests/integration/` fixture.** Anything that round-trips through the guest: a hypercall request whose *reply* drives the next step, guest-side struct packing, a native-toolchain build. The harness can fake the *shape* of the round-trip (see `drive(responses=...)`), but not the guest's real behaviour. Don't pretend it can.

When in doubt, test the part that is pure host logic and stub the guest edge.

18.2 The harness (`penguin.testing`)

A pyplugin can't just be imported and instantiated: its module and class body reference the live `plugins` manager (`@plugins.portalcall(...)`, `plugins.subscribe(...)`, sibling `plugins.<name>` calls) that is normally late-bound against a running portal. The harness stands up a **null backend** so the *real* loader and the *real* `Plugin` construction protocol run against doubles you control.

18.2.1 `load_pyplugin(...)` — construct a plugin, ready to drive

```
from penguin.testing import load_pyplugin

lp = load_pyplugin(
    "pyplugins/analysis/netbinds.py",
    outdir=tmp_path,                    # where the plugin writes its files
    args={"shutdown_on_www": False},    # the conf['plugins'][name] block
    doubles={"mem": my_mem_double},     # sibling plugins you want to control
)
lp.dispatch("igloo_ipv4_bind", cpu=None, port=port_be, is_steam=True)
```

(continues on next page)

(continued from previous page)

```
lp.finalize()                              # runs uninit() -> flushes final files
assert "httpd" in (tmp_path / "netbinds.csv").read_text()
```

- **doubles=** maps a sibling name (`mem`, `kffi`, `portal`, ...) to a test double. Any sibling you *don't* provide resolves to a `RecorderStub`: a recording no-op that's safe to call/decorate/index, so unrelated sibling calls don't blow up. Inspect what got called via `lp.calls`.
- **`lp.dispatch(event, ...)`** invokes every handler the plugin subscribed to that event (class-body `@plugins.subscribe` handlers are bound to the instance for you). `lp.subscriptions` / `lp.published` expose what the plugin wired up and emitted.
- **`lp.finalize()`** runs the plugin's `uninit()` — many plugins flush their final output file there.
- **`call_init=False`** stops construction after `__preinit__` (skips `__init__`). Use it when `__init__` does host-impossible I/O (e.g. `nvram2` shells out to `clang-20`): the class still imports so class-body decorators register, and your test sets the handful of attributes the handlers need.

18.2.2 `drive(gen, responses=..., collect=True)` — run a portal generator

Many handlers are generators that `yield` a `PortalCmd` (or `yield from` a sibling API) and resume on the reply. `drive` pumps such a generator to completion:

```
from penguin.testing import drive
import hyper.consts as consts

ret, yielded = drive(lp.plugin._get_or_create_proc_dir("a/b"),
                     responses=[10, 20], collect=True)
# responses are fed to successive yields (the "guest replies")
assert [c.op for c in yielded] == [           # assert the RIGHT commands issued
    consts.HYPER_OP.HYPER_OP_PROCFS_CREATE_OR_LOOKUP_DIR,
    consts.HYPER_OP.HYPER_OP_PROCFS_CREATE_OR_LOOKUP_DIR,
]
assert ret == 20                              # deepest id threaded back out
```

Compare `cmd.op` against the enum *member*, not a literal — see `real_isf=`.

18.2.3 `dispatch_syscall(...)` — drive a `@syscalls.syscall` hook

`@plugins.syscalls.syscall(...)` registrations are recorded; drive one (they're portal generators) with:

```
lp.dispatch_syscall("kill", regs, proto, sysno, sig, on_return=False,
                    responses=[...])
```

Sibling API calls the handler makes (`plugins.mem.read_bytes`, `plugins.osi.*`) are satisfied by *generator doubles* that `yield from ()` and `return` a canned value; `panda.ffi.cast` is modelled as identity.

18.2.4 `load_module(...)` — for logic that isn't a `Plugin`

Some testable logic lives in plain classes (e.g. the `hyperfile/models/*` mixins), not a `Plugin` subclass. `load_module(path, doubles=...)` imports the *module* under its real dotted name (so `from .base import ...` resolves) with the null manager bound as `plugins`, and returns `(module, manager)`. Instantiate the class and `drive()` its generator methods directly.

18.3 Plugins behind the FFI-enum boundary: `real_isf=`

A plugin that imports `hyper.portal` / `hyper.consts` / `apis.syscalls` transitively builds its enums (`HYPER_OP`, `value_filter_type`, …) from the driver's **ISF** (the DWARF-derived symbol format `apis.kffi` reads at runtime). To let the *genuine* `hyper.consts` build with **real** enum values, pass `real_isf=`:

```
def test_something(tmp_path, igloo_ko_isf):        # session fixture (see below)
    lp = load_pyplugin(DEVFS, outdir=tmp_path, real_isf=igloo_ko_isf)
    import hyper.consts as consts
    # cmd.op now carries the true op number from the driver's DWARF
    assert cmd.op == consts.HYPER_OP.HYPER_OP_DEVFS_CREATE_OR_LOOKUP_DIR
```

- The **`igloo_ko_isf` pytest fixture** (in `tests/unit/conftest.py`) resolves the ISF — `PENGUIN_TEST_IGLOO_KO_ISF` env var → local cache → **download `igloo_driver.tar.gz` for the `IGLOO_DRIVER_VERSION` pinned in the Dockerfile** (not `:latest`) and extract one arch → nix store — and `skip`s cleanly when offline with nothing cached. Take it as a test argument.
- **No checked-in enum fixture.** The ISF *is* the source of truth, pinned to the driver release, so the enums can't drift out of date. This also loads the whole driver type universe (structs, not just enums), so type-reading plugins work. Enums/most driver types are arch-invariant, so one arch (`armel`) suffices.
- If your plugin also calls `kffi.new(...)` / `kffi.callback(...)` (struct packing — a guest ABI concern), subclass `RealKffi` and stub just those to return a fixed blob, then pass it as `doubles={"kffi": MyKffi([isf])}`. The `real_isf=` value still makes `hyper.consts` real; your subclass wins for the packing methods. See `test_devfs.py` for the canonical shape.

The `[test]` extra in `src/pyproject.toml` provides `dwarffi` (matching the Dockerfile pin), so the ISF loads with the same reader the image uses.

18.4 Running the tests

Everything runs on a bare host with only the `[test]` extra — no image:

```
python3 -m venv .venv && . .venv/bin/activate
pip install -e './src[test]' -e './pengutils'      # pengutils is a sibling package
pytest tests/unit -q
# coverage over plugin logic:
pip install pytest-cov
pytest tests/unit --cov=pyplugins --cov-report=term-missing:skip-covered
```

Validate in a **fresh** venv with only the `[test]` extra (not your working venv), so a missing dependency can't hide behind something already installed.

18.5 Where to look for patterns

- `tests/unit/test_pyplugin_harness.py` — the reference proof (netbinds: events → `netbinds.csv`, dedup, lifecycle CSV on `finalize()`).
- `tests/unit/test_devfs.py` / `test_procfs.py` / `test_sysfs.py` / `test_sysctl.py` — the pseudofile registrars: `real_isf=` + a `RealKffi` subclass + asserting the right `PortalCmd` op.
- `tests/unit/test_read_models.py` (and `write`/`seek`/`ioctl`) — `load_module` driving the pseudofile model mixins.
- `tests/unit/test_real_consts.py` — the `real_isf=` mechanism itself.

- `tests/unit/PYPLUGIN_COVERAGE_PLAN.md` — the living status/tracker of what is covered and what is deliberately out of scope, and why.

CHAPTER

NINETEEN

UPROBES PLUGIN SYSTEM

The Uprobes plugin provides a flexible interface for monitoring, filtering, and intercepting user-space function calls in guest processes via the IGLOO hypervisor and portal interface. This enables dynamic tracing, security monitoring, and program analysis at the function level in user applications and libraries.

19.1 Overview

The uprobes system operates through a multi-layer architecture:

1. **Kernel Layer**: IGLOO kernel module registers uprobes at specified file offsets or symbols, with process and PID filtering.
2. **Hypervisor Layer**: Portal system manages communication between guest and host, relaying probe events.
3. **Plugin Layer**: Python plugins register uprobes and process events via callback functions.

19.2 Quick Start

```
from penguin import plugins, Plugin

class MyUprobeMonitor(Plugin):
    def __init__(self):
        # Register a uprobe on 'main' in /usr/bin/myapp for all processes named 'target'
        @plugins.uprobes.uprobe("/usr/bin/myapp", "main", process_filter="target")
        def on_main_entry(pt_regs):
            print(f"main() called! PC={pt_regs.pc:#x}")

        # Register a return probe on a library function
        @plugins.uprobes.uretprobe("/usr/lib/libfoo.so", "foo_func", process_filter=
↪"target")
        def on_foo_func_return(pt_regs):
            print(f"foo_func returned! Return value: {pt_regs.retval}")
```

19.3 Core Concepts

19.3.1 Uprobe Events

A uprobe event is generated when a monitored function is entered or returned from. Each event provides access to the CPU register state (`pt_regs`) at the probe point.

19.3.2 Probe Registration

Probes are registered using the `@uprobes.uprobe()` or `@uprobes.uretprobe()` decorators, supporting flexible filtering and symbol/offset selection.

```
@uprobes.uprobe(
    path="/usr/lib/libfoo.so",
    symbol="foo_func",
    process_filter="target_app",
    on_enter=True,
    on_return=False,
    pid_filter=1234
)
def my_handler(pt_regs):
    # Handler code here
    pass
```

19.4 API Reference

19.4.1 Decorator Parameters

`path` (str or None)

Path to the executable or library file. Supports wildcards (*). If `None`, will search all known libraries for the symbol.

`symbol` (str or int)

Function name (string) or file offset (integer) at which to place the probe.

`process_filter` (Optional[str])

Filter by process name (comm):

```
@uprobes.uprobe("/usr/bin/myapp", "main", process_filter="target")
def monitor_main(pt_regs):
    pass
```

`pid_filter` (Optional[int])

Filter by specific process ID:

```
@uprobes.uprobe("/usr/bin/myapp", "main", pid_filter=1234)
def monitor_pid(pt_regs):
    pass
```

`on_enter` / `on_return` (bool)

Control when the probe triggers:

```
# Entry only (default)
@uprobes.uprobe("/usr/bin/myapp", "main", on_enter=True)

# Return only
@uprobes.uretprobe("/usr/bin/myapp", "main", on_return=True)
```

(continues on next page)

(continued from previous page)

```
# Both entry and return
@uprobes.uprobe("/usr/bin/myapp", "main", on_enter=True, on_return=True)
```

fail_register_ok (bool)

If True, do not raise an error if the symbol is not found.

19.4.2 Handler Function Signature

```
def handler(pt_regs):
    """
    pt_regs: Register state at probe point (see below)
    """
```

19.5 pt_regs Object

The `pt_regs` parameter provides access to the CPU register state at the probe point:

```
def my_handler(pt_regs):
    print(f"PC: {pt_regs.pc:#x}")
    print(f"SP: {pt_regs.sp:#x}")
    print(f"Return value: {pt_regs.retval}")  # On return probes
    # Access general-purpose registers as needed
```

19.6 Practical Examples

19.6.1 Monitor Function Entry

```
@uprobes.uprobe("/usr/bin/myapp", "main")
def on_main_entry(pt_regs):
    print(f"main() called at PC={pt_regs.pc:#x}")
```

19.6.2 Monitor Function Return

```
@uprobes.uretprobe("/usr/lib/libfoo.so", "foo_func")
def on_foo_func_return(pt_regs):
    print(f"foo_func returned, retval={pt_regs.retval}")
```

19.6.3 Wildcard Library Matching

```
@uprobes.uprobe("libssl*", "SSL_read")
def monitor_ssl_read(pt_regs):
    print("SSL_read called!")
```

19.6.4 Register on All Libraries Containing a Symbol

```
@uprobes.uprobe(None, "malloc")
def monitor_malloc(pt_regs):
    print("malloc called in some library!")
```

19.6.5 Offset-based Probes

```
@uprobes.uprobe("/usr/bin/myapp", 0x1234)
def on_offset(pt_regs):
    print("Hit offset 0x1234 in myapp")
```

19.6.6 Process and PID Filtering

```
@uprobes.uprobe("/usr/bin/myapp", "main", process_filter="target", pid_filter=4321)
def filtered_handler(pt_regs):
    print("main() called by target process with PID 4321")
```

19.7 Performance Considerations

- **Specific probes**: Register only the functions you need to minimize overhead.
- **Process/PID filtering**: Reduces the number of events delivered to the host.
- **Wildcard/pathless probes**: May register probes in many libraries; use sparingly.

19.8 Error Handling

- Always check for symbol resolution errors; use `fail_register_ok=True` to ignore missing symbols.
- Handle missing or invalid register state gracefully in handlers.

19.9 Hook Management

19.9.1 Dynamic Control

Enable/disable probes at runtime:

```
# Unregister a probe by ID
yield from uprobes.unregister(probe_id)
```

19.9.2 Probe Information

Access probe configuration:

```
probe_id = plugins.uprobes._func_to_probe_id[my_handler]
info = plugins.uprobes.probe_info[probe_id]
print(f"Probe at {info['path']}:{info['offset']}")
```

19.10 Integration with Other Plugins

19.10.1 With Portal Plugin

Uprobes are managed via the portal system and can be combined with other portal-based plugins for richer context.

19.10.2 With OSI Plugin

Combine with OS introspection for process metadata:

```
def handler(pt_regs):
    proc = yield from plugins.OSI.get_proc()
    print(f"Probe hit in process {proc.name} (PID {proc.pid})")
```

19.11 Testing

The system supports automated tests for probe registration, event delivery, and filtering.

```
@uprobes.uprobe("/usr/bin/testprog", "main", process_filter="testproc")
def test_probe(pt_regs):
    assert pt_regs.pc != 0
```

19.12 Troubleshooting

19.12.1 Common Issues

1. **Probe not triggering**: Check path and symbol normalization, process filters, and PID filters.
2. **Symbol not found**: Ensure the symbols database is present and up to date.
3. **Multiple matches**: Wildcard or pathless probes may register in multiple libraries.

19.12.2 Debugging

Enable debug logging:

```
import logging
logging.getLogger("uprobes").setLevel(logging.DEBUG)
```

Print probe event details:

```
def debug_handler(pt_regs):
    print(f"PC: {pt_regs.pc:#x}, Registers: {pt_regs.regs}")
```

19.13 Best Practices

1. **Be specific**: Register probes only where needed.
2. **Filter early**: Use process and PID filters to reduce event volume.
3. **Handle errors**: Always expect symbol resolution or memory access to fail.
4. **Cache expensive lookups**: If reading memory or symbols, cache results.
5. **Test thoroughly**: Use the test framework to verify probe behavior.

6. **Monitor performance**: Excessive probes can impact guest performance.

The uprobes system provides a powerful foundation for dynamic user-space analysis, security monitoring, and program understanding in virtualized environments.

CHAPTER

TWENTY

EXAMPLE REHOSTING WORKFLOW

This guide walks through the process of rehosting the StrideLinx router.

After installing `penguin` and `fw2tar` as described in the README, download a copy of the target firmware from https://ftp.automationdirect.com/pub/stride_firmware_5.3.174.zip.

Rename `stride_firmware_5.3.174.zip` to `stride.bin` for convenience.

20.1 Convert firmware blob to root filesystem archive

To begin, we'll convert the firmware blob into a root filesystem archive with `fw2tar`. This utility handles unpacking files from a wide variety of storage formats while preserving the extracted file's permissions and symlinks. To learn more about the utility or try non-standard options, you can run `fw2tar --help`. To enable all extractors you can run `fw2tar --extractors binwalk,unblob yourfw.bin`. For our use case, we'll just run the utility on the firmware we downloaded with no special options:

```
fw2tar stride.bin
```

After a few seconds, you should see a message saying the best filesystem was created as `stride.rootfs.tar.gz`.

You can use the command `tar tvf stride.rootfs.tar.gz` to view the contents of that archive:

```
drwxr-xr-x root/root         0 2018-12-31 19:00 ./
drwxrwxrwx root/root         0 2018-12-31 19:00 ./alternate/
drwxrwxrwx root/root         0 2018-12-31 19:00 ./bin/
lrwxrwxrwx root/root         0 2018-12-31 19:00 ./bin/ash -> busybox
-rwxr-xr-x root/root    504332 2018-12-31 19:00 ./bin/busybox
```

If `fw2tar` had failed to extract the filesystem, you could have tried creating or finding your own extraction utilities and then generated a rootfs archive of that extraction (with correct permissions and a root of `./`). But it works most of the time, so that's not likely something you'll ever need to do.

20.2 Initialize penguin project from root filesystem archive

Now that we have our filesystem, we can begin the rehosting process by initializing a new penguin project for it.

```
penguin init stride.rootfs.tar.gz
```

This command will create a new project in `./projects/stride` which will contain a `config.yaml` which precisely specifies how this firmware is to be rehosted. Note that the paths logged by Penguin may include `/host_...` these paths are mapped between the container and your host machine but are based on defaults or the paths you've specified on the command line. For example, the command above will report the project is created at `/host_projects/stride` which corresponds to `./projects/stride`.

When a project is initialized, Penguin uses a **static analysis** of the filesystem to generate an initial **configuration** specifying the rehosting process for this firmware.The static analysis results are stored in `./projects/stride/static/` and the configuration is stored at `./projects/stride/config.yaml`.

20.2.1 Static analysis outputs

`./projects/stride/base/` holds `fs.tar.gz`, a copy of the input filesystem archive.

Each static analysis writes its result to `./projects/stride/static/` under the name of the analysis that produced it:

- `InitFinder.yaml`: Statically-identified *init* programs, shortest path first.
- `EnvFinder.yaml`: Environment variables you may later need to set, including the same init candidates under `igloo_init`.
- `PseudofileFinder.yaml`: Statically-identified `/dev` and `/proc` files that you may later need to model in the rehosting process.
- `InterfaceFinder.yaml`: Statically-identified network interface names.
- `ArchId.yaml` / `KernelVersionFinder.yaml`: The detected architecture, and the kernel version chosen for it.
- `LibrarySymbols.json.xz`: Every exported library function with its name and offset.
- `manifest.yaml`: What each analysis produced, including any that failed.

The values those analyses feed into the configuration are applied as generated patches under `./projects/stride/static_patches/` — including the NVRAM keys recovered for the system (`nvram.*.yaml`, where `nvram.04_defaults` is generic and the others are specific to the firmware under analysis).

20.2.2 Configuration

The format of autogenerated configuration `./projects/stride/config.yaml` is documented in *docs/schema_doc.md*, but the following sections and fields are worth highlighting:

`core`: This section indicates project-wide settings such as the architecture. Within this section there are 3 notable options:

- `force_www`: if enabled, the rehosting will agressively attempt to start standard webservers
- `strace`: if enabled, every process in the system will have its system calls traced and logged to the output.
- `show_output`: if this is set to true, console output will be shown on standard out of penguin. Otherwise console output will be logged into the results directory at `console.log`

`static_files`: This section specifies static modifications to make to the root filesystem before boot. Of particular note is the `/igloo/init` script that you can edit to control what happens in the system before the specified *init* program runs (i.e., that you set in `env.igloo_init`).

`env`: This section stores the kernel boot arguments passed to the system. Of particular note is the `igloo_init` field which is used by penguin to select the *init* program to run during boot. A default is chosen for you and recorded in `static_patches/base.yaml`; to change it, set `env.igloo_init` in a patch (see *the playbook* — editing `config.yaml` directly has no effect, because patches are applied over it). The candidates are listed in `static/InitFinder.yaml`.

`pseudofiles`: This section specifies pseudofiles in `/dev`, `/sys`, and `/proc` to model in the rehosting. Note that you may see yaml variables used where a dictionary is first defined as `&id001` and then later referenced as `*id001`.

`nvram`: This is a set of keys and corresponding values to initialize the system's non-volatile RAM (NVRAM) with. These are contributed by generated patches under `static_patches/` — `nvram.04_defaults.yaml` carries generic values, and the other `nvram.*.yaml` patches carry values recovered from this firmware. (`nvram.csv` in a run's results directory is a different thing: the NVRAM accesses observed at runtime.)

`netdevs`: This is a list of network device names to configure within the guest

20.3 Run your configuration and collect output:

Run the auto-generated configuration:

```
penguin run projects/stride/config.yaml
```

This rehosting will run until you terminate the command or the system shuts down. It will not log the firmware's console output by default, but it will inform you whenever new network services start in the guest and how to reach them. You should expect to see output like:

```
20:25:34 penguin INFO Running config /host_projects/stride/config.yaml
20:25:34 penguin INFO Saving results to /host_projects/stride/results/0
20:25:34 penguin INFO Note messages referencing /host paths reflect automatically-mapped␣
↪shared directories based on your command line arguments
20:25:37 penguin.gen_image INFO Generating new image from config...
20:25:38 penguin.gen_image WARNING Deleting existing file /etc/hosts to replace it
copying from tar archive /host_projects/stride/qcows/fs_out.tar
20:25:42 penguin.runner INFO Logging console output to /host_projects/stride/results/0/
↪console.log
PANDA[core]:os_familyno=2 bits=32 os_details=generic
PANDA[syscalls2]:using profile for linux arm
PANDA[osi_linux]:W> failed to read task.switch_task_hook_addr
PANDA[osi_linux]:W> kernelinfo bytes [20-23] not read
20:25:42 penguin.runner INFO Loading plugins
20:25:42 plugins.core INFO Root shell will be available at: 192.168.0.2:4321
20:25:42 plugins.core INFO Connect with: telnet 192.168.0.2 4321
20:25:50 penguin.runner INFO Launching rehosting
20:27:49 plugins.VPN INFO            inetd binds tcp 0.0.0.0:7        reach it at 192.
↪168.0.2:7
20:27:49 plugins.VPN INFO            inetd binds udp 0.0.0.0:7        reach it at 192.
↪168.0.2:54933
20:27:49 plugins.VPN INFO     7 is already in use
20:27:49 plugins.VPN INFO            inetd binds tcp 0.0.0.0:9        reach it at 192.
↪168.0.2:9
20:27:49 plugins.VPN INFO            inetd binds udp 0.0.0.0:9        reach it at 192.
↪168.0.2:38301
20:27:49 plugins.VPN INFO     9 is already in use
20:27:49 plugins.VPN INFO            inetd binds tcp 0.0.0.0:23       reach it at 192.
↪168.0.2:23
20:28:18 plugins.VPN INFO            snmpd binds udp 0.0.0.0:161      reach it at 192.
↪168.0.2:161
```

20.3.1 Dynamic analysis results

By default the `penguin run` command will store dynamic analysis results in `<project directory>/results/<auto-incrementing number>` so the first run will generate output in `./projects/stride/results/0`. Launch a second terminal to examine results while your rehosting continues to running.

- Examine the console output by looking at `./projects/stride/results/0/console.log`. To view the output as it is updated you can run `tail -f ./projects/stride/results/0/console.log`.

- Examine dynamically-traced shell script execution at `./projects/stride/results/0/shell_cov_trace.csv` and note that concrete values for each variable are included in the trace.
- After the run completes, `./projects/stride/results/0/summary.json` holds a single machine-readable digest of the run (total and per-metric scores, network binds, unmodeled pseudofile count, panic flag, wall-clock) aggregated from the other output files — handy for scripting and CI, e.g. `jq .score results/latest/summary.json`.

Note that some files will not be created or populated until the emulation terminates. To learn more about the outputs created in this directory, check out *docs/plugins.md*.

20.3.2 Root shell

After examining these files, use your second terminal to connect to the root shell with `telnet` at the IP address and port that was specified when you launched the `penguin run` command. After launching `telnet` you may need to push enter to see a prompt

```
$ telnet 192.168.1.2 4321
Trying 192.168.1.2...
Connected to 192.168.1.2.
Escape character is '^]'.

~ #
```

This is a penguin-provided root shell into the running firmware. From here you can run commands inside the guest. Note that the root shell will prioritize running commands from a penguin-provided busybox binary over guest binaries. This ensures that standard commands are available regardless of what the firmware provides. But if you'd like to run the guest's version of a specific command such as `httpd` you should either run it from the absolute path in the guest filesystem or run `unalias httpd` first.

While you're connected to the guest shell, try running commands like `ps` and `whoami` to take a look around the system you're connected to. To disconnect you should press `ctrl + ]` and then `ctrl + d`.

20.3.3 Network connectivity with VPN

After allowing your rehosting to run for a few minutes, you should see output about various processes binding to network IPs and ports. For each bind, another IP and port will be logged indicating how you can connect to that service from your host machine:

```
20:27:49 plugins.VPN INFO           inetd binds tcp 0.0.0.0:23       reach it at 192.
↪168.0.2:23
20:28:18 plugins.VPN INFO           snmpd binds udp 0.0.0.0:161      reach it at 192.
↪168.0.2:161
```

When you send traffic to the specified IP address (i.e., 192.168.0.2), it will be bridged into the running firmware as if you connected to the bound IP listed.

20.3.4 Shut down rehosting

In the terminal where you ran `penguin run` press `ctrl + c` to gracefully shut down the rehosting. The shutdown may take up to 30s as dynamic analysis data must be written to disk at this time.

20.4 Update configuration

At this point you have run an auto-generated rehosting for the stride firmware. But you may have noticed that the VPN logs didn't indicate a webserver was reachable on port 80. Let's go through the *iterative rehosting process* to try finding and fixing errors related to this.

In our initial results directory we can examine `env_missing.yaml` and see that two unset environment variables were referenced: `sxid` and `sxserno`. If we examine `console.log` we see a number of errors about `No configuration for 'SXID'` so we'll start by trying to fix this.

20.4.1 Finding SXID

We could now manually analyze binaries and scripts in the filesystem to understand what the `sxid` variable is used for and what it controls. But we'll instead use a dynamic analysis.

Edit `projects/stride/config.yaml` and within the `env` section add the `sxid` keys as shown:

```
env:
  igloo_init: /sbin/init
  sxid: DYNVALDYNVALDYNVAL
```

Here we're setting `sxid` to a magic value that we'll dynamically detect comparisons to. We can re-run our rehosting for a few minutes. Note that this time it will log to `projects/stride/results/1`.

```
penguin run projects/stride/config.yaml
```

While it runs examine the console.log file and wait until you see message saying "sxid not found." At that point (or after a few minutes) press `ctrl + c` to stop the run.

Examine the result file `env_cmp.txt` to view strings that were compared against the magic value:

```
$ cat projects/stride/results/1/env_cmp.txt
0150_5MS-MDM-1
0150S_5MS-5SC
H
```

Either of the first two values will be valid, so update your configuration's env section to be:

```
env:
  igloo_init: /sbin/init
  sxid: 0150_5MS-MDM-1
```

20.4.2 New errors with SXID

Now run your updated configuration, this time it will log to `projects/stride/results/2`

```
penguin run projects/stride/config.yaml
```

After a few minutes (or when you see an error about "unable to open DSA"), press `ctrl + c` to terminate the run.

Compare the output from your inintial run (0) versus the that run (2) using a diffing tool:

```
git diff --no-index --word-diff projects/stride/results/0/console.log projects/stride/
↪results/2/console.log
```

Scroll past the lines that only have different timestamps and you should see that other messages are changing. For example "SkippingNo configuration for SXID '0190_???'" is gone and new errors like "Dsa: Failed to open /dev/dsa: No such file or directory" are present.

To learn more about what's happening with that device, we can examine `projects/stride/results/2/pseudofiles_failures.yaml` which shows that the system tried to open that device many times while our rehosting was running.

20.4.3 Adding `/dev/dsa`

We can add the device `/dev/dsa` into our rehosting by updating the `pseudofiles` section of our configuration. At this point we don't know what the system was trying to do with the device, so we'll just add it and not yet configure antyhing else

Update your configuration's pseudofiles section to look like the following. You can delete the default pseudofile models that were already present or leave them alone.

```
pseudofiles:
  /dev/dsa: {}
```

Now run your rehosting again, this time it will log to `projects/stride/results/3`:

```
penguin run projects/stride/config.yaml
```

After a few minures (or when you see an error about "ioctl()-error -1"), press `ctrl + c` to terminate the run and examine `projects/stride/results/3/pseudofiles_failures.yaml`. Note that you can even examine the pseudofile_failures file while the system is running.

20.4.4 IOCTL modeling

Now that we've added the `/dev/dsa` device, we don't see any `open` failures and we should see that some `ioctl` errors are reported instead for ioctl number 1074029569 (0x40046401).

```
/dev/dsa:
  impact: {hits: 40, distinct_callers: 1, crashing_callers: 0, crashing_callers_by_name:␣
↪0}
  callers: [dsad:412]
  events:
    ioctl:
      1074029569:
        count: 40
  suggest:
    ioctl:
      '*': {model: return_const, val: 0}
```

Entries are ranked by impact (crashing callers, then distinct callers, then raw hits), and each comes with a `suggest` block: a heuristic starting model you can paste directly under the path in your config's `pseudofiles:` section.

We can update our `/dev/dsa` pseudofile to simply return 0 or "success" on this ioctl:

```
pseudofiles:
  /dev/dsa:
    ioctl:
      1074029569:
        model: return_const
        val: 0
```

and run again. If we repeat this process (results directory 4), we'll see other IOCTLs are issued on the device such as 3221513218 (0xc0046402). We could keep adding models for each observed ioctl, or we can take a heavy-handed approach and say any ioctl issued on `/dev/dsa` should return 0.

```
pseudofiles:
  /dev/dsa:
    ioctl:
      "*":
        model: return_const
        val: 0
```

20.4.5 Webserver starts

With the config changes described above, a `lighttpd` webserver should start after a minute.

In total the changes we made to the auto-generated configuration are adding `sxid` into the `env` section and `/dev/dsa` into `pseudofiles:

```
env:
  igloo_init: /sbin/init
  sxid: 0150_5MS-MDM-1
pseudofiles:
  /dev/dsa:
    ioctl:
      "*":
        model: return_const
        val: 0
```

With these changes the VPN should log that a lighttpd server is listening:

```
21:14:41 plugins.VPN INFO         lighttpd binds tcp [::]:80          reach it at 192.
↪168.0.2:80
21:14:41 plugins.VPN INFO         lighttpd binds tcp [::]:443         reach it at 192.
↪168.0.2:443
```

If you are running penguin on your local machine, you can try using a web browser to reach http://192.168.0.2 and log in with `admin` and `admin`. If penguin is running on a remote machine you can run the following command to get a long response:

```
curl -u admin:admin 192.168.0.2/cgi-bin/menubar.cgi
```

If you'd like to connect to the rehosting from a remote machine you can use ssh port forwarding. For example, to make localhost:9000 on your desktop connect to `192.168.0.2` port 80 on the machine where you're running penguin, you could run:

```
user@desktop$ ssh -L 9000:192.168.0.2:80 user@your_penguin_dev_machine -N
```

CHAPTER

TWENTYONE

API REFERENCE

21.1 penguin package

21.1.1 Subpackages

penguin.penguin_config package

penguin.penguin_config.**dump_config**(*config*, *path*)

Write penguin config to path TODO: If we have a config that includes patches we should validate *after* patches. For now we allow empty arch and kernel with patches filling them in later, but validation doesn't check this

penguin.penguin_config.**hash_yaml_config**(*config*)

Given a config dict, generate a hash

Parameters
config (*dict*)

penguin.penguin_config.**load_config**(*proj_dir*, *path*, *validate=True*, *resolved_kernel=None*, *verbose=False*)

Load penguin config from path

penguin.penguin_config.**load_unpatched_config**(*path*)

Load a configuration without applying any patches. No validation.

Subpackages

penguin.penguin_config.versions package

Submodules

penguin.penguin_config.versions.v2 module

class penguin.penguin_config.versions.v2.**V2**

Bases: object

auto_fix()

change_description = '\n All libnvram functions now have a `libinject_` prefix.\n This prevents overriding more library functions than intended.\n '

example_old_config = {'lib_inject': {'aliases': {'nvram_get_state': 'nvram_get_int', 'nvram_load': 'nvram_init', 'nvram_loaddefault': 'true', 'nvram_restore_default': 'nvram_reset', 'nvram_set_state': 'nvram_set_int'}}}

```
fix_guide = "\n In `lib_inject.aliases`,\n\n 1. Change `'true'` to
`libinject_ret_1`,\n `'false'` to `libinject_ret_0`,\n `'true1'` to
`libinject_ret_1_arg`,\n and `'false1'` to `libinject_ret_0_arg`.\n\n 2. Replace
`nvram_loaddefault` in the right side with `libinject_ret_1`.\n\n 3. Add the
`libinject_` prefix to names of libnvram functions.\n\n 4. Add aliases of the form
`X: libinject_X` for all the prefixed functions.\n https://github.com/rehosting/
libnvram/commit/5cdd5156c777f497361d86c1c7f166ed9005a6f3\n "

new_aliases = {'WAN_ith_CONFIG_GET': 'libinject_WAN_ith_CONFIG_GET', 'apmib_get':
'libinject_apmib_get', 'apmib_set': 'libinject_apmib_set', 'envram_commit':
'libinject_envram_commit', 'envram_default': 'libinject_envram_default',
'envram_get': 'libinject_envram_get', 'envram_getf': 'libinject_envram_getf',
'envram_load': 'libinject_envram_load', 'envram_match': 'libinject_envram_match',
'envram_safe_load': 'libinject_envram_safe_load', 'envram_set':
'libinject_envram_set', 'envram_setf': 'libinject_envram_setf', 'envram_unset':
'libinject_envram_unset', 'getenv': 'libinject_getenv', 'nvram_bufget':
'libinject_nvram_bufget', 'nvram_bufset': 'libinject_nvram_bufset', 'nvram_clear':
'libinject_nvram_clear', 'nvram_close': 'libinject_nvram_close', 'nvram_commit':
'libinject_nvram_commit', 'nvram_default_get': 'libinject_nvram_default_get',
'nvram_get': 'libinject_nvram_get', 'nvram_get_adv': 'libinject_nvram_get_adv',
'nvram_get_buf': 'libinject_nvram_get_buf', 'nvram_get_int':
'libinject_nvram_get_int', 'nvram_get_nvramspace':
'libinject_nvram_get_nvramspace', 'nvram_getall': 'libinject_nvram_getall',
'nvram_getall_adv': 'libinject_nvram_getall_adv', 'nvram_init':
'libinject_nvram_init', 'nvram_invmatch': 'libinject_nvram_invmatch',
'nvram_list_add': 'libinject_nvram_list_add', 'nvram_list_del':
'libinject_nvram_list_del', 'nvram_list_exist': 'libinject_nvram_list_exist',
'nvram_match': 'libinject_nvram_match', 'nvram_nget': 'libinject_nvram_nget',
'nvram_nmatch': 'libinject_nvram_nmatch', 'nvram_nset': 'libinject_nvram_nset',
'nvram_nset_int': 'libinject_nvram_nset_int', 'nvram_parse_nvram_from_file':
'libinject_parse_nvram_from_file', 'nvram_reset': 'libinject_nvram_reset',
'nvram_safe_get': 'libinject_nvram_safe_get', 'nvram_safe_unset':
'libinject_nvram_safe_unset', 'nvram_set': 'libinject_nvram_set', 'nvram_set_adv':
'libinject_nvram_set_adv', 'nvram_set_int': 'libinject_nvram_set_int',
'nvram_state': 'libinject_nvram_state', 'nvram_unset': 'libinject_nvram_unset',
'strcmp': 'libinject_strcmp', 'strncmp': 'libinject_strncmp', 'strstr':
'libinject_strstr'}

num = 2
```

Submodules

penguin.penguin_config.errors module

Human-friendly rendering of Pydantic ValidationErrors for Penguin configs.

*structure.Main(**config)* raises a *pydantic.ValidationError* whose default string form is a wall of text with URLs and no indication of which config/patch file is at fault. *format_validation_error* turns it into a short, located, optionally-colorized report that names the offending option, the file that set it (via the patch *origin_map*), what was wrong, and the allowed values.

penguin.penguin_config.errors.**format_validation_error**(*exc*, ** (Keyword-only parameters separator (PEP 3102))*, *config_path=None*, *origin_map=None*, *root_model=None*, *header='Config validation failed:'*)

Render a pydantic `ValidationError` into a friendly multi-line report.

`config_path` is the main config file (used as a fallback source). `origin_map` is the {dotted.path: source_file} map produced by `patch_config` during load, used to name which file set each option. `root_model` is the model the error paths are relative to (defaults to `structure.Main`); pass a plugin's `Args` model for plugin-arg errors.

penguin.penguin_config.gen_docs module

class penguin.penguin_config.gen_docs.**DocsField**(*type_*, *merge_behavior*, *title*, *description*, *default*, *examples*)

Bases: `object`

Information about a field of the config, for generating docs

Parameters

- **type_** (type)
- **merge_behavior** (*str | None*)
- **title** (*str | None*)
- **description** (*str | None*)
- **default** (*PydanticUndefinedType | Any*)
- **examples** (*tuple[Any]*)

default: PydanticUndefinedType | Any

description: str | None

examples: tuple[Any]

from_field()

Create a *DocsField* from a Pydantic *Field*

Return type
DocsField

from_type()

Create a *DocsField* from a Python type, which should probably inherit *BaseModel* or *RootModel*

Parameters
type_ (type)

Return type
DocsField

merge(*other*)

Create a DocsField by combining two DocsFields, using the second to fill in gaps in the first.

Parameters
other (DocsField)

Return type
DocsField

merge_behavior: str | None

title: *str | None*

type_: *type*

penguin.penguin_config.gen_docs.**gen_all_plugin_args_docs**(*plugin_path=None*, *manager=None*, *panda=None*, *level=1*, *show_skipped=False*, *static=False*)

Render a single markdown reference for every plugin that declares an `Args` schema under `plugin_path` (default: the schema's `core.plugin_path`).

With `static=True` the declared `Args` are read via `ast` without importing the plugins, so coverage is complete even outside a live emulator — this is what the CLI uses. Otherwise plugins are imported: pass `manager`/`panda` (the live singletons, as the docgen plugin does) for full, high-fidelity coverage; without them import-based discovery is best-effort and runtime-dependent plugins are skipped.

`level` sets the heading depth of the page title (and shifts each plugin's sub-heading to `level + 1`) so this can nest under another section. When `show_skipped` is true, the count of plugins that could not be introspected is rendered as a visible note (rather than an HTML comment).

penguin.penguin_config.gen_docs.**gen_docs**(*path=[]*, *docs_field=DocsField(type_=<class 'penguin.penguin_config.structure.Main'>, merge_behavior=None, title='Penguin Configuration', description='Configuration file for config-file-based rehosting with IGLOO', default=PydanticUndefined, examples=[])*)

Generate docs for config format starting from the field at the given path

penguin.penguin_config.gen_docs.**gen_docs_compact_field_table**(*fields*)

For fields that do not have any nested structure and only simple types, generate a compact table to make the docs easier to read.

penguin.penguin_config.gen_docs.**gen_docs_field**(*path*, *docs_field*, *include_type=True*)

Generate docs for a single field of the config

penguin.penguin_config.gen_docs.**gen_docs_literal_arg**(*a*)

penguin.penguin_config.gen_docs.**gen_docs_type_name**(*t*)

Convert the Python type *t* to a string for use in generated docs.

penguin.penguin_config.gen_docs.**gen_docs_yaml_dump**(*x*)

Convert x to YAML for use in generated docs.

We can't use yaml.dump(x) alone for this, because it appends "

... ".

penguin.penguin_config.gen_docs.**gen_plugin_args_docs**(*name*, *args_model*, *deprecation_note=True*, *level=1*)

Render a plugin's declared `Args` model as a markdown section.

`args_model` is a `PluginArgs` subclass; we render one row per field with its type, default, required-ness, and description. Set `deprecation_note` to False to omit the per-plugin first-class-syntax note (the aggregate reference states it once instead). `level` sets the markdown heading depth of the plugin's section header (1 → # for a standalone page, 2 → ## when nested under an aggregate page's single # title).

penguin.penguin_config.gen_docs.**gen_plugin_args_docs_from_specs**(*name*, *arg_specs*, *deprecation_note=True*, *level=1*)

Render a plugin's arguments from statically-extracted `ArgSpec``s (see ``plugin_manager.discover_declaring_plugins_static`) — the import-free path, so plugins that can't be imported outside a live emulator are still covered.

Types and defaults are rendered from source (e.g. `List[str]`, `false`) rather than via pydantic, so output is slightly lower-fidelity than the model-based *gen_plugin_args_docs()* but available without a runtime.

penguin.penguin_config.gen_docs.**list_sections**()

Return [(name, title)] for the top-level config sections.

penguin.penguin_config.gen_docs.**main**()

penguin.penguin_config.gen_docs.**resolve_section**(*dotted*)

Resolve a dotted config-section path (e.g. "core", "pseudofiles.read", "pseudofiles.read.const_buf") to the type at that location, or None.

penguin.penguin_config.gen_docs.**resolve_section_docs_field**(*dotted*)

Resolve a dotted config-section path to a *DocsField*, carrying the field-level metadata (title, description, examples) of the section.

resolve_section returns only the bare type, which loses metadata that lives on the *Field* rather than the model — e.g. the *plugins* section is a plain *dict[str, Plugin]* whose title only exists on its *Field*. Using *DocsField.from_type* on that bare dict yields a title-less field and makes *gen_docs* raise. Resolving through the owning field keeps the metadata so rendering matches the full-tree docs. Returns None if the path doesn't resolve.

penguin.penguin_config.gen_docs.**type_has_simple_name**(*ty*)

Determine whether a type is a regular Python type and not a Pydantic model class.

penguin.penguin_config.structure module

class penguin.penguin_config.structure.**BinaryPatchEntry**(*, *file_offset*, *hex_bytes=None*, *asm=None*, *mode=None*, *expect=None*, *on_mismatch='fail'*, *why=None*, *tag=None*)

Bases: `PartialModelMixin`, `BaseModel`

A single edit within a `binary_patch` action: bytes to write at one file offset, optionally guarded by an `expect` check. Multiple entries can target one file via the action's `patches` list; they are applied host-side to one buffer in a single pass, and overlapping write ranges are rejected.

Parameters

- **file_offset** (*Annotated[int, Ge(ge=0)]*)
- **hex_bytes** (*Annotated[str, AfterValidator(func=~penguin.penguin_config.structure._validate_hex_string)] | None*)
- **asm** (*str | None*)
- **mode** (*str | None*)
- **expect** (*Annotated[str, AfterValidator(func=~penguin.penguin_config.structure._validate_hex_string)] | None*)
- **on_mismatch** (*Literal['fail', 'skip', 'warn']*)
- **why** (*str | None*)
- **tag** (*str | None*)

asm: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Assembly code to write at offset (runs through keystone)', examples=['nop', 'mov r0, #0xdeadbeef'])]

expect: Annotated[Annotated[str, AfterValidator(func=_validate_hex_string)] | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Expected bytes at offset before patching (hex string)', description='If set, the current bytes at file_offset are compared against this hex string (over its own length, which may differ from the patch length) before the patch is written. If the bytes at the offset already equal the patch bytes, the patch is skipped (idempotent re-run); otherwise the on_mismatch policy applies.', examples=['0102 0304', 'DEADBEEF'])]

file_offset: Annotated[int, FieldInfo(annotation=NoneType, required=True, title='File offset (integer)', metadata=[Ge(ge=0)])]

hex_bytes: Annotated[Annotated[str, AfterValidator(func=_validate_hex_string)] | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Bytes to write at offset (hex string)', examples=['DEADBEEF', '90 90'])]

mode: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Assembly mode', examples=['arm', 'thumb'])]

model_config = {'extra': 'forbid', 'title': 'Binary patch entry'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

on_mismatch: Annotated[Literal['fail', 'skip', 'warn'], FieldInfo(annotation=NoneType, required=False, default='fail', title="Policy when 'expect' does not match", description="fail: abort the run (default, safest). skip: leave the file unpatched and continue. warn: log a warning and write the patch anyway. Only meaningful when 'expect' is set.")]

tag: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title="Label grouping related patches, recorded in the run's binary_patches.yaml", examples=['secureboot'])]

why: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title="Rationale for this patch, recorded in the run's binary_patches.yaml", examples=['NOP out the secure-boot check'])]

class penguin.penguin_config.structure.**Core**(*, *arch=None*, *kernel=None*, *fs='./base/fs.tar.gz'*, *plugin_path='/pyplugins'*, *root_shell=False*, *root_shell_backend='vsock'*, *analysis_scope='firmware'*, *strace=False*, *ltrace=False*, *gdbserver=None*, *snapshot=None*, *force_www=False*, *cpu=None*, *show_output=False*, *log_file=None*, *immutable=True*, *network=False*, *shared_dir=None*, *core_dumps=None*, *version*, *auto_patching=True*, *guest_cmd=False*, *execution_mode='qemu'*, *extra_qemu_args=None*, *kernel_cmdline_append=None*, *mem='2G'*, *kernel_quiet=True*, *smp=1*, *timeout=None*, *graphics=False*, *allow_reboot=False*, *startup_script=None*)

Bases: `PartialModelMixin`, `BaseModel`

Core configuration options for this rehosting

Parameters

- **arch** (*Literal['armel', 'arm', 'armle', 'aarch64', 'arm64', 'mipsel', 'mipseb', 'mipsbe', 'mips64el', 'mips64eb', 'mips64be', 'powerpc', 'ppc', 'powerpc64', 'ppc64', 'powerpc64le', 'ppc64le', 'powerpc64el', 'ppc64el', 'riscv64', 'riscv', 'rv64', 'loongarch64', 'loongarch', 'la64', 'x86_64', 'intel64', 'amd64', 'x86-64', 'x64'] | None*)
- **kernel** (*str | None*)
- **fs** (*str | None*)
- **plugin_path** (*str*)
- **root_shell** (*bool*)
- **root_shell_backend** (*Literal['telnet', 'vsock']*)
- **analysis_scope** (*bool | str*)
- **strace** (*bool | list[str]*)
- **ltrace** (*bool | list[str]*)
- **gdbserver** (*GDBServerPrograms | None*)
- **snapshot** (Snapshot | *None*)
- **force_www** (*bool*)
- **cpu** (*str | None*)
- **show_output** (*bool*)
- **log_file** (*str | None*)
- **immutable** (*bool*)
- **network** (*bool*)
- **shared_dir** (*bool | str |* SharedDir | *None*)
- **core_dumps** (*bool | str |* CoreDumps | *None*)
- **version** (*Literal['1.0.0', 2]*)
- **auto_patching** (*bool*)
- **guest_cmd** (*bool*)
- **execution_mode** (*Literal['qemu', 'kvm']*)
- **extra_qemu_args** (StrSepSpace | *None*)
- **kernel_cmdline_append** (StrSepSpace | *None*)
- **mem** (*str | None*)
- **kernel_quiet** (*bool*)
- **smp** (*int | None*)
- **timeout** (*int | None*)
- **graphics** (*bool*)
- **allow_reboot** (*bool*)
- **startup_script** (*str | None*)

allow_reboot: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Allow the guest to reboot', description="If False (default), QEMU is launched with '-no-reboot' so a guest-initiated reboot terminates the emulation (one boot per run). If True, '-no-reboot' is omitted so QEMU resets the machine in place and the guest reboots within the same run. Persistent drives and host-file-backed MTD devices survive the reset; stateful plugins must tolerate a second guest init.", examples=[False, True])]

analysis_scope: Annotated[bool | str, FieldInfo(annotation=NoneType, required=False, default='firmware', title='Scope of per-process analysis', description="Which processes the per-process analysis loggers capture. Affects the syscall/exec-derived loggers (syscalls, exec, read/write, ficd, interfaces) and busybox shell coverage; netbinds always reports for every process regardless of this setting. Recognized values: 'firmware' (default) captures only the firmware-under-analysis process subtree, excluding Penguin's own infrastructure (boot machinery and the vpnguin/console/guesthopper helpers); 'none' captures every process, including Penguin infrastructure; 'infra' inverts the firmware filter to capture only Penguin's own tools. Booleans are accepted for backward compatibility: true == 'firmware', false == 'none'. The field is a string so further interpretations can be added without a schema change.", examples=['firmware', 'none', 'infra'])]

arch: Annotated[Literal['armel', 'arm', 'armle', 'aarch64', 'arm64', 'mipsel', 'mipseb', 'mipsbe', 'mips64el', 'mips64eb', 'mips64be', 'powerpc', 'ppc', 'powerpc64', 'ppc64', 'powerpc64le', 'ppc64le', 'powerpc64el', 'ppc64el', 'riscv64', 'riscv', 'rv64', 'loongarch64', 'loongarch', 'la64', 'x86_64', 'intel64', 'amd64', 'x86-64', 'x64'] | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Architecture of guest', description='Canonical name or an accepted alias (normalized at load, e.g. intel64 -> x86_64).', examples=['x86_64', 'armel', 'aarch64', 'mipsel', 'mipseb', 'mips64el', 'powerpc64le'])]

auto_patching: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=True, title='Enable automatic patching', description='Whether to automatically apply patches named patch_*.yaml or from patches/*.yaml in the project directory', examples=[False, True])]

core_dumps: %p', {'lock': False}])]

cpu: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='CPU model', description='Specify non-default QEMU CPU model')]

execution_mode: Annotated[Literal['qemu', 'kvm'], FieldInfo(annotation=NoneType, required=False, default='qemu', title='Execution Mode', description='The execution backend to use for the guest (qemu for TCG, kvm for hardware acceleration)', examples=['qemu', 'kvm'])]

extra_qemu_args: Annotated[*StrSepSpace* | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Extra QEMU arguments', description='A list of additional QEMU command-line arguments to use when booting the guest', examples=['-vnc :0 -vga std -device usb-kbd -device usb-tablet'])]

force_www: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Try to force webserver start', description='Whether to try forcing webserver start', examples=[False, True])]

fs: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default='./base/fs.tar.gz', title='Project-relative path to filesystem tarball', examples=['base/fs.tar.gz'])]

gdbserver: GDBServerPrograms | None

graphics: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Enable graphics', description='Whether to enable graphics in the guest', examples=[False, True])]

guest_cmd: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Enable running commands in the guest', description='When enabled, starts the guesthopper daemon in the guest that the host can use to run commands over vsock', examples=[False, True])]

immutable: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=True, title='Enable immutable mode', description='Whether to run the guest filesystem in immutable mode', examples=[False, True])]

kernel: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Path to kernel image', examples=['/igloo_static/kernels/zImage.armel', '/igloo_static/kernels/zImage.arm64', '/igloo_static/kernels/vmlinux.mipsel', '/igloo_static/kernels/vmlinux.mipseb', '/igloo_static/kernels/vmlinux.mips64el', '/igloo_static/kernels/vmlinux.mips64eb'])]

kernel_cmdline_append: Annotated[*StrSepSpace* | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Extra kernel command-line tokens', description='Tokens appended verbatim to the kernel command line (-append). Use this to set real kernel parameters or anything you need on /proc/cmdline. Unlike `env`, these are never diverted to the early-boot env blob, so they always reach the kernel cmdline. They count against the per-arch COMMAND_LINE_SIZE budget (256 bytes on MIPS), so penguin warns/errors if they overflow it.', examples=['nokaslr mem=256M', 'igloo_debug=1'])]

kernel_quiet: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=True, title='Whether to include quiet flag in kernel command line', description='If true, the kernel command line will include the quiet flag, otherwise all kernel boot messages will be printed to the console', examples=[False, True])]

log_file: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Penguin log file', description='If set, write penguin/plugin logger output to this file (relative paths resolve under the results dir).', examples=['penguin.log'])]

ltrace: Annotated[bool | list[str], FieldInfo(annotation=NoneType, required=False, default=False, title='Enable ltrace', description='If true, run ltrace for entire system starting from init. If names of programs, enable ltrace only for those programs.', examples=[False, True, ['lighttpd']])]

mem: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default='2G', title='Panda Memory Value', description='Allows users to customize memory allocation for guest', examples=['16K', '512M', '1G', '2G'])]

model_config = {'extra': 'forbid', 'title': 'Core configuration options'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

network: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Connect guest to network', description='Whether to connect the guest to the network', examples=[False, True])]

plugin_path: Annotated[str, FieldInfo(annotation=NoneType, required=False, default='/pyplugins', title='Path to search for PyPlugins', examples=['/pyplugins'])]

root_shell: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Enable root shell', description='Whether to enable a root shell into the guest', examples=[False, True])]

root_shell_backend: Annotated[Literal['telnet', 'vsock'], FieldInfo(annotation=NoneType, required=False, default='vsock', title='Root shell backend', description="Which console backend serves core.root_shell. 'vsock' (default) runs an on-demand pty over the guest command channel (guesthopper), freeing the ttyS1 serial line; it requires the vsock transport, so core.guest_cmd is enabled automatically and the VPN transport must be available (with vpn disabled, this falls back to 'telnet'). 'telnet' uses the legacy QEMU -serial telnet console on ttyS1.", examples=['vsock', 'telnet'])]

shared_dir: ', examples=[True, 'my_shared_directory', {'path': 'shared', 'msize': 131072}])]

show_output: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='Write serial to stdout', description='Whether to print QEMU serial output to stdout instead of writing to a log file', examples=[False, True])]

smp: Annotated[int | None, FieldInfo(annotation=NoneType, required=False, default=1, title='Number of CPUs', description='Number of CPUs to emulate in the guest (Warning: This can break things)', examples=[1, 2, 4])]

snapshot: *Snapshot* | None

startup_script: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Inline guest startup script', description="Shell script body dropped into /igloo/init.d to run during guest boot. Installed under a name that sorts after other init.d entries so it runs last. A '#!/igloo/utils/sh' shebang is prepended automatically.", examples=['ip link set eth0 up\nudhcpc -i eth0\n'])]

strace: Annotated[bool | list[str], FieldInfo(annotation=NoneType, required=False, default=False, title='Enable strace', description='If true, run strace for entire system starting from init. If names of programs, enable strace only for those programs.', examples=[False, True, ['lighttpd']])]

timeout: Annotated[int | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Run timeout (seconds)', description='If set, automatically shut the guest down after this many seconds. Overridden by the --timeout CLI flag. No timeout when unset.', examples=[60, 300])]

version: Annotated[Literal['1.0.0', 2], FieldInfo(annotation=NoneType, required=True, title='Config format version', description='Version of the config file format')]

class penguin.penguin_config.structure.**CoreDumps**(*, *lock=True*, *pattern=None*)

Bases: PartialModelMixin, BaseModel

Guest core-dump capture configuration.

Accepted in core.core_dumps as true (enable with defaults), a string (shorthand for pattern), or this object. When enabled, penguin points core_pattern at /igloo/core_dumps (a symlink into the shared mount) and brings that mount up even when core.shared_dir is unset.

Parameters

- **lock** (*bool*)
- **pattern** (*str | None*)

lock: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=True, title='Lock core_pattern', description="Install a sysctl pseudofile that eats guest writes to core_pattern so dumps can't be redirected. Set false to let the guest firmware keep its own core_pattern.", examples=[True, False])]

model_config = {'extra': 'forbid', 'title': 'Core dump configuration'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

pattern: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='core_pattern override', description='Override the core_pattern string. Unset uses /igloo/core_dumps/core_%e.%p.', examples=['/igloo/core_dumps/core_%e.%p.%t'])]

class penguin.penguin_config.structure.**ExternalNetwork**(*, *mac='52:54:00:12:34:56'*, *pcap=None*)

Bases: PartialModelMixin, BaseModel

Configuration for NAT for external connections

Parameters

- **mac** (*str | None*)
- **pcap** (*bool | None*)

mac: str | None

model_config = {'extra': 'forbid', 'title': 'Set up NAT for outgoing connections'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

pcap: bool | None

class penguin.penguin_config.structure.**InitPluginEntry**(*, *enabled=True*, ***extra_data*)

Bases: PartialModelMixin, BaseModel

One init plugin's record/settings in the init_plugins section.

Parameters

- **enabled** (*bool*)
- **extra_data** (*Any*)

enabled: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=True, title='Run this init plugin during penguin refresh', description='Set to false to skip this plugin entirely when re-running init analyses with `penguin refresh`.')]

model_config = {'extra': 'allow', 'title': 'Init plugin'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

class penguin.penguin_config.structure.**LibInject**(*, *enabled=True*, *aliases=None*, *extra=None*, *stubs=None*)

Bases: PartialModelMixin, BaseModel

Library functions to be intercepted

Parameters

- **enabled** (*bool | None*)
- **aliases** (*LibInjectAliases | None*)
- **extra** (StrLines | *None*)
- **stubs** (Stubs | *None*)

aliases: , nvram_get)'})]

enabled: Annotated[bool | None, FieldInfo(annotation=NoneType, required=False, default=True, title='Enables LD_PRELOAD in init script', description='Whether to enable LD_PRELOAD variable in init script', examples=[False, True])]

extra: Annotated[*StrLines* | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Extra injected library code', description='Custom source code for library functions to intercept and model')]

model_config = {'extra': 'forbid', 'title': 'Injected library configuration'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

stubs: Annotated[*Stubs* | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Declarative symbol stubs', description="Force library/object symbols to return a constant. Compiles down to generated C shims plus 'aliases' entries; a symbol may not appear in both 'stubs' and 'aliases'.")]

class penguin.penguin_config.structure.**Main**(*, *core*, *patches=None*, *vars=None*, *env*, *pseudofiles*, *nvram*, *netdevs=[]*, *uboot_env=None*, *blocked_signals=[]*, *lib_inject*, *static_files*, *plugins*, *init_plugins=None*, *network=None*)

Bases: PartialModelMixin, BaseModel

Configuration file for config-file-based rehosting with IGLOO

Parameters

- **core** (Core)
- **patches** (*Patches | None*)
- **vars** (*dict[str, Any] | None*)
- **env** (*Env*)

- **pseudofiles** (Pseudofiles)
- **nvram** (*NVRAM*)
- **netdevs** (*List[str]*)
- **uboot_env** (*UBootEnv | None*)
- **blocked_signals** (*List[int]*)
- **lib_inject** (LibInject)
- **static_files** (StaticFiles)
- **plugins** (*dict[str,* Plugin*]*)
- **init_plugins** (*dict[str,* InitPluginEntry*]*)
- **network** (Network *| None*)

blocked_signals: List[int]

core: *Core*

env: Env

init_plugins: Annotated[dict[str, *InitPluginEntry*], FieldInfo(annotation=NoneType, required=False, default=None, title='Init plugins', description='The init plugins that generated this project (recorded by `penguin init`). Drives which plugins re-run on `penguin refresh`; newly available plugins are appended when they run.')]

lib_inject: *LibInject*

model_config = {'extra': 'forbid', 'title': 'Penguin Configuration'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

netdevs: List[str]

network: *Network* | None

nvram: NVRAM

patches: Patches | None

plugins: Annotated[dict[str, *Plugin*], FieldInfo(annotation=NoneType, required=True, title='Plugins')]

pseudofiles: *Pseudofiles*

static_files: *StaticFiles*

uboot_env: UBootEnv | None

vars: ', examples=[{'webroot': '/www', 'libdir': '/lib/{{ arch }}'}])]

class penguin.penguin_config.structure.**Network**(*, *external*=<*factory*>)

Bases: PartialModelMixin, BaseModel

Configuration for networks to attach to guest

Parameters

external (ExternalNetwork)

external: *ExternalNetwork*

model_config = {'extra': 'forbid', 'title': 'Network Configuration'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

class penguin.penguin_config.structure.**Plugin**(*, *description=None*, *depends_on=None*, *enabled=True*, *version=None*, ***extra_data*)

Bases: PartialModelMixin, BaseModel

Parameters

- **description** (*str | None*)
- **depends_on** (*str | None*)
- **enabled** (*bool*)
- **version** (*str | None*)
- **extra_data** (*Any*)

depends_on: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Plugin dependency')]

description: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Plugin description')]

enabled: Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=True, title='Enable this plugin (default depends on plugin)')]

model_config = {'extra': 'allow', 'title': 'Plugin'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

version: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Plugin version')]

class penguin.penguin_config.structure.**Pseudofile**(*, *name=None*, *size=None*, *plugin=None*, *read=None*, *write=None*, *ioctl=None*, *poll=None*, *lseek=None*, *mmap=None*, *open=None*, *release=None*, *compat_ioctl=None*, *flush=None*, *fsync=None*, *fasync=None*, *lock=None*, *read_iter=None*, *write_iter=None*, *get_unmapped_area=None*)

Bases: PartialModelMixin, BaseModel

How to emulate a device file

Parameters

- **name** (*str | None*)
- **size** (*int | None*)
- **plugin** (*str | None*)
- **read** (*Read | None*)
- **write** (*Write | None*)
- **ioctl** (*Ioctls | None*)

- **poll** (*Poll | None*)
- **lseek** (*Seek | None*)
- **mmap** (*Mmap | None*)
- **open** (*Open | None*)
- **release** (*Release | None*)
- **compat_ioctl** (*CompatIoctl | None*)
- **flush** (*Flush | None*)
- **fsync** (*Fsync | None*)
- **fasync** (*Fasync | None*)
- **lock** (*Lock | None*)
- **read_iter** (*ReadIter | None*)
- **write_iter** (*WriteIter | None*)
- **get_unmapped_area** (*GetUnmappedArea | None*)

compat_ioctl: CompatIoctl | None

fasync: Fasync | None

flush: Flush | None

fsync: Fsync | None

get_unmapped_area: GetUnmappedArea | None

ioctl: Ioctls | None

lock: Lock | None

lseek: Seek | None

mmap: Mmap | None

model_config = {'extra': 'forbid', 'title': 'File emulation spec'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

name: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='MTD name', description='Name of an MTD device (ignored for non-mtd)', examples=['flash', 'uboot'])]

open: Open | None

plugin: Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Single backing class', description="Name of a single backing class that owns the whole file_operations surface (read/write/ioctl/poll) for this node. Reference a built-in backing by name, or a user class as 'file:ClassName' (the file is found via the normal pyplugin search path). When set, the per-domain read/write/ioctl/poll keys are ignored - the class owns them all.", examples=['my_backing:SerialBacking'])]

poll: Poll | None

read: *Read | None*

read_iter: *ReadIter | None*

release: *Release | None*

size: *Annotated[int | None, FieldInfo(annotation=NoneType, required=False, default=None, title='File size', description='Size of the pseudofile to be reported by stat(). This must be specified for mmap() on the pseudofile to work.', examples=[1, 4096])]*

write: *Write | None*

write_iter: *WriteIter | None*

class penguin.penguin_config.structure.**Pseudofiles**(*root=PydanticUndefined*)

Bases: RootModel

Device files to emulate in the guest

Parameters
root (*RootModelRootType*)

model_config = {'title': 'Pseudo-files'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

root: *dict[str, Pseudofile]*

class penguin.penguin_config.structure.**SharedDir**(*, *path='shared'*, *host_path=None*, *msize=None*)

Bases: PartialModelMixin, BaseModel

Host<->guest shared directory (9p) configuration.

Accepted in `core.shared_dir` as `true` (enable with defaults), a string (shorthand for `path`), or this object. Core dumps ride the same single mount (see `core.core_dumps`); they do not need this feature enabled.

Parameters

- **path** (*str*)
- **host_path** (*str | None*)
- **msize** (*int | None*)

host_path: *Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Absolute host directory to share', description='If set, share this absolute host directory instead of a results-relative one (path is ignored).', examples=['/data/fixtures'])]*

model_config = {'extra': 'forbid', 'title': 'Shared directory configuration'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

msize: *Annotated[int | None, FieldInfo(annotation=NoneType, required=False, default=None, title='9p msize override', description='Override the 9p transport buffer size. Unset uses the default 8MB with an automatic fallback to 128KB on memory-tight guests.', examples=[8192000, 131072])]*

path: `Annotated[str, FieldInfo(annotation=NoneType, required=False, default='shared', title='Results-relative share directory', description="Directory shared into the guest at /igloo/shared. Resolved under the run's results dir unless core.shared_dir.host_path is set.", examples=['shared', 'my_shared_directory'])]`

class penguin.penguin_config.structure.**Snapshot**(*, *backend='internal'*, *tag='boot'*, *save_at=None*, *boot_from=None*, *stop_after_save=False*)

Bases: `PartialModelMixin`, `BaseModel`

VM snapshot (savevm/loadvm) configuration.

Snapshotting is *active* whenever `save_at` or `boot_from` is set — there is no separate enable flag. When active, the guest runs on a persistent qcow2 overlay (rather than the throwaway immutable overlay) so an internal VM snapshot can be saved and later restored. Saving a snapshot at a chosen point lets a later run boot directly from that state instead of re-booting the firmware.

Parameters

- **backend** (*Literal['internal', 'file']*)
- **tag** (*str*)
- **save_at** (*Literal['readiness', 'manual'] | None*)
- **boot_from** (*str | None*)
- **stop_after_save** (*bool*)

backend: `Annotated[Literal['internal', 'file'], FieldInfo(annotation=NoneType, required=False, default='internal', title='Snapshot backend', description="'internal' stores the snapshot inside the qcow2 overlay (savevm/loadvm). 'file' (not yet implemented) writes a standalone migration file bundle.", examples=['internal', 'file'])]`

boot_from: `Annotated[str | None, FieldInfo(annotation=NoneType, required=False, default=None, title='Snapshot tag to boot from', description='If set, restore this internal snapshot at startup (-loadvm).', examples=['boot'])]`

model_config = `{'extra': 'forbid', 'title': 'Snapshot configuration'}`

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

save_at: `Annotated[Literal['readiness', 'manual'] | None, FieldInfo(annotation=NoneType, required=False, default=None, title='When to save the snapshot', description="'readiness' saves once the guest reaches steady state; 'manual' arms the Snapshot plugin to save on request (via guest_cmd / hypercall). None disables saving.", examples=['readiness', 'manual'])]`

stop_after_save: `Annotated[bool, FieldInfo(annotation=NoneType, required=False, default=False, title='End the run after saving', description='Shut the guest down immediately after the snapshot is saved.', examples=[False, True])]`

tag: `Annotated[str, FieldInfo(annotation=NoneType, required=False, default='boot', title='Snapshot tag', description='Name of the internal VM snapshot to save and/or restore.', examples=['boot', 'post_init'])]`

class penguin.penguin_config.structure.**StaticFiles**(*root=PydanticUndefined*)

Bases: `RootModel`

Files to create in the guest filesystem

Parameters
root (*RootModelRootType*)

model_config = {'json_schema_extra': {'examples': [{}, {'/path/to/file': {'contents': 'Hello world!', 'type': 'file'}}, {'/path/to/symlink/source': {'target': '/path/to/symlink/target', 'type': 'symlink'}}, {'/dev/some_device': {'devtype': 'char', 'major': 1, 'minor': 2, 'mode': 438, 'type': 'dev'}}]}, 'title': 'Static files'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

root: dict[str, StaticFileAction]

class penguin.penguin_config.structure.**StrLines**(*root=PydanticUndefined*)

Bases: *StrSep*

Parameters
root (*RootModelRootType*)

model_config = {}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

separator = '\n'

class penguin.penguin_config.structure.**StrSep**(*root=PydanticUndefined*)

Bases: RootModel

Parameters
root (*RootModelRootType*)

merge(*other*)

classmethod merge_behavior()

model_config = {}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

root: str

separator: ClassVar = None

class penguin.penguin_config.structure.**StrSepSpace**(*root=PydanticUndefined*)

Bases: *StrSep*

Parameters
root (*RootModelRootType*)

model_config = {}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

separator = ' '

class penguin.penguin_config.structure.**StubLibrary**(*root=PydanticUndefined*)

Bases: RootModel

Symbols (or globs) to stub within one library/object. The library key is an organizational label and the glob-resolution target; aliasing itself is global (`--defsym`).

Parameters
root (*RootModelRootType*)

model_config = {'title': 'Per-library symbol stubs'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

root: dict[str, StubAction]

class penguin.penguin_config.structure.**Stubs**(*root=PydanticUndefined*)

Bases: RootModel

Declarative symbol stubs, grouped by library/object.

Each key is a library/object (an absolute guest path like `/lib/libc.so` or a bare basename like `libX.so` searched for in the rootfs). Each value maps a symbol key to a stub action. A symbol key is either a symbol name, a glob such as `nvram_*` (expanded against the library's exported symbols at build time; symbol-return stubs only), or `symbol@offset` (assembly-body stubs only, where `offset` is added to the symbol address).

Parameters
root (*RootModelRootType*)

model_config = {'json_schema_extra': {'examples': [{}, {'/lib/libc.so': {'memcpy': {'guard_null_args': [0, 1], 'return': 0}}, 'libX.so': {'get_flag': {'return': 0}, 'hw_probe': {'body': 'movs r0, #0\nbx lr', 'mode': 'thumb'}, 'nvram_*': {'return': 0}}}]}, 'title': 'Declarative symbol stubs'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

root: dict[str, *StubLibrary*]

penguin.penguin_config.structure.**normalize_hex_string**(*value*)

Normalize a user-supplied hex byte string: strip whitespace, allow an optional `0x`/`0X` prefix, and drop internal spaces. Returns lowercase hex with no separators. Raises `ValueError` if it is not valid hex or has an odd digit count. Shared by the schema validator and the patch applier so both accept exactly the same inputs.

Parameters
value (*str*)

Return type
str

penguin.penguin_config.templating module

Jinja2 "meta variable" templating for Penguin configs.

Lets a config or patch reference values like `{{ arch }}` / `{{ core.arch }}`, any other `core.*` scalar, user-defined `vars:`, and `{{ kernel_version }}` so that changing one value (e.g. `core.arch`) flows everywhere it is used.

Substitution runs per-file on the parsed YAML dict, before patch merging: each file resolves against the main config's `core`/`arch` plus its own and the main config's `vars`. `kernel_version` is special — the real kernel path is only resolved late, so first-pass rendering replaces `{{ kernel_version }}` with a sentinel that `resolve_kernel_version` substitutes once it is known.

Configs that contain no `{{ }}`/`{% %}` are returned untouched.

exception penguin.penguin_config.templating.**TemplateError**

Bases: Exception

Raised when a config template references an undefined var or is malformed.

penguin.penguin_config.templating.**build_context**(*raw_config*, *extra=None*, *env=None*, *where='vars'*)

Build the Jinja context for a config/patch dict.

Exposes `arch`, the arch-derived `arch_dir` / `dylib_dir` static-layout subdirs, `core` (so `{{ core.mem }}` works), the late-bound `kernel_version` sentinel, anything in `extra` (e.g. the main config's context when rendering a patch), and the file's own `vars:` (which may themselves reference earlier vars / arch).

penguin.penguin_config.templating.**render_config**(*raw_config*, *extra=None*, *env=None*, *where='config'*)

Render a raw config/patch dict in place-equivalent fashion, returning the rendered dict and the context used (so callers can reuse it for patches).

penguin.penguin_config.templating.**resolve_kernel_version**(*obj*, *kernel_version*)

Second pass: replace the kernel_version sentinel with the resolved value.

penguin.penguin_config.templating.**substitute**(*obj*, *ctx*, *env=None*, *where='config'*)

Recursively render every string leaf of `obj` with the Jinja context.

penguin.testing package

Host-side test harness for driving pyplugins in place (no PANDA/guest).

See *penguin.testing.harness* for the design. Public API:

from penguin.testing import load_pyplugin

class penguin.testing.**LoadedPlugin**(*plugin*, *manager*, *panda*, *log*)

Bases: `object`

A constructed plugin plus the null backend it was built against.

Attributes

plugin
[the constructed plugin instance (drive its handlers directly, or use] *dispatch()*).

manager
[the *NullManager* (inspect `.subscriptions`, `.published`,] `.registrations`).

panda : the *NullPanda*. calls : the shared recorder log of stub calls `(path, args, kwargs)`.

dispatch(*event*, **args*, ***kwargs*)

Invoke every handler the plugin subscribed to `event`.

Mirrors the manager delivering an event to subscribers. Returns the list of handler return values. Raises if no handler is subscribed to `event` (a silent no-op usually means a typo in the event name).

Parameters

- **event** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
List[*Any*]

dispatch_syscall(*name*, **args*, *on_return=None*, *responses=None*)

Invoke every `@plugins.syscalls.syscall(...)` hook registered for `name` and drive each (they are portal generators) to completion.

`name` matches by bare syscall name (`sys_kill`/`kill`/ `on_sys_kill_enter` all select `kill`). Pass `on_return=True/False` to disambiguate enter vs return hooks. Returns each handler's final value. Raises if no hook matches. The handler args are the syscall-callback args (`regs, proto, syscall, *syscall_args`); sibling API calls the handler makes (`plugins.mem`/`plugins.osi`) must be satisfied by generator `doubles`.

Parameters

- **name** (*str*)
- **args** (*Any*)
- **on_return** (*bool | None*)
- **responses** (*List[Any] | None*)

Return type
List[*Any*]

finalize()

Run the plugin's teardown (`uninit`) if it has one — many plugins flush their final output file there (e.g. netbinds' lifecycle CSV).

Return type
None

property published: List[Tuple[Any, str, tuple, dict]]

property subscriptions: List[Tuple[Any, str, Callable]]

Parameters

- **plugin** (Plugin)
- **manager** (NullManager)
- **panda** (NullPanda)
- **log** (*List[Tuple]*)

class penguin.testing.**NullManager**(*args*, *doubles*, *panda*, *log*)

Bases: `object`

Null backend standing in for `IGLOOPluginManager`.

Records `subscribe`/`register`/`publish`/`portal_publish` calls, resolves registered doubles by name, and falls back to a recording stub for everything else. It never tries to load a real plugin (no recursion).

Parameters

- **args** (*Dict[str, Any]*)
- **doubles** (*Dict[str, Any]*)
- **panda** (NullPanda)
- **log** (*List[Tuple]*)

get_plugin_by_name(*name*)

Parameters
name (*str*)

Return type
Any | None

portal_publish(*plugin*, *event*, **args*, ***kwargs*)

Parameters

- **plugin** (*Any*)
- **event** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

publish(*plugin*, *event*, **args*, ***kwargs*)

Parameters

- **plugin** (*Any*)
- **event** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

register(*plugin*, *event*, **a*, ***k*)

Parameters

- **plugin** (*Any*)
- **event** (*str*)

Return type
None

subscribe(*publisher*, *event*, *callback=None*, **a*, ***k*)

Record a subscription. Mirrors the real manager: with no `callback` it returns a decorator (the class-body `@plugins.subscribe(pub, event)` form), otherwise it records directly.

Parameters

- **publisher** (*Any*)
- **event** (*str*)
- **callback** (*Callable*)

class penguin.testing.**NullPanda**(*endianness='little'*, *log=None*)

Bases: `object`

Stand-in for `self.panda`: records callback registrations, exposes a configurable `endianness` and a minimal `ffi`. Any other attribute is a *RecorderStub*.

Parameters

- **endianness** (*str*)

- **log** (*Optional[List[Tuple]]*)

class penguin.testing.**RealKffi**(*isf_paths*)

Bases: object

A real dwarffi-backed stand-in for the kffi plugin, exposing just the enum/type surface hyper.consts and type-reading plugins need. Built from one or more real ISF paths; get_enum_dict/get_type mirror apis/kffi.py so the genuine hyper.consts imports against real values.

igloo_base_hypercalls is defined in igloo_base and is not emitted into the driver ISF (and the kernel cosi ISF that used to carry it is being retired), so its single ABI-fixed member is supplied here — the one enum with no host-reachable ISF home.

get_enum_dict(*name*)

Parameters
name (*str*)

Return type
Dict[str, int]

get_type(*name*)

class penguin.testing.**RecorderStub**(*path='plugins'*, *log=None*)

Bases: object

Recursive no-op that records attribute access and calls.

Stands in for any plugin-manager attribute the harness has not been given a real double for (sibling plugins, plugins.Events, nested APIs). Behaves as a decorator (@x and @x(...)), an empty container, and a callable, so class/module-body and __init__ code touching it does not blow up. Every call is appended to the shared log for assertions.

Parameters

- **path** (*str*)
- **log** (*Optional[List[Tuple]]*)

penguin.testing.**drive**(*gen*, *responses=None*, *collect=False*)

Run a portal-style generator handler to completion and return its value.

Penguin's syscall-return / OSI / mem handlers are generators that yield from sibling API calls (plugins.mem.read_bytes, plugins.osi.*); at runtime the portal fulfils each yielded request and resumes the generator. Host-side, the sibling *doubles* are themselves generators that yield nothing and return a canned value, so yield from resolves immediately and this pump just exhausts the outer generator (running its side effects) and returns its final value. responses (optional) is fed in order to the values the generator yields (in order); exhausted responses send None.

With collect=True returns (return_value, yielded) where yielded is the list of values the generator yielded — e.g. the PortalCmds a handler emits — so a test can assert the plugin issues the *right* portal command (compare cmd.op against the same enum member) with the right args, even though the fake enum's numeric value is meaningless.

Parameters

- **gen** (*Any*)
- **responses** (*List[Any] | None*)
- **collect** (*bool*)

Return type
Any

penguin.testing.**install_real_consts**(*isf_paths*)

Build a real dwarffi-backed *RealKffi* from isf_paths and clear cached hyper.consts so the genuine module rebuilds against it.

Register the returned object as the kffi double (load_pyplugin does this for you via real_isf=): the real hyper.consts then imports through the normal loader with real enum values.

Return type
RealKffi

penguin.testing.**load_module**(*path_or_name*, *, *doubles=None*, *pyplugins_dir=None*, *endianness='little'*, *real_isf=None*)

Import a pyplugin *module* (not necessarily a Plugin subclass) with a null plugins bound, and return (module, manager).

Use this for modules whose testable logic lives in plain classes/functions rather than a Plugin (e.g. the hyperfile/models/* read/write mixins): import once, then instantiate the classes directly and drive their generator methods with *drive()*, resolving plugins.<name> against doubles.

The module is imported under its real dotted name (hyperfile.models.read) so intra-package from .base import ... resolves; penguin.plugins is set to the null manager for the duration of the import so module-body / from penguin import plugins bindings resolve against it.

Parameters

- **path_or_name** (*str*)
- **doubles** (*Dict[str, Any] | None*)
- **pyplugins_dir** (*str | None*)
- **endianness** (*str*)

Return type
Tuple[*Any*, NullManager]

penguin.testing.**load_pyplugin**(*path_or_name*, *, *args=None*, *outdir=None*, *doubles=None*, *class_name=None*, *endianness='little'*, *pyplugins_dir=None*, *call_init=True*, *real_isf=None*)

Load and construct a pyplugin against a null backend, ready to drive.

Parameters

path_or_name
[filesystem path to the plugin .py (recommended), or a bare] module basename resolved against pyplugins_dir.

args
[the plugin's arguments (the conf['plugins'][name] block). outdir] is injected automatically when given.

outdir
[output directory the plugin writes into (a tmp dir in tests). Added to] args as outdir.

doubles
[mapping of sibling name -> test double, resolved by] plugins.<name> / get_plugin_by_name. Unlisted siblings become a recording *RecorderStub*.

class_name
[which `Plugin` subclass to construct when the module defines] more than one. Defaults to the sole subclass.

endianness : `self.panda.endianness` value ("little" or "big"). pyplugins_dir : directory to resolve a bare `path_or_name` against. call_init : run the plugin's `__init__` (default). Set False for plugins

whose `__init__` does host-impossible I/O (e.g. nvram2 shells out to `clang-20` to compile lib_inject) — the class still imports (so class-body `@subscribe`/`@syscall` decorators register), the instance is `__new__`/`__preinit__`-wired, and the test sets the handful of attributes the handlers need before driving them.

real_isf
[path (or list of paths, or `"auto"`) to a real `igloo.ko` ISF,] for plugins behind the FFI-enum boundary (those importing `apis.syscalls`/`hyper`, e.g. `analysis/interfaces`). Registers a real `dwarffi`-backed `kffi` double so the genuine `hyper.consts` builds against **real** enum values (see *`install_real_consts()`*). `"auto"` resolves via *`resolve_igloo_ko_isf()`*.

Parameters

- **path_or_name** (*str*)
- **args** (*Dict[str, Any] | None*)
- **outdir** (*str | None*)
- **doubles** (*Dict[str, Any] | None*)
- **class_name** (*str | None*)
- **endianness** (*str*)
- **pyplugins_dir** (*str | None*)
- **call_init** (*bool*)

Return type
LoadedPlugin

penguin.testing.**resolve_igloo_ko_isf**(*arch='armel'*, *version=None*)

Locate an `igloo.ko.<arch>.json.xz` ISF, or return None if unavailable.

Resolution order:

1. `PENGUIN_TEST_IGLOO_KO_ISF` env var (explicit path).
2. The local cache for *this exact version* (a prior download).
3. Download `igloo_driver.tar.gz` for the flake.nix-pinned igloo_driver release (or `version`) and extract the one ISF.
4. The nix store (dev machines).

The cache is keyed by version (`.isf_cache/<version>/igloo.ko.<arch>...`), not by arch alone: a stale ISF from an *earlier* pin must never satisfy a lookup after `IGLOO_DRIVER_VERSION` is bumped, or a bump would be silently masked (the test would run against the wrong ABI). A bump simply misses the old cache entry and re-downloads.

Returns None (rather than raising) when offline with nothing cached, so tests can `skip` cleanly.

Parameters

- **arch** (*str*)
- **version** (*str | None*)

Return type
str | None

Submodules

penguin.testing.harness module

In-place pyplugin test harness (`penguin.testing`)

Drive a Penguin pyplugin **where it lives**, with no PANDA, no guest, and no emulation, so its host-side logic (the files it writes, the events it emits) can be tested by a fast host-side unit test instead of a per-arch guest boot.

The problem this solves

A pyplugin cannot simply be imported and instantiated standalone: its module and class body reference the live `plugins` manager (`@plugins.portalcall(...)`, `plugins.subscribe(plugins.Events, ...)`, sibling `plugins.<name>` calls), which are normally late-bound against a running PANDA/portal. Importing without a manager raises (see the recursion in `IGLOOPluginManager.__getattr__`); the old approach was a bespoke `sys.modules` stub per test file.

This harness replaces that with **one reusable null backend**:

- `NullManager` stands in for the plugin manager. Known siblings resolve to test doubles you register; unknown ones resolve to a recording no-op (`RecorderStub`) so decorators and sibling calls are harmless and observable.
- `NullPanda` stands in for the emulator handle (`self.panda`). It records callback registrations and exposes a configurable `endianness`.
- The plugin is loaded via the *real* loader (`_exec_plugin_module`) and constructed via the real `Plugin` protocol (`__new__` → `__preinit__` → `__init__`), so behaviour matches production construction.

Scope

Faithful for plugins that **consume events and write host files** (the `analysis/` writers, pseudofile models, loggers). A plugin that round-trips through the guest (hypercall request → guest action → response) is out of scope and stays a `tests/integration/` fixture — register a hand-written double for the piece that talks to the guest if you must, or don't use the harness.

Example

```
from penguin.testing import load_pyplugin

lp = load_pyplugin(
    "pyplugins/analysis/netbinds.py",
    outdir=tmp_path,
    args={"shutdown_on_www": False},
)
lp.dispatch("igloo_ipv4_setup", cpu=None, procname="httpd", sin_addr=0)
lp.dispatch("igloo_ipv4_bind", cpu=None, port=port_be, is_steam=True)
lp.finalize()  # runs uninit() -> lifecycle CSV
assert "httpd" in (tmp_path / "netbinds.csv").read_text()
```

class penguin.testing.harness.**LoadedPlugin**(*plugin*, *manager*, *panda*, *log*)

Bases: `object`

A constructed plugin plus the null backend it was built against.

Attributes

plugin
[the constructed plugin instance (drive its handlers directly, or use] *dispatch()*).

manager
[the *NullManager* (inspect .subscriptions, .published,] .registrations).

panda : the *NullPanda*. calls : the shared recorder log of stub calls (path, args, kwargs).

dispatch(*event*, **args*, ***kwargs*)

Invoke every handler the plugin subscribed to event.

Mirrors the manager delivering an event to subscribers. Returns the list of handler return values. Raises if no handler is subscribed to event (a silent no-op usually means a typo in the event name).

Parameters

- **event** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
List[*Any*]

dispatch_syscall(*name*, **args*, *on_return=None*, *responses=None*)

Invoke every @plugins.syscalls.syscall(...) hook registered for name and drive each (they are portal generators) to completion.

name matches by bare syscall name (sys_kill/kill/ on_sys_kill_enter all select kill). Pass on_return=True/False to disambiguate enter vs return hooks. Returns each handler's final value. Raises if no hook matches. The handler args are the syscall-callback args (regs, proto, syscall, *syscall_args); sibling API calls the handler makes (plugins.mem/plugins.osi) must be satisfied by generator doubles.

Parameters

- **name** (*str*)
- **args** (*Any*)
- **on_return** (*bool | None*)
- **responses** (*List[Any] | None*)

Return type
List[*Any*]

finalize()

Run the plugin's teardown (uninit) if it has one — many plugins flush their final output file there (e.g. netbinds' lifecycle CSV).

Return type
None

property published: List[Tuple[Any, str, tuple, dict]]

property subscriptions: List[Tuple[Any, str, Callable]]

Parameters

- **plugin** (Plugin)
- **manager** (NullManager)
- **panda** (NullPanda)
- **log** (*List[Tuple]*)

class penguin.testing.harness.**NullManager**(*args*, *doubles*, *panda*, *log*)

Bases: object

Null backend standing in for IGLOOPluginManager.

Records subscribe/register/publish/portal_publish calls, resolves registered doubles by name, and falls back to a recording stub for everything else. It never tries to load a real plugin (no recursion).

Parameters

- **args** (*Dict[str, Any]*)
- **doubles** (*Dict[str, Any]*)
- **panda** (NullPanda)
- **log** (*List[Tuple]*)

get_plugin_by_name(*name*)

Parameters
name (*str*)

Return type
Any | None

portal_publish(*plugin*, *event*, **args*, ***kwargs*)

Parameters

- **plugin** (*Any*)
- **event** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

publish(*plugin*, *event*, **args*, ***kwargs*)

Parameters

- **plugin** (*Any*)
- **event** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

register(*plugin*, *event*, **a*, ***k*)

Parameters

- **plugin** (*Any*)
- **event** (*str*)

Return type
None

`subscribe`(*publisher*, *event*, *callback=None*, **a*, ***k*)

Record a subscription. Mirrors the real manager: with no `callback` it returns a decorator (the class-body `@plugins.subscribe(pub, event)` form), otherwise it records directly.

Parameters

- **publisher** (*Any*)
- **event** (*str*)
- **callback** (*Callable*)

class penguin.testing.harness.**NullPanda**(*endianness='little'*, *log=None*)

Bases: `object`

Stand-in for `self.panda`: records callback registrations, exposes a configurable `endianness` and a minimal `ffi`. Any other attribute is a *RecorderStub*.

Parameters

- **endianness** (*str*)
- **log** (*Optional[List[Tuple]]*)

class penguin.testing.harness.**RealKffi**(*isf_paths*)

Bases: `object`

A real `dwarffi`-backed stand-in for the `kffi` plugin, exposing just the enum/type surface `hyper.consts` and type-reading plugins need. Built from one or more real ISF paths; `get_enum_dict`/`get_type` mirror `apis/kffi.py` so the genuine `hyper.consts` imports against real values.

`igloo_base_hypercalls` is defined in igloo_base and is not emitted into the driver ISF (and the kernel `cosi` ISF that used to carry it is being retired), so its single ABI-fixed member is supplied here — the one enum with no host-reachable ISF home.

`get_enum_dict`(*name*)

Parameters
name (*str*)

Return type
Dict[str, int]

`get_type`(*name*)

class penguin.testing.harness.**RecorderStub**(*path='plugins'*, *log=None*)

Bases: `object`

Recursive no-op that records attribute access and calls.

Stands in for any plugin-manager attribute the harness has not been given a real double for (sibling plugins, `plugins.Events`, nested APIs). Behaves as a decorator (`@x` and `@x(...)`), an empty container, and a callable, so class/module-body and `__init__` code touching it does not blow up. Every call is appended to the shared `log` for assertions.

Parameters

- **path** (*str*)
- **log** (*Optional[List[Tuple]]*)

penguin.testing.harness.**drive**(*gen*, *responses=None*, *collect=False*)

Run a portal-style generator handler to completion and return its value.

Penguin's syscall-return / OSI / mem handlers are generators that `yield from` sibling API calls (`plugins.mem.read_bytes`, `plugins.osi.*`); at runtime the portal fulfils each yielded request and resumes the generator. Host-side, the sibling *doubles* are themselves generators that yield nothing and `return` a canned value, so `yield from` resolves immediately and this pump just exhausts the outer generator (running its side effects) and returns its final value. `responses` (optional) is fed in order to the values the generator yields (in order); exhausted responses send `None`.

With `collect=True` returns `(return_value, yielded)` where `yielded` is the list of values the generator yielded — e.g. the `PortalCmd`s a handler emits — so a test can assert the plugin issues the *right* portal command (compare `cmd.op` against the same enum member) with the right args, even though the fake enum's numeric value is meaningless.

Parameters

- **gen** (*Any*)
- **responses** (*List[Any] | None*)
- **collect** (*bool*)

Return type
Any

penguin.testing.harness.**install_real_consts**(*isf_paths*)

Build a real `dwarffi`-backed *RealKffi* from `isf_paths` and clear cached `hyper.consts` so the genuine module rebuilds against it.

Register the returned object as the `kffi` double (`load_pyplugin` does this for you via `real_isf=`): the real `hyper.consts` then imports through the normal loader with real enum values.

Return type
RealKffi

penguin.testing.harness.**load_module**(*path_or_name*, ***, *doubles=None*, *pyplugins_dir=None*, *endianness='little'*, *real_isf=None*)

Import a pyplugin *module* (not necessarily a `Plugin` subclass) with a null `plugins` bound, and return `(module, manager)`.

Use this for modules whose testable logic lives in plain classes/functions rather than a `Plugin` (e.g. the `hyperfile/models/*` read/write mixins): import once, then instantiate the classes directly and drive their generator methods with *drive()*, resolving `plugins.<name>` against `doubles`.

The module is imported under its real dotted name (`hyperfile.models.read`) so intra-package `from .base import ...` resolves; `penguin.plugins` is set to the null manager for the duration of the import so module-body / `from penguin import plugins` bindings resolve against it.

Parameters

- **path_or_name** (*str*)
- **doubles** (*Dict[str, Any] | None*)
- **pyplugins_dir** (*str | None*)
- **endianness** (*str*)

Return type
Tuple[*Any*, NullManager]

penguin.testing.harness.**load_pyplugin**(*path_or_name*, ***, *args=None*, *outdir=None*, *doubles=None*, *class_name=None*, *endianness='little'*, *pyplugins_dir=None*, *call_init=True*, *real_isf=None*)

Load and construct a pyplugin against a null backend, ready to drive.

Parameters

path_or_name
[filesystem path to the plugin `.py` (recommended), or a bare] module basename resolved against `pyplugins_dir`.

args
[the plugin's arguments (the `conf['plugins'][name]` block). `outdir`] is injected automatically when given.

outdir
[output directory the plugin writes into (a tmp dir in tests). Added to] `args` as `outdir`.

doubles
[mapping of sibling name -> test double, resolved by] `plugins.<name>` / `get_plugin_by_name`. Unlisted siblings become a recording *`RecorderStub`*.

class_name
[which `Plugin` subclass to construct when the module defines] more than one. Defaults to the sole subclass.

endianness : `self.panda.endianness` value ("little" or "big"). pyplugins_dir : directory to resolve a bare `path_or_name` against. call_init : run the plugin's `__init__` (default). Set False for plugins

whose `__init__` does host-impossible I/O (e.g. nvram2 shells out to `clang-20` to compile lib_inject) — the class still imports (so class-body `@subscribe`/`@syscall` decorators register), the instance is `__new__`/`__preinit__`-wired, and the test sets the handful of attributes the handlers need before driving them.

real_isf
[path (or list of paths, or `"auto"`) to a real `igloo.ko` ISF,] for plugins behind the FFI-enum boundary (those importing `apis.syscalls`/`hyper`, e.g. `analysis/interfaces`). Registers a real `dwarffi`-backed `kffi` double so the genuine `hyper.consts` builds against **real** enum values (see *`install_real_consts()`*). `"auto"` resolves via *`resolve_igloo_ko_isf()`*.

Parameters

- **path_or_name** (*str*)
- **args** (*Dict[str, Any] | None*)
- **outdir** (*str | None*)
- **doubles** (*Dict[str, Any] | None*)
- **class_name** (*str | None*)
- **endianness** (*str*)
- **pyplugins_dir** (*str | None*)
- **call_init** (*bool*)

Return type
LoadedPlugin

penguin.testing.harness.**resolve_igloo_ko_isf**(*arch='armel'*, *version=None*)

Locate an `igloo.ko.<arch>.json.xz` ISF, or return None if unavailable.

Resolution order:

1. `PENGUIN_TEST_IGLOO_KO_ISF` env var (explicit path).
2. The local cache for *this exact version* (a prior download).
3. Download `igloo_driver.tar.gz` for the flake.nix-pinned igloo_driver release (or `version`) and extract the one ISF.
4. The nix store (dev machines).

The cache is keyed by version (`.isf_cache/<version>/igloo.ko.<arch>...`), not by arch alone: a stale ISF from an *earlier* pin must never satisfy a lookup after `IGLOO_DRIVER_VERSION` is bumped, or a bump would be silently masked (the test would run against the wrong ABI). A bump simply misses the old cache entry and re-downloads.

Returns None (rather than raising) when offline with nothing cached, so tests can `skip` cleanly.

Parameters

- **arch** (*str*)
- **version** (*str | None*)

Return type
str | None

21.1.2 Submodules

penguin.abi_info module

penguin.abi_info.**arch_abi_info**(*arch*)

Look up ABI info for a config arch name or any accepted alias.

Normalizes via the arch registry so callers may pass e.g. `intel64` or `x86_64` (both resolve to the canonical `x86_64` entry).

Parameters
arch (*str*)

Return type
dict

penguin.analyses module

class penguin.analyses.**PenguinAnalysis**(*debug=False*)

Bases: `ABC`

Parameters
debug (*bool*)

ANALYSIS_TYPE = None

abstractmethod get_potential_mitigations(*config*, *failure*)

Given a configuration and the global state, what potential mitigations could be deployed?

Parameters

- **config** (Configuration)
- **failure** (Failure)

Return type
List[Mitigation]

abstractmethod implement_mitigation(*config*, *failure*, *mitigation*)

Given a configuration a fail cause and a mitigation, return a new configuration with the mitigation applied.

Parameters

- **config** (Configuration)
- **failure** (Failure)
- **mitigation** (Mitigation)

Return type
Configuration

abstractmethod parse_failures(*output_dir*)

Given a run's output directory, parse the failures as reported by our corresponding PyPlugin and return as a list

Return type
List[Failure]

penguin.arch module

class penguin.arch.**ArchInfo**(*arch: str | None = None*, *abi: str = 'default'*, *bits: int | None = None*, *endianness: str | None = None*, *description: str | None = None*)

Bases: object

Parameters

- **arch** (*str | None*)
- **abi** (*str*)
- **bits** (*int | None*)
- **endianness** (*str | None*)
- **description** (*str | None*)

abi: str = 'default'

arch: str | None = None

bits: int | None = None

description: str | None = None

endianness: str | None = None

penguin.arch.**arch_end**(*value*)

penguin.arch.**arch_filter**(*elf*)

penguin.arch.**get_dylib_subdir**(*arch_name*)

Map a config arch name (or alias) to the subdirectory under <static>/dylibs holding that arch's prebuilt dynamic libraries.

Thin wrapper over the arch registry (the single source of truth), kept for backward compatibility with existing callers / the {{ dylib_dir }} template variable.

Parameters
arch_name (*str*)

Return type
str

penguin.arch_registry module

penguin.arch_registry

Single source of truth for architecture naming in Penguin.

Historically arch names were spelled differently in a half-dozen tables (`q_config.qemu_configs`, `utils.get_arch_subdir`, `arch.get_dylib_subdir`, `dropin_compile.DYLIB_DIRS`, `abi_info.ARCH_ABI_INFO`, `qemu_compat._normalize_arch_name`), with drift and bugs between them. This module defines one *ArchSpec* per architecture holding every per-namespace name, plus alias handling so a user may write any common spelling.

The canonical x86-64 name is `x86_64` (`intel64` is an accepted alias), matching the on-disk asset layout (dylibs/x86_64, kernels vmlinux.x86_64, ...).

This module is intentionally **dependency-free** (stdlib only) so that `utils`, `arch`, `q_config`, `dropin_compile`, `abi_info` and the config `templating` module can all import it without import cycles or heavy dependencies. It must NOT touch the filesystem — the canonical/alias on-disk fallback resolver lives in `utils.resolve_arch_asset`.

class penguin.arch_registry.**ArchSpec**(*canonical*, *aliases=()*, *arch_subdir=''*, *dylib_subdir=''*, *_kmod_subdir=None*, *qemu_arch=None*, *qemu_machine=None*, *cpu=None*, *_kconf_group=None*, *kernel_fmt='vmlinux'*, *_kernel_whole=None*, *serial=(204, 65)*, *console_replacement=None*, *command_line_size=4096*, *endianness='little'*, *_abi_key=None*)

Bases: `object`

All naming/config facts about one architecture.

Parameters

- **canonical** (*str*)
- **aliases** (*Tuple[str, ...]*)
- **arch_subdir** (*str*)
- **dylib_subdir** (*str*)
- **_kmod_subdir** (*str | None*)
- **qemu_arch** (*str | None*)
- **qemu_machine** (*str | None*)
- **cpu** (*str | None*)
- **_kconf_group** (*str | None*)
- **kernel_fmt** (*str*)
- **_kernel_whole** (*str | None*)
- **serial** (*Tuple[int, int]*)
- **console_replacement** (*Tuple[str, str] | None*)
- **command_line_size** (*int*)
- **endianness** (*str*)

- **_abi_key** (*str | None*)

property abi_key: str

aliases: Tuple[str, ...] = ()

arch_subdir: str = ''

canonical: str

command_line_size: int = 4096

console_replacement: Tuple[str, str] | None = None

cpu: str | None = None

dylib_subdir: str = ''

endianness: str = 'little'

property kconf_group: str

kernel_fmt: str = 'vmlinux'

property kernel_whole: str

property kmod_subdir: str

property panda_arch: str

qemu_arch: str | None = None

qemu_machine: str | None = None

serial: Tuple[int, int] = (204, 65)

penguin.arch_registry.**all_names**()

Canonical names + every alias — exactly the set the schema Literal accepts.

Return type
list

penguin.arch_registry.**arch_subdir**(*name*)

Parameters
name (*str*)

Return type
str

penguin.arch_registry.**canonical_names**()

Just the canonical names, in registry order.

Return type
list

penguin.arch_registry.**dylib_subdir**(*name*)

Parameters
name (*str*)

Return type
str

penguin.arch_registry.**is_known**(*name*)

Parameters
name (*str*)

Return type
bool

penguin.arch_registry.**kmod_subdir**(*name*)

Parameters
name (*str*)

Return type
str

penguin.arch_registry.**normalize_arch**(*name*)

Map any accepted spelling to its canonical config arch name.

Parameters
name (*str*)

Return type
str

penguin.arch_registry.**qemu_arch**(*name*)

Parameters
name (*str*)

Return type
str

penguin.arch_registry.**spec**(*name*)

Return the *ArchSpec* for a canonical name or any accepted alias.

Parameters
name (*str*)

Return type
ArchSpec

penguin.boot_env module

penguin.boot_env

Partitioning and delivery of penguin's boot-time guest environment.

Most of Penguin's internal env knobs (`ROOT_SHELL`, `igloo_init`, the `IGLOO_*` family, ...) are consumed only by guest *userspace* – the `source.d` boot scripts, `/igloo/init`, and `LD_PRELOAD` constructors – and only after `igloo.ko` has been inserted. So rather than smuggling them on the kernel command line (which is capped at `COMMAND_LINE_SIZE` – as low as 256B on MIPS, where it silently truncates), we deliver them in an early-boot env blob that `preinit.sh` sources into PID1's environment, whence they propagate to the whole guest process tree exactly as cmdline env did.

A few knobs are the exception: they are read by the *kernel* itself at boot, before any userspace runs, so they MUST reach `/proc/cmdline` and cannot be diverted to the blob (which is fetched over the portal and sourced only after `insmod`). These are listed in `KERNEL_CMDLINE_KNOBS` and always stay on the cmdline. `igloo_task_size` (the user/kernel VM-split boundary the igloo kernel parses to size the guest address space) is one such knob – diverting it silently gave targets that rely on it the wrong VM split.

The kernel command line then carries only what must be there: `root=`, `init=`, `panic=`, `console=`, `rw`, plus any *firmware-expected* params – user/config `env:` entries that a vendor init may read from `/proc/cmdline` (penguin can't know which, so it leaves all non-internal env on the cmdline).

This module is intentionally dependency-light (stdlib only) so both the host runner (*penguin.penguin_run*) and the boot-time file server (`pyplugins/core/live_image.py`) can import it without import cycles.

penguin.boot_env.**is_penguin_internal_env**(*key*)

Return True if `key` is a penguin-internal knob delivered via the boot env blob (off the kernel cmdline).

Matches the `igloo_`/`IGLOO_` namespace plus the small set of historically un-prefixed knobs in `PENGUIN_INTERNAL_ENV` – EXCEPT the kernel-parsed knobs in `KERNEL_CMDLINE_KNOBS`, which are read by the kernel at boot and must stay on the cmdline.

Parameters
key (*str*)

Return type
bool

penguin.boot_env.**partition_boot_env**(*env*)

Split `conf['env']` into `(cmdline_env, blob_env)`.

- `blob_env` – penguin-internal knobs, delivered early in boot off the cmdline (see *render_env_blob()*).
- `cmdline_env` – everything else (firmware-expected / user params), which stays on the kernel command line.

Insertion order is preserved within each bucket.

Parameters
env (*dict*)

Return type
tuple[dict, dict]

penguin.boot_env.**render_env_blob**(*blob_env*)

Render `blob_env` as a POSIX-sh snippet of `export K=V` lines for `preinit.sh` to source.

Values are shell-quoted. Valueless (`None`) knobs are skipped, matching the kernel's old behavior where a bare `key` cmdline token (no =) never became an environment variable.

Parameters
blob_env (*dict*)

Return type
str

penguin.common module

class penguin.common.**HexInt**

Bases: `int`

An int that dumps as hex (`0x...`) — for addresses, masks, etc.

class penguin.common.**OctInt**

Bases: `int`

An int that dumps as octal (`0o...`) — for file permission modes.

class penguin.common.**PathHighlightingFormatter**(*fmt=None*, *datefmt=None*, *style='%'*, *level_styles=None*, *field_styles=None*)

Bases: ColoredFormatter

format(*record*)

Apply level-specific styling to log records.

Parameters
record – A LogRecord object.

Returns
The result of logging.Formatter.format().

This method injects ANSI escape sequences that are specific to the level of each log record (because such logic cannot be expressed in the syntax of a log format string). It works by making a copy of the log record, changing the *msg* field inside the copy and passing the copy into the format() method of the base class.

penguin.common.**dict_to_frozenset**(*d*)

penguin.common.**frozenset_to_dict**(*fs*)

penguin.common.**getColoredLogger**(*name*)

Get or create a coloredlogger at INFO.

penguin.common.**get_inits_from_proj**(*proj_dir*)

Given a project directory, return the statically-identified init candidates from static/InitFinder.yaml (a list, shortest path first).

Returns an empty list when the file is absent or empty – callers do *len(options)* / *options[0]*, so never None.

penguin.common.**hash_yaml**(*section_to_hash*)

penguin.common.**literal_presenter**(*dumper*, *data*)

penguin.common.**patch_config**(*logger*, *base_config*, *patch*, *patch_name='patch'*, *origin_map=None*, *verbose=False*)

penguin.common.**redirect_logs_to**(*handler*)

Replace StreamHandlers with *handler* for all current and future penguin/plugins/config loggers.

Parameters
handler (*Handler*)

Return type
None

penguin.common.**style_config_for_dump**(*obj*, *_key=None*)

Return a copy of a config with ints wrapped for readable YAML output: permission (mode) fields as octal, large/address-like ints as hex, and everything else left as plain base-10 ints. Booleans are passed through untouched (bool is an int subclass but must stay true/false).

penguin.compose module

penguin.compose — multi-system firmware rehosting orchestration.

Reads a compose.yaml that describes multiple devices and the virtual networks connecting them, then starts all devices in parallel. Devices communicate via QEMU's socket/mcast backend: each compose network is a UDP multicast group that provides a true L2 broadcast domain with no host privileges required.

class penguin.compose.**ComposeConfig**(*version: int*, *networks: dict[str,* penguin.compose.NetworkSpec*]*, *devices: dict[str,* penguin.compose.DeviceConfig*]*, *output_base_dir: str | None = None*)

Bases: object

Parameters

- **version** (*int*)
- **networks** (*dict[str,* NetworkSpec*]*)
- **devices** (*dict[str,* DeviceConfig*]*)
- **output_base_dir** (*str | None*)

devices: dict[str, *DeviceConfig*]

networks: dict[str, *NetworkSpec*]

output_base_dir: str | None = None

version: int

class penguin.compose.**DeviceConfig**(*name: str*, *proj_dir: str*, *config_path: str*, *networks: list[*penguin.compose.DeviceNetAttachment*]* = <*factory*>, *config_overrides: dict* = <*factory*>)

Bases: object

Parameters

- **name** (*str*)
- **proj_dir** (*str*)
- **config_path** (*str*)
- **networks** (*list[*DeviceNetAttachment*]*)
- **config_overrides** (*dict*)

config_overrides: dict

config_path: str

name: str

networks: list[*DeviceNetAttachment*]

proj_dir: str

class penguin.compose.**DeviceNetAttachment**(*network_name: str*, *iface: str*, *ip: str | None = None*, *mac: str | None = None*)

Bases: object

Parameters

- **network_name** (*str*)
- **iface** (*str*)
- **ip** (*str | None*)
- **mac** (*str | None*)

iface: str

ip: str | None = None

mac: str | None = None

network_name: str

class penguin.compose.**NetworkSpec**(*name: str*, *subnet: str | None = None*, *port: int = 0*)

Bases: object

Parameters

- **name** (*str*)
- **subnet** (*str | None*)
- **port** (*int*)

name: str

port: int = 0

subnet: str | None = None

class penguin.compose.**RuntimeEndpointSpec**(*telnet_port_base: int*, *telnet_port_count: int*, *vsock_cid: int*)

Bases: object

Parameters

- **telnet_port_base** (*int*)
- **telnet_port_count** (*int*)
- **vsock_cid** (*int*)

telnet_port_base: int

telnet_port_count: int

vsock_cid: int

penguin.compose.**load_compose**(*compose_path*)

Parse and validate a compose.yaml. Raises ValueError on bad input.

Parameters
compose_path (*str*)

Return type
ComposeConfig

penguin.compose.**run_compose**(*compose_path*, *output_dir*, *timeout*, *force*, *verbose*)

Parse compose.yaml, start all devices in parallel, and collect results.

Parameters

- **compose_path** (*str*)
- **output_dir** (*str | None*)
- **timeout** (*int | None*)
- **force** (*bool*)
- **verbose** (*bool*)

Return type
None

penguin.compose.**scaffold_compose**(*device_projects*, *name=None*)

Generate a fresh compose.yaml from a list of project directories.

Each entry of `device_projects` must be a path to a directory containing a `config.yaml`. The scaffold creates a single `lan` network on `192.168.1.0/24` and assigns each device a sequential IP starting at `192.168.1.1/24`.

The scaffold directory is `<parent-of-first-proj-parent>/compose_projects/<name>/`. If `name` is omitted it defaults to the device basenames joined with `_` (e.g. projects `foo` + `bar` → `foo_bar`). Refuses if the directory already exists. Returns the absolute path to the new `compose.yaml`.

Parameters

- **device_projects** (*list[str]*)
- **name** (*str | None*)

Return type
str

penguin.config_patchers module

penguin.config_patchers

Patch-generation helpers (tar archives, file searches, NVRAM parsing, binary shims) shared by penguin core and the init plugins in pyplugins/init/ (the patch-generating classes themselves live there).

class penguin.config_patchers.**FileHelper**

Bases: `object`

static exists(*tmp_dir*, *target*)

Check if the target exists within the extracted filesystem in tmp_dir, handling symlinks correctly.

Parameters

- **tmp_dir** (*str*) – The root of the extracted filesystem (e.g., /tmp/extracted)
- **target** (*str*) – The target path to check (e.g., /foo/zoo)

Returns
True if the target exists within tmp_dir, False otherwise

Return type
bool

static find_executables(*tmp_dir*, *target_dirs=None*, *index=None*)

Executable files outside /igloo whose path ends with one of target_dirs. Pass an InitContext.file_index to avoid re-walking.

Parameters

- **tmp_dir** (*str*)
- **target_dirs** (*set[str] | None*)

static find_shell_scripts(*tmp_dir*, *index=None*)

Executable *.sh files outside /igloo. Pass an InitContext.file_index to avoid re-walking.

Parameters
tmp_dir (*str*)

static find_strings_in_file(*file_path*, *pattern*)

Parameters

- **file_path** (*str*)
- **pattern** (*str*)

Return type
list[str]

class penguin.config_patchers.**NvramHelper**

Bases: object

static nvram_config_analysis(*fs_path*, *full_path=True*, *index=None*)

Parameters

- **fs_path** (*str*)
- **full_path** (*bool*)

Return type
dict[str, str]

static parse_nvram_file(*path*, *f*)

Parse a NVRAM file and return key-value pairs.

Parameters

- **path** (*str*) – Path to NVRAM file.
- **f** – File object.

Returns
Dictionary of key-value pairs.

Return type
dict

class penguin.config_patchers.**TarHelper**

Bases: object

Collection of static method to help find files in a tar archive

static get_all_members(*tarfile_path*)

Parameters
tarfile_path (*str*)

static get_directory_members(*tarfile_path*)

Parameters
tarfile_path (*str*)

Return type
set[str]

static get_file_members(*tarfile_path*)

Parameters
tarfile_path (*str*)

Return type
set[str]

static get_other_members(*tarfile_path*)

Parameters
tarfile_path (*str*)

static get_symlink_members(*tarfile_path*)

Parameters
tarfile_path (*str*)

Return type
dict[str, str]

penguin.config_patchers.**make_shims**(*files*, *shim_targets*)

Identify binaries in the guest FS that we want to shim and add symlinks to go from guest bin -> igloo bin into our config.

Parameters
shim_targets (*dict[str, str]*)

Return type
dict

penguin.defaults module

penguin.defaults

Default values and configuration templates for the Penguin emulation environment.

This module provides default scripts, plugin settings, device models, and library injection mappings used throughout Penguin.

penguin.dropin_compile module

penguin.dropin_compile.**compile_init_c_dropin**(*proj_dir*, *init_dir*, *source_path*, *config*)

Compile an init.d/*.c drop-in and return its project-local binary path.

Parameters

- **proj_dir** (*str*)
- **init_dir** (*str*)
- **source_path** (*str*)
- **config** (*Dict[str, Any]*)

Return type
str

penguin.gen_config module

penguin.gen_config

Configuration generation utilities for the Penguin emulation environment.

This module provides functions and classes for analyzing extracted filesystems, running static analyses, generating configuration files and patches, and handling config creation via CLI or programmatically.

class penguin.gen_config.**ConfigBuilder**(*fs_archive*, *output_dir*, *jobs=None*, *init_plugin_dirs=()*, *enable=()*, *disable=()*)

Bases: object

Given a filesystem and an output directory, create a configuration and initialize the output directory as necessary.

Parameters

- **fs_archive** (*str*)
- **output_dir** (*str | Path*)
- **jobs** (*int | None*)

PATCH_DIR: str = 'static_patches'

generate_initial_config(*patches*, *init_plugins=None*)

Generate the initial configuration dictionary.

Parameters

- **patches** (*dict*) – Dictionary of patches.
- **init_plugins** (*dict | None*) – Init plugin record for the init_plugins section.

Returns
Initial configuration dictionary.

Return type
dict

penguin.gen_config.**extract_fs_archive**(*archive*, *dest*)

Extract a root filesystem archive and normalize permissions so the current (fakeroot) user can read everything.

Parameters

- **archive** (*str | Path*)
- **dest** (*str | Path*)

Return type
None

penguin.gen_config.**fakeroot_gen_config**(*fs*, *out*, *artifacts_dir*, *verbose*, *jobs=None*, *init_plugin_dirs=()*, *enable=()*, *disable=()*)

Run config generation under fakeroot.

Parameters

- **fs** (*str*) – Path to filesystem archive.
- **out** (*str*) – Path to output config file.
- **artifacts_dir** (*str*) – Path to artifacts directory.
- **verbose** (*int*) – Verbosity level.
- **jobs** (*int | None*) – Init plugin thread pool size.
- **init_plugin_dirs** – Extra directories to search for init plugins.
- **enable** – Init plugin names to force-enable.
- **disable** – Init plugin names to skip entirely.

Returns
Path to generated config file or None.

Return type
str or None

penguin.gen_config.**fakeroot_refresh**(*proj_dir*, *verbose=0*, *jobs=None*, *init_plugin_dirs=()*, *enable=()*, *disable=()*, *only=()*, *update_config=True*)

Run *gen_config refresh* under fakeroot (always in a fresh subprocess so the plugin-manager singleton never collides with a running emulation).

Returns
True if the refresh subprocess succeeded.

Parameters

- **proj_dir** (*str*)
- **verbose** (*int*)
- **jobs** (*int | None*)
- **update_config** (*bool*)

Return type
bool

penguin.gen_config.**init_plugin_search_dirs**(*proj_dir*, *extra_dirs=()*)

Directories scanned for init plugins, lowest to highest precedence: built-ins under <plugin_path>/init, then the project's plugins.d/, then any explicitly passed directories.

Parameters
proj_dir (*str | Path*)

Return type
list

penguin.gen_config.**initialize_and_build_config**(*fs*, *out=None*, *artifacts_dir=None*, *jobs=None*, *init_plugin_dirs=()*, *enable=()*, *disable=()*)

Given a filesystem as a .tar.gz analyze it to create a configuration file. Out is the path to the config to produce. Use artifacts_dir as scratch space while analyzing.

Parameters

- **fs** (*str*) – Path to filesystem archive (.tar.gz).
- **out** (*str or None*) – Path to output config file.
- **artifacts_dir** (*str or None*) – Path to artifacts directory.
- **jobs** (*int | None*) – Init plugin thread pool size (default: cpu count).
- **init_plugin_dirs** – Extra directories to search for init plugins.
- **enable** – Init plugin names to force-enable (their patches join the config's patches list even if disabled by default).
- **disable** – Init plugin names to skip entirely.

Returns
Path to generated config file.

Return type
str

Raises

- **RuntimeError** – If firmware file is not found.

- **ValueError** – If input file is not a .tar.gz archive.

penguin.gen_config.**refresh_project**(*proj_dir*, *jobs=None*, *init_plugin_dirs=()*, *enable=()*, *disable=()*, *only=()*, *update_config=True*)

Re-run init plugins inside an existing project, refreshing static/ and static_patches/ and (optionally) reconciling the config's patches list.

The user's config.yaml is never rewritten wholesale: only the *patches:* and *init_plugins:* sections are updated, after backing the file up to config.yaml.bak. Patch files the user edited since generation are preserved (new content lands in <name>.yaml.new).

The config's init_plugins: section is the source of truth for which plugins run; entries with enabled=false are skipped. CLI –enable/–disable override per-invocation, and –only restricts execution to the named plugins (everything else keeps its current outputs; shared analyses are still computed on demand).

Returns
True on success.

Parameters

- **proj_dir** (*str*)
- **jobs** (*int | None*)
- **update_config** (*bool*)

Return type
bool

penguin.gen_image module

penguin.gen_image

Filesystem image generation utilities for the Penguin emulation environment.

This module provides functions for generating disk images, adding required files, and handling image creation via CLI or programmatically.

penguin.gen_image.**fakeroot_gen_image**(*fs*, *out*, *artifacts*, *proj_dir*, *config*)

Run image generation under fakeroot.

Parameters

- **fs** (*str*) – Path to input filesystem tarball.
- **out** (*str*) – Path to output qcow image.
- **artifacts** (*str*) – Path to artifacts directory.
- **proj_dir** (*str*) – Path to project directory.
- **config** (*str*) – Path to config file.

Returns
Path to generated image.

Return type
str

Raises
Exception – If image generation fails.

penguin.gen_image.**get_mount_type**(*path*)

Get the filesystem type of the mount at the given path.

Parameters
path (*str*) – Path to check.

Returns
Filesystem type string or None.

Return type
str or None

penguin.gen_image.**make_image**(*fs*, *out*, *artifacts*, *config*)

Generate a new disk image from the given filesystem and configuration.

Parameters

- **fs** (*str*) – Path to input filesystem tarball.
- **out** (*str*) – Path to output qcow image.
- **artifacts** (*str or None*) – Path to artifacts directory.
- **config** (*dict*) – Configuration dictionary.

Return type
None

penguin.gen_image.**tar_add_min_files**(*tf_path*, *config*)

Add minimal required files to a tar archive for the image.

Parameters

- **tf_path** (*str*) – Path to the tar file.
- **config** (*dict*) – Configuration dictionary.

Return type
None

penguin.gen_image.**tar_add_tool_closure**(*tf_path*, *config*)

Bake the per-arch debugging-tool runtime closure into the base image.

The closure (pristine nixpkgs glibc python3/strace/gdbserver/ltrace/iptables) is large (~150-300MB, ~8.4k files). It is image-level, content-stable data, so we bake it into the cached base qcow2 once instead of re-shipping it into every guest via live_image on every boot. Its store paths land at /igloo/nix/store/...; the per-tool /igloo/utils/<tool> wrappers (still staged per-run by live_image) bind-mount /igloo/nix onto /nix and exec the in-store binary, so the wrappers work unchanged against the baked store.

The base qcow2 is cache-keyed by hash_image_inputs, which folds in the closure manifest hash, so a closure change rebuilds the image while configs and explore iterations reuse it. Absent closure (older images / arches without one) is skipped gracefully.

Parameters

- **tf_path** (*str*) – Path to the (uncompressed) tar archive to append into.
- **config** (*dict*) – Configuration dictionary.

Return type
None

penguin.graphs module

class penguin.graphs.**Configuration**(*name*, *info=None*, *exclusive=None*)

Bases: *GraphNode*

class penguin.graphs.**ConfigurationGraph**(*base_config*)

Bases: object

Parameters
base_config (Configuration)

add_dependencies(*parent_config*, *child_config*)

Parameters

- **parent_config** (Configuration)
- **child_config** (Configuration)

add_derived_configuration(*derived_config*, *parent_config*, *mitigation*)

Add a new configuration derived from a specific mitigation and parent configuration.

Parameters

- **derived_config** (GraphNode)
- **parent_config** (GraphNode)
- **mitigation** (GraphNode)

add_edge(*from_node*, *to_node*, *weight=1.0*, *unknown=False*, *delta=None*)

Parameters

- **from_node** (GraphNode)
- **to_node** (GraphNode)
- **weight** (*float*)
- **unknown** (*bool*)
- **delta** (*str*)

add_node(*node*)

Parameters
node (GraphNode)

calculate_expected_config_health(*cc*)

Given a config, calculate the expected health score of that config. Note this ignores any actual health score we've seen for this config.

Return type
float

create_config_png(*file_path*)

Parameters
file_path (*str*)

create_html(*file_path*)

Create an interactive html page of the graph

Args:
file_path (str): The file path where the HTML image will be saved.

Parameters
file_path (*str*)

create_png(*file_path*)

Create a PNG image of the graph with enhanced visual features.

Args:
file_path (str): The file path where the PNG image will be saved.

Parameters
file_path (*str*)

static determine_edge_type(*from_node*, *to_node*)

Parameters

- **from_node** (GraphNode)
- **to_node** (GraphNode)

static find_delta(*derived*, *parent*, *prefix=''*)

Given two dicts, create a string representation of the difference between them.

find_unexplored_configurations(*exclude=None*, *potential=None*)

Find all configurations that have not been run yet.

If exclude is provided, we'll exclude those configurations from the search. If parent is set, we'll only consider direct descendants of that parent.

Returns:
list: A list of configuration IDs that have not been linked to any failures or mitigations.

Parameters

- **exclude** (*Set[*Configuration*]*)
- **potential** (*List[*Configuration*] | None*)

Return type
List[Configuration]

get_all_configurations()

Get all configurations in our graph

Return type
List[Configuration]

get_best_run_configuration()

Find the configuration with the highest health score - not estimated, actual.

Return type
Configuration | None

get_child_configs(*config*)

Parameters
config (Configuration)

Return type
List[Configuration]

`get_config_depth`(*config*)

Given a config, find its depth in the graph

Parameters
config (Configuration)

Return type
int

`get_existing_node`(*new_node*)

Parameters
new_node (GraphNode)

Return type
GraphNode | None

`get_existing_node_or_self`(*new_node*)

Given a new node, return the existing node in the graph if it exists. Otherwise return the new node. Check with hashing of node object

Parameters
new_node (GraphNode)

Return type
GraphNode

`get_node`(*node_id*)

Get a node from the graph.

Args:
node_id (str): The ID of the node to retrieve.

Returns:
GraphNode: The node with the given ID.

Parameters
node_id (*UUID*)

Return type
GraphNode

`get_parent_config`(*config*)

Given a config, find its parent config. Returns None if it's the root config

Parameters
config (Configuration)

Return type
Configuration | None

`get_parent_failure`(*config*)

Given a config, find its parent failure. Returns None if it's the root config

Parameters
config (Configuration)

Return type
Failure | None

get_parent_mitigation(*config*)

Given a config, find its parent mitigation. Returns None if it's the root config

Parameters
config (Configuration)

Return type
Mitigation | None

get_root_config()

Return type
Configuration | None

has_edge(*from_node*, *to_node*)

Parameters

- **from_node** (GraphNode)
- **to_node** (GraphNode)

has_node(*node*)

Parameters
node (GraphNode)

mitigations_for(*failure*)

Given a failure, return a list of mitigations that could be applied

node_has_predecessor(*node*)

Check if a given node has a predecessor

Parameters
node (GraphNode)

report_config_run(*config*, *health_score*)

After we've run a configuration we have its health score.

For all but the root config, we'll have a chain: parent config -> failure -> mitigation -> this config and there's also an edge from parent config -> this config We make two updates for weight: at parent config -> this config we directly set the weight as this edge is only considered once. Then at the failure->mitigation edge we add the weight to a list of weights, and update the weight to be the average of the list. This is because mitigations are tested multiple times.

The goal is to tune our weights such that 1) From the parent config, we'll be able to select child configs with high health scores 2) From the failure we'll be able to select mitigations with high health scores

Parameters

- **config** (Configuration)
- **health_score** (*float*)

save_graph(*file_path*)

Save the graph to a file using pickle.

Args:
file_path (str): The file path where the graph should be saved.

Parameters
file_path (*str*)

set_cc_edge_weight(*from_node*, *to_node*, *new_weight*)

Set the weight of an edge between two configurations in the graph.

Args:
from_node (str): The starting node of the edge. to_node (str): The ending node of the edge. new_weight (float): The new weight to assign to the edge.

Parameters

- **from_node** (Configuration)
- **to_node** (Configuration)
- **new_weight** (*float*)

update_parent_fail_mit_weight(*config*, *health_score*)

Parameters

- **config** (Configuration)
- **health_score** (*float*)

class penguin.graphs.ConfigurationManager(*base_config*)

Bases: object

Parameters
base_config (Configuration)

calculate_config_depth(*cc*)

run_configuration(*config*, *weight*, *run_config_f*, *find_mitigations_f*, *find_new_configs_f*, *logger=None*)

Run a given configuration to get a list of failure and a health score. Update the graph with the new information to set weights Add new failures and mitigations to the graph

Parameters

- **config** (Configuration)
- **weight** (*float*)
- **run_config_f** (*Callable[[*Configuration*], Tuple[List[*Failure*], float]]*)
- **find_mitigations_f** (*Callable[[*Failure, Configuration*], List[*Mitigation*]]*)
- **find_new_configs_f** (*Callable[[*Failure, Mitigation, Configuration*], List[*Configuration*]]*)
- **logger** (*Callable[[str], None] | None*)

run_exploration_cycle(*run_config_f*, *find_mitigations_f*, *find_new_configs_f*, *logger=None*)

Get the best config and run it. Hold lock while selecting. While we're running, ensure config is in self.pending_runs

Parameters

- **run_config_f** (*Callable[[*Configuration*], Tuple[List[*Failure*], float]]*)
- **find_mitigations_f** (*Callable[[*Failure, Configuration*], List[*Mitigation*]]*)
- **find_new_configs_f** (*Callable[[*Failure, Mitigation, Configuration*], List[*Configuration*]]*)

- **logger**(*Callable[[str], None] | None*)

select_best_config()

First try finding an un-run+non-pending config that's derived from a mitigation we've never run before. Prioritize by expected health score.

Priority init > pseudofile > netdev > signals > env vars

If we've run every mitigation, just select the best config based on expected health score.

Return type
Tuple[Configuration | None, float]

select_best_config_orig()

— DEPRECATED —

Select the best configuration to run next. Node can't have been run before

Just return the first unexplored config for now

For each un-run node, we look at its parent config and parent mitigation. We can calculate an expected weight based on these two as: expected_weight = parent_config_weight + parent_mitigation_weight

We support biasing this calculation when we get results that are better than expected from equally-likely-to-be-good mitigations. E.g., if we add two inits and just run one, we bias the health score of the other to be comprably weighed instead of falling behind and never getting run.

We do this for inits, dynamically-discovered env vars, and ioctl models.

XXX: We assume these nodes all show up at the same time - i.e., after an exclusive nodes runs. If that changes later, we'll need to better track these

Call with self.lock held!

Return type
Tuple[Configuration | None, float]

stringify_state()

Return a string representation of the current graph.

Organize configs by run idx. Report deltas from parent Report score/pending/estimated for each config

stringify_state2()

Return a string representation of the current graph. Each node should have a line. Under each node we'll indent and list out edges

class penguin.graphs.**Failure**(*name*, *type*, *info=None*, *patch_name=None*)

Bases: *GraphNode*

to_dict()

class penguin.graphs.**GraphNode**(*name*)

Bases: object

Base class for all graph nodes

to_dict()

class penguin.graphs.**Mitigation**(*name*, *type*, *info=None*, *exclusive=None*, *patch=None*, *failure_name=None*)

Bases: *GraphNode*

penguin.graphs.**get_global_mitigation_weight**(*mitigation_type*)

Global hyperparameter for how much we should prioritize a given mitigation type. Intuition is that we should check all inits.

With new approach of searching across a generation, we'll always cover the whole init generation. So now we can leave these all equal?

TODO: it seems like pseudofiles are generally more important than env variables or blocking signals. Perhaps our search should be more of a:

If any untested pseudofile failure mitigations - select If any untested env failure mitigations - select (dynval, then apply) If any untested signal failure mitigations - select

Then select highest estimated scores of remaining un-run nodes.

This would ensure we create devices (straightforward and often good), before we go into expensive dynval tests that infrequently work

Parameters
mitigation_type (*str*)

Return type
float

penguin.graphs.**run_test**()

Use stubs to simulate running configs, identifying failures, finding mitigations, and applying them.

penguin.init_plugin module

penguin.init_plugin

Base class and helpers for init-time (config generation) plugins.

Init plugins run during `penguin init` (and `penguin refresh`), before any emulation exists, inside the fakeroot config-generation subprocess. They are ordinary pyplugins (subclasses of *penguin.plugin_manager.Plugin*) with `panda=None`, so they get the plugin manager, logging, args, and inter-plugin access (`self.plugins.X`) for free.

An init plugin can do either or both of:

- expose shared analysis results as *cached_analysis()* methods, computed lazily and exactly once no matter how many other plugins (or threads) access them, e.g. `self.plugins.ArchId.arch`;
- produce a config patch by setting `patch_name` and implementing *InitPlugin.patch()*, rendered to `<proj>/static_patches/<patch_name>.yaml`.

Plugins run concurrently. There is no declared dependency graph: a plugin that touches another plugin's cached analysis simply blocks until it is computed. Circular accesses raise `InitAnalysisCycleError` instead of deadlocking.

class penguin.init_plugin.**FileEntry**(*path*, *rel*, *name*, *is_file*, *is_symlink*, *executable*)

Bases: `NamedTuple`

One file from the extracted filesystem, as seen by os.walk.

Parameters

- **path** (*str*)
- **rel** (*str*)
- **name** (*str*)
- **is_file** (*bool*)
- **is_symlink** (*bool*)

- **executable** (*bool*)

executable: bool

Alias for field number 5

is_file: bool

Alias for field number 3

is_symlink: bool

Alias for field number 4

name: str

Alias for field number 2

path: str

Alias for field number 0

rel: str

Alias for field number 1

class penguin.init_plugin.**FileIndex**(*root*)

Bases: object

A single os.walk pass over the extracted filesystem, shared by all init plugins via ctx.file_index so each doesn't re-walk (and re-stat) the whole tree. Semantics match the walks it replaces: only file entries (no directories), symlinks not followed into.

Parameters
root (*str | Path*)

executables()

files()

Regular files (symlink targets resolved).

exception penguin.init_plugin.**InitAnalysisCycleError**

Bases: RuntimeError

Raised when cached analyses circularly depend on each other.

class penguin.init_plugin.**InitContext**(*fs_archive*, *extracted_fs*, *proj_dir*, *static_dir*, *patch_dir*, *options=None*)

Bases: object

Shared, read-mostly context handed to every init plugin.

Variables

- **fs_archive** – path to the project's base/fs.tar.gz
- **extracted_fs** – directory the filesystem is extracted into (temporary, removed after the run)
- **proj_dir** – the project directory results are written into
- **static_dir** – proj_dir/static (analysis results)
- **patch_dir** – proj_dir/static_patches (rendered patches)
- **options** – CLI/runner options (free-form dict)

Parameters

- **fs_archive** (*str | Path*)
- **extracted_fs** (*str | Path*)
- **proj_dir** (*str | Path*)
- **static_dir** (*str | Path*)
- **patch_dir** (*str | Path*)
- **options** (*Dict[str, Any]*)

property archive_files: list

All tar members of `fs_archive`, read once and shared.

property file_index: *FileIndex*

A shared one-pass *FileIndex* over `extracted_fs`.

options: Dict[str, Any]

patches_snapshot()

The accumulated `{patch_name: (patch_dict, enabled)}` map.

Only available to plugins with `consumes_patches = True` (they run in a post phase after all other patches are generated).

Return type
Dict[str, tuple]

class penguin.init_plugin.**InitPlugin**

Bases: *Plugin*

Base class for init-time plugins.

Class attributes:

- `patch_name`: set to produce a patch file (`static_patches/<patch_name>.yaml`).
- `order`: position of this plugin's patch in the generated config's `patches:` list. Later patches override earlier ones on conflicting keys. Built-ins use 10..990; the default (1000) renders user patches after all built-ins.
- `consumes_patches`: if True, run in a post phase after all other patches are generated; `ctx.patches_snapshot()` becomes available.
- `serializer`: how *static_result()* is persisted under `static/<ClassName>.*` — `"yaml"` (default) or `"json_xz"`.

No plugin failure stops config generation: an exception is logged, recorded in the manifest, skips this plugin's outputs, and (for a failed cached analysis) fails the plugins that consume it — init carries on with everything else.

Instances may set `self.enabled = False` (e.g. in `__init__` or `patch()`) to generate their patch file but leave it out of the config's `patches:` list.

The runner assigns `self.ctx` (an *InitContext*) to every loaded init plugin before execution starts, so cached analyses can use it too.

consumes_patches: ClassVar[bool] = False

ctx: *InitContext*

enabled: bool = True

order: ClassVar[int] = 1000

patch(*ctx*)

Produce this plugin's config patch as a dict (or None/empty for no patch). Only meaningful if `patch_name` is set.

Parameters
ctx (`InitContext`)

Return type
dict | None

patch_name: ClassVar[str | None] = None

serializer: ClassVar[str] = 'yaml'

static_result()

Analysis result to persist to `static/<ClassName>.yaml` (or `.json.xz`). Return None (default) to persist nothing.

Return type
Any

class penguin.init_plugin.**cached_analysis**(*func*)

Bases: `object`

Descriptor: thread-safe, compute-once property for shared analysis results.

The first accessor computes the value on its own thread; concurrent accessors block until it is ready. A circular access — directly re-entrant or through a chain of plugins across threads — raises *InitAnalysisCycleError* instead of deadlocking. Exceptions are cached and re-raised for every subsequent accessor so a failed analysis fails all of its consumers consistently.

Usage:

```
class ArchId(InitPlugin):
    @cached_analysis
    def arch(self) -> str:
        ...  # expensive, runs once
```

attrname: str | None

penguin.init_runner module

penguin.init_runner

Executes init plugins (see *penguin.init_plugin*) to produce a project's static analysis results and config patches.

Execution model:

- all selected plugin classes are loaded up-front (serially, on the calling thread) through the IGLOO plugin manager with `panda=None`;
- every plugin's work (`patch()` + `static_result()`) is submitted to a thread pool; shared analyses are lazy `cached_analysis` attributes, so cross-plugin data dependencies serialize naturally on first access;
- plugins with `consumes_patches = True` run afterwards, sequentially in `(order, patch_name)` order, with `ctx.patches_snapshot()` available;

- the rendered `patches:` list order is `(order, patch_name)` — stable and independent of execution timing, because later patches override earlier ones when the config is loaded.

Failure model: no individual plugin failure stops config generation. An exception is logged, recorded in `static/manifest.yaml`, and the run continues without that plugin's outputs; a failed cached analysis fails every plugin that consumes it (each recorded the same way). A summary of failures is logged at the end of the run.

class penguin.init_runner.**InitPluginRunner**(*plugin_classes*, *ctx*, *manager=None*, *jobs=None*, *skip=()*)

Bases: `object`

Run a set of init plugin classes against an extracted filesystem.

Parameters

- **plugin_classes** (*List[Type[*InitPlugin*]]*) – InitPlugin subclasses to run.
- **ctx** (InitContext) – shared `InitContext`.
- **manager** – plugin manager to load through (defaults to the process-wide singleton).
- **jobs** (*Optional[int]*) – thread pool size (defaults to `os.cpu_count()`).

load_plugins(*extra_args=None*)

Initialize the manager (panda=None) and load every plugin class. Returns the loaded instances in input order.

Parameters
extra_args (*Dict[str, Any] | None*)

Return type
List[InitPlugin]

merge_previous_manifest(*previous*)

Carry forward manifest entries for plugins that did not run this time (refresh with a skip set).

Parameters
previous (*Dict[str, dict]*)

Return type
None

render_patches(*patches*, *previous_manifest=None*)

Write non-empty patches to `patch_dir/<name>.yaml` (including disabled ones, matching historic behavior).

When `previous_manifest` is given (refresh), a patch file whose on-disk content no longer matches the hash penguin recorded when it generated it is treated as user-edited: it is preserved and the new content is written next to it as `<name>.yaml.new`.

Returns `{"written": [...], "preserved": [...]}` patch names.

Parameters

- **patches** (*Dict[str, tuple]*)
- **previous_manifest** (*Dict[str, dict] | None*)

Return type
Dict[str, list]

run()

Execute all plugins and return `{patch_name: (patch_dict, enabled)}` ordered by `(order, patch_name)`. Persists static results, patch files, and the manifest as a side effect.

Return type
Dict[str, tuple]

penguin.init_runner.**discover_init_plugins**(*search_dirs*, *disable=()*)

Find InitPlugin subclasses in .py files under the given directories (searched recursively, in order — a class found in a later directory shadows a same-named class from an earlier one, so project-local plugins override built-ins).

Files are imported via the introspection path (no live plugin manager needed); only classes *defined* in each file are considered, so imported bases like InitPlugin are ignored. A file that fails to import is skipped with a warning rather than aborting init.

Parameters

- **search_dirs** – directories to scan, lowest to highest precedence.
- **disable** – plugin class names to exclude (case/underscore-insensitive).

Returns
plugin classes sorted by class name.

Return type
List[*Type*[InitPlugin]]

penguin.init_runner.**load_manifest**(*static_dir*)

Read static/manifest.yaml; empty dict if absent or unreadable.

Parameters
static_dir(*str | Path*)

Return type
Dict[str, dict]

penguin.manager module

penguin.manager

Management utilities for running and scoring Penguin emulation experiments.

This module provides functions and classes for calculating experiment scores, handling subprocess execution and timeouts, and managing Panda emulation runs.

Functions

- calculate_score

Classes

- PandaRunner

class penguin.manager.**PandaRunner**

Bases: object

Wrapper class for running penguin_run in a subprocess.

This class manages subprocess execution, timeouts, and signal handling to ensure robust experiment runs even in the presence of crashes or hangs.

catch_and_forward_sigint(*p*)

Install a SIGINT handler that escalates to SIGKILL on repeated Ctrl+C.

Parameters
p (*subprocess.Popen*) – Subprocess to forward signals to.

Return type
None

run(*conf_yaml*, *proj_dir*, *out_dir*, *init=None*, *timeout=None*, *show_output=False*, *verbose=False*, *resolved_kernel=None*, *extra_env=None*)

Run the penguin emulation experiment in a subprocess.

Parameters

- **conf_yaml** (*str*) – Path to configuration YAML file.
- **proj_dir** (*str*) – Project directory.
- **out_dir** (*str*) – Output directory.
- **init** (*str or None*) – Optional init script.
- **timeout** (*int or None*) – Optional timeout in seconds.
- **show_output** (*bool*) – Whether to show output.
- **verbose** (*bool*) – Whether to enable verbose output.
- **resolved_kernel** (*str or None*) – Optional resolved kernel path.
- **extra_env** (*dict | None*)

Raises
RuntimeError – If the run was not successful.

Return type
None

penguin.manager.**calculate_score**(*result_dir*, *have_console=True*)

Return a dict of the distinct metrics we care about name: value.

This function loads experiment results and computes a score based on various health and coverage metrics. It handles missing or corrupt files gracefully by logging warnings and defaulting scores to 0.

Parameters

- **result_dir** (*str*) – Directory containing experiment results.
- **have_console** (*bool*) – Whether console output is available.

Returns
Dictionary of score metrics.

Raises
RuntimeError – If the critical .ran file is missing, indicating a failed run.

Return type
dict[str, int]

penguin.penguin_prep module

penguin.penguin_prep.**add_lib_inject_all_abis**(*conf*)

Add lib_inject for all supported ABIs to /igloo

penguin.penguin_prep.**add_lib_inject_for_abi**(*config*, *abi*)

Compile lib_inject for the ABI and put it in /igloo

penguin.penguin_prep.**prep_config**(*conf*)

penguin.penguin_run module

penguin.penguin_run.**check_cmdline_size**(*cmdline*, *archend*, *logger*)

Guard against silent kernel-cmdline truncation.

The kernel copies at most `COMMAND_LINE_SIZE - 1` bytes of `-append` into `boot_command_line` (one byte reserved for the NUL terminator) and **silently drops the rest** — so an over-long cmdline quietly loses env knobs and produces a baffling rehost. We warn as we approach the cap and raise once we'd exceed it, rather than letting the kernel truncate in silence. See the per-arch `command_line_size` in *penguin.arch_registry*.

Parameters

- **cmdline** (*str*)
- **archend** (*str*)

Return type
None

penguin.penguin_run.**find_free_port**()

penguin.penguin_run.**main**()

penguin.penguin_run.**print_to_log**(*out*, *err*)

penguin.penguin_run.**redirect_stdout_stderr**(*stdout_path*, *stderr_path*)

penguin.penguin_run.**render_kernel_append**(*append_parts*, *env*, *extra_cmdline=''*)

Assemble the kernel `-append` string.

`append_parts` is the original whitespace-split append (e.g. `root=...`, `console=...` that QEMU/penguin already put there); `env` is the full `conf["env"]`; `extra_cmdline` is `core.kernel_cmdline_append` (raw tokens the user wants on the cmdline verbatim).

Only the *firmware-expected* portion of the env reaches the cmdline: *penguin.boot_env.partition_boot_env()* strips penguin's internal knobs (`ROOT_SHELL`, `igloo_init`, `IGLOO_*`, …), which are instead delivered over the portal as an early-boot env blob (see `pyplugins/core/live_image.py` + `preinit.sh`). What remains on the cmdline is user/config `env:` entries that a vendor init may read from `/proc/cmdline`.

Layout is `critical_args + config_args + extra_cmdline + rest_args`:

- critical args (`root=`, `init=`, `panic=1`, any `console=`, `rw`) go first and cannot be clobbered by config;
- then the firmware-expected env;
- then `core.kernel_cmdline_append` tokens, verbatim and never diverted to the env blob (the explicit "put this on the kernel cmdline" channel);
- then whatever else was originally on the append.

Pure / side-effect free so it can be unit-tested and length-checked (see *check_cmdline_size()*) before it is handed to the kernel.

Parameters

- **append_parts** (*list[str]*)
- **env** (*dict*)
- **extra_cmdline** (*str*)

Return type
str

penguin.penguin_run.**run_config**(*proj_dir*, *conf_yaml*, *out_dir=None*, *logger=None*, *init=None*, *timeout=None*, *show_output=False*, *verbose=False*, *resolved_kernel=None*)

conf_yaml a path to our config within proj_dir proj_dir contains config.yaml out_dir stores results and a copy of config.yaml

penguin.plugin_manager module

plugin_manager.py - IGLOO Plugin Manager for Penguin

This module provides the IGLOOPluginManager and Plugin base class for the Penguin emulation environment.

It is responsible for:

- Discovering, loading, and unloading plugin classes.
- Managing plugin lifecycles and dependencies.
- Providing a singleton `plugins` object for global plugin access.
- Registering, subscribing, and publishing plugin events.
- Supporting Penguin Plugin interfaces.
- Providing utility functions for plugin name resolution and file discovery.

Arguments

- `panda`: The active emulator compatibility object.
- `args` (`dict`): Dictionary of arguments and configuration for plugins.

Plugin Interface

Plugins should subclass *`Plugin`* and will be automatically discovered and managed. Plugins can register, subscribe, and publish events using the `plugins` singleton:

```
plugins.register(plugin_instance, "event_name")
plugins.subscribe(other_plugin, "event_name", callback)
plugins.publish(plugin_instance, "event_name", *args, **kwargs)
```

Plugins can be loaded by name or class, and arguments can be passed via the plugin manager.

Overall Purpose

The plugin manager provides a flexible, extensible, and event-driven system for managing plugins in the Penguin emulation environment, enabling modular analysis, automation, and extension of the emulation workflow.

class penguin.plugin_manager.**ArgSpec**(*name*, *type*, *default*, *required*, *description*)

Bases: `tuple`

default

Alias for field number 2

description

Alias for field number 4

name

Alias for field number 0

required

Alias for field number 3

type

Alias for field number 1

class penguin.plugin_manager.**ArgsBox**(*args*)

Bases: object

Parameters
args (*Dict[str, Any]*)

args

get(*key*, *default=None*)

Parameters

- **key** (*str*)
- **default** (*Any*)

Return type
Any

get_bool(*key*, *default=False*)

Get a boolean argument value by name.

Parameters

- **key** (*str*) – The argument name.
- **default** (*bool*) – Default value if the argument is not set.

Returns
The argument value interpreted as a boolean.

Return type
bool

class penguin.plugin_manager.**IGLOOPluginManager**

Bases: object

Singleton class that manages the loading, unloading, and interaction with plugins.

Provides event registration, subscription, publishing, and plugin lifecycle management.

Return type
IGLOOPluginManager

aliases: Dict[str, str]

get_arg(*arg_name*)

Get an argument value by name.

Parameters
arg_name (*str*) – The argument name.

Returns
The argument value or None if not set.

Return type
Any

get_arg_bool(*arg_name*, *default=False*)

Returns True if the argument is set and has a truthy value.

Parameters

- **arg_name** (*bool*) – The name of the argument to retrieve.
- **default** (*bool*) – The default value to return if the argument is not set.

Returns
True if the argument exists and has a truthy value, False otherwise.

Return type
bool

Raises
ValueError – If the argument exists but has an unsupported type.

get_plugin_by_name(*plugin_name*)

Retrieve a loaded plugin by name.

Parameters
plugin_name (*str*) – Name of the plugin.

Returns
The plugin instance if found, else None.

Return type
Plugin or None

initialize(*panda*, *args*)

Initialize the plugin manager with an emulator compatibility object and arguments.

Parameters

- **panda** (*Any*) – Emulator compatibility object exposed to plugins as self.panda.
- **args** (*Dict[str, Any]*) – Dictionary of arguments.

Return type
None

instance = <penguin.plugin_manager.IGLOOPluginManager object>

load(*pluginclasses*, *args=None*)

Load one or more plugin classes.

Parameters

- **pluginclasses** (*Union[Type[T], List[Type[T]], Tuple[str, List[str]]]*) – Plugin class(es) or (file, classnames) tuple.
- **args** (*Dict[str, Any], optional*) – Arguments to pass to the plugins.

Return type
None

`load_all`(*plugin_file*, *args=None*)

Load all Plugin classes from a Python file. If no Plugin classes are found, load as ScriptingPlugin.

Parameters

- **plugin_file** (*str*) – Path to the Python file.
- **args** (*Optional[Dict[str, Any]]*) – Arguments to pass to the Plugin.

Returns
List of Plugin class names loaded from the file.

Return type
List[str]

Raises
ValueError – If the plugin file cannot be loaded.

`load_plugin`(*plugin_name*, *extra_args=None*)

Load a plugin by name.

Parameters

- **plugin_name** (*str*) – Name of the plugin to load.
- **extra_args** (*Dict[str, Any]*)

Raises
ValueError – If plugin loading fails.

Return type
None

`load_plugins`(*conf_plugins*)

Load multiple plugins from a list of names.

Parameters
conf_plugins (*List[str]*) – List of plugin names to load.

Return type
None

`plugin_cbs: Dict[Plugin, Dict[str, List[Callable]]]`

`plugins: Dict[str, Plugin]`

`portal_publish`(*plugin*, *event*, **args*, ***kwargs*)

Publish an event to all registered callbacks for a plugin event, handling generators properly.

Parameters

- **plugin** (Plugin) – The plugin instance.
- **event** (*str*) – Event name.
- **args** (*Any*) – Positional arguments for callbacks.
- **kwargs** (*Any*) – Keyword arguments for callbacks.

Return type
Iterator

publish(*plugin*, *event*, **args*, ***kwargs*)

Publish an event to all registered callbacks for a plugin event.

Parameters

- **plugin** (Plugin) – The plugin instance.
- **event** (*str*) – Event name.
- **args** (*Any*) – Positional arguments for callbacks.
- **kwargs** (*Any*) – Keyword arguments for callbacks.

Return type
None

register(*plugin*, *event*, *register_notify=None*)

Register a plugin event for callbacks.

Parameters

- **plugin** (Plugin) – The plugin instance.
- **event** (*str*) – Event name.
- **register_notify** (*Callable, optional*) – Optional callback for registration notification.

Return type
None

registered_cbs: Dict[Tuple[*Plugin*, str], Callable]

property resources: str

Returns the path to the plugin resources directory.

Returns
Path to the resources directory.

Return type
str

subscribe(*plugin*, *event*, *callback=None*)

Subscribe a callback to a plugin event. Can also be used as a decorator if callback is not provided.

Parameters

- **plugin** (Plugin) – The plugin instance.
- **event** (*str*) – Event name.
- **callback** (*Callable, optional*) – Callback function.

Return type
Callable | None

Usage:

```
@plugins.subscribe(plugin, "event_name")
def handler(...):
    ...

# or
```

(continues on next page)

(continued from previous page)

```
plugins.subscribe(plugin, "event_name", handler)
```

unload(*pluginclass*)

Unload a plugin by class or name.

Parameters
pluginclass (*Union[Type[Plugin], str]*) – Plugin class or name.

Raises
ValueError – If the argument is not a loaded plugin.

Return type
None

unload_all()

Unload all loaded plugins in reverse order of load time.

Return type
None

class penguin.plugin_manager.**Plugin**

Bases: object

Base class for all IGLOO plugins.

Plugins should inherit from this class to be managed by the plugin manager. Provides argument access, logging, and emulator compatibility object access.

Args: Type[*PluginArgs*] | None = None

classmethod declares_args()

True if this class declares its own Args schema (not inherited).

Return type
bool

ensure_init()

Ensure that the plugin is fully initialized before use.

get_arg(*arg_name*)

Get an argument value by name.

Parameters
arg_name (*str*) – The argument name.

Returns
The argument value or None if not set.

Return type
Any

get_arg_bool(*arg_name*, *default=False*)

Returns True if the argument is set and has a truthy value.

Parameters

- **arg_name** (*bool*) – The name of the argument to retrieve.
- **default** (*bool*) – The default value to return if the argument is not set.

Returns
True if the argument exists and has a truthy value, False otherwise.

Return type
bool

Raises
ValueError – If the argument exists but has an unsupported type.

`load_state`(*data*)

Opt-in: rehydrate state previously returned by *save_state()*.

Called on restore before *on_restore()*. No-op by default.

Return type
None

`property name: str`

Returns the name of this plugin, which is its class name.

Returns
The class name of this plugin.

Return type
str

`on_restore`(*tag*)

Hook called after a VM snapshot is restored (or booted via -loadvm).

Called after *load_state()* has run, so any state captured at save time is already available. Override to rehydrate host-side state (e.g. re-establish bridges, replay recorded bindings) so it matches the restored guest. No-op by default.

Parameters
tag (*str*)

Return type
None

`on_snapshot`(*tag*)

Hook called after a VM snapshot is saved.

Override to persist any host-side state that must be reproduced when the snapshot is later restored. No-op by default. Plugins may also subscribe to the Snapshot plugin's `on_snapshot` event instead.

Parameters
tag (*str*)

Return type
None

`reset_state`()

Opt-in: reset host-side state to a pristine baseline.

Reserved for fork/restore-many (restoring the same point repeatedly), where each restore needs a clean host-side slate (fresh counters, port maps, etc.). Not exercised by once-and-continue restore. No-op by default; this is the designed seam, not yet driven by any caller.

Return type
None

save_state()

Opt-in: return JSON-serialisable host-side state to bundle with a snapshot, or None to save nothing (default).

A snapshot does not capture host-side plugin state; plugins that hold in-memory state needed for continuity after a cross-process restore should return it here. File-backed state (e.g. NVRAM's nvram_state.yaml) can self-heal via the statedir and need not be returned. The returned value is stored in the snapshot's host sidecar and handed back to *load_state()* on restore.

class penguin.plugin_manager.**PluginArgs**

Bases: BaseModel

Base class for declaring a plugin's accepted arguments.

Declare a nested Args class inheriting from PluginArgs on a plugin to opt into argument validation, defaults, schema/docs generation, and the first-class top-level config syntax (pluginname: {...}). Plugins that do not declare Args keep the legacy behavior: any args are accepted and no defaults are applied.

Example:

```
class Kmods(Plugin):
    class Args(PluginArgs):
        allowlist: list[str] = []
        quiet: bool = Field(False, description="Reduce logging verbosity")
```

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

class penguin.plugin_manager.**ScriptingPlugin**(**args*, ***kwargs*)

Bases: *Plugin*

A plugin that loads and executes a Python script as its __init__.

The script will have access to plugins and self (the plugin instance).

Parameters

- **args** (*Any*)
- **kwargs** (*Any*)

property name: str

Returns the name of this plugin, which is its class name.

Returns
The class name of this plugin.

Return type
str

script: str | None = None

uninit()

Uninitialize the plugin, if needed.

This method can be overridden by subclasses to perform cleanup.

Return type
None

penguin.plugin_manager.**camel_to_snake**(*name*)

Convert CamelCase to snake_case.

Parameters
name (*str*) – The CamelCase string to convert

Returns
The converted snake_case string

Return type
str

penguin.plugin_manager.**discover_declaring_plugins**(*plugin_path*, *manager=None*, *panda=None*)

Walk `plugin_path` and return every plugin that declares an `Args` schema.

Parameters

- **manager** – If given (the live `plugins` singleton from inside a real penguin process), imports run with the real runtime bound instead of the no-op stub, so plugins that read runtime state at import time (kernel-FFI enums via `plugins.kffi`, etc.) resolve and aren't skipped. Pass it from the docgen plugin for full coverage; omit it for best-effort standalone enumeration.
- **panda** – The live emulator/panda object to bind alongside `manager`.
- **plugin_path** (*str*)

Returns
`(found, skipped)` where `found` is a sorted list of `(config_name, args_model)` (`config_name` is the file stem users put in `plugins:`) and `skipped` is a sorted list of files that could not be imported. With `manager` unset, many runtime-dependent plugins land in `skipped`.

Return type
Tuple[*List*[*Tuple*[str, *Type*[PluginArgs]]], *List*[str]]

penguin.plugin_manager.**discover_declaring_plugins_static**(*plugin_path*)

Like *discover_declaring_plugins()*, but reads each plugin's declared `Args` via `ast` **without importing** — so runtime-dependent plugins (kernel-FFI enums, sibling imports, a live emulator) are covered too, exactly as config-load arg validation already does.

Returns
`(found, skipped)` where `found` is a sorted list of `(config_name, [ArgSpec])` and `skipped` lists files that failed to parse. Lower-fidelity than the imported model (types/defaults are rendered from source, not pydantic), but complete without a runtime.

Parameters
plugin_path (*str*)

Return type
Tuple[*List*[*Tuple*[str, *List*[ArgSpec]]], *List*[str]]

penguin.plugin_manager.**find_local_plugins**(*plugin_names*, *proj_dir*)

Find all local plugin files for a given list of plugin names.

Parameters

- **plugin_names** (*List[str]*) – List of plugin names to search for.
- **proj_dir** (*str*) – The project directory.

Returns
List of valid local file paths for the discovered plugins.

Return type
List[str]

penguin.plugin_manager.**find_plugin_by_name**(*plugin_name*, *proj_dir*, *plugin_path*)

Find a plugin file by name, trying various naming conventions.

Parameters

- **plugin_name** (*str*) – The name of the plugin to find
- **proj_dir** (*str*) – The project directory
- **plugin_path** (*str*) – The plugin path

Returns
Tuple of (file_path, is_local_plugin)

Return type
Tuple[str, bool]

Raises
ValueError – If the plugin cannot be found

penguin.plugin_manager.**gen_search_locations**(*plugin_name*, *proj_dir*, *plugin_path*)

Generate a list of possible file paths to look for a plugin.

Parameters

- **plugin_name** (*str*) – The name of the plugin to search for
- **proj_dir** (*str*) – The project directory
- **plugin_path** (*str*) – The plugin path

Returns
List of possible file paths to search for the plugin

Return type
List[str]

penguin.plugin_manager.**get_plugin_args_model**(*name*, *proj_dir*, *plugin_path*)

Return the declared `Args` model for plugin `name`, or None if the plugin can't be found, can't be imported, or doesn't declare an `Args` schema.

Parameters

- **name** (*str*)
- **proj_dir** (*str*)
- **plugin_path** (*str*)

Return type
Type[PluginArgs] | None

penguin.plugin_manager.**get_plugin_class**(*name*, *proj_dir*, *plugin_path*)

Resolve a plugin name to its `Plugin` subclass without instantiating it, or None if it cannot be discovered/imported. Best effort and side-effect tolerant: any failure returns None so callers never regress config load.

Parameters

- **name** (*str*)
- **proj_dir** (*str*)
- **plugin_path** (*str*)

Return type
type | None

penguin.plugin_manager.**interpret_bool**(*val*)

Interpret a value as a boolean, supporting bool, str, and int types.

Parameters
val (*Any*) – The value to interpret.

Returns
The interpreted boolean value.

Return type
bool

Raises
ValueError – If the value has an unsupported type.

penguin.plugin_manager.**plugin_declared_arg_fields**(*name*, *proj_dir*, *plugin_path*)

Statically determine whether plugin `name` declares an `Args` schema, and if so the set of field names it declares — **without importing the module**.

Parses the plugin file with `ast` and looks for a nested `class Args(PluginArgs)`. Returns the declared field names (possibly empty) if found, else `None`. Because it never executes plugin code, it is safe to call during config load in the run's own process (unlike importing, which drags in sibling/third-party modules — see `_exec_plugin_module`).

Parameters

- **name** (*str*)
- **proj_dir** (*str*)
- **plugin_path** (*str*)

Return type
set | None

penguin.plugin_manager.**plugin_declared_arg_specs**(*name*, *proj_dir*, *plugin_path*)

Return the declared `ArgSpec`s for plugin ``name` via `ast`, without importing it — the rich counterpart to *plugin_declared_arg_fields()*.

Returns the (possibly empty) spec list if the plugin declares an `Args` schema, else `None` (plugin not found, unparseable, or no `Args`).

Parameters

- **name** (*str*)
- **proj_dir** (*str*)
- **plugin_path** (*str*)

Return type
List[ArgSpec] | None

penguin.plugin_manager.**resolve_bound_method_from_class**(*f*, *manager=None*)

Resolve a method from a class given a function reference.

Parameters

- **f** (*Callable*) – The function reference to resolve.
- **manager** (*Any*)

Returns
The resolved method or the original function if not found.

Return type
Callable

penguin.plugin_manager.**snake_to_camel**(*name*)

Convert snake_case to CamelCase.

Parameters
name (*str*) – The snake_case string to convert

Returns
The converted CamelCase string

Return type
str

penguin.q_config module

penguin.q_config

QEMU configuration utilities for the Penguin emulation environment.

This module provides architecture-specific QEMU configuration mappings and a helper function to load the configuration for a given architecture.

penguin.q_config.**load_q_config**(*conf*)

Load the QEMU configuration for the given architecture.

Parameters
conf (*dict*) – Configuration dictionary containing 'core' and 'arch' keys.

Returns
A fresh QEMU configuration dictionary for the specified architecture.

Return type
dict[str, str]

Raises
ValueError – If the architecture is unknown or has no QEMU machine.

penguin.run_summary module

penguin.run_summary

Aggregate a completed run's scattered output artifacts into a single machine-readable `summary.json` in the result directory.

A run's signal is spread across many output files (`health_final.yaml`, `netbinds.csv`, `pseudofiles_failures.yaml`, `scores.txt`, ...). This module reads the artifacts that already exist — it never recomputes them — and writes one digest that humans triaging a run and outer loops (explore / CI / escalation) can parse without re-opening everything.

Schema (`schema_version` 1)

- `score` (float): total score, the sum of the `scores` values (same number written to `score.txt`).
- `scores` (dict): the per-metric dict from `calculate_score`, verbatim.
- `binds` (list): one entry per working service bind from `netbinds.csv`: `{proc, proto, ipvn, ip, port, time}` plus `pid` when the CSV carries a pid column and `{state, closed_time}` when it carries lifecycle

columns. Only `listening/closed` sockets are included — `pending/ transient` binds are not working services and are excluded.

- `crashes` (list): entries from `crashes.yaml` when present, else `[]`.
- `unmodeled_pseudofiles` (int|null): number of distinct paths in `pseudofiles_failures.yaml`; `null` if the tracker didn't run.
- `stalled_threads` (int|null): number of blocked threads reported in `waitgraph.yaml`; `null` if stall diagnostics didn't run.
- `panic` (bool): kernel panic observed (derived from the `nopanic` score metric).
- `wallclock_s` (float|null): wallclock duration of the emulation run, measured by the caller; `null` if unknown.
- `suggest` (list): remediation suggestions. Always present (stable key for consumers); currently always empty, populated as suggestion/escalation work lands.

Fields sourced from optional artifacts distinguish "producer didn't run" (`null`) from "producer ran and found nothing" (`0` / `[]`).

`summary.json` is written only for runs that completed and scored — outer loops must treat a missing `summary.json` as "run did not complete".

This file is *what happened* in a run. Its sibling `run_manifest.yaml` (what was run: input hashes, config identity) is a separate artifact with a different lifecycle — the manifest describes inputs and can be written even when a run dies; the summary exists only once a run completed and scored.

Functions

- build_run_summary
- write_run_summary

penguin.run_summary.**build_run_summary**(*result_dir*, *scores*, *wallclock_s=None*)

Aggregate a completed run's artifacts into the summary dict.

Parameters

- **result_dir** (*str*) – Directory containing the run's output artifacts.
- **scores** (*dict[str, int]*) – Per-metric score dict from `calculate_score` (embedded verbatim; the total is derived from it, matching `score.txt`).
- **wallclock_s** (*float | None*) – Emulation wallclock seconds, if the caller measured it.

Returns

The summary dict (see module docstring for the schema).

Return type

dict

penguin.run_summary.**write_run_summary**(*result_dir*, *scores*, *wallclock_s=None*)

Build and write `summary.json` into `result_dir`. Returns its path.

Parameters

- **result_dir** (*str*)
- **scores** (*dict[str, int]*)
- **wallclock_s** (*float | None*)

Return type

str

penguin.static_analyses module

penguin.static_analyses

Filesystem-analysis helpers shared by penguin core and the init plugins in pyplugins/init/ (the analysis classes themselves live there).

class penguin.static_analyses.**FileSystemHelper**

Bases: object

static find_regex(*target_regex*, *extract_root*, *ignore=None*)

Search the filesystem for matches to a regex pattern using ripgrep.

Parameters

- **target_regex** (*Pattern*) – Compiled regex pattern to match.
- **extract_root** (*str*) – Root directory to search.
- **ignore** (*list | tuple | None*) – Optional list/tuple of matches to ignore.

Returns
Dict of {match: {"count": int, "files": [str]}}

Return type
dict

penguin.static_analyses.**find_kernel_versions**(*extract_dir*)

Find kernel versions under <extract_dir>/lib/modules and select the best-matching available kernel. Shared by the KernelVersionFinder init plugin and runtime kernel resolution (penguin.utils).

Parameters
extract_dir (*str*) – Directory containing an extracted filesystem.

Returns
Dict with potential and selected kernel versions.

Return type
dict[str, list[str] | str]

penguin.static_analyses.**is_kernel_version**(*name*)

Check if a string matches a kernel version pattern.

Parameters
name (*str*) – Version string.

Returns
True if matches kernel version pattern.

Return type
bool

penguin.static_analyses.**select_best_kernel**(*kernel_versions*)

Select the most recent kernel version and match to available kernels.

Parameters
kernel_versions (*set[str]*) – Iterable of kernel version strings.

Returns
Best matching kernel version string.

Return type
str

penguin.stubs module

Declarative symbol stubs -> generated C shims + `lib_inject` aliases.

A `lib_inject.stubs` config section stubs out library/object symbols. It has two families, both of which compile down to existing machinery rather than inventing a new one:

- **Symbol return** (`return/type/guard_null_args`) generates one tiny C function per stubbed symbol plus a linker `--defsym` alias, fed through the same clang build in `pyplugins/interventions/nvram2.py`. This is a global, LD_PRELOAD-based replacement.
- **Assembly body** (`body` at a `symbol` or `symbol@offset` key) resolves the symbol to a file offset (statically, from the rootfs) and emits a `static_files` `binary_patch` action – delegating on-disk patching to its single owner (`pyplugins/core/live_image.py`).

This module is deliberately pure/host-side (no penguin runtime imports) so the codegen can be unit-tested directly. The library-symbol resolver used for glob expansion is injectable: *`make_fs_resolver()`* provides the production default that reads a project rootfs tarball, and tests can pass a plain callable returning a set of names.

Config shape (already validated by `structure.Stubs`), as a plain dict:

```
{
    "libX.so":     {"get_flag": {"return": 0}, "nvram_*": {"return": 0}},
    "/lib/libc.so": {"memcpy": {"guard_null_args": [0, 1], "return": 0}},
}
```

exception penguin.stubs.**StubError**

Bases: `Exception`

Raised for any invalid or unsatisfiable `stubs` configuration.

penguin.stubs.**check_precedence**(*stubs*, *existing_aliases*, *resolver=None*)

Raise *`StubError`* if any stubbed symbol also appears in `lib_inject.aliases` (a symbol must be owned by exactly one of them).

penguin.stubs.**expand**(*stubs*, *resolver=None*)

Flatten the symbol-return family of a `stubs` config into a list of `(symbol, action, library_key)` tuples, expanding glob keys against the library's exported symbols via `resolver`. Assembly-body stubs are ignored here (see *`generate_patches()`*). Deterministically ordered. Raises *`StubError`* on a glob that matches nothing, a duplicated symbol, an `@` in a symbol-return key, or a glob with no resolver supplied.

penguin.stubs.**generate**(*stubs*, *resolver=None*, *existing_aliases=None*)

Compile a `stubs` config into generated C shims and `--defsym` aliases.

Returns `(files, aliases)` where `files` maps a filename -> C source (to write into the generated shim directory) and `aliases` maps a real symbol name -> generated shim function name. If `existing_aliases` is provided, the precedence rule (no symbol in both `stubs` and `aliases`) is enforced.

penguin.stubs.**generate_patches**(*stubs*, *offset_resolver*)

Compile the assembly-body family of a `stubs` config into `binary_patch` edits, grouped by target guest file.

Returns `dict[guest_path, list[entry]]` where each entry is a `BinaryPatchEntry` dict (`file_offset` + `asm` + optional `mode` / `expect` + provenance). `offset_resolver(library_key, symbol)` must return `(guest_path, file_offset)`. Raises *`StubError`* on a glob key, a malformed key, or an unresolved symbol.

penguin.stubs.**make_fs_offset_resolver**(*fs_tar_path*)

Return `resolve(library_key, symbol) -> (guest_path, file_offset)` for the assembly-body form, reading the ELF out of a project rootfs tarball. Raises *`StubError`* when the library or symbol can't be resolved.

penguin.stubs.**make_fs_resolver**(*fs_tar_path*)

Return `resolve(library_key) -> set[str]` reading exported symbols out of a project rootfs tarball. Used for glob expansion. Raises *StubError* when the library is missing, ambiguous, or stripped.

penguin.stubs.**merge_patches_into_static_files**(*static_files*, *patches*)

Merge `generate_patches` output into a config `static_files` dict.

For each guest file, appends the edits to an existing `binary_patch` action (coalescing into its `patches` list) or creates a new one. Raises *StubError* if the path already holds a non-`binary_patch` action.

penguin.stubs.**parse_symbol_key**(*symkey*)

Split a body-stub key `symbol` or `symbol@offset` into `(symbol, offset)`. `offset` accepts hex (`0x..`) or decimal and defaults to 0. Raises *StubError* on a malformed key.

penguin.stubs.**parse_symbol_trace**(*stderr*, *symbols*)

Find non-synthetic definitions of *symbols* in –trace-symbol output.

Returns `{symbol: [provider, ...]}` for symbols that something other than the generated `--defsym` alias also defines – i.e. a stub and a hand-written `lib_inject.d/` drop-in both claiming one symbol. Nothing to report is an empty dict.

penguin.stubs.**trace_symbol_flags**(*symbols*)

Linker flags asking for a –trace-symbol report on each of *symbols*.

penguin.stubs.**write_files**(*gen_dir*, *files*)

Write generated shim files into `gen_dir`, replacing any previous contents so removed stubs don't linger. Returns the sorted list of written `.c` paths (as strings).

penguin.utils module

penguin.utils

Utility functions and classes for the Penguin emulation environment.

This module provides helpers for file hashing, weighted lists, atomic counters, filesystem and kernel management, plugin analysis loading, and mitigation provider discovery.

class penguin.utils.**AtomicCounter**(*initial=0*)

Bases: `object`

Parameters
initial (*int*)

decrement()

Atomically decrement the counter.

Returns
The new value.

Return type
int

get()

Get the current value atomically.

Returns
The current value.

Return type
int

increment()

Atomically increment the counter.

Returns
The new value.

Return type
int

value: int

penguin.utils.**LEGACY_KEY_FS_PATH = 'base/fs.tar.gz'**

The project-relative filesystem path whose cache key is left BYTE-COMPATIBLE with earlier releases. `penguin init` writes `base/fs.tar.gz` and penguin_config fills `core.fs` with `./base/fs.tar.gz`, so nearly every project's filesystem lives here; leaving this one path alone means the fix below does not invalidate their already-built images.

This is safe precisely because a project can hold at most ONE archive at this path, so the legacy key can never be the key SHARED by two different filesystems – which is the substitution being fixed. Any second archive in a project necessarily has some other path, and gets that path folded in.

penguin.utils.**REAL_run_command_with_output**(*cmd*, *stdout_file*, *stderr_file*)

Run a command and write output to files.

Parameters

- **cmd** (*list[str]*) – Command as a list of arguments.
- **stdout_file** (*str*) – Path to stdout file.
- **stderr_file** (*str*) – Path to stderr file.

Returns
Tuple of (error_msg, stderr) or (None, None) on success.

Return type
tuple[Optional[str], Optional[str]]

class penguin.utils.**WeightedItem**(*item*, *weight*)

Bases: `object`

Parameters

- **item** (*Any*)
- **weight** (*float*)

item: Any

weight: float

class penguin.utils.**WeightedList**

Bases: `object`

insert(*weight*, *item*)

Insert an item with a given weight.

Parameters

- **weight** (*float*) – The weight for the item.
- **item** (*Any*) – The item to insert.

Return type
None

items: List[*WeightedItem*]

lock: lock

pop()

Pop the item with the highest weight.

Returns
Tuple of (weight, item) or (None, None) if empty.

Return type
tuple[Optional[float], Optional[Any]]

penguin.utils.**boot_fingerprint**(*proj_dir*, *conf*)

Coarse fingerprint of the *boot-frozen* inputs that a VM snapshot is bound to. A snapshot captures guest state produced by these inputs, so a restore is only coherent if they still match.

This is intentionally broader than *hash_image_inputs()* (which omits the kernel and the boot/append args) and intentionally *coarse*: it freezes whole device-ish config sections rather than tracking which individual NVRAM keys or pseudofiles the guest actually touched. That can refuse a restore that would in fact have been fine; per-access precision can be layered on later.

Mutable-on-restore config (analysis/logging/breakpoint plugins, and the response logic of already-wired pseudofiles) is deliberately excluded.

Parameters

- **proj_dir** (*str*) – Project directory.
- **conf** (*Dict[str, Any]*) – Configuration dictionary.

Returns
Hex digest string.

Return type
str

penguin.utils.**construct_empty_fs**(*path*)

Construct an empty filesystem archive at the given path.

Parameters
path (*str*) – Path to create the archive.

Return type
None

penguin.utils.**get_arch_dir**(*config*)

Get the full architecture directory path.

Parameters
config (*dict[str, Any]*) – Configuration dictionary.

Returns
Directory path string.

Return type
str

penguin.utils.**get_arch_subdir**(*config*)

Get the architecture subdirectory name for the given config.

Parameters
config (*dict[str, Any]*) – Configuration dictionary.

Returns
Architecture subdirectory string.

Return type
str

penguin.utils.**get_available_kernel_versions**()

Scan /igloo_static/kernels and return a list of available kernel versions as tuples of integers, e.g., [(5, 15, 0), (6, 1, 55)]

Returns
List of kernel version tuples.

Return type
list[tuple[int, ...]]

penguin.utils.**get_driver_kmod_path**(*config*)

Get the path to the driver kernel module.

Parameters
config (*dict[str, Any]*) – Configuration dictionary.

Returns
Path string.

Return type
str

penguin.utils.**get_file_hash**(*filepath*)

Compute the MD5 hash of a file.

Parameters
filepath (*str*) – Path to the file.

Returns
Hex digest string.

Return type
str

penguin.utils.**get_kernel**(*conf*, *proj_dir*)

Get the kernel path for the given configuration and project directory.

Parameters

- **conf** (*dict*) – Configuration dictionary.
- **proj_dir** (*str*) – Project directory.

Returns
Path to the kernel file.

Return type
str

Raises
ValueError – If kernel cannot be found or multiple kernels found.

penguin.utils.**get_mitigation_providers**(*config*)

Given a config, pull out all the enabled mitigation providers, load them and return a dict of {ANALYSIS_TYPE: analysis class object}

Skip plugins that are disabled in config. Raise an error if version of a plugin mismatches the config version.

Parameters
config (*dict*) – Configuration dictionary.

Returns
Dictionary mapping ANALYSIS_TYPE to analysis class object.

Return type
dict[str, Any]

Raises
ValueError – On version mismatch.

penguin.utils.**get_mount_type**(*path*)

Get the filesystem type of the mount at the given path.

Parameters
path (*str*) – Path to check.

Returns
Filesystem type string or None.

Return type
str or None

penguin.utils.**get_penguin_kernel_version**(*conf*)

Extract kernel version tuple from conf['core']['kernel'].

Expected path format:
/igloo_static/kernels/<VERSION>/<kernel_filename>

<VERSION> may have a suffix (e.g., 6.1.55-custom); the suffix after the first '-' is ignored. Only leading dot-separated numeric components are returned.

Parameters
conf (*dict*) – Configuration dictionary.

Returns
Tuple of integers representing the version, e.g., (6, 1, 55)

Return type
tuple[int, ...]

Raises
ValueError – If the version cannot be determined.

penguin.utils.**hash_image_inputs**(*proj_dir*, *conf*)

Create a hash of all the inputs of the image creation process.

For the base filesystem this is its modification time, plus – unless it sits at the default `base/fs.tar.gz` – the path it was named by and its size.

It used to be the mtime ALONE, which left the key blind to WHICH filesystem was being built: two different archives that shared an mtime produced the same key, so the image built from the first was silently reused for the second and the run booted a filesystem the config did not ask for. Archives sharing an mtime is not exotic – one extraction, copy or checkout pass stamps everything it writes with the same timestamp – and nothing downstream can notice, because the config, the logs and the output directory all still name the archive that was requested.

The default path keeps its old key so that existing projects do not all rebuild; see LEGACY_KEY_FS_PATH for why that cannot reintroduce the substitution. Path and size come from stat data the mtime already required, so this adds no I/O.

Not covered: an archive replaced in place, under the same name, with the same size and timestamp. Hashing the contents would close that, at the cost of reading an archive that can be tens of gigabytes on every invocation – a poor trade for the residual risk.

We specifically do NOT include the busybox binary in this hash despite its potential effect on the image because we expect them to be fairly standard at least over the very small number of lines.

Parameters

- **proj_dir** (*str*) – Project directory.
- **conf** (*dict[str, Any]*) – Configuration dictionary.

Returns
Hex digest string.

Return type
str

penguin.utils.**read_output_files**(*stdout_file*, *stderr_file*)

Read the contents of stdout and stderr files.

Parameters

- **stdout_file** (*str*) – Path to stdout file.
- **stderr_file** (*str*) – Path to stderr file.

Returns
Tuple of (stdout_data, stderr_data).

Return type
tuple[str, str]

penguin.utils.**resolve_arch_asset**(*arch*, *base_dir*, *prefix=''*, *suffix=''*)

Resolve the on-disk name of an asset that is named by the *config* arch (e.g. guesthopper.<arch>, sysroots/<arch>), preferring the canonical name and falling back to the first alias whose file/dir actually exists.

This lets the canonical name flip to x86_64 while still working against assets/build artifacts that are still named with an alias (intel64) until the producing build/sibling repo catches up.

Parameters

- **arch** (*str*) – A config arch name or alias.
- **base_dir** (*str*) – Directory the asset lives in.
- **prefix** (*str*) – Filename prefix before the arch token (e.g. "guesthopper.").
- **suffix** (*str*) – Filename suffix after the arch token.

Returns
The arch token (canonical or alias) to use; canonical if none exist.

Return type
str

penguin.utils.**run_command_with_output**(*cmd*, *ignore1*, *ignore2*)

Run a command and capture stdout/stderr.

Parameters

- **cmd** (*list[str]*) – Command as a list of arguments.
- **ignore1** (*Any*) – Ignored.
- **ignore2** (*Any*) – Ignored.

Returns
Tuple of (None, None) on success, or (-1, "", error_msg) on exception.

Return type
tuple[Any, Any]

penguin.utils_cli module

Implementation of *penguin utils <subcommand>*.

The compose commands inspect compose results metadata and, when run inside the active compose container, /proc. They are intentionally filesystem based so the same command still gives useful connection details after a compose run exits.

penguin.utils_cli.**list_instances**(*compose_dir*)

Inventory devices under a compose results run directory.

Parameters
compose_dir (*str*)

Return type
list[dict]

21.2 pengutils package

21.2.1 Pengutils

General-purpose utilities, data models, and CLI tools for the Penguin system. Provides reusable functions, database models, and helpers for querying, filtering, and analyzing Penguin and interacting with a running system.

Usage

Pengutils can be used in two main ways:

1. **Inside the Penguin container**: All utilities and CLI tools are available by default when running inside the official Penguin container environment.
2. **Outside the Penguin container**: You can install pengutils from source and use its modules and CLI tools independently. Clone the repository and install with pip:

```
git clone https://github.com/rehosting/penguin.git
cd penguin/pengutils
pip install .
```

This allows you to run pengutils commands and import its modules in your own Python environment, provided you have access to the Penguin results database.

Once installed you can run a series of CLI tools for querying and processing Penguin data, as well as importing its data models and utilities in your own Python scripts.

21.2.2 Subpackages

pengutils.events package

Pengutils Package

Pengutils is a general-purpose utility and data modeling package for the Penguin system. It provides SQLAlchemy ORM models for event data, as well as a growing set of utilities, helpers, and CLI tools for querying, filtering, and processing Penguin database information.

Usage

You can use pengutils:

1. **Inside the Penguin container**: All models and CLI tools are available by default.
2. **Outside the Penguin container**: Clone the repository and install from source to use the models and CLI tools independently:

```
git clone https://github.com/rehosting/penguin.git
cd penguin/pengutils
pip install .
```

Scope

- Data models for core Penguin event types (e.g., Read, Write, Syscall, Exec).
- Utility functions for working with Penguin data.
- CLI tools for querying, filtering, and exporting results.
- Helpers for common database and analysis tasks.

Example programmatic usage

```
from pengutils.events import Event, Read, Write, Syscall, Exec

# Querying events (using SQLAlchemy session):
session.query(Read).filter(Read.procname == "myproc").all()
```

Example CLI usage

```
execs --procname myproc --fd 3 --filename log.txt --output results.txt
reads --procname myproc --fd 4 --filename input.txt --output reads.txt
writes --procname myproc --fd 5 --filename output.txt --output writes.txt
syscalls --procname myproc --syscall open --errors --output syscalls.txt
fds --procname myproc --fd 3 --output fds.txt
tasks --results ./results/latest --output tasks.txt
```

Available Types

- `Event`: Base class for all events.
- `Read`: File read events.
- `Write`: File write events.

- `Syscall`: System call events.
- `Exec`: Process execution events.

Utilities

For CLI tools and helpers, see the `pengutils.utils` subpackage.

Submodules

pengutils.events.base module

Events Base Models

This module defines the SQLAlchemy base classes and the core *Event* model for the penguin event database. All event types inherit from *Event*, which provides polymorphic mapping for different event subtypes.

Example usage

```
from pengutils.base import Event, Base
```

Classes

- Base: Declarative base for all ORM models.
- Event: Base class for all event records, supporting polymorphic identity.

Table Structure

The *event* table contains: - id: Primary key for the event. - type: Polymorphic type identifier. - procname: Name of the process involved in the event. - proc_id: Process ID.

class pengutils.events.base.**Base**(***kwargs*)

Bases: `DeclarativeBase`

Declarative base class for all ORM models.

Parameters
kwargs (*Any*)

metadata: ClassVar[MetaData] = MetaData()

Refers to the `_schema.MetaData` collection that will be used for new `_schema.Table` objects.

See also

orm_declarative_metadata

registry: ClassVar[registry] = <sqlalchemy.orm.decl_api.registry object>

Refers to the `_orm.registry` in use where new `_orm.Mapper` objects will be associated.

class pengutils.events.base.**Event**(***kwargs*)

Bases: *Base*

Base class for all event records in the database.

Attributes

id
[int] Primary key for the event.

type
[str] Polymorphic type identifier for the event.

procname
[str] Name of the process involved in the event.

proc_id
[int] Process ID associated with the event.

SQLAlchemy polymorphic mapping is used to allow subclasses to represent different event types.

`id: Mapped[int]`

`proc_id: Mapped[int]`

`procname: Mapped[str]`

`type: Mapped[str]`

pengutils.events.types module

Event Types

This module defines the main event types for the penguin event database, each as a subclass of *Event*. These types represent different kinds of system events (read, write, syscall, exec) and are mapped to corresponding tables in the database using SQLAlchemy ORM.

Example usage

```
from pengutils.events.types import Read, Write, Syscall, Exec
```

Classes

- Read: Represents a file read event.
- Write: Represents a file write event.
- Syscall: Represents a syscall event with arguments and return value.
- Exec: Represents an exec event (process execution).

Each class provides a `__str__` method for human-readable representation.

Table Structure

Each event type is mapped to its own table and linked to the base `event` table via a foreign key.

`class pengutils.events.types.Exec(**kwargs)`
Bases: *Event*

Exec Event

Represents a process execution (exec) event.

Attributes

id
[int] Primary key, foreign key to event.id.

calltree
[str] Call tree information.

argc
[str] Argument count.

argv
[str] Argument values.

envp
[str] Environment variables.

euid
[int] Effective user ID.

egid
[int] Effective group ID.

argc: Mapped[str]

argv: Mapped[str]

calltree: Mapped[str]

egid: Mapped[int]

envp: Mapped[str]

euid: Mapped[int]

id: Mapped[int]

proc_id: Mapped[int]

procname: Mapped[str]

type: Mapped[str]

class pengutils.events.types.**ProcExit**(***kwargs*)
Bases: *Event*

ProcExit Event

A process/thread exit, recorded from the exit/exit_group syscall. Identity is denormalized from the syscall event (no memory read of the dying task). Pairs with *ProcStart* by `(pid, create_time)` to close out a process in the derived tree.

Attributes

pid
[int] Exiting process id.

create_time
[int] Process identity key (see *ProcStart*). May be 0 if an older driver did not denormalize it, in which case consumers fall back to the unique live (pid).

code
[int] Exit status argument passed to exit / exit_group.

reason
[str] "exit" or "exit_group".

code: Mapped[int]

create_time: Mapped[int]

id: Mapped[int]

pid: Mapped[int]

proc_id: Mapped[int]

procname: Mapped[str]

reason: Mapped[str]

type: Mapped[str]

class pengutils.events.types.**ProcStart**(***kwargs*)
Bases: *Event*

ProcStart Event

A process becoming visible (via exec) to the process-tree model. Lean and identity-focused – argv/env live on *Exec*; this carries only what the genealogy needs. Re-exec appends another row with the same (pid, create_time); consumers coalesce by that key.

Attributes

pid
[int] Process id.

ppid
[int] Parent process id (from osi_proc.ppid).

create_time
[int] Kernel task creation timestamp; stable across execve, so (pid, create_time) is a durable process identity across pid reuse.

comm
[str] Kernel task->comm at exec time (the executed program's basename may differ; procname on the base row carries the fuller name).

uid, gid, euid, egid
[int] Credentials at exec time.

comm: Mapped[str]

create_time: Mapped[int]

egid: Mapped[int]

euid: Mapped[int]

gid: Mapped[int]

id: Mapped[int]

pid: Mapped[int]

ppid: Mapped[int]

proc_id: Mapped[int]

procname: Mapped[str]

type: Mapped[str]

uid: Mapped[int]

class pengutils.events.types.**Read**(***kwargs*)

Bases: *Event*

Read Event

Represents a file read event.

Attributes

id
: [int] Primary key, foreign key to event.id.

fd
: [int] File descriptor read from.

fname
: [str] Name of the file read.

buffer
: [Optional[bytes]] Contents read from the file.

buffer: Mapped[bytes | None]

fd: Mapped[int]

fname: Mapped[str]

id: Mapped[int]

proc_id: Mapped[int]

procname: Mapped[str]

type: Mapped[str]

class pengutils.events.types.**Syscall**(***kwargs*)

Bases: *Event*

Syscall Event

Represents a syscall event, including arguments and return value.

Attributes

id
[int] Primary key, foreign key to event.id.

name
[str] Name of the syscall.

retno
[Optional[int]] Return value of the syscall.

retno_repr
[Optional[str]] String representation of the return value.

arg0-arg5
[Optional[int]] Argument values.

arg0_repr-arg5_repr
[Optional[str]] String representations of arguments.

```
arg0: Mapped[int | None]

arg0_repr: Mapped[str | None]

arg1: Mapped[int | None]

arg1_repr: Mapped[str | None]

arg2: Mapped[int | None]

arg2_repr: Mapped[str | None]

arg3: Mapped[int | None]

arg3_repr: Mapped[str | None]

arg4: Mapped[int | None]

arg4_repr: Mapped[str | None]

arg5: Mapped[int | None]

arg5_repr: Mapped[str | None]

id: Mapped[int]

name: Mapped[str]

proc_id: Mapped[int]

procname: Mapped[str]

retno: Mapped[int | None]

retno_repr: Mapped[str | None]
```

type: Mapped[str]

class pengutils.events.types.**Write**(***kwargs*)

Bases: *Event*

Write Event

Represents a file write event.

Attributes

id
[int] Primary key, foreign key to event.id.

fd
[int] File descriptor written to.

fname
[Optional[str]] Name of the file written.

buffer
[Optional[bytes]] Contents written to the file.

buffer: Mapped[bytes | None]

fd: Mapped[int]

fname: Mapped[str | None]

id: Mapped[int]

proc_id: Mapped[int]

procname: Mapped[str]

type: Mapped[str]

pengutils.utils package

Pengutils Utils Package

This package contains general-purpose utility modules for querying, filtering, and processing data from the Penguin database. It provides reusable functions and CLI tools for working with event data (execution, syscall, read, write, file descriptor events) as well as other Penguin-related analysis and database tasks.

Usage

You can run these utilities:

1. **Inside the Penguin container**: All CLI tools and modules are available by default.
2. **Outside the Penguin container**: Clone the repository and install from source to use the utilities independently:

```
git clone https://github.com/rehosting/penguin.git
cd penguin/pengutils
pip install .
```

Database Expectations

All CLI commands expect to find a `plugins.db` SQLite database inside the specified results directory (default: `./results/latest/`). This database is generated by Penguin's data collection pipeline and contains all relevant data for querying. If `plugins.db` is missing, commands will fail with an error.

Running Commands

Commands should be run from the root of your Penguin workspace or any directory where the `results` folder is accessible. You can specify a different results directory using the `--results` option if your data is stored elsewhere.

Example CLI usage

```
execs --procname myproc --fd 3 --filename log.txt --output results.txt
reads --procname myproc --fd 4 --filename input.txt --output reads.txt
writes --procname myproc --fd 5 --filename output.txt --output writes.txt
syscalls --procname myproc --syscall open --errors --output syscalls.txt
fds --procname myproc --fd 3 --output fds.txt
tasks --results ./results/latest --output tasks.txt
```

Each module provides command-line interfaces and helper functions for specific Penguin data types and analysis tasks.

Submodules

pengutils.utils.cli_breakpoint module

Breakpoint CLI

This script provides a command-line interface (CLI) for managing breakpoints, syscall traces, and uprobes via the Penguin RemoteCtrl Plugin Unix socket. It combines functionality for listing, disabling, and creating various event hooks.

Example usage

```
# List breakpoints
cli_breakpoint.py list --sock results/latest/remotectrl.sock

# Disable a breakpoint
cli_breakpoint.py disable --sock results/latest/remotectrl.sock --id 3

# Add a syscall trace
cli_breakpoint.py syscall --name sys_write --action "print %fd, %s, %d"

# Add a uprobe with custom output file
cli_breakpoint.py uprobe --path /lib/libc.so.6 --symbol malloc --action "print %d" --
↪output malloc.log
```

Options

Common options: - `--sock`: Path to plugin socket (default: results/latest/remotectrl.sock)

See individual commands for specific options.

pengutils.utils.cli_db module

Database CLI

This script consolidates various database query utilities into a single command-line interface. It allows querying execution events, file descriptor usage, file reads/writes, syscalls, and unique tasks (process names) from the Penguin RemoteCtrl Plugin database.

Example usage

```
# Query tasks
cli_db.py tasks --results ./results/latest

# Query execs
cli_db.py execs --procname myproc --fd 3

# Query syscalls
cli_db.py syscalls --errors

# Query reads/writes
cli_db.py reads --filename config.txt
cli_db.py writes --fd 1

# Query unique FDs
cli_db.py fds --follow
```

Options

Common options: - `--results`: Path to results folder (default: `./results/latest/`) - `--output`: Output file (default: `/dev/stdout`)

See individual commands for specific filters and options.

pengutils.utils.cli_db.**exec_filter**(*sess*, *procname*, *fd*, *filename*)

pengutils.utils.cli_db.**read_filter**(*sess*, *procname*, *fd*, *filename*)

pengutils.utils.cli_db.**syscall_filter**(*sess*, *procname*, *syscall*, *errors*)

pengutils.utils.cli_db.**write_filter**(*sess*, *procname*, *fd*, *filename*)

pengutils.utils.cli_guest_cmd module

Guest Command CLI

This script provides a command-line interface (CLI) for running a command in a rehosted guest via a vsock Unix socket. It is inspired by guest_cmd.py and uses Click for the CLI interface.

Example usage

```
cli_guest_cmd.py --socket /tmp/vsocket --port 12341234 "ls -l /"
```

Options

- `--socket`: Path to vsock Unix socket (default: auto-search for 'vsocket' in /tmp/*)
- `--port`: Vsock port number (default: 12341234)
- `command`: Command to run in the guest (required)

pengutils.utils.cli_pengutils module

Main CLI Dispatcher

This module provides a dynamic CLI that discovers and allows execution of other registered console_scripts in the pengutils package.

class pengutils.utils.cli_pengutils.**PenguinCLI**(*name=None*, *commands=None*, *invoke_without_command=False*, *no_args_is_help=None*, *subcommand_metavar=None*, *chain=False*, *result_callback=None*, ***kwargs*)

Bases: `_MultiCommand`

Parameters

- **name** (*str | None*)
- **commands** (*cabc.MutableMapping[str, Command] | cabc.Sequence[Command] | None*)
- **invoke_without_command** (*bool*)
- **no_args_is_help** (*bool | None*)
- **subcommand_metavar** (*str | None*)
- **chain** (*bool*)
- **result_callback** (*t.Callable[..., t.Any] | None*)
- **kwargs** (*t.Any*)

get_command(*ctx*, *name*)

Load and return the requested command object.

list_commands(*ctx*)

Dynamically list commands registered in setup.cfg under console_scripts.

pengutils.utils.cli_plugins module

Plugins CLI

This script provides a command-line interface (CLI) for managing plugins via the Penguin RemoteCtrl Plugin Unix socket. It uses Click for the CLI interface.

Example usage

```
plugins load plugin_name [key=value ...]
plugins enable plugin_name [key=value ...]
plugins disable plugin_name [key=value ...]
```

Options

- `--sock`: Path to plugin socket (default: /workspace/results/latest/remotectrl.sock)

pengutils.utils.cli_plugins.**parse_extra_args**(*args_tuple*)

Parses a tuple of arguments into a dictionary. Supports 'key=value' for key-value pairs. Supports 'flag' (no equals sign) as {flag: True}. Performs basic type inference for booleans and integers.

pengutils.utils.cli_snapshot module

Snapshot CLI

Host-driven VM snapshotting via the Penguin RemoteCtrl Plugin Unix socket. Requests are *armed* at a safe execution boundary (default: the next syscall return) rather than captured at the instant the command is received.

Example usage

```
# Save at the next syscall boundary (default), tag "warm"
cli_snapshot.py save --tag warm

# Save at the next syscall boundary in a specific process
cli_snapshot.py save --tag warm --proc lighttpd

# Save when execution reaches a symbol
cli_snapshot.py save --tag warm --when symbol --symbol main --proc lighttpd

# Save immediately (next main-loop tick)
cli_snapshot.py save --tag warm --when now

# Restore a snapshot into the running guest
cli_snapshot.py load --tag warm
```

Options

- `--sock`: Path to plugin socket (default: results/latest/remotectrl.sock)

pengutils.utils.util_base module

Utility Base Functions

This module provides utility functions for wrapping and executing filtered queries on event databases. It is designed to be used by CLI scripts for querying and outputting event data, supporting flexible filtering and output options.

Example usage

```
from pengutils.utils.util_base import wrapper

def my_filter(sess, arg1, arg2):
    # ...filter logic...
    return query

wrapper(results_path, output_path, print_procname, follow, my_filter, (arg1, arg2))
```

Functions

- wrapper: Handles session setup, filtering, and output for event queries.

pengutils.utils.util_base.**get_default_results_path**()

Get the default results path.

Returns

str
Default results path.

pengutils.utils.util_base.**wrapper**(*results*, *output*, *print_procname*, *follow*, *filter_func*, *args*)

Wrapper function to execute a filtered query and output results.

Parameters

results
[str] Path to results folder.

output
[str] Output file path (default: `/dev/stdout`).

print_procname
[bool] Whether to print the process name in output.

follow
[bool] Whether to show latest results as they appear.

filter_func
[callable] Function to filter the query. Should accept (sess, `*args`).

args
[tuple] Arguments to pass to the filter function.

Returns

None

pengutils.utils.util_events module

pengutils.utils.util_events.**get_default_socket_path**()

Return type
str

pengutils.utils.util_events.**in_container**()

Return type
bool

pengutils.utils.util_events.**send_command**(*data=None*, *sock=None*)

Parameters

- **data** (*dict*)
- **sock** (*str*)

Return type
dict | None

21.3 pyplugins package

21.3.1 PyPlugins package

This package contains optional, pluggable features for Penguin. It is discovered and loaded by the main application at runtime.

Importing in PyPlugins

To import modules inside a PyPlugin, use standard package imports. For example:

```
from apis.events import EVENTS
```

Where the `apis` directory exists in the `pyplugins` directory.

For general architecture and usage information, see the repository documentation.

21.3.2 Subpackages

pyplugins.actuation package

Actuation Plugins for Penguin

The `actuation` package contains plugins that automate actions and service interactions within the Penguin emulation environment. These plugins can perform tasks such as network scanning, web content fetching, and VPN bridging, enabling automated analysis and testing of guest services exposed to the host.

Each plugin is designed to be used within the Penguin plugin system and may subscribe to or publish events for automation workflows.

Submodules

pyplugins.actuation.fetch_web module

FetchWeb Plugin (fetch_web.py) for Penguin

This module provides the FetchWeb plugin, which automatically fetches web pages from guest services exposed to the host (typically on ports 80 and 443) via the VPN plugin. It listens for 'on_bind' events published by the VPN plugin and attempts to retrieve web content from the corresponding host ports. The plugin is responsible for:

- Subscribing to VPN 'on_bind' events to detect new web services exposed to the host.
- Enqueuing fetch tasks for HTTP/HTTPS ports (80, 443) and processing them in a worker thread.
- Fetching web content using wget and logging the response size and entropy.
- Optionally shutting down the emulation after a successful fetch or after failure, based on arguments.

Arguments

- outdir (str): Output directory for storing fetched web content.
- shutdown_after_www (bool, optional): If True, shut down emulation after a successful fetch.
- shutdown_on_failure (bool, optional): If True, shut down emulation if no responsive servers are found.

Plugin Interface

- Subscribes to the VPN plugin's 'on_bind' event to trigger web fetches.
- Does not provide a direct interface for other plugins, but writes fetched content to files in the output directory.

Overall Purpose

The FetchWeb plugin automates the retrieval and analysis of web content from guest services exposed to the host, aiding in service validation and content inspection during emulation.

class pyplugins.actuation.fetch_web.**FetchWeb**

Bases: *Plugin*

class Args(*, *fetch_delay=None*, *shutdown_after_www=False*, *shutdown_on_failure=False*)

Bases: *PluginArgs*

Parameters

- **fetch_delay** (*int | None*)
- **shutdown_after_www** (*bool*)
- **shutdown_on_failure** (*bool*)

fetch_delay: int | None

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

shutdown_after_www: bool

shutdown_on_failure: bool

fetch(*guest_ip*, *host_ip*, *guest_port*, *host_port*, *log_file_name*)

Fetch web content from the specified host port and log the response.

Args:

guest_ip (str): Guest IP address. host_ip (str): Host IP address. guest_port (int): Guest port. host_port (int): Host port. log_file_name (str): Path to the file for storing fetched content.

Returns:

bool: True if fetch was successful, False otherwise.

Parameters

- **guest_ip** (*str*)
- **host_ip** (*str*)
- **guest_port** (*int*)
- **host_port** (*int*)
- **log_file_name** (*str*)

Return type

bool

fetchweb_on_bind(*proto*, *guest_ip*, *guest_port*, *host_port*, *host_ip*, *procname*)

Handle a new bind event from the VPN plugin and enqueue a fetch task for web ports.

Args:
proto (str): Protocol (e.g., 'tcp'). guest_ip (str): Guest IP address. guest_port (int): Guest port. host_port (int): Host port mapped to the guest service. host_ip (str): Host IP address. procname (str): Name of the process binding the port.

Parameters

- **proto** (*str*)
- **guest_ip** (*str*)
- **guest_port** (*int*)
- **host_port** (*int*)
- **host_ip** (*str*)
- **procname** (*str*)

Return type
None

worker()

Worker thread that processes fetch tasks from the queue.

Return type
None

pyplugins.actuation.fetch_web.**calculate_entropy**(*buffer*)

Calculate the Shannon entropy of a byte buffer.

Args:
buffer (bytes): The data buffer to analyze.

Returns:
float: The calculated entropy value.

Parameters
buffer (*bytes*)

Return type
float

pyplugins.actuation.nmap module

Nmap Plugin (nmap.py) for Penguin

This module provides the Nmap plugin, which automatically performs service and vulnerability scans on guest services exposed to the host via the VPN plugin. It listens for 'on_bind' events published by the VPN plugin and launches nmap scans against the corresponding host ports.

Features

- Subscribes to VPN 'on_bind' events to detect new guest services exposed to the host.
- Launches nmap scans (in a separate thread) for each new TCP service, storing results as XML files in the output directory.

- Supports custom nmap configurations if present.
- Manages and cleans up subprocesses for running nmap scans.

Arguments

- None

Plugin Interface

- Subscribes to the VPN plugin's 'on_bind' event to trigger scans.
- Does not provide a direct interface for other plugins, but writes scan results to files in the output directory.

Overall Purpose

The Nmap plugin automates the discovery and analysis of guest services exposed to the host, aiding in security assessment and service enumeration during emulation.

class pyplugins.actuation.nmap.**Nmap**

Bases: *Plugin*

cleanup_subprocesses()

Terminate and clean up all running nmap subprocesses.

Return type
None

nmap_on_bind(*proto*, *guest_ip*, *guest_port*, *host_port*, *host_ip*, *procname*)

Handle a new bind event from the VPN plugin and launch an nmap scan if appropriate.

Args:
proto (str): Protocol (e.g., 'tcp'). guest_ip (str): Guest IP address. guest_port (int): Guest port. host_port (int): Host port mapped to the guest service. host_ip (str): Host IP address. procname (str): Name of the process binding the port.

Parameters

- **proto** (*str*)
- **guest_ip** (*str*)
- **guest_port** (*int*)
- **host_port** (*int*)
- **host_ip** (*str*)
- **procname** (*str*)

Return type
None

scan_thread(*host_ip*, *guest_port*, *host_port*, *log_file_name*)

Run an nmap scan against the specified host port and save results.

Args:
host_ip (str): Host IP address. guest_port (int): Guest port. host_port (int): Host port. log_file_name (str): Path to the XML log file for scan results.

Parameters

- **host_ip** (*str*)
- **guest_port** (*int*)
- **host_port** (*int*)
- **log_file_name** (*str*)

Return type
None

uninit()

Cleanup subprocesses on plugin unload.

Return type
None

pyplugins.actuation.run_on_bind module

RunOnBind Plugin (run_on_bind.py) for Penguin

Executes user-defined host or guest commands when a guest service binds a port that is exposed to the host by the VPN plugin.

Execution model

Bind events are filtered, de-duplicated per endpoint, and queued. A single worker thread drains that queue, so command runs are serialized, two endpoints binding at once can never produce concurrent *guest_cmd.py* invocations.

An attempt runs every configured command and succeeds if ANY of them does, since configurations routinely include commands that only apply to some targets. The first successful attempt is the last: the plugin stops accepting endpoints and, if *shutdown_after_cmd* is set, ends the analysis. A failed attempt is not terminal – the next endpoint to bind gets its own try, uncapped, which keeps a run from wedging when the first endpoint to appear (often a loopback bind) isn't the one that works.

Every command has a hard timeout, and every command's outcome is recorded whether it passed or failed.

Arguments

- outdir (str): Output directory for command-output artifacts.
- **commands (str, list, or dict, REQUIRED): Command(s) to run after a matching bind event.**
  Loading the plugin without commands raises at init.

  Supports three formats:

  1. **Single string (runs in guest mode by default):**
     commands: python3 example.py
  2. **List of strings (runs in guest mode by default):**

     commands:

     - python3 analyze.py
     - python3 report.py
  3. **Structured format with explicit mode specification:**

     commands:

- mode: host run:
  - python3 ./test.py
- mode: guest run:
  - echo “test” > /tmp/test.txt

Mixed lists of strings and dictionaries are not supported.

Mode options:

- host: Run commands on the host system (inside the container), using the project directory penguin supplies as the working directory.
- guest: Run commands inside the emulated guest via the *guest_cmd.py* wrapper (default). Requires *guest_cmd: true* under *core* in config.yaml.

Host commands also receive the host port that the VPN plugin mapped the binding guest service to. That port cannot be hardcoded in config: the VPN plugin remaps it whenever the guest port is privileged or already taken. It is supplied two ways:

- *{host_port}* anywhere in the command string is replaced with the port before the command is split and run.
- *PENGUIN_HOST_PORT* is set in the command’s environment.

The bridge listens on 0.0.0.0 inside the container, so *127.0.0.1:<host_port>* always reaches the service. Guest commands run inside the guest and talk to the guest port directly, so they are left verbatim.

Example:

commands:

- mode: host run:
  - curl -sf http://127.0.0.1:{host_port}/
  - python3 ./probe.py –port {host_port}

- **endpoints (list[str], optional): Guest endpoints that may trigger the commands,**
  formatted as *guest_ip:guest_port*. Either component may be * to match anything. If omitted, any bind event passing the protocol/port filters may trigger.

  Example:

  endpoints:

  - “0.0.0.0:80”
  - “*:8080”

  Prefer a wildcard over an exact IP: an allowlist that never matches means the commands never run and nothing ever ends the run.

- **ports (list[int], optional): Guest ports that may trigger the commands.**
  Default: *[80, 443]*, the common web-UI ports, so an unfiltered config does not fire on the first unrelated bind – a target whose httpd serves UPnP on 1900 well before it binds 80 would otherwise trigger on the wrong service. Set an explicit list for other services, or *[]* to accept any port: an empty list disables the filter rather than matching nothing.
- proto (str, optional): Protocol to match. Default: ‘tcp’. Use ‘*’ for any.
- delay (int, optional): Delay in seconds after a bind before running commands. Default: 20.
- **timeout (int, optional): Per-command timeout in seconds. Default: 120. A command that**
  exceeds it is killed and counted as a failure.

- shutdown_after_cmd (bool, optional): If True, end emulation after the first successful run.

Plugin Interface

- Subscribes to the VPN plugin's 'on_bind' event.
- Writes command output to *<outdir>/run_on_bind_output.txt*, one block per command per attempt, each carrying a *Status:* of SUCCESS or FAILED.
- Exposes *succeeded* (bool) and *attempts* (int) for other plugins or tests to inspect.

class pyplugins.actuation.run_on_bind.**RunOnBind**

Bases: *Plugin*

class Args(*, *commands*, *endpoints=None*, *ports=[80, 443]*, *proto='tcp'*, *delay=None*, *timeout=None*, *shutdown_after_cmd=False*)

Bases: *PluginArgs*

Parameters

- **commands** (*Any*)
- **endpoints** (*List[str] | None*)
- **ports** (*List[int] | None*)
- **proto** (*str*)
- **delay** (*int | None*)
- **timeout** (*int | None*)
- **shutdown_after_cmd** (*bool*)

commands: Any

delay: int | None

endpoints: List[str] | None

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

ports: List[int] | None

proto: str

shutdown_after_cmd: bool

timeout: int | None

on_bind_handler(*proto*, *guest_ip*, *guest_port*, *host_port*, *host_ip*, *procname*)

Handle a bind event from the VPN plugin and, if it matches, queue an attempt.

Args:

proto (str): Protocol (e.g., 'tcp'). guest_ip (str): Guest IP address. guest_port (int): Guest port. host_port (int): Host port mapped to the guest service. host_ip (str): Host IP address. procname (str): Name of the process binding the port.

Parameters

- **proto** (*str*)
- **guest_ip** (*str*)
- **guest_port** (*int*)
- **host_port** (*int*)
- **host_ip** (*str*)
- **procname** (*str*)

Return type
None

run_commands(*guest_endpoint*, *host_port*)

Execute every configured command once and report whether any of them succeeded.

A single success resolves the run. Configurations routinely include commands that only apply to some targets, so requiring every one to pass would keep retrying endpoints on account of commands that were never going to work.

Args:
guest_endpoint (str): Endpoint that triggered this run, for the output artifact. host_port (int): Host port mapped to the triggering guest service,

handed to host commands via *{host_port}* and *PENGUIN_HOST_PORT*.

Returns:
bool: True if at least one command succeeded.

Parameters

- **guest_endpoint** (*str*)
- **host_port** (*int*)

Return type
bool

worker()

Drain queued endpoints one at a time, stopping at the first successful run.

Serializing here is deliberate: concurrent guest_cmd.py invocations against the same guest are a reliable way to wedge the emulation.

Return type
None

pyplugins.actuation.run_on_bind.**endpoint_matches**(*guest_ip*, *guest_port*, *allowlist*)

Check a bind event against the endpoint allowlist.

An empty allowlist matches everything. Either component of an allowlist entry may be '*'.

Parameters

- **guest_ip** (*str*)
- **guest_port** (*int*)
- **allowlist** (*List[str]*)

Return type
bool

pyplugins.actuation.run_on_bind.**normalize_commands**(*spec*)

Normalize the three supported *commands* formats into a single structured form.

Args:
spec: A string, a list of strings, a list of {mode, cmd} dicts, or None.

Returns:
list[dict]: Entries of the form {"mode": "host"|"guest", "cmd": [str, ...]}. None and empty input map to an empty list; the plugin, not this function, decides whether that is an error.

Raises:
ValueError: If the spec is malformed or mixes formats.

Parameters
spec (*Any*)

Return type
List[*Dict*[str, *Any*]]

pyplugins.actuation.run_on_bind.**parse_endpoints**(*entries*)

Validate and normalize the *endpoints* allowlist.

Args:
entries: List of 'guest_ip:guest_port' strings, where either side may be '*'.

Returns:
list[str]: The validated entries.

Raises:
ValueError: If any entry is malformed.

Parameters
entries (*List[str] | None*)

Return type
List[str]

pyplugins.actuation.vpn module

VPN Plugin (vpn.py) for Penguin

This module provides the VPN plugin, which enables vsock-based VPN bridging between the emulated guest and the host. It manages port mappings, source IP spoofing, and dynamic forwarding of guest network services to the host.

Features

- Launches and manages host-side VPN and vsock bridge processes.
- Handles port mappings from guest to host, including privileged ports and user-defined mappings.
- Supports source IP spoofing for guest services.
- Dynamically bridges guest network binds to the host and exposes them.
- Logs and tracks active network bridges and listeners.
- Maintains an `endpoints.txt` listing (one `host -> guest` line per bridge) and a `guest_ip` file in the output directory, regenerated as bridges are established.
- Cleans up VPN processes on exit.

Arguments

- log (bool, optional): Enable logging of VPN traffic.
- pcap (bool, optional): Enable PCAP capture of VPN traffic.
- verbose (bool, optional): Enable verbose logging.
- **IGLOO_VPN_PORT_MAPS (str, optional): Comma-separated port mapping rules.**

  This variable allows explicit mapping of guest services to host ports. The format is:
  <proto>:<host_port>:<guest_ip>:<guest_port>

  For example:
  IGLOO_VPN_PORT_MAPS="TCP:80:192.168.0.1:80,udp:20002:192.168.0.1:20002"

  This maps TCP port 80 on the host to TCP port 80 on the guest at 192.168.0.1, and UDP port 20002 on the host to UDP port 20002 on the guest at 192.168.0.1. If not provided as an argument, the plugin will look for it in the environment variables. If neither is set, default port mapping logic is used.
- spoof (dict, optional): Source IP spoofing configuration.
- conf (dict): Configuration dictionary for the emulation environment.

Plugin Interface

- Registers for the "on_bind" event to dynamically bridge guest network binds.
- Publishes "on_bind" events for other plugins to react to new host-exposed services.
- Writes bridge information to a CSV file in the output directory.

Overall Purpose

The VPN plugin enables flexible, dynamic, and secure exposure of guest network services to the host, supporting advanced features like port mapping and source IP spoofing, and integrates with the Penguin plugin system for event-driven networking.

class pyplugins.actuation.vpn.**VPN**(*panda*)

Bases: *Plugin*

class Args(*, *IGLOO_VPN_PORT_MAPS=None*, *log=False*, *pcap=False*, *spoof=None*, *interfaces=None*, *routes=None*, *default_interface=None*)

Bases: *PluginArgs*

Parameters

- **IGLOO_VPN_PORT_MAPS** (*str | None*)
- **log** (*bool*)
- **pcap** (*bool*)
- **spoof** (*Dict | None*)
- **interfaces** (*Dict | None*)
- **routes** (*Dict | None*)
- **default_interface** (*str | None*)

IGLOO_VPN_PORT_MAPS: str | None

default_interface: str | None

interfaces: Dict | None

log: bool

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

pcap: bool

routes: Dict | None

spoof: Dict | None

bridge(*sock_type*, *ip*, *guest_port*, *procname*, *ipvn*)

Bridge a guest socket to the host, set up event and return host port.

Args:
sock_type (str): Socket type. ip (str): Guest IP address. guest_port (int): Guest port. procname (str): Process name. ipvn (int): IP version.

Returns:
int: Host port mapped to the guest socket.

Parameters

- **sock_type** (*str*)
- **ip** (*str*)
- **guest_port** (*int*)
- **procname** (*str*)
- **ipvn** (*int*)

Return type
int

ensure_dev_has_ip(*ip*, *dev*, *ipvn*)

Ensure the specified device has the given IP address.

Args:
ip (str): IP address to assign. dev (str): Device name. ipvn (int): IP version (4 or 6).

Raises:
RuntimeError: If unable to query or assign the IP address.

Parameters

- **ip** (*str*)
- **dev** (*str*)
- **ipvn** (*int*)

Return type
None

launch_host_vpn(*CID*, *socket_path*, *uds_path*, *log=False*, *pcap=False*)

Launch vhost-device-vsock and VPN on host.

Args:

CID (int): Context ID for vsock. socket_path (str): Path to the vsock socket. uds_path (str): Path to the Unix domain socket for QEMU. log (bool, optional): Enable logging of VPN traffic. pcap (bool, optional): Enable PCAP capture of VPN traffic.

Parameters

- **CID** (*int*)
- **socket_path** (*str*)
- **uds_path** (*str*)
- **log** (*bool*)
- **pcap** (*bool*)

Return type

None

load_state(*data*)

Stash bridge state captured at snapshot time; replayed in on_restore.

Return type

None

map_bound_socket(*sock_type*, *ip*, *guest_port*, *procname*)

Map a guest socket to a host port, handling privileged ports and collisions.

Args:

sock_type (str): Socket type. ip (str): Guest IP address. guest_port (int): Guest port. procname (str): Process name.

Returns:

int: Host port mapped to the guest socket.

Parameters

- **sock_type** (*str*)
- **ip** (*str*)
- **guest_port** (*int*)
- **procname** (*str*)

Return type

int

on_bind(*sock_type*, *ipvn*, *ip*, *port*, *procname*)

Handle a new bind event from the guest, set up bridges and publish events.

Args:

sock_type (str): Socket type (e.g., 'tcp', 'udp'). ipvn (int): IP version (4 or 6). ip (str): Guest IP address. port (int): Guest port. procname (str): Process name binding the port.

Parameters

- **sock_type** (*str*)

- **ipvn** (*int*)
- **ip** (*str*)
- **port** (*int*)
- **procname** (*str*)

Return type
None

on_restore(*tag*)

Re-establish bridges after a snapshot restore.

The restored guest is already past its binds, so NetBinds won't re-emit on_bind events. We replay the recorded bridges, pinning each to the same host port via fixed_maps so host-side URLs stay stable across restore. Live connections are not (and cannot be) replayed.

Parameters
tag (*str*)

Return type
None

save_state()

Capture the active bridges (and IP-discovery state) so they can be re-established on the same host ports after a snapshot restore.

uninit()

Cleanup and terminate VPN and vsock bridge processes.

Return type
None

pyplugins.actuation.vpn.**guest_cmd**(*cmd*, *timeout=30.0*)

Run a command in the guest using guesthopper.

Bounded on both ends: `--timeout 20` caps the command guest-side, and the subprocess `timeout` caps the host-side wait, so a guest agent that is not up yet or has wedged can't block the caller forever (this runs during bridge setup). A timeout is surfaced as a failed run, not an exception, so callers take their normal error path.

Args:
cmd (str): Command to run in the guest. timeout (float): Max seconds to wait host-side.

Returns:
subprocess.CompletedProcess: Result of the command execution.

Parameters

- **cmd** (*str*)
- **timeout** (*float*)

Return type
CompletedProcess

pyplugins.actuation.vpn.**kill_vpn**()

Kill all running VPN processes registered in running_vpns.

Return type
None

pyplugins.analysis package

Submodules

pyplugins.analysis.crashes module

Crashes Plugin (crashes.py) for Penguin

This module provides the Crashes plugin, which records fatal signal deliveries to guest processes – userland crashes – to `crashes.yaml` in the output directory. Before this, the only crash signal Penguin recorded was a kernel panic; a guest service dying from SIGSEGV left no artifact.

The plugin registers guest signal-delivery hooks (via the SignalMonitor plugin / igloo_driver) for a configurable set of fatal-by-default signals and aggregates deliveries, de-duplicating identical (process, signal, pc) triples with a count – mirroring how NetBinds de-duplicates binds.

The recorded `pc` comes from the target task's saved userspace register frame (`task_pt_regs`) at signal-delivery time. For synchronous faults (SIGSEGV/SIGBUS/SIGILL/SIGFPE) that frame is the exception trap frame, so the pc is the faulting instruction address. For asynchronous signals (e.g. a `kill`) it is wherever the task last entered the kernel.

Output `crashes.yaml`:

```
crashes:
- proc: httpd
  pid: 412
  signal: 11
  signame: SIGSEGV
  pc: '0x004013a8'
  time: 12.481      # host wall-clock seconds since emulation start
  count: 3          # de-duplicated identical (proc, signal, pc)
```

The file is rewritten on every recorded delivery (crashes are rare), so it is always current; an empty `crashes: []` is written at startup so downstream consumers can rely on the file existing.

Caveats

- **Deliveries, not terminations.** The underlying driver hook fires when a signal is *dequeued for delivery*, before the kernel applies its disposition, and the event does not say whether a userspace handler is installed. A process that catches SIGSEGV/SIGABRT/etc. and survives (e.g. a runtime using SIGSEGV for GC barriers, or a daemon with an abort handler) is still recorded. Treat a crashes.yaml row as "a fatal-class signal was delivered", not proof the process died.
- Deliveries another subscriber has already marked dropped (`event.drop`) are skipped, but publish order between subscribers is not deterministic, so a drop made *after* this plugin runs is still recorded.
- `time` is host wall-clock seconds since this plugin initialized (i.e. emulation start), not guest uptime.

Arguments

- signals (list of str, optional): Signal names to record. Defaults to the signals whose default action is to terminate with a core dump and that indicate a program fault: SIGSEGV, SIGBUS, SIGILL, SIGABRT, SIGFPE, SIGSYS. Names are resolved per guest architecture (MIPS numbering differs), so always configure by name, not number.

Overall Purpose

A crashing service is one of the most common reasons a rehost "runs" but produces no bound port. This plugin makes those crashes visible in the output directory and in the run score (see `manager.calculate_score`).

class pyplugins.analysis.crashes.**Crashes**

Bases: *Plugin*

class Args(*, *signals=['SIGSEGV', 'SIGBUS', 'SIGILL', 'SIGABRT', 'SIGFPE', 'SIGSYS']*)

Bases: *PluginArgs*

Parameters
signals (*list[str]*)

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

signals: list[str]

on_signal_deliver(*cpu*, *event*)

Record a fatal signal delivery. SignalMonitor publishes every hooked delivery (including ones registered by other plugins), so filter to our watched set here.

uninit()

write_report()

pyplugins.analysis.env module

class pyplugins.analysis.env.**EnvTracker**

Bases: *Plugin*

Track environment variables that appear to be read and store them in missing_output if they aren't in our env

addvar(*cpu*, *match*)

mtd_addvar(*cpu*, *match*)

on_getenv(*cpu*, *s*)

on_strstr(*cpu*, *s1*, *s2*)

uboot_addvar(*cpu*, *match*)

static uboot_var_interesting(*var*)

uninit()

static var_interesting(*var*)

class pyplugins.analysis.env.**TargetCmp**

Bases: *Plugin*

Our 2nd pyplugin for environemnt tracking.

Here we examine config to see if any env variable is set to our magic value. If so, we'll track values it's comapred to using the C++ targetcmp plugin. We'll also try to detect comparisons with library hooks, though those don't have much success.

Most of the time this plugin doesn't do anything since there's no magic value in our env

static filter_env_var_values(*target_key*, *values*)

on_string_compare(*cpu*, *s*)

LD_PRELOAD based hooks for strcmp/strncmp the guest strcmp/strncmps s to our DYNVAL string

uninit()

pyplugins.analysis.ficd module

class pyplugins.analysis.ficd.**FICD**

Bases: *Plugin*

FICD metric based on Pandawan, see https://github.com/BUseclab/Pandawan/blob/main/plugins/pandawan/ficd.py The goal here is to produce a faithful representation of that metric in PENGUIN

"To this end, FICD considers that a firmware image reached Ifin in if no previously unseen (i.e., unique) tasks are launched within tf seconds. We refer to tf as the time frame parameter" "In our re-hosting experiments we use three (Py)PANDA plugins (coverage, syscalls_logger, and SyscallToKmodTracer) along with the FICD plugin, which results in the optimal tf = 220sec and tf = 300sec"

class Args(*, *stop_on_if=False*)

Bases: *PluginArgs*

Parameters
stop_on_if (*bool*)

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

stop_on_if: bool

on_exec(*newproc*)

reset()

uninit()

pyplugins.analysis.health module

class pyplugins.analysis.health.**Health**

Bases: *Plugin*

health_detect_opens(*cpu*, *fname*, *fd*)

health_exec_event(*event*)

health_on_bind(*cpu*, *is_ipv4*, *port*)

increment_event(*event*)

Increment the score for the given event

log_dev_open(*fname*)

on_ipv4_bind(*cpu*, *port*, *is_steam*)

on_ipv6_bind(*cpu*, *port*, *is_steam*)

uninit()

pyplugins.analysis.hooklogger module

HookLogger Plugin

"Easy mode" for dynamic binary instrumentation: register uprobes and syscall hooks from printf-style *action strings* instead of writing callbacks.

> sys_read(%fd, %s:out, %d) = %d

It handles per-arch argument retrieval, memory dereferencing, endianness, and pairing entry/exit probes to capture return values.

The action-string grammar, the format specifiers, deferred (`:out`) resolution, the filters, and both drivers (the `breakpoint` CLI and the Python API) are documented in `docs/hooklogger.md`:

> penguin docs –filename hooklogger.md

Keep that page as the reference; this docstring is deliberately a summary so the two cannot drift.

class pyplugins.analysis.hooklogger.**HookLogger**

Bases: *Plugin*

HookLogger Plugin

The "easy mode" for dynamic instrumentation.

Transforms simple action strings (e.g., 'print(%s, %d)') into complex, reliable uprobes and syscall hooks.

Features: - Parses action strings to determine what data to capture. - Automatically handles memory resolution (dereferencing pointers) at entry. - **Deferred Resolution**: Use '%s:out' to capture a pointer at entry but read

> the string at return (useful for output buffers like in read()).

- Manages context stacks to safely print return values ('func() = ret').

disable_all()

disable_hook(*hook_id*)

list_hooks()

register_syscall(*name*, *action_str*, *pid_filter=None*, *process_filter=None*, *logfile=None*)

register_uprobe(*path*, *symbol*, *action_str*, *pid_filter=None*, *process_filter=None*, *logfile=None*)

pyplugins.analysis.indiv_debug module

class pyplugins.analysis.indiv_debug.**IndivDebug**

Bases: *Plugin*

Plugin for debugging individual programs

This plugin is for running debugging tools such as strace, ltrace, and gdbserver on individual processes rather than the entire system. You might want to do this if, for example, you want to debug the HTTP server process without the overhead or loss of fidelity from tracing the entire system.

pyplugins.analysis.indiv_debug.**guest_cmd**(*cmd*)

Run a command in the guest, logging output to a file.

These debug helpers – gdbserver, strace/ltrace, and the blocking wait-for-attach loops – run for the whole debug session, well past the agent's default command timeout. Opt out of the cap with –timeout 0 so the agent never kills them out from under us.

pyplugins.analysis.interfaces module

Interfaces Plugin (interfaces.py) for Penguin

This module provides the Interfaces plugin, which tracks and logs missing network interfaces and failing ioctl operations in the guest environment. It is intended for use with the Penguin analysis framework and is implemented as a plugin.

Features

- Detects and logs missing network interfaces accessed by the guest.
- Tracks and logs failing ioctl operations related to network interfaces.
- Integrates with syscall and exec event hooks for comprehensive coverage.
- Supports filtering and validation of interface names.

Usage

The plugin is loaded by the Penguin framework and responds to relevant syscall and exec events.

Arguments

- outdir: Output directory for logs.
- conf: Configuration dictionary.
- verbose: Enables debug logging.

Output

- Logs missing interfaces to *iface.log*.
- Logs failing ioctl operations to *iface_ioctl.log*.

Example

```
from penguin import plugins
plugins.load("analysis.interfaces", outdir="/tmp", verbose=True)
```

class pyplugins.analysis.interfaces.**Interfaces**

Bases: *Plugin*

after_ioctl(*regs*, *proto*, *syscall*, *fd*, *request*, *arg*)

failing_ioctl(*ioctl*, *iface*, *rv*)

handle_interface(*iface*)

iface_on_exec(*event*)

pyplugins.analysis.netbinds module

NetBinds Plugin (netbinds.py) for Penguin

This module provides the NetBinds plugin, which monitors and records the full lifecycle of network services within the guest during emulation. It tracks both IPv4 and IPv6 sockets, recording when they are *bound* (opened) and when they are *released* (closed), and publishes 'on_bind' events for other plugins to react to new network services.

Lifecycle tracking

Each socket (keyed by IP version, socket type, address, and port) moves through a small state machine:

- `pending` – bound, but not yet announced. A new bind is held for `announce_debounce_s` seconds before 'on_bind' is published, so short-lived binds never reach downstream actuation (VPN bridging, nmap, fetchweb, ...). A socket still pending when the run ends keeps this state in the final CSVs: it was bound at shutdown but never confirmed as a listener, so consumers should not count it as a working service.
- `listening` – bound and announced: the bind survived the announce window.
- `transient` – bound and released *within* the announce window. Transient binds are never announced; they appear in `netbinds.csv` (state `transient`) so analyses can see them without treating them as working services.
- `closed` – was listening, later released for good.

Both debounce windows are measured in **host wall-clock time** (like every other timestamp this plugin records): guest/emulation time is not exposed to pyplugins, so the transient-vs-listening classification of a short-lived bind depends on emulation speed and is not exactly reproducible across hosts. The in-guest test fixtures set `announce_debounce_s: 0` for this reason; the debounce machinery itself is covered by host-side unit tests.

Announced services frequently flap: a process may close and re-bind the same address in rapid succession (e.g. supervisord restarting a daemon, or a server that briefly rebinds during startup). To avoid treating every flap as a genuine close, NetBinds also applies a *close debounce*:

- When a listening socket is released, the close is held as *pending* rather than being finalized immediately.
- If the same socket is re-bound within `debounce_period` seconds, the close is cancelled and the event is recorded as a *flap* (the service is considered to have stayed up continuously).
- A pending close older than `debounce_period` is *finalized* into a real close. Finalization happens opportunistically when later events arrive, and unconditionally when the plugin unloads.

A socket that flaps `transient_threshold` or more times during the run is also flagged transient in the lifecycle summary, marking it as an unstable service worth attention.

Known limitation: the guest kernel only reports TCP releases for sockets in the LISTEN state (accepted connections share the listener's local port, so their teardown must not be reported). A TCP socket that binds but closes without ever listening therefore never emits a release and remains `listening`/`pending` until the end of the run.

Snapshot / restore

A restored VM is already past its binds, so it never re-issues the bind hypercalls this plugin watches. NetBinds therefore participates in the snapshot host-state protocol (`save_state` / `load_state` / `on_restore`): the socket lifecycle is bundled with the snapshot and rebuilt on restore, so `netbinds.csv`, the lifecycle summary, and `give_list` stay accurate across a (possibly cross-process) restore. Already-announced services are rebuilt silently – downstream consumers such as the VPN bridge replay their own saved state, so re-announcing would double-actuate – while a service still within its announce window at snapshot time is a ground-truth listener in the restored guest and is announced on restore.

Features

- Subscribes to low-level bind/setup/release events for IPv4 and IPv6 sockets.
- Tracks and deduplicates first-seen binds (process name, IP version, socket type, IP, port), as before.
- Logs announced binds and summary statistics to CSV files in the output directory (`netbinds.csv`, `netbinds_summary.csv`). `netbinds.csv` carries `state` and `closed_time` columns and is rewritten on unload with each socket's final state (including never-announced transient binds).

- Logs every open/flap/transient/close transition to `netbind_events.csv` and writes a per-socket lifecycle summary to `netbinds_lifecycle.csv` on unload.
- Publishes 'on_bind' events for other plugins (such as VPN, Nmap, FetchWeb) after the announce debounce.
- Optionally shuts down emulation when a web service (port 80) is announced.

Arguments

- shutdown_on_www (bool, optional): If True, shut down emulation when a bind on port 80 is announced (i.e. survives the announce debounce).
- announce_debounce_s (float, optional): Seconds a new bind is held before being announced via 'on_bind'. A release within this window marks the bind transient and suppresses the announcement. <= 0 announces immediately. Default: 2.0.
- debounce_period (float, optional): Seconds a close is held pending before being treated as a real close. A re-bind within this window is a flap. Default: 2.0.
- transient_threshold (int, optional): Number of flaps at which a socket is labelled transient. Default: 3.

Plugin Interface

- Publishes 'on_bind' events with (sock_type, ipvn, ip, port, procname) for other plugins to consume. (Semantics unchanged apart from the announce debounce: fired once per first-seen unique bind that survives the window.)
- Listens to low-level system bind/setup/release events.
- Writes bind logs, a lifecycle event log, and summaries to the output dir.

Overall Purpose

The NetBinds plugin provides a comprehensive record of the network services started – and stopped – by the guest, enabling automation, analysis, and integration with other actuation plugins.

class pyplugins.analysis.netbinds.**NetBinds**

Bases: *Plugin*

class Args(*, *shutdown_on_www=False*, *announce_debounce_s=2.0*, *debounce_period=2.0*, *transient_threshold=3*)

Bases: *PluginArgs*

Parameters

- **shutdown_on_www** (*bool*)
- **announce_debounce_s** (*float*)
- **debounce_period** (*float*)
- **transient_threshold** (*int*)

announce_debounce_s: float

debounce_period: float

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

shutdown_on_www: bool

transient_threshold: int

give_list()

Return the list of currently-active binds: listening, pending announcement, or closed-but-pending within the close debounce window.

Returns:
list: A list of dictionaries, each representing an active bind.

load_state(*data*)

Stash lifecycle state captured at snapshot time. Applied in on_restore(), which runs after every plugin has loaded.

Return type
None

on_bind(*cpu*, *procname*, *is_ipv4*, *is_stream*, *port_pid*, *sin_addr*)

Handle a completed bind event: decode it and feed the socket lifecycle. Announcement (CSV row, 'on_bind' publication, shutdown_on_www) happens after the announce debounce – see record_open.

Args:
cpu: The CPU core where the event occurred. procname: The name of the process that performed the bind. is_ipv4: Boolean indicating if this is an IPv4 bind. is_stream: Boolean indicating if this is a stream (TCP) bind. port_pid: "port:pid" string, port in network byte order. sin_addr: The IP address being bound, in network byte order.

Return type
None

on_ipv4_bind(*cpu*, *port*, *is_steam*)

Handle IPv4 bind event, trigger on_bind and clear pending state.

Args:
cpu: The CPU core where the event occurred. port: The port number being bound, in host byte order. is_steam: Boolean indicating if this is a stream (TCP) bind.

Return type
None

on_ipv4_release(*cpu*, *ip_port*, *is_stream*)

Handle IPv4 socket release event, record a (debounced) close.

Args:
cpu: The CPU core where the event occurred. ip_port: The IP:port string of the released socket. is_stream: Boolean indicating if this was a stream (TCP) socket.

Return type
None

on_ipv4_setup(*cpu*, *procname*, *sin_addr*)

Handle IPv4 socket setup event, record pending bind state.

Args:
cpu: The CPU core where the event occurred. procname: The name of the process attempting the bind. sin_addr: The IPv4 address being bound, in network byte order.

Return type
None

`on_ipv6_bind`(*cpu*, *port*, *is_steam*)

Handle IPv6 bind event, trigger on_bind and clear pending state.

Args:
cpu: The CPU core where the event occurred. port: The port number being bound, in host byte order. is_steam: Boolean indicating if this is a stream (TCP) bind.

Return type
None

`on_ipv6_release`(*cpu*, *ip_port*, *is_stream*)

Handle IPv6 socket release event, record a (debounced) close.

Args:
cpu: The CPU core where the event occurred. ip_port: The IP:port string of the released socket, e.g. "[::1]:80". is_stream: Boolean indicating if this was a stream (TCP) socket.

Return type
None

`on_ipv6_setup`(*cpu*, *procname*, *sinaddr_addr*)

Handle IPv6 socket setup event, record pending bind state.

Args:
cpu: The CPU core where the event occurred. procname: The name of the process attempting the bind. sinaddr_addr: The memory address where the IPv6 address is stored.

Return type
None

`on_restore`(*tag*)

Rehydrate the socket lifecycle after a snapshot restore.

Already-announced sockets (listening/closed) are restored *silently*: downstream consumers such as the VPN bridge replay their own saved state on restore, so re-publishing 'on_bind' for them would double-actuate. Sockets still 'pending' (bound but never announced) at snapshot time are ground-truth listeners in the restored guest that no downstream plugin saved, so they are promoted to 'listening' and announced now.

Parameters
tag (*str*)

Return type
None

`record_close`(*ipvn*, *sock_type*, *ip*, *port*)

Record a release (close) in the socket lifecycle. A release within the announce window marks the bind transient (never announced); a release of a listening socket is held pending until the close debounce elapses (see _sweep_pending_closes and uninit), so a quick re-bind can cancel it as a flap.

Return type
None

`record_open`(*now*, *time_delta*, *procname*, *ipvn*, *sock_type*, *ip*, *port*, *pid*)

Record a bind (open) in the socket lifecycle. New sockets are held in 'pending' for the announce debounce; re-binds shortly after a close are treated as flaps rather than new services.

Return type
None

`report_bind_info`(*time_delta*, *procname*, *ipvn*, *sock_type*, *ip*, *port*, *pid*)

Log bind details and summary statistics to disk. Called at announce time, so the row is written with state 'listening'; final states are filled in when the file is rewritten on unload.

Args:
time_delta: The time since emulation start when the bind was announced. procname: The name of the process that performed the bind. ipvn: The IP version (4 or 6) of the bind. sock_type: The type of socket (TCP or UDP). ip: The IP address being bound. port: The port number being bound.

Return type
None

`save_state`()

Capture the socket lifecycle so a cross-process snapshot restore can reproduce netbinds.csv, the lifecycle summary, and give_list().

A VM snapshot restores the *guest*, which is then already past its binds, so it never re-issues the bind hypercalls NetBinds reacts to. Without this, a restored run would start blank and every pre-snapshot service would vanish from the record. Non-serialisable / host-clock fields (the announce timer thread and the absolute pending_since / pending_close instants) are dropped – they cannot survive a cross-process, possibly cross-host restore; see on_restore.

`uninit`()

Finalize the socket lifecycle on unload and write the lifecycle summary.

Announce timers are cancelled; sockets still pending announcement keep the 'pending' state in the final CSVs – they were bound when the run ended but never confirmed as listeners (and never published). Any pending closes are finalized at their real (pending) close time, and any sockets still open at shutdown have their uptime closed out at the end of the run.

Return type
None

pyplugins.analysis.processes module

Processes Plugin (processes.py) for Penguin

A small, queryable model of the guest's **process tree** – the first consumer of the OSI suite that builds a system-wide view rather than reading one pid at a time. This is *slice 1* of the "system cartography" epic (threads/kthreads, fd/peer graph, maps, CPU and ptregs are deliberate follow-ons – see `Follow-ons` below).

Two audiences, one model

AI / the MCP agent (#835) get a stable, documented, JSON-serializable query API – three **live** portal generators (drive with `yield from` while the guest runs):

- `processes.get(pid)` -> one flat record (dict) or `None`
- `processes.tree()` -> `{"roots": [node, ...]}` (nested; each node has `children`)
- `processes.snapshot()` -> `{"processes": [record, ...], "tree": {...}}`

These read the whole process set in **one** kernel-side walk (`OSI.get_all_procs` -> `HYPER_OP_OSI_PROC_ALL`): the driver walks `for_each_process` under `rcu_read_lock` and returns a slim node (pid, ppid, create_time, ids, comm) per process, so a tree snapshot is a single RCU-consistent transaction rather than 1 + N per-pid reads that could tear.

Users get a legible artifact. Process lifecycle is recorded to the event **database** (`plugins.db`) as it happens, and at teardown the plugin materializes a derived `system_map.yaml` from it: structured per-process records plus a rendered ASCII tree embedded as a literal block:

```
schema_version: 1
generated_by: processes
process_count: 3
processes:
- {pid: 1, ppid: 0, name: init, create_time: 100, exec_count: 1, exit: null, ...}
...
tree: |
  init (1)
  `- httpd (400)
     `- status.cgi (517)  [exit: 0]
```

Persistence model (DB-backed)

The plugin depends on the `DB` logger plugin and emits two lean events (`pengutils.events`):

- `exec_event` (from Execs) -> a `ProcStart` row (identity + genealogy: pid, ppid, create_time, comm, ids). argv/env stay on the existing `Exec` event; this row carries only what the tree needs. Re-exec appends another `ProcStart` with the same `(pid, create_time)`; the derived view coalesces by that key.
- `exit` / `exit_group` syscalls -> a `ProcExit` row. Identity comes off the syscall event (`syscall.pid` / `syscall.create_time`, denormalized by the driver from `current`), so exit recording costs **zero** portal round-trips and never reads the dying task's memory.

`system_map.yaml` is the derived view: `ProcStart LEFT JOIN ProcExit` on `(pid, create_time)`, rendered at teardown. Because plugins unload in reverse load order (this plugin's `uninit` runs before the DB's own final flush), the teardown path calls `plugins.db.flush()` first to make buffered rows visible.

Driver-side safety

Both the live walk and the exit path touch only kernel `task_struct` state – `get_all_procs` copies `task->comm` (like `get_proc`) and skips `!task->mm` (kthreads = slice 2); the exit path reads a denormalized pid. No `access_remote_vm` on userspace, so nothing faults on an exiting/stopped context (the class of bug behind the rv130 `read_procargs` panic).

Arguments

- `outdir` (str): output directory (supplied by the framework).
- `write_map` (bool, default True): write `system_map.yaml` at teardown.

Follow-ons (out of scope for slice 1)

Threads + kernel threads (the one further driver change), fd + peer/resource graph, maps + loaded-lib inventory, CPU/scheduling, `get_ptregs` – slices 2-6 of the epic.

class pyplugins.analysis.processes.**Processes**

Bases: *Plugin*

Maintains and queries the guest process tree (slice 1: tree + get).

get(*pid=None*)

Return a flat record for `pid` (or the current process if None).

Live read via `get_proc` – kernel `task_struct/comm` only, safe against exiting contexts. Returns `None` if the pid is not present.

Return schema (all ints except `name`):

```
{pid, ppid, name, create_time, start_time, uid, gid, euid, egid}
```

Parameters
pid (*int | None*)

Return type
Generator[*Any*, None, *Dict*[str, *Any*] | None]

on_exec_event(*event*)

Record a process from an `exec_event` as a `ProcStart` row.

The payload carries `proc` (osi_proc: pid/ppid/create_time/ids) and the new program's `procname/argv`; identity is from `proc` (stable kernel fields), the display name from the exec payload.

Parameters
event (*Any*)

Return type
None

on_exit(*regs*, *proto*, *syscall*, *error_code*)

Thread exit -> ProcExit row. Registered in `__init__` only when the do_exit hook is not in use; do_exit supersedes it otherwise.

Parameters

- **regs** (*Any*)
- **proto** (*Any*)
- **syscall** (*Any*)
- **error_code** (*int*)

Return type
Generator[*Any*, None, None]

on_exit_group(*regs*, *proto*, *syscall*, *error_code*)

Whole-process exit -> ProcExit row. Registered in `__init__` only when the do_exit hook is not in use.

Parameters

- **regs** (*Any*)
- **proto** (*Any*)
- **syscall** (*Any*)
- **error_code** (*int*)

Return type
Generator[*Any*, None, None]

on_proc_exit(*cpu*, *event*)

Authoritative task-exit -> ProcExit row (do_exit kprobe).

Fires once per user-process death for exit, exit_group, AND fatal signals alike, carrying the real exit code (wait-status encoded) and the dying task's create_time – so it needs no (pid, create_time) reconstruction and cannot false-positive on a caught-and-survived signal. Active only when use_do_exit is set; supersedes the syscall hooks and the signal heuristic.

Parameters

- **cpu** (*Any*)
- **event** (*Any*)

Return type
None

on_signal_deliver(*cpu*, *event*)

Fatal-signal death -> ProcExit row.

Covers the process kills that exit/exit_group never see (SIGSEGV and friends). Runs in the signal_monitor's (non-portal) publish, so it only appends a buffered row – no guest read. event.drop means another subscriber bypassed the delivery (e.g. SIGILL emulation), so the process is not dying; skip it. Deduped per pid: the first fatal delivery wins (a dying process may be hit more than once).

Parameters

- **cpu** (*Any*)
- **event** (*Any*)

Return type
None

snapshot()

Return {"processes": [flat records], "tree": {...}} – the live "dump the whole process view" call for MCP/tooling.

Return type
Generator[*Any*, None, *Dict*[str, *Any*]]

tree()

Return the live process tree as {"roots": [node, ...]}.

Each node is a get() record plus a children list. A process whose parent is not among the reported leaders (or ppid 0) becomes a root. Cycles and self-parenting are broken defensively.

Return type
Generator[*Any*, None, *Dict*[str, *Any*]]

uninit()

Materialize system_map.yaml from the DB on unload.

Flushes the DB first (plugins unload in reverse load order, so our uninit runs before the DB's own final flush) then derives the tree from the ProcStart/ProcExit rows.

Return type
None

pyplugins.analysis.repl module

class pyplugins.analysis.repl.**Repl**(*panda*)

Bases: *Plugin*

repl()

Generator-capable IPython REPL

Call with *yield from repl()*. Use *%yield_from* and *%yield_* line commands in place of *yield from* and *yield*.

repl_thread(*command_queue*, *result_queue*, *local*)

Function for running the IPython REPL in a separate thread. The command queue is for sending *yield* and *yield from* commands to the generator, and the result queue is for getting the results back.

pyplugins.apis package

Penguin APIs Package

This package contains core API plugins for the Penguin framework, providing interfaces for interacting with guest memory, user-space probes, and other hypervisor-level features.

Overview

- **mem.py**: Safe and efficient guest memory access and manipulation.
- **uprobes.py**: User-space probe (uprobe) registration and event handling.
- *(Other API modules may be present for additional functionality.)*

These APIs are designed to be used as plugins, enabling advanced instrumentation, tracing, and analysis of guest systems in a modular and extensible way.

—

Submodules

pyplugins.apis.events module

Events API Plugin

This plugin provides a generic interface for handling hypercall-based events in the penguin system. It maps event numbers (MAGIC values) to named events and their argument types, sets up hypercall handlers, and allows other plugins to register callbacks for these events.

Purpose

- Maps hypercall event numbers to named events and argument types.
- Sets up hypercall handlers that parse arguments and publish events.
- Allows plugins to register for named events and receive parsed arguments.

Usage

This plugin is loaded automatically as part of the penguin plugin system. Other plugins can register for named events (e.g., igloo_open, igloo_string_cmp) and receive callbacks with parsed arguments.

Event Format

Each event is published with the parsed arguments as specified in the EVENTS mapping.

```
EVENTS = {
    MAGIC_NUMBER: ('event_name', (arg_type1, arg_type2, ...)),
    ...
}
```

For example:

```
0xB335A535: ('igloo_send_hypercall', (None, int, int)),
```

Example

```
from penguin import plugins

def on_open(cpu, filename, flags):
    print(f"Open: {filename} (flags={flags})")

plugins.subscribe(plugin_instance, "igloo_open", on_open)
```

class pyplugins.apis.events.**Events**

Bases: *Plugin*

Events Plugin

Handles hypercall-based events and publishes them as named events for other plugins to subscribe to.

Features

- Registers all known event names for notification.
- Sets up hypercall handlers for each event.
- Publishes events with parsed arguments to subscribers.

register_notify(*name*, *callback*)

Register a callback for an event.

Parameters

name
: [str] The event name.

callback
: [callable] The callback function to register.

Registers the callback for the specified event name. Raises a ValueError if the event name is not found.

pyplugins.apis.execs module

Execs

This plugin provides a generic interface for monitoring process execution events in the guest. It subscribes to syscall events for both `execve` and `execveat`, extracts execution details (process name, arguments, environment), and publishes this information to listeners via the plugin event system. Other plugins can subscribe to these events for custom analysis, logging, or automation.

Arguments

- `outdir` (`str`): Output directory for any optional logging or artifacts.

Plugin Interface

Publishes `exec_event` with a dictionary containing execution details:

```
{
    'procname': str or None,      # Name of the executed program (target of exec),␣
↪resolved via OSI if AT_EMPTY_PATH
    'proc': Wrapper,              # Process info wrapper for the process being exec'd
    'argv': list[str],            # Argument vector for the new program
    'envp': dict[str, str],       # Environment for the new program
    'raw_args': tuple,            # Raw syscall arguments to the handler
    'parent': Wrapper or None,    # Process info wrapper for the parent process
    'retval': int,                # Return value of the syscall (negative on failure)
}
```

Both execve and execveat syscalls are tracked and normalized into this unified event format.

Overall Purpose

The Execs plugin provides a flexible, extensible interface for tracking process execution events in the guest, enabling downstream plugins to implement their own analysis or response logic.

class pyplugins.apis.execs.**Execs**

Bases: *Plugin*

Execs Plugin

Monitors execve and execveat syscalls and publishes normalized execution events.

Publishes 'exec_event' with a dictionary containing:

procname (str or None): Name of the executed program (target of exec), resolved via OSI if AT_EMPTY_PATH proc (Wrapper): Process info wrapper for the process being exec'd argv (List[str]): Argument vector for the new program envp (Dict[str, str]): Environment for the new program raw_args (tuple): Raw syscall arguments to the handler parent (Wrapper or None): Process info wrapper for the parent process retval (int): Return value of the syscall (negative on failure)

on_execve(*regs*, *proto*, *syscall*, *fname_ptr*, *argv_ptr*, *envp_ptr*)

Callback for execve syscall. Delegates to shared handler.

Returns

None

Parameters

- **regs** (PtRegsWrapper)
- **proto** (*Any*)
- **syscall** (*int*)
- **fname_ptr** (*int*)
- **argv_ptr** (*int*)
- **envp_ptr** (*int*)

Return type
Generator[*Any*, None, None]

on_execveat(*regs*, *proto*, *syscall*, *dirfd*, *fname_ptr*, *argv_ptr*, *envp_ptr*, *flags*)

Callback for execveat syscall. Delegates to shared handler.

Returns

None

Parameters

- **regs** (PtRegsWrapper)
- **proto** (*Any*)
- **syscall** (*int*)
- **dirfd** (*int*)
- **fname_ptr** (*int*)
- **argv_ptr** (*int*)
- **envp_ptr** (*int*)
- **flags** (*int*)

Return type
Generator[*Any*, None, None]

pyplugins.apis.exit_monitor module

Exit Monitor Plugin (exit_monitor.py)

Surfaces guest task-exit events from the igloo_driver `do_exit` kprobe. Unlike the exit/exit_group *syscall* hooks, this fires on the kernel task-exit path, so it also captures deaths that never issue an exit syscall – fatal signals (SIGSEGV/SIGKILL/…), OOM kills, kernel-forced exits – each with the real, wait(2)-status-encoded exit code. One event per user process (reported at thread-group-leader death), so it maps 1:1 onto the leader-only process model.

This is the authoritative process-exit source: it replaces the host-side signal-death heuristic (which cannot distinguish a caught SIGSEGV from a fatal one) and needs no per-signal guesswork.

Usage

```
from penguin import plugins

@plugins.subscribe(plugins.exit_monitor, "proc_exit")
def on_exit(cpu, event):
    # event.code is the raw do_exit() code in wait-status encoding.
    print(f"pid {event.pid} ({event.comm}) exited, code={event.code:#x}")

plugins.exit_monitor.enable()   # opt in; costs nothing until enabled
```

class pyplugins.apis.exit_monitor.**ExitEvent**(*ee*)

Bases: `object`

Wrapper for struct exit_event from the guest driver.

comm

property core_dumped: bool

property exit_status: int

Normal-exit status (WEXITSTATUS); meaningful only if not signaled.

property signaled: bool

True if the process was killed by a signal (WIFSIGNALED).

property termsig: int

Signal number that killed the process, or 0 (WTERMSIG).

class pyplugins.apis.exit_monitor.**ExitMonitor**

Bases: *Plugin*

Interacts with the igloo_driver do_exit kprobe to surface task exits.

disable()

Return type
bool

enable()

Enable the guest do_exit hook (idempotent). Opt-in: nothing fires until at least one plugin calls this.

Return type
bool

pyplugins.apis.fs module

FS API Plugin

This module provides the FS plugin for the penguin framework, enabling interaction with the guest filesystem. It exposes methods for reading, writing, and querying files and directories in the guest, and can be used by other plugins to perform filesystem operations.

Features

- Read files from the guest filesystem.
- Write data to files in the guest.
- Execute programs in the guest environment.

- Supports chunked operations for large files.
- Abstracts guest filesystem access for analysis and automation.

Example Usage

```
from penguin import plugins

# Read a file from the guest
content = yield from plugins.fs.read_file("/etc/passwd")

# Write to a file in the guest
yield from plugins.fs.write_file("/tmp/test.txt", "hello world")

# Execute a program in the guest
yield from plugins.fs.exec_program("/bin/ls", argv=["ls", "-l", "/"])
```

class pyplugins.apis.fs.**FS**

Bases: *Plugin*

FS Plugin

Provides methods for interacting with the guest filesystem, including reading, writing, and listing files and directories.

Methods

read_file
Read a file from the guest filesystem.

write_file
Write data to a file in the guest filesystem.

exec_program
Execute a program in the guest environment.

Note

All methods are generated and their signatures and types are enforced.

exec_program(*exe_path=None*, *argv=None*, *envp=None*, *wait=False*)

Execute a program in the guest environment.

Parameters

exe_path
[str, optional] Path to executable. If not provided, uses *argv[0]*.

argv
[list of str, optional] List of arguments (including program name as first arg).

envp
[dict of str, optional] Dictionary of environment variables.

wait
[bool, optional] Whether to wait for program to complete.

Returns

int
Return code from execution.

Raises

Exception
If the program cannot be executed.

Executes a program in the guest using the kernel's *call_usermodehelper* function. Optionally waits for completion.

> **Note:** This method is generated and type-checked.

Parameters

- **exe_path** (*str*)
- **argv** (*list[str]*)
- **envp** (*dict[str, str]*)
- **wait** (*bool*)

Return type
int

read_file(*fname*, *size=None*, *offset=0*)

Read a file from the guest filesystem.

Parameters

fname
[str] Path to the file in the guest.

size
[int, optional] Size limit. If None, reads entire file.

offset
[int, optional] Offset in bytes where to start reading (default: 0).

Returns

bytes
The file data as bytes.

Raises

Exception
If the file cannot be read.

Reads the specified file from the guest filesystem, optionally limiting the read to a specific size and offset. If *size* is not specified, the entire file is read in chunks.

> **Note:** This method is generated and type-checked.

Parameters

- **fname** (*str*)

- **size** (*int*)
- **offset** (*int*)

Return type
bytes

write_file(*fname*, *data*, *offset=0*)

Write data to a file in the guest filesystem.

Parameters

fname
[str] Path to the file in the guest.

data
[bytes or str] Data to write to the file.

offset
[int, optional] Offset in bytes where to start writing (default: 0).

Returns

int
Number of bytes written.

Raises

Exception
If the file cannot be written.

Overwrites the file if it exists, or creates it if it does not. Handles large writes in chunks.

> **Note:** This method is generated and type-checked.

Parameters

- **fname** (*str*)
- **data** (*bytes | str*)
- **offset** (*int*)

Return type
int

pyplugins.apis.hypercall module

class pyplugins.apis.hypercall.**Hypercall**

Bases: *Plugin*

QEMU backend hypercall registry.

This replaces the old PANDA C hypercall plugin with a Penguin pyplugin that owns the magic-number to callback mapping. qemu_compat only forwards backend hypercall exits here.

bind_qemu_compat(*qemu_compat*)

Return type
None

dispatch(*cpu*, *nr*, *ret_ptr*)

Parameters
nr (*int*)

Return type
int

hypercall(*nr*)

Parameters
nr (*int*)

Return type
Callable[[*Callable*], *Callable*]

register(*nr*, *func*)

Parameters

- **nr** (*int*)
- **func** (*Callable*)

Return type
Callable

pyplugins.apis.kffi module

KFFI API Plugin

This module provides the KFFI (Kernel Foreign Function Interface) plugin for the Penguin framework. It enables calling kernel-space functions and interacting with kernel memory from user-space plugins. The KFFI plugin abstracts low-level kernel function invocation, argument marshalling, and result handling, allowing other plugins to perform advanced kernel introspection and manipulation.

Features

- Call arbitrary kernel functions with specified arguments.
- Read and write kernel memory.
- Marshal arguments and results between user-space and kernel-space.
- Supports type-safe function signatures and return values.
- Compile and inject dynamic C-structs/typedefs on the fly using DWARFFI.

Example usage

```
from penguin import plugins

# Call a kernel function (e.g., do_sys_open) with arguments
result = yield from plugins.kffi.call_function("do_sys_open", "/etc/passwd", 0, 0)
```

(continues on next page)

(continued from previous page)

```
# Read kernel memory at a specific address
data = yield from plugins.kffi.read_kernel_memory(0xffff888000000000, 64)

# Write to kernel memory
yield from plugins.kffi.write_kernel_memory(0xffff888000000000, b"\x90\x90\x90\x90")

# Load a custom struct layout into the emulator's architecture
plugins.kffi.cdef("struct my_payload { int a; char b[10]; };")
```

class pyplugins.apis.kffi.**KFFI**

Bases: *Plugin*

KFFI Plugin

Provides methods for calling kernel functions and interacting with kernel memory.

Methods

- `call_function`: Call a kernel function with arguments.

call(*func*, **args*)

Parameters

- **func** (*int | str*)
- **args** (*Any*)

Return type

Generator[*Any*, *Any*, *Any*]

call_kernel_function(*func*, **args*)

Call a kernel function dynamically with the given arguments.

This uses the FFI mechanism to directly call kernel functions. CAUTION: This is extremely powerful and can easily crash the kernel if used incorrectly. Only call functions that are safe to call from arbitrary contexts.

Args:

func (int or str): Function address or name. *args (Any): Arguments to pass to the function (max 8).

Returns:

Any: Return value from the kernel function, or None if call fails.

Note:

This leaks memory. We should have a better policy on that.

Parameters

- **func** (*int | str*)
- **args** (*Any*)

Return type

Generator[*Any*, *Any*, *Any*]

callback(*func*, *func_type=None*)

Register a trampoline callback and return an integer guest virtual address.

Immediately generates the trampoline, sets up the interrupt handler, and returns an integer address.

Parameters
func_type (*Dict | Any | None*)

Return type
Generator[*Any*, *Any*, *Any*]

cdef(*source*)

Compile C definitions on the fly and load them into DWARFFI. Automatically handles architecture-specific compiler flags, musl headers, and caches the ISF output to speed up subsequent runs.

Args:
source (str): The C code containing structs/enums/typedefs to compile.

Parameters
source (*str*)

Return type
None

deref(*ptr*)

Dereference a pointer to a type.

Args:
ptr (Ptr): Pointer object.

Returns:
Any: Value pointed to, or None if pointer is null.

Parameters
ptr (*Ptr*)

Return type
Generator[*Any*, *Any*, *Any*]

from_buffer(*type_*, *buf*, *instance_offset_in_buffer=0*)

Create an instance of a type from a buffer.

Args:
type_ (str): Name of the type. buf (bytes): Buffer containing the data. instance_offset_in_buffer (int): Offset in buffer (default: 0).

Returns:
Any: Instance of the type.

Parameters

- **type_** (*str*)
- **buf** (*bytes*)
- **instance_offset_in_buffer** (*int*)

Return type
Any

`generate_trampoline()`

Request a trampoline from the kernel via portal.

Returns:
dict: Keys include `tramp_id`, `tramp_addr`, and `status`.

Return type
Generator[*Any*, *Any*, *Any*]

`get_callback_id(f)`

Get the trampoline ID for a registered callback function or trampoline address.

Args:
f (int | Any): Callback function or trampoline address.

Returns:
Optional[int]: Trampoline ID, or None if not found.

Parameters
f (*int | Any*)

Return type
int | None

`get_enum_dict(enum_name)`

Get dictionary of enum constants.

Args:
enum_name (str): Name of the enum.

Returns:
Wrapper: Wrapper containing enum constants.

Parameters
enum_name (*str*)

Return type
Wrapper

`get_field_casted(struct, field)`

Get a field from a struct, casted to its declared CFFI type.

Args:
struct (Any): Struct instance. field (str): Field name.

Returns:
Any: Field value, or None if error occurs.

Parameters

- **struct** (*Any*)
- **field** (*str*)

Return type
Any

`get_function_address`(*function*)

Get the address of a kernel function.

Args:
function (str): Name of the function.

Returns:
Optional[int]: Address of the function, or None if not found.

Parameters
function (*str*)

Return type
int | None

`get_struct_size`(*struct_name*)

Get the size of a struct.

Args:
struct_name (str): Name of the struct.

Returns:
Optional[int]: Size of the struct, or None if not found.

Parameters
struct_name (*str*)

Return type
int | None

`kallsyms_lookup`(*symbol*)

Look up a kernel symbol address using the `kallsyms_lookup` portal operation (simplified).

Args:
symbol (str): Name of the symbol to look up.

Returns:
Any: Address of the symbol as int, or None if not found.

Parameters
symbol (*str*)

Return type
Generator[*Any*, *Any*, *Any*]

`kfree`(*addr*)

Free memory in the kernel using `kfree`.

Args:
addr (Union[int, Ptr]): Address of memory to free.

Returns:
None

Parameters
addr (*int | Ptr*)

Return type
Generator[*Any*, *Any*, *Any*]

kmalloc(*size*)

Allocate memory in the kernel using `kmalloc`.

Args:
size (Union[int, Ptr]): Size of memory to allocate.

Returns:
Any: Address of allocated memory, or None if allocation fails.

Parameters
size (*int | Ptr*)

Return type
Generator[*Any*, *Any*, *Any*]

new(*type_*, *init=None*)

Create a new instance of a type.

Args:
type_ (str): Name of the type. init (Any): Initial value for the instance (optional).

Returns:
Any: Instance of the type, or None if type not found.

Parameters

- **type_** (*str*)
- **init** (*Any*)

Return type
Any

read_field(*addr*, *type_*, *field*)

Read a single field of a struct from kernel memory.

Parameters

- **addr** (*int | Ptr*)
- **type_** (*str*)
- **field** (*str*)

Return type
Generator[*Any*, *Any*, *Any*]

read_type(*addr*, *type_*)

Read a type from kernel memory.

Args:
addr (int): Address to read from. type_ (str): Name of the type.

Returns:
Any: Instance of the type, or None if read fails.

Parameters

- **addr** (*int*)
- **type_** (*str*)

Return type
Generator[*Any*, *Any*, *Any*]

read_type_panda(*cpu*, *addr*, *type_*)

Read a type from kernel memory using PANDA.

Args:
cpu (Any): CPU context. addr (int): Address to read from. type_ (str): Name of the type.

Returns:
Any: Instance of the type, or None if read fails.

Parameters

- **cpu** (*Any*)
- **addr** (*int*)
- **type_** (*str*)

Return type
Any

ref(*thing*)

Gets the address of an ffi type'd object (usually a struct)

Args:
thing (Any): Object.

Returns:
int: The address, or None if no address attribute.

Parameters
thing (*Any*)

Return type
int | None

sizeof(*struct_name*)

Alias for get_struct_size.

Args:
struct_name (str): Name of the struct.

Returns:
Optional[int]: Size of the struct, or None if not found.

Parameters
struct_name (*str*)

Return type
int | None

write_field(*addr*, *type_*, *field*, *val*)

Write a single field of a struct in kernel memory. Automatically handles offsets, endianness, and bitfield masking.

Parameters

- **addr** (*int | Ptr*)

- **type_** (*str*)
- **field** (*str*)
- **val** (*Any*)

Return type
Generator[*Any*, *Any*, None]

pyplugins.apis.kprobes module

Kprobes Plugin System

The Kprobes plugin provides a flexible interface for monitoring, filtering, and intercepting kernel-space function calls in the guest via the IGLOO hypervisor and portal interface. This enables dynamic kernel tracing, security monitoring, and program analysis at the function level inside the guest kernel.

Overview

The kprobes system operates through a multi-layer architecture:

1. **Kernel Layer**: The IGLOO kernel module registers kprobes on kernel functions identified by symbol name. The guest kernel resolves the symbol via `kallsyms`, applies an optional byte offset, and supports process and PID filtering.
2. **Hypervisor Layer**: The portal system manages communication between guest and host, relaying probe events.
3. **Plugin Layer**: Python plugins register kprobes and process events via callback functions.

Unlike uprobes, which hook user-space code by file path and offset, kprobes hook **kernel** functions by symbol name. There is no path or address resolution on the host side; the symbol string is sent to the guest kernel, which resolves it.

Quick Start

```
from penguin import plugins, Plugin

class MyKprobeMonitor(Plugin):
    def __init__(self):
        # Register a kprobe on the kernel function 'do_filp_open'
        @plugins.kprobes.kprobe(symbol="do_filp_open")
        def on_open_entry(pt_regs):
            print(f"do_filp_open() called! PC={pt_regs.pc:#x}")

        # Register a return probe on a kernel function
        @plugins.kprobes.kretprobe(symbol="do_filp_open")
        def on_open_return(pt_regs):
            print(f"do_filp_open returned! Return value: {pt_regs.retval}")
```

Core Concepts

Kprobe Events

A kprobe event is generated when a monitored kernel function is entered or returned from. Each event provides access to the CPU register state (`pt_regs`) at the probe point.

Probe Registration

Probes are registered using the `@kprobes.kprobe()` or `@kprobes.kretprobe()` decorators, supporting symbol-based selection, an optional byte offset, and process / PID filtering.

```
@kprobes.kprobe(
    symbol="do_filp_open",
    process_filter="target_app",
    on_enter=True,
    on_return=False,
    pid_filter=1234
)
def my_handler(pt_regs):
    # Handler code here
    pass
```

API Reference

Decorator Parameters

`symbol` (str)

Kernel symbol name to probe. The guest kernel resolves it via `kallsyms`. Some kernel functions carry compiler-generated suffixes (e.g. `do_execveat_common.isra.0`); register on the concrete name. You can discover the real name with:

```
for name in plugins.kffi.ffi.search_symbols("do_execveat_common*"):
    ...
```

`offset` (int)

Byte offset from the symbol at which to place the probe (default `0`). Return probes (`kretprobe`) force `offset=0`.

> **MIPS caveat — use `offset=0` (function entry).** MIPS has no hardware single-step, so the kernel single-steps a probed instruction *out of line* (it copies the instruction to a scratch slot, executes it there, then resumes). For instructions in the middle of a function this can resume with corrupted CPU state and crash the probed thread. In testing, an entry probe (`offset=0`) and `kretprobe` on a function are rock-solid on all MIPS variants, but a non-zero offset into some functions (e.g. `vfs_read+4`) reliably Oopses the guest kernel (`get_signal` NULL-deref → SIGSEGV → panic), while the identical probe on ARM/x86 runs fine. This is an inherent MIPS kprobe (single-step-out-of-line) limitation, not specific to Penguin. Prefer entry probes / kretprobes on MIPS; treat non-zero offsets as unsupported there.

`process_filter` (Optional[str])

Filter by process name (comm):

```
@kprobes.kprobe(symbol="do_filp_open", process_filter="target")
def monitor_open(pt_regs):
    pass
```

pid_filter (Optional[int])

Filter by specific process ID:

```
@kprobes.kprobe(symbol="do_filp_open", pid_filter=1234)
def monitor_pid(pt_regs):
    pass
```

on_enter / on_return (bool)

Control when the probe triggers:

```
# Entry only (default)
@kprobes.kprobe(symbol="do_filp_open", on_enter=True)

# Return only
@kprobes.kretprobe(symbol="do_filp_open")

# Both entry and return
@kprobes.kprobe(symbol="do_filp_open", on_enter=True, on_return=True)
```

When a probe is registered for both entry and return, the handler is invoked twice. Use the injected `is_enter` argument (see below) to distinguish the two.

read_only (bool)

If True, modifications to `pt_regs` made in the handler are not written back to the guest.

fail_register_ok (bool)

If True, do not raise an error if registration fails.

Handler Function Signature

The handler always receives `pt_regs` as its first argument. Additional context can be requested using "signature sugar": declaring extra positional or keyword parameters injects context values.

```
def handler(pt_regs):
    """pt_regs: register state at the probe point."""

def handler(pt_regs, is_enter):
    """is_enter: True on function entry, False on return."""

def handler(pt_regs, is_enter, tgid_pid):
    """tgid_pid: (tgid << 32) | tid for the triggering task."""

def handler(pt_regs, **ctx):
    """ctx contains is_enter and tgid_pid."""
```

The injection rules:

- 1 extra positional arg -> `is_enter`
- 2 extra positional args -> `is_enter`, `tgid_pid`
- Parameters explicitly named `is_enter` / `tgid_pid` are bound by name.

- A **kwargs parameter receives is_enter and tgid_pid.

Handlers may be plain functions or generators (yield from) so they can perform portal memory reads.

pt_regs Object

The pt_regs parameter provides architecture-agnostic access to the CPU register state at the probe point:

```
def my_handler(pt_regs):
    print(f"PC: {pt_regs.pc:#x}")
    print(f"SP: {pt_regs.sp:#x}")
    print(f"Return value: {pt_regs.get_return_value()}")  # On return probes
    a, b = pt_regs.get_args(2)                              # First two args
```

Note that kernel functions follow the kernel calling convention; get_args returns register-passed arguments for the architecture.

Practical Examples

Inspect a kernel path open

```
@kprobes.kprobe(symbol="do_filp_open")
def on_open(pt_regs):
    args = pt_regs.get_args(5)
    struct_filename = yield from plugins.kffi.read_type(args[1], "struct filename")
    path = yield from plugins.mem.read_str(struct_filename.name.address)
    print(f"opening {path}")
```

Check a kernel return value

```
@kprobes.kprobe(symbol="do_filp_open", on_enter=False, on_return=True)
def on_open_return(pt_regs):
    retval = int(plugins.kprobes.panda.ffi.cast("target_long", pt_regs.get_return_
↪value()))
    if retval == -2:  # -ENOENT
        print("open failed: no such file")
    if False:
        yield
```

Probe a symbol with a suffix

```
for name in plugins.kffi.ffi.search_symbols("do_execveat_common*"):
    plugins.kprobes.kprobe(symbol=name)(on_exec)
    break
```

Hook Management

Unregistering

Probes can be unregistered by handle (the decorator return value), by function, or by name:

```
handle = kprobes.kprobe(symbol="do_filp_open")(on_open)
...
kprobes.unregister(handle)          # by handle
kprobes.unregister(on_open)         # by function
kprobes.unregister("on_open")       # by name
```

Integration with Other Plugins

With KFFI

Use `plugins.kffi.read_type` / `read_type_panda` to decode kernel structs referenced by probe arguments, and `plugins.kffi.ffi.search_symbols` to resolve symbol names.

With OSI

Combine with OS introspection for process metadata:

```
def handler(pt_regs):
    proc = yield from plugins.osi.get_proc()
    print(f"kprobe hit in process {proc.name} (PID {proc.pid})")
```

Best Practices

1. **Be specific**: Probe only the kernel functions you need to minimize overhead.
2. **Filter early**: Use process and PID filters to reduce event volume.
3. **Mind symbol suffixes**: Inlined/cloned kernel functions may carry `.isra.N`, `.constprop.N`, etc. — resolve the concrete name.
4. **Handle errors**: Memory reads and symbol resolution can fail; guard accordingly.
5. **Use `is_enter`**: For combined entry+return probes, branch on `is_enter`.

The kprobes system provides a powerful foundation for dynamic kernel analysis, security monitoring, and program understanding in virtualized environments.

class pyplugins.apis.kprobes.**Kprobes**

Bases: *Plugin*

Kprobes Plugin

Provides an interface for registering and handling kernel-space probes (kprobes).

kprobe(*symbol*, *offset=0*, *process_filter=None*, *on_enter=True*, *on_return=False*, *pid_filter=None*, *read_only=False*, *fail_register_ok=False*)

Decorator to register a kprobe on a kernel function by symbol name.

The guest kernel resolves the symbol via kallsyms at registration time.

Parameters

symbol
[str] Kernel symbol name (resolved via kallsyms in the guest).

offset
[int] Offset (in bytes) from the symbol at which to place the probe (default: 0).

process_filter
[Optional[str]] Process name (comm) to filter events.

on_enter
[bool] Trigger on function entry (default: True).

on_return
[bool] Trigger on function return (default: False).

pid_filter
[Optional[int]] PID to filter events for a specific process.

read_only: bool
If True, register state modifications are not written back.

fail_register_ok
[bool] If True, silently ignore registration failures.

Returns

Callable[[Callable], Callable]
Decorator function that registers the kprobe.

Parameters

- **symbol** (*str*)
- **offset** (*int*)
- **process_filter** (*str | None*)
- **on_enter** (*bool*)
- **on_return** (*bool*)
- **pid_filter** (*int | None*)
- **read_only** (*bool*)
- **fail_register_ok** (*bool*)

Return type
Callable[[*Callable*], *Callable*]

kretprobe(*symbol*, ***kwargs*)

Parameters
symbol (*str*)

Return type
Callable

on_fs_init()

unregister(*target*)

Unregister a kprobe by handle, function, or name.

Args:
target: The handle (returned by decorator), function, or name of the kprobe to unregister.

Parameters
target (*Callable | str*)

pyplugins.apis.mem module

Memory Access Plugin (mem.py)

This module provides the Mem plugin for the Penguin framework, enabling safe and efficient reading and writing of guest memory via the hypervisor portal. It abstracts low-level memory operations, supporting chunked access, endianness handling, and architecture-specific pointer sizes. The plugin is designed to work with both PANDA and non-PANDA environments, and provides utilities for reading and writing bytes, integers, pointers, strings, arrays, and more.

Features

- Read and write arbitrary bytes to guest memory.
- Read and write integers, longs, words, and pointers with correct endianness.
- Read and write null-terminated and UTF-8 strings.
- Read arrays of integers and lists of pointers or strings.
- Memory comparison and memset utilities.
- Handles chunking for large memory operations.
- Supports both PANDA and hypervisor portal backends.

Example Usage

```
from penguin import plugins

# Read 16 bytes from address 0x1000
data = yield from plugins.mem.read_bytes(0x1000, 16)

# Write a string to memory
yield from plugins.mem.write_str(0x2000, "hello world")

# Read a pointer-sized value
ptr = yield from plugins.mem.read_ptr(0x3000)

# Compare two memory regions
equal = yield from plugins.mem.memcmp(0x4000, 0x5000, 32)
```

class pyplugins.apis.mem.**Mem**

Bases: *Plugin*

Mem Plugin

Provides guest memory access and manipulation via the hypervisor portal.

Reads never fail

By default every read here is a total function. Memory the guest cannot supply – unmapped, swapped out, a bad pointer, a driver-side failure – is reported as NUL bytes (or an empty/truncated string), never as an exception and never as `None`. That is deliberate: it keeps try/except out of hundreds of call sites, lets a generator read a pointer without a guard around it, and stops one plugin dereferencing garbage from taking a whole run down.

The cost is that a fabricated zero is indistinguishable from a real one. Usually that is the right trade. Sometimes it is not:

- a pointer that reads 0 looks like a genuine NULL, so a list walk stops at the first unmapped page and reports a short list as a complete one;
- a struct in unreadable memory decodes to an all-zero struct, which looks exactly like a valid zero-initialised one;
- a read-modify-write on such a struct writes the fabricated zeros *back*, so a bad read becomes a bad write;
- two unreadable regions compare equal.

For those cases every reader takes `required=True`, which returns `None` instead of fabricating. It is still not an exception – the caller checks a value, it does not wrap the call – and the default behaviour is untouched, so passing it is a local decision at the one call site that needs it:

```
# fine: a wrong value here is a wrong log line
val = yield from plugins.mem.read_int(addr)

# not fine: this decides whether to write to the guest
val = yield from plugins.mem.read_int(addr, required=True)
if val is None:
    return
```

`required` propagates through the derived readers, so `read_char_ptrlist(..., required=True)` fails if any pointer or any string underneath it could not be read.

Attributes

endian_format
[str] Endianness format for struct packing/unpacking.

try_panda
[bool] Whether to attempt PANDA-based memory access.

copy_buf_guest(*data*)

Copy a buffer into guest kernel memory and return the guest address.

Uses the HYPER_OP_COPY_BUF_GUEST portal command to allocate and copy a buffer into guest kernel memory. Handles chunking if the buffer is larger than the portal region.

Parameters

data
[bytes] The data to copy into guest memory.

Returns

int
Guest address of the copied buffer, or 0 on failure.

Parameters
data (*bytes*)

Return type
Generator[*Any*, *Any*, int]

memcmp(*addr1*, *addr2*, *size*, *pid=None*, *required=False*)

Compare two regions of guest memory for equality.

Compares two regions of guest memory for equality.

Parameters

addr1
[int or Ptr] First address.

addr2
[int or Ptr] Second address.

size
[int] Number of bytes to compare.

pid
[int, optional] Process ID for context.

Returns

bool
True if memory regions are equal, False otherwise.

Parameters

- **addr1** (*int | Ptr*)
- **addr2** (*int | Ptr*)
- **size** (*int*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, bool | None]

memset(*addr*, *value*, *size*, *pid=None*)

Set a region of guest memory to a specific byte value.

Sets a region of guest memory to a specific byte value.

Parameters

addr
[int or Ptr] Address to start setting.

value
[int] Byte value to set (0-255).

size
[int] Number of bytes to set.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **value** (*int*)
- **size** (*int*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

read(*addr*, *size=None*, *fmt=None*, *pid=None*, *required=False*)
Smart dispatcher for memory reads. Automatically infers sizes and types if a dwarffi Ptr is provided.

Parameters

addr
[int or Ptr] The source address or a dwarffi Ptr object.

size
[int, optional] Number of bytes (or elements) to read. Can be inferred from Ptr.

fmt
[type or str, optional] The requested return format (bytes, str, int, list, 'ptr'). Default is bytes.

pid
[int, optional] Process ID for context.

required
[bool, optional] Forwarded to whichever reader this dispatches to. Default False: unreadable memory is reported as zeros/an empty string. Pass True to get `None` instead. See the class docstring.

Returns

Any
The read data in the requested format, or `None` when `required` is set and the read could not be satisfied.

Parameters

- **addr** (*int | Ptr*)
- **size** (*int | None*)
- **fmt** (type | *str | None*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, *Any*]

`read_byte`(*addr*, *pid=None*, *required=False*)

Read a single byte from guest memory.

Reads a single byte from guest memory.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

required
[bool, optional] Whether the caller needs to know that the read failed. Default False: unreadable memory reads as zero and `None` is never returned. Pass True to get `None` instead.

Returns

int or None
Byte value read (0-255). `None` only when `required` is set and the read failed; otherwise unreadable memory reads as 0.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, int | None]

`read_bytes`(*addr*, *size*, *pid=None*, *required=False*)

Reads bytes from guest memory. Optimized with a Fast Path for single-chunk reads.

Parameters

addr
[int or Ptr] Address to read from.

size
[int] Number of bytes to read.

pid
[int, optional] Process ID for context.

required
[bool, optional] Whether the caller needs to know that the read failed. Default False, which is the total contract described in the class docstring: unreadable memory is reported as NUL bytes and the caller cannot fail. Pass True where a fabricated zero would be taken for real data – see the class docstring for when that matters.

Returns

bytes or None
Exactly `size` bytes. `None` only when `required` is set and the read could not be satisfied in full.

Parameters

- **addr** (*int | Ptr*)
- **size** (*int*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, bytes | None]

read_bytes_panda(*cpu*, *addr*, *size*)

Optimized PANDA read.

Parameters

- **addr** (*int | Ptr*)
- **size** (*int*)

Return type
bytes

read_char_ptrlist(*addr*, *length*, *pid=None*, *required=False*)

Read a list of null-terminated strings from a list of pointers.

Reads a list of null-terminated strings from a list of pointers in guest memory.

Parameters

addr
[int or Ptr] Address to start reading pointer list from.

length
[int] Maximum number of pointers to read.

pid
[int, optional] Process ID for context.

Returns

list of str
List of strings read from memory.

Parameters

- **addr** (*int | Ptr*)
- **length** (*int*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, *List*[str] | None]

`read_int`(*addr*, *pid=None*, *required=False*)

Read a 4-byte integer from guest memory.

Reads a 4-byte integer from guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

required
[bool, optional] Whether the caller needs to know that the read failed. Default False: unreadable memory reads as zero and `None` is never returned. Pass True to get `None` instead.

Returns

int or None
Integer value read. `None` only when `required` is set and the read failed; otherwise unreadable memory reads as 0.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, int | None]

`read_int_array`(*addr*, *count*, *pid=None*, *required=False*)

Read an array of 4-byte integers from guest memory.

Reads an array of 4-byte integers from guest memory.

Parameters

addr
[int or Ptr] Address to start reading from.

count
[int] Number of integers to read.

pid
[int, optional] Process ID for context.

Returns

list of int
List of integers read from memory.

Parameters

- **addr** (*int | Ptr*)
- **count** (*int*)

- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, *List*[int] | None]

read_long(*addr*, *pid=None*, *required=False*)

Read an 8-byte long integer from guest memory.

Reads an 8-byte long integer from guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

required
[bool, optional] Whether the caller needs to know that the read failed. Default False: unreadable memory reads as zero and `None` is never returned. Pass True to get `None` instead.

Returns

int or None
Long value read. `None` only when `required` is set and the read failed; otherwise unreadable memory reads as 0.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, int | None]

read_long_array(*addr*, *count*, *pid=None*, *required=False*)

Read an array of 8-byte long integers from guest memory.

Reads an array of 8-byte long integers from guest memory.

Parameters

addr
[int or Ptr] Address to start reading from.

count
[int] Number of longs to read.

pid
[int, optional] Process ID for context.

Returns

list of int
List of long integers read from memory.

Parameters

- **addr** (*int | Ptr*)
- **count** (*int*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, *List*[int] | None]

read_ptr(*addr*, *pid=None*, *required=False*)

Read a pointer-sized value from guest memory.

Reads a pointer-sized value from guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

Returns

int or None
Pointer value read. `None` only when `required` is set and the read failed; otherwise an unreadable pointer reads as 0, which is indistinguishable from a genuine NULL.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, int | None]

read_ptr_array(*addr*, *pid=None*)

Read a NULL-terminated array of pointers to strings from guest memory using the portal's optimized handler.

Uses the HYPER_OP_READ_PTR_ARRAY portal command for efficient reading.

Parameters

addr
[int or Ptr] Address of the pointer array in guest memory.

pid
[int, optional] Process ID for context.

Returns

list of str
List of strings read from the array.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, *List*[str]]

read_ptrlist(*addr*, *length*, *pid=None*, *required=False*)

Read a list of pointer values from guest memory.

Reads a list of pointer values from guest memory.

Parameters

addr
[int or Ptr] Address to start reading from.

length
[int] Maximum number of pointers to read.

pid
[int, optional] Process ID for context.

Returns

list of int
List of pointer values.

Parameters

- **addr** (*int | Ptr*)
- **length** (*int*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, *List*[int] | None]

read_str(*addr*, *pid=None*, *required=False*)

Read a null-terminated string from guest memory.

Reads a null-terminated string from guest memory at a specified address. Optimized to read in page-aligned chunks to minimize overhead, with fallback to PortalCmd if memory is unmapped in the emulator.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

required
[bool, optional] Whether the caller needs to know that the string is incomplete. Default False, which is the total contract: a read that fails part-way, or a string longer than one portal region, comes back truncated and indistinguishable from a short string. Pass True to get `None` in both of those cases instead.

Returns

str or None
String read from memory. `None` only when `required` is set and the string could not be read in full – including a NULL `addr`, a failed read part-way through, and hitting the size cap without finding a terminator.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, str | None]

`read_str_panda`(*cpu*, *addr*)

Read a null-terminated string from guest memory using PANDA only.

Reads a null-terminated string from guest memory at a specified address, using PANDA's virtual_memory_read in page-aligned chunks. Never falls back to the portal.

Parameters

cpu : Any (CPUState) addr : int or Ptr

Address to read from.

Returns

str
String read from memory.

Parameters
addr (*int | Ptr*)

Return type
str

`read_uint64_array`(*addr*, *count*, *pid=None*, *required=False*)

Read an array of 8-byte unsigned integers from guest memory.

Reads an array of 8-byte unsigned integers from guest memory.

Parameters

addr
[int or Ptr] Address to start reading from.

count
[int] Number of uint64s to read.

pid
[int, optional] Process ID for context.

Returns

list of int
List of uint64s read from memory.

Parameters

- **addr** (*int | Ptr*)
- **count** (*int*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, *List*[int] | None]

read_utf8_str(*addr*, *pid=None*, *required=False*)

Read a null-terminated UTF-8 string from guest memory.

Reads a null-terminated UTF-8 string from guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

Returns

str
UTF-8 string read from memory.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, str | None]

read_word(*addr*, *pid=None*, *required=False*)

Read a 2-byte word from guest memory.

Reads a 2-byte word from guest memory.

Parameters

addr
[int or Ptr] Address to read from.

pid
[int, optional] Process ID for context.

required
[bool, optional] Whether the caller needs to know that the read failed. Default False: unreadable memory reads as zero and `None` is never returned. Pass True to get `None` instead.

Returns

int or None
Word value read. `None` only when `required` is set and the read failed; otherwise unreadable memory reads as 0.

Parameters

- **addr** (*int | Ptr*)
- **pid** (*int | None*)
- **required** (*bool*)

Return type
Generator[*Any*, *Any*, int | None]

write(*addr*, *data*, *size=None*, *pid=None*)

Smart dispatcher for memory writes. Examines the address type and data type to automatically select the correct write method.

Parameters

addr
[int or Ptr] The destination address or a dwarffi Ptr object.

data
[Any] The data to write (bytes, str, int, list, or dict/object for Ptrs).

size
[int, optional] Explicit size override for integer writes.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **data** (*Any*)
- **size** (*int | None*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

write_byte(*addr*, *value*, *pid=None*)

Write a single byte to guest memory.

Writes a single byte to guest memory.

Parameters

addr
[int or Ptr] Address to write to.

value
[int] Byte value to write (0-255).

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written (should be 1).

Parameters

- **addr** (*int | Ptr*)
- **value** (*int*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

write_bytes(*addr*, *data*, *pid=None*)

Write bytes to guest memory.

Writes bytes to guest memory at a specified address, handling chunking for large data.

Parameters

addr
[int or Ptr] Address to write to.

data
[bytes] Data to write.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **data** (*bytes*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

write_bytes_panda(*cpu*, *addr*, *data*)
Write a bytearray into memory at the specified physical/virtual address

Parameters

- **addr** (*int | Ptr*)
- **data** (*bytes*)

Return type
None

write_deref(*ptr*, *value*, *pid=None*)
Write a Python value or dictionary directly to the memory address pointed to by a dwarffi Ptr. Automatically handles size, endianness, padding, and nested struct serialization via dwarffi's type system.

Parameters

ptr
[Ptr] A dwarffi pointer object pointing to the destination address.

value
[Any] The value to write. Can be a primitive (int, bytes) or a dict for deep struct initialization.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **ptr** (*Ptr*)
- **value** (*Any*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

write_int(*addr*, *value*, *pid=None*)

Write a 4-byte integer to guest memory.

Writes a 4-byte integer to guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to write to.

value
[int] Integer value to write.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **value** (*int*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

write_int_array(*addr*, *values*, *pid=None*)

Write an array of 4-byte integers to guest memory.

Writes an array of 4-byte integers to guest memory.

Parameters

addr
[int or Ptr] Address to start writing to.

values
[list of int] List of integers to write.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **values** (*List[int]*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

`write_long`(*addr*, *value*, *pid=None*)

Write an 8-byte long integer to guest memory.

Writes an 8-byte long integer to guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to write to.

value
[int] Long value to write.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **value** (*int*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

`write_ptr`(*addr*, *value*, *pid=None*)

Write a pointer-sized value to guest memory.

Writes a pointer-sized value to guest memory at a specified address.

Parameters

addr
[int or Ptr] Address to write to.

value
[int] Pointer value to write.

pid
[int, optional] Process ID for context.

Returns

None

Parameters

- **addr** (*int | Ptr*)
- **value** (*int*)

- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, None]

write_str(*addr*, *string*, *null_terminate=True*, *pid=None*)

Write a string to guest memory.

Writes a string to guest memory at a specified address, optionally null-terminated.

Parameters

addr
[int or Ptr] Address to write to.

string
[str or bytes] String to write.

null_terminate
[bool, optional] Whether to append a null terminator (default: True).

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written.

Parameters

- **addr** (*int | Ptr*)
- **string** (*str | bytes*)
- **null_terminate** (*bool*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

write_word(*addr*, *value*, *pid=None*)

Write a 2-byte word to guest memory.

Writes a 2-byte word to guest memory.

Parameters

addr
[int or Ptr] Address to write to.

value
[int] Word value to write.

pid
[int, optional] Process ID for context.

Returns

int
Number of bytes written (should be 2).

Parameters

- **addr** (*int | Ptr*)
- **value** (*int*)
- **pid** (*int | None*)

Return type
Generator[*Any*, *Any*, int]

pyplugins.apis.net module

class pyplugins.apis.net.**Netdev**(*name*, **args*, ***kwargs*)

Bases: object

Base class for all network devices. Custom netdev plugins should inherit from this.

Parameters
name (*str*)

ALLOW_DELETE: bool = True

property logger

name: str

netdev_ptr: int

setup(*netdev_struct*)
Called when the netdev is initialized in the kernel.

Return type
Iterator | None

class pyplugins.apis.net.**Netdevs**

Bases: *Plugin*

RUNTIME_CREATED_IFACE = re.compile('^(?:br|bridge|bond|vlan)\\d|^br-')

VALID_IFACE_PATTERN = re.compile('^(?!\\.\\.?$)[a-zA-Z0-9_.-]+$')

ensure_netdev_from_path(*filepath*)
Analyzes a normalized pseudofile path and extracts the network interface name.

Parameters
filepath (*str*)

get_netdev_state(*name*)
Get the state (up/down) of a network device. Returns True if up, False if down, or None if not found.

Parameters
name (*str*)

Return type
Generator[PortalCmd, int | None, bool | None]

lookup_netdev(*name*)

Look up a network device by name using the portal. Returns the pointer to net_device struct or None if not found.

Parameters
name (*str*)

Return type
Generator[PortalCmd, int | None, int | None]

register(*name*, *backing_class=None*, *exist_ok=False*, **args*, ***kwargs*)

Parameters

- **name** (*str*)
- **backing_class** (*Type[Netdev] | None*)
- **exist_ok** (*bool*)

register_netdev(*name*, *backing_class=None*, *exist_ok=False*, **args*, ***kwargs*)

Register a network device with the given name.

Parameters

- **name** (*str*)
- **backing_class** (*Type[Netdev] | None*)
- **exist_ok** (*bool*)

set_netdev_state(*name*, *up*)

Set the state (up/down) of a network device. Returns True if successful, False otherwise.

Parameters

- **name** (*str*)
- **up** (*bool*)

Return type
Generator[PortalCmd, int | None, bool | None]

pyplugins.apis.osi module

OSI Plugin (osi.py) for Penguin

This module provides the OSI plugin for the Penguin framework, enabling coroutine-based access to guest OS state via the hypervisor portal. It allows querying of process information, file descriptors, memory mappings, process handles, arguments, and environment variables. The plugin is designed for advanced analysis, automation, and plugin interoperability.

Features

- Retrieve process arguments, environment variables, and process names.
- List open file descriptors and resolve their names.
- Query memory mappings and locate mappings by address.
- Fetch process handles and detailed process information.

- All methods are coroutine-based and return wrapper objects or lists for further inspection.

Example Usage

```
from penguin import plugins

# Get process name
procname = yield from plugins.OSI.get_proc_name(pid)

# Get process arguments
args = yield from plugins.OSI.get_args(pid)

# Get environment variables
env = yield from plugins.OSI.get_env(pid)

# Get open file descriptors
fds = yield from plugins.OSI.get_fds(pid)

# Get memory mappings
mappings = yield from plugins.OSI.get_mappings(pid)
```

Purpose

The OSI plugin enables flexible, efficient, and scriptable access to guest OS state, supporting advanced analysis, automation, and plugin interoperability in the Penguin environment.

class pyplugins.apis.osi.**OSI**

Bases: *Plugin*

OSI Plugin

Provides coroutine-based methods for querying OS-level information (processes, FDs, mappings) from the guest via the hypervisor portal.

Attributes

logger

[object] Logger for debug and error messages.

get_all_procs()

Retrieve full `osi_proc` records for every user process in one kernel-side walk (`HYPER_OP_OSI_PROC_ALL`).

This is the bulk equivalent of calling *get_proc()* for each pid from *get_proc_handles()*: the driver walks `for_each_process` under `rcu_read_lock` and returns the whole set (paginated), so a process-tree snapshot costs `ceil(N / procs-per-page)` round-trips and is internally consistent rather than torn across N separate reads. Names come from `task->comm` (kernel memory only), so it is safe against exiting contexts. Kernel threads are omitted (they have no `mm`).

Returns

List[Wrapper]
One wrapper per process, each carrying the `osi_proc` fields (`pid`, `ppid`, `uid` …) plus a decoded `.name` (comm).

Return type
Generator[*Any*, None, *List*[Wrapper]]

`get_args`(*pid=None*)

Get the argument list for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

Returns

List[str]
List of argument strings (empty if not found).

Parameters
`pid` (*int | None*)

Return type
Generator[*Any*, None, *List*[str]]

`get_cwd`(*pid=None*)

Get the current working directory for a process.

Parameters
`pid` (*int | None*)

Return type
Generator[*Any*, None, str | None]

`get_env`(*pid=None*)

Get the environment variables for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

Returns

Dict[str, str]
Dictionary of environment variables (empty if not found).

Parameters
`pid` (*int | None*)

Return type
Generator[*Any*, None, *Dict*[str, str]]

`get_fd_name`(*fd*, *pid=None*)

Get the filename for a specific file descriptor.

Uses the efficient get_fds function to retrieve information for a specific file descriptor.

Parameters

fd
[int] File descriptor number. Passing AT_FDCWD (-100) resolves the Current Working Directory.

pid
[int, optional] Process ID, or None for current process.

Returns

str or None
The file descriptor name, or None if not found.

Parameters

- **fd** (*int*)
- **pid** (*int | None*)

Return type
Generator[*Any*, None, str | None]

`get_fds`(*pid=None*, *start_fd=0*, *count=None*)

Retrieve file descriptors for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

start_fd
[int, optional] FD number to start listing from (default: 0).

count
[int, optional] Maximum number of file descriptors to return (None for all).

Returns

List[Wrapper]
List of file descriptor objects with fd and name properties (empty if not found).

Parameters

- **pid** (*int | None*)
- **start_fd** (*int*)
- **count** (*int | None*)

Return type
Generator[*Any*, None, *List*[Wrapper]]

`get_mapping_by_addr`(*addr*)

Get the memory mapping containing a specific address.

Parameters

addr
[int] Address to look up.

Returns

MappingWrapper or None
Mapping containing the address, or None if not found.

Parameters
addr (*int*)

Return type
Generator[*Any*, None, MappingWrapper | None]

`get_mappings`(*pid=None*)

Get memory mappings for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

Returns

MappingsWrapper
Wrapper containing all memory mappings (empty if not found).

Parameters
pid (*int | None*)

Return type
Generator[*Any*, None, MappingsWrapper]

`get_proc`(*pid=None*)

Get detailed process information for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

Returns

Wrapper or None
Process information wrapper object, or None if not found.

Parameters
pid (*int | None*)

Return type
Generator[*Any*, None, Wrapper | None]

`get_proc_exe`(*pid=None*)

Get the full executable path for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

Returns

Wrapper or None
Process information wrapper object, or None if not found.

Parameters
`pid` (*int | None*)

Return type
Generator[*Any*, None, str | None]

`get_proc_handles`()

Retrieve a list of process handles from the kernel.

Returns

List[Wrapper]
List of process handle objects with properties: pid, taskd, start_time (empty if not found).

Return type
Generator[*Any*, None, *List*[Wrapper]]

`get_proc_name`(*pid=None*)

Get the process name (first argument) for a process.

Parameters

pid
[int, optional] Process ID, or None for current process.

Returns

str
Process name or '[???]' if not found.

Parameters
`pid` (*int | None*)

Return type
Generator[*Any*, None, str]

pyplugins.apis.portalcall module

PortalCall Plugin (portalcall.py) for Penguin

This module provides the PortalCall plugin for the Penguin framework, enabling registration and handling of portalcalls via syscall filtering. It allows plugins to register handlers for specific portalcall magic values and process arguments from guest syscalls.

Features

- Register portalcall handlers for specific magic values.
- Handle portalcall syscalls and dispatch to registered handlers.
- Safely read arguments from guest memory and return results.
- Supports coroutine-style handler functions.

Example Usage

```
from penguin import plugins

@plugins.portalcall.portalcall(0x12345678)
def my_portalcall_handler(arg1, arg2):
    # Process arguments and return result
    return arg1 + arg2
```

Classes

- PortalCall: Main plugin class for portalcall registration and handling.

class pyplugins.apis.portalcall.**PortalCall**

Bases: *Plugin*

Plugin that provides an interface to register and handle portalcalls via syscall filtering.

portalcall(*user_magic*)

Decorator to register a portalcall handler for a given user_magic value.

Parameters

user_magic (*int*)

pyplugins.apis.qmp module

QMP command and event API plugin.

Provides both directions of the QMP control plane:

- **Inbound** (`command`): owns the mapping of custom QMP command names to Python handlers, mirroring `apis.hypercall.Hypercall`. `qemu_compat` installs a single C-level QMP trampoline and forwards each unrecognized QMP command here; this plugin resolves the command name to a registered handler and marshals the result.
- **Outbound** (`emit_event`): pushes an unsolicited QMP event of any name to every connected QMP monitor, for notifying a host-side consumer of something that happened in the guest.

The C trampoline is installed **lazily**: it is wired up only when the first command is registered (usually in a consumer plugin's `__init__`), so a loaded-but-unused Qmp plugin never touches the QEMU callback.

This plugin is **opt-in**: unlike `Hypercall` it is not loaded automatically. List `qmp` in the config `plugins` to enable it. On load it opens a QMP socket for QEMU itself – it appends `-qmp unix:<outdir>/qmp.sock,...` to the QEMU argv (which penguin has assembled but not yet launched) – so no `penguin_run`/config-schema change is needed. The socket path can be overridden with the `socket` arg; `server`/`nowait` keep QEMU from blocking on a client.

Registration reads like every other plugin-facing capability:

```
from penguin import plugins

@plugins.qmp.command("penguin-snapshot-save")
def save(args):
    plugins.snapshot.save(args["name"])
    return {"saved": args["name"]}
```

A handler receives the decoded `arguments` object (a dict) and returns one of:

- `None` / `False` – decline the command. If no handler claims it, QEMU reports `CommandNotFound`.
- `True` – handled, empty `{"return": {}}` response.
- any JSON-serializable object – used as the QMP `return` value.

Emitting an event is the other direction, and needs no registration:

```
plugins.qmp.emit_event("MY_EVENT", {"pid": pid, "path": path})
```

Every connected client receives:

```
{"event": "MY_EVENT",
 "data": {"pid": 1234, "path": "/bin/sh"},
 "timestamp": {"seconds": ..., "microseconds": ...}}
```

The name is used as the QMP `event` member verbatim, so any event name can be emitted without a QAPI schema entry – and, having no schema entry, these events are never rate-limited or coalesced by QEMU. The payload must be a dict, since the QMP spec defines an event's `data` member as a JSON object.

Unlike command handlers, `emit_event` is safe to call from guest context – the QEMU side defers the work to a main-loop bottom half. It is fire-and-forget: there is no delivery confirmation, and an event emitted before a client connects is not buffered for it. An arbitrary name can collide with a built-in QEMU event (`STOP`, `RESET`, ...), so picking a distinctive name is the caller's responsibility.

Handler contract / caveats

- Handlers run **on the QEMU main loop with the BQL held** (QMP dispatch is not in guest context). A handler that blocks waiting on the guest freezes timers and I/O – keep them short and non-blocking.
- Because dispatch is on the main loop rather than in a guest hypercall, guest-memory access via the portal (`yield from plugins.mem.read_*`) is **not** serviceable here. A handler that returns a generator is rejected explicitly (logged) rather than silently failing to serialize.
- The current QEMU ABI (`penguin_qmp_cb_t`) can only express "handled + result string" or "not handled". It cannot carry a structured `{"error": {...}}`, so a handler that raises or returns a non-serializable value is reported to the client as `CommandNotFound` (with the real cause logged host-side). A dedicated error out-param in the QEMU fork (`char **error`) would let a handler surface a real `GenericError`; that ABI change is intentionally deferred so nothing depends on it yet.

class pyplugins.apis.qmp.**Qmp**

Bases: *Plugin*

QMP custom-command registry (host-side control plane).

Replaces the compat layer's single global `cb_qmp` handler with a proper registry so multiple plugins can each own distinct QMP commands.

class Args(*, *socket='qmp.sock'*)

Bases: *PluginArgs*

Parameters
socket (*str*)

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

socket: str

bind_qemu_compat(*qemu_compat*)

Attach to a QemuCompat instance.

The C-level QMP trampoline is installed lazily – only once at least one command is registered (see register()). Binding a compat that already has commands to serve installs immediately; otherwise it waits, so a loaded-but-unused Qmp plugin never touches the QEMU callback.

Return type
None

command(*name*)

Decorator: register the wrapped function for QMP command `name`.

Parameters
name (*str*)

Return type
Callable[[*Callable*], *Callable*]

dispatch(*command*, *args*)

Resolve and invoke the handler for `command`.

Returns the handler's result (None/False/True/JSON-able object) for the compat trampoline to marshal, or None if no handler is registered (so QEMU reports CommandNotFound). Handler exceptions propagate to the trampoline, which logs and declines.

Parameters

- **command** (*str*)
- **args** (*Any*)

Return type
Any

emit_event(*name*, *data=None*)

Emit an arbitrary QMP event to every connected QMP monitor.

The outbound counterpart to *command()*. Where a command handler answers a client's request, this pushes an unsolicited notification:

```
plugins.qmp.emit_event("MY_EVENT", {"pid": pid, "path": path})
```

arrives at every connected client as:

```
{"event": "MY_EVENT",
 "data": {"pid": 1234, "path": "/bin/sh"},
 "timestamp": {...}}
```

`name` becomes the QMP `event` member verbatim, so any event name can be emitted without a QAPI schema entry. Because there is no schema entry there is no QAPIEvent ordinal, so these events bypass QEMU's per-event rate limiting and are never throttled or coalesced.

`data` is an optional payload. The QMP spec defines an event's `data` member as a JSON *object*, so it must be a dict; pass None for no payload. A timestamp is added automatically.

Unlike command handlers, this is safe to call from guest context: the QEMU side defers to a main-loop bottom half, so it does not take the BQL and does not block the calling vCPU on the monitor write. It is fire-and-forget – there is no delivery confirmation, and events emitted before a client connects are not buffered for it.

Note that an arbitrary name can collide with a built-in QEMU event (e.g. `STOP`, `RESET`). Picking a distinctive name is the caller's responsibility.

Parameters

- **name** (*str*)
- **data** (*Dict[str, Any] | None*)

Return type
None

handlers: Dict[str, Callable]

register(*name*, *func*)

Register `func` as the handler for QMP command `name`.

Raises if `name` is already registered: silently replacing a handler (the old compat-layer behavior) means the second plugin to register would knock the first offline with no warning.

Parameters

- **name** (*str*)
- **func** (*Callable*)

Return type
Callable

pyplugins.apis.send_hypercall module

SendHypercall Plugin (send_hypercall.py) for Penguin

This module provides the SendHypercall plugin for the Penguin framework, enabling the registration and handling of custom hypercalls from the guest OS. It allows plugins to subscribe to specific hypercall events, receive arguments from the guest, process them, and return results back to the guest memory. The plugin is designed for extensibility and safe event-driven communication between guest and host.

Features

- Register and handle custom hypercall events from the guest.
- Safely read arguments and write results to guest memory.
- Supports both string and bytes output.

- Handles architecture-specific pointer sizes and endianness.
- Provides error handling and logging for robust operation.

Example Usage

```
from penguin import plugins

def my_hypercall_handler(arg1, arg2):
    # Process arguments and return (retval, output)
    return 0, f"Received: {arg1}, {arg2}"

# Direct registration
plugins.send_hypercall.subscribe("mycmd", my_hypercall_handler)

# Decorator usage
@plugins.send_hypercall.subscribe("mycmd2")
def my_hypercall_handler2(arg1):
    # Process arguments and return (retval, output)
    return 0, f"Handled by decorator: {arg1}"

# You can also use a bound method as a decorator:
class MyPlugin:
    @plugins.send_hypercall.subscribe("mycmd3")
    def my_method(self, arg):
        return 0, f"Handled in class: {arg}"
```

class pyplugins.apis.send_hypercall.**SendHypercall**

Bases: *Plugin*

SendHypercall Plugin

Handles registration and processing of custom hypercall events from the guest OS.

Attributes

outdir
: [str] Output directory for plugin data.

registered_events
: [Dict[str, Callable[..., Tuple[int, Union[str, bytes]]]]] Registered event handlers.

on_send_hypercall(*cpu*, *buf_addr*, *buf_num_ptrs*)

Handle an incoming hypercall from the guest.

Reads arguments from guest memory, dispatches to the registered handler, and writes the result back.

Parameters

cpu
: [Any] CPU context from PANDA.

buf_addr
: [int] Address of the pointer array in guest memory.

buf_num_ptrs
[int] Number of pointers in the array.

Returns

None

Parameters

- **cpu** (*Any*)
- **buf_addr** (*int*)
- **buf_num_ptrs** (*int*)

Return type
None

subscribe(*event*, *callback=None*)

Register a callback for a specific hypercall event.

Can be used as a decorator or called directly.

Parameters

event
[str] Event name to subscribe to.

callback
[Callable[..., Tuple[int, Union[str, bytes]]], optional] Callback function that processes the event.

Raises

ValueError
If already subscribed to the event.

Returns

decorator or None
If used as a decorator, returns the decorator function. If called directly, returns None.

Parameters

- **event** (*str*)
- **callback** (*Callable[[...], Tuple[int, str | bytes]]*)

pyplugins.apis.signal_monitor module

Signal Monitor Plugin (signal_monitor.py)

This module provides the Signal Monitor plugin for the Penguin framework. It allows monitoring and interception of signal deliveries in the guest OS. Plugins can register for signal events and optionally drop catchable deliveries or modify process state. Default-fatal signals without a userspace handler may begin process exit before the delivery hook is reached; block signal-send syscalls when the target process must be preserved.

Usage

```
from penguin import plugins

@plugins.subscribe(plugins.signal_monitor, "signal_deliver")
def on_signal(cpu, event):
    print(f"Signal {event.sig} delivered to {event.comm} (PID {event.pid})")
    if event.sig == 4: # SIGILL
        print("Intercepted SIGILL! Bypassing...")
        event.drop = True
        event.regs.set_pc(event.regs.get_pc() + 4) # Advance PC (assuming 4-byte␣
↪instruction)

# Register a hook for all signals
plugins.signal_monitor.register_hook()
```

class pyplugins.apis.signal_monitor.**SignalEvent**(*se*)

Bases: `object`

Wrapper for struct signal_event from the guest driver.

comm

regs

class pyplugins.apis.signal_monitor.**SignalMonitor**

Bases: *Plugin*

Signal Monitor Plugin

Interacts with the igloo_driver to monitor and intercept signals.

register_hook(*sig=0*, *pid=None*, *procname=None*)

Register a signal hook in the guest driver.

Parameters

sig
: [int, optional] Signal number to hook (0 for all). Default is 0.

pid
: [int, optional] Filter by process ID.

procname
: [str, optional] Filter by process name (comm).

Returns

bool
: True if the hook registration was queued.

Parameters

- **sig** (*int*)
- **pid** (*int | None*)
- **procname** (*str | None*)

Return type
bool

unregister_hook(*handle*)

Unregister a signal hook.

Parameters

handle
[int] The handle returned by register_hook.

Returns

bool
True if the unregister command was queued.

Parameters
handle (*int*)

Return type
bool

pyplugins.apis.signals module

Signals Plugin (signals.py) for Penguin

This module provides the Signals plugin for the Penguin framework, enabling the triggering of core dumps, crash snapshots, and custom signals in the guest OS via the hypervisor portal. It is useful for debugging, analysis, and automation of guest process state.

Features

- Trigger a full snapshot and core dump in the guest.
- Send SIGABRT or custom signals to the current process in the guest.
- Coroutine-based API for integration with other Penguin plugins.

Example Usage

```
from penguin import plugins

# Trigger a full snapshot and core dump
pid = yield from plugins.signals.crash_snapshot()

# Send SIGABRT to the current process
pid = yield from plugins.signals.self_abort()

# Send a custom signal
pid = yield from plugins.signals.self_signal("SIGTERM")
```

class pyplugins.apis.signals.**Signals**

Bases: *Plugin*

Signals Plugin

Provides methods to trigger core dumps, crash snapshots, and send signals to guest processes via the hypervisor portal.

`crash_snapshot()`

Create a snapshot and core dump in the guest (default dump mode).

Returns

Optional[int]
PID of the process that received the signal, or error code, or None on failure.

Return type
Generator[*Any*, None, int | None]

`dump(mode=0, signal=0)`

Trigger a core dump or signal in the guest.

Parameters

mode
[int, optional] Dump mode (0=full snapshot and coredump, 1=self abort, 2=custom signal). Default is 0.

signal
[int, optional] Signal number to send (only used with mode=2). Default is 0.

Returns

Optional[int]
PID of the process that received the signal, or error code, or None on failure.

Parameters

- **mode** (*int*)
- **signal** (*int*)

Return type
Generator[*Any*, None, int | None]

`self_abort()`

Send SIGABRT to the current process in the guest.

Returns

Optional[int]
PID of the process that received SIGABRT, or error code, or None on failure.

Return type
Generator[*Any*, None, int | None]

`self_signal(signal)`

Send a custom signal to the current process in the guest.

Parameters

signal
[int] Signal number (1-31) or name to send.

Returns

Optional[int]
PID of the process that received the signal, or error code, or None on failure.

Raises

ValueError
If the signal number is not between 1 and 31 or the signal name is unrecognized.

Parameters
signal (*int | str*)

Return type
Generator[*Any*, None, int | None]

`signal_name_to_num`(*name*)

Parameters
name (*str*)

Return type
int | None

pyplugins.apis.static_fs module

`class` pyplugins.apis.static_fs.**StaticFS**
Bases: *Plugin*

`exists`(*path*)

Parameters
path (*str*)

Return type
bool

`get_size`(*path*, *transparent=frozenset({})*)
Returns the size of the file in bytes without opening it.

`transparent` (see `_resolve_path()`) lets in-place action types such as `binary_patch` resolve to the underlying file's size instead of the action's (0-byte) placeholder.

Parameters

- **path** (*str*)
- **transparent** (*frozenset*)

Return type
int | None

get_xattr(*path*, *key*)

Parameters

- **path** (*str*)
- **key** (*str*)

Return type
bytes | None

list(*path*)

Parameters
path (*str*)

Return type
list[str] | None

list_mode(*path*)

Parameters
path (*str*)

Return type
Dict[str, int] | None

list_xattr(*path*)

Parameters
path (*str*)

Return type
list[str]

open(*path*)

Parameters
path (*str*)

Return type
IO[bytes] | None

read(*path*, *size*, *offset=0*)

Parameters

- **path** (*str*)
- **size** (*int*)
- **offset** (*int*)

Return type
bytes | None

statfs(*path=None*)

Returns a dictionary with keys named like the POSIX statvfs struct. For static files, returns default values. For ratarmountcore files, delegates to the underlying mount source.

Parameters
path (*str | None*)

Return type
Dict[str, *Any*]

uninit()

Cleanly closes the underlying MountSource and forces a disk sync.

pyplugins.apis.symbols module

Symbols Plugin (symbols.py) for Penguin

This module provides the Symbols plugin for the Penguin framework, serving as a robust, centralized service for resolving binary symbols to file offsets. It allows other plugins and scripts to locate functions and variables within guest executables and shared libraries, even in challenging scenarios like stripped binaries or non-standard architectures.

Features

- **Robust Forward Lookup:** Resolves symbol names to file offsets using a tiered strategy: 1. Pre-computed JSON cache (fastest). 2. Native `nm` utility (static and dynamic tables). 3. PyELFtools (section parsing). 4. Manual PT_DYNAMIC segment parsing (handles "sstripped" binaries with no section headers). 5. `readelf` fallbacks, including MIPS GOT scraping for embedded targets.
- **Reverse Resolution:** Maps file offsets back to the nearest symbol name (useful for stack trace generation).
- **Address Resolution:** Maps virtual addresses to file offsets (useful for handling raw addresses provided by users).
- **Introspection:** Methods to list, filter, and bulk-load symbols for specific binaries.
- **Architecture Aware:** Automatically handles absolute addressing (ET_EXEC) vs relative offsets (ET_DYN) and architecture-specific symbol tables.

Example Usage

```
from penguin import plugins

# 1. Forward Lookup: Get the binary path and file offset for a function
#    (Useful for placing hooks or uprobes)
path, offset = plugins.Symbols.lookup("/usr/bin/httpd", "httpGetEnv")
if offset:
    print(f"Function located at offset {hex(offset)} in {path}")

# 2. Address Resolution: Convert a virtual address to a file offset
offset = plugins.Symbols.resolve_addr("/usr/bin/httpd", 0x400500)

# 3. Reverse Lookup: Identify a function from a crash/instruction pointer
name, dist = plugins.Symbols.resolve_offset("/usr/lib/libc.so.0", 0x12345)
```

Purpose

The Symbols plugin bridges the gap between high-level analysis names (functions) and low-level binary offsets required for instrumentation.

class pyplugins.apis.symbols.**Symbols**

Bases: *Plugin*

Symbols Plugin

A central service for resolving symbol names to file offsets within the guest.

find_all(*symbol*)

Search for a symbol in ALL known libraries in the database.

Parameters

symbol
[str] Symbol name to look up.

Returns

List[Tuple[str, int]]
A list of (library_path, file_offset) for every occurrence of the symbol.

Parameters
symbol (*str*)

Return type
List[*Tuple*[str, int]]

get_offset(*path*, *symbol*)

Parameters

- **path** (*str*)
- **symbol** (*str*)

Return type
int | None

list_symbols(*path*, *filter_str=None*)

Returns a list of all symbol names found in the binary. Optional filter_str performs a substring match.

Parameters

- **path** (*str*)
- **filter_str** (*str | None*)

Return type
List[str]

load_symbols(*path*)

Force-loads symbols for a binary into the cache. Useful for pre-warming the cache or debugging what symbols are detected.

Parameters
path (*str*)

Return type
Dict[str, int]

lookup(*path*, *symbol*)

Resolve a symbol name to a specific library path and file offset.

Parameters

path
[str] Path to binary (supports wildcards like "*/libc.so*").

symbol
[str] Symbol name to look up.

Returns

Tuple[str, int] or (None, None)

Parameters

- **path** (*str*)
- **symbol** (*str*)

Return type
Tuple[str | None, int | None]

resolve_addr(*path*, *vaddr*, *base_addr=None*)

Resolves a virtual address to a file offset.

If *base_addr* is provided, the offset is calculated as *vaddr* - *base_addr*. Otherwise, it attempts to map the virtual address to a file offset using ELF segments. If that fails, it retries by assuming common base addresses (e.g., 0x400000) and checking if the adjusted address maps to a segment or valid file offset.

Returns the file offset or None if resolution fails.

Parameters

- **path** (*str*)
- **vaddr** (*int*)
- **base_addr** (*int | None*)

Return type
int | None

resolve_offset(*path*, *offset*)

Reverse lookup: Given a file offset, find the nearest preceding symbol. Returns (SymbolName, DistanceFromStart)

Example: resolve_offset("/bin/httpd", 0x4005) -> ("main", 5)

Parameters

- **path** (*str*)
- **offset** (*int*)

Return type
Tuple[str, int] | None

pyplugins.apis.syscalls module

Syscalls Plugin System

The Syscalls plugin provides a comprehensive interface for monitoring, filtering, and intercepting system calls in the guest via the IGLOO hypervisor interface. This system enables real-time analysis of guest behavior, security monitoring, and dynamic program analysis.

Overview

The syscalls system operates through a multi-layer architecture:

1. **Kernel Layer**: IGLOO kernel module hooks into syscall entry/return points
2. **Hypervisor Layer**: Portal system manages communication between guest and host
3. **Plugin Layer**: Python plugins register callbacks and process syscall events

Quick Start

```
from penguin import plugins, Plugin

class MySyscallMonitor(Plugin):
    def __init__(self):
        # Register syscall hooks in __init__
        plugins.syscalls.syscall("on_sys_execve_enter")(self.monitor_execve)
        plugins.syscalls.syscall("on_all_sys_enter", comm_filter="target_process")(self.
↪monitor_all)

    def monitor_execve(self, pt_regs, proto, syscall, filename, argv, envp):
        """Monitor process execution"""
        filename_str = yield from plugins.mem.read_str(filename)
        print(f"Process executing: {filename_str}")

    def monitor_all(self, pt_regs, proto, syscall):
        """Monitor all syscalls for a specific process"""
        print(f"Syscall: {proto.name} by {syscall.task.comm}")
```

Core Concepts

Syscall Events

Every syscall generates events that can be intercepted at two points:

- **Entry (on_enter)**: Before the syscall executes in the kernel
- **Return (on_return)**: After the syscall completes

Hook Registration

Hooks are registered using the `@syscalls.syscall()` decorator with flexible filtering options:

```
@syscalls.syscall(
    name_or_pattern="sys_openat",
    on_enter=True,
    on_return=True,
    comm_filter="target_app",
    pid_filter=1234,
    arg_filters=[None, ValueFilter.exact(0x42)],
    retval_filter=ValueFilter.success()
)
def my_handler(pt_regs, proto, syscall, *args):
    # Handler code here
    pass
```

API Reference

Decorator Parameters

name_or_pattern (Optional[str])

Specifies which syscalls to hook. Supports multiple formats:

```
# Specific syscall
@syscalls.syscall("sys_open")

# Pattern-based (hsyscall format)
@syscalls.syscall("on_sys_execve_enter")
@syscalls.syscall("on_sys_openat_return")

# All syscalls
@syscalls.syscall("on_all_sys_enter")

# Unknown syscalls (those without metadata)
@syscalls.syscall("on_unknown_sys_enter")
```

on_enter / on_return (Optional[bool])

Control when the hook triggers:

```
# Entry only (default if neither specified)
@syscalls.syscall("sys_open", on_enter=True)

# Return only
@syscalls.syscall("sys_open", on_return=True)

# Both entry and return
@syscalls.syscall("sys_open", on_enter=True, on_return=True)
```

comm_filter (Optional[str])

Filter by process name:

```
@syscalls.syscall("sys_write", comm_filter="nginx")
def monitor_nginx_writes(pt_regs, proto, syscall, fd, buf, count):
    pass
```

pid_filter (Optional[int])

Filter by specific process ID:

```
@syscalls.syscall("sys_read", pid_filter=1234)
def monitor_specific_process(pt_regs, proto, syscall, fd, buf, count):
    pass
```

Advanced Filtering with ValueFilter

The `ValueFilter` class provides sophisticated argument and return value filtering:

```
from apis.syscalls import ValueFilter

# Exact match
ValueFilter.exact(42)

# Numeric comparisons
ValueFilter.greater(100)
ValueFilter.less_equal(1000)
ValueFilter.range(10, 20)

# Success/error filtering
ValueFilter.success()  # >= 0
ValueFilter.error()    # < 0

# Bitwise operations
ValueFilter.bitmask_set(0x0001)    # All specified bits set
ValueFilter.bitmask_clear(0x0002)  # All specified bits clear
```

arg_filters (Optional[List[Any]])

Filter by syscall arguments:

```
@syscalls.syscall(
    "sys_ioctl",
    arg_filters=[
        None,                          # arg0: no filter
        ValueFilter.exact(0xabcd),     # arg1: exact match
        ValueFilter.range(100, 200)    # arg2: range filter
    ]
)
def filtered_ioctl(pt_regs, proto, syscall, fd, cmd, arg):
    pass
```

retval_filter (Optional[Any])

Filter by return value:

```
# Only successful calls
@syscalls.syscall("sys_open", on_return=True, retval_filter=ValueFilter.success())

# Specific error codes
@syscalls.syscall("sys_open", on_return=True, retval_filter=ValueFilter.exact(-2))  #␣
↪ENOENT

# Backward compatibility with simple values
@syscalls.syscall("sys_getpid", on_return=True, retval_filter=1234)
```

Handler Function Signatures

Standard Handlers

```
def handler(regs, proto, syscall, *args):
    """
    regs: Register/context object (formerly pt_regs, now typically a PtRegsWrapper)
        - Contains the CPU register state at the time of the syscall.
        - Provides access to general-purpose registers, PC, SP, etc.
        - Used for advanced introspection or argument extraction.
    proto: SyscallPrototype with metadata (name, types, arg names)
    syscall: SyscallEvent object with runtime data
    *args: Unpacked syscall arguments (for known syscalls)
    """
```

All-Syscall Handlers

```
def all_handler(regs, proto, syscall):
    """
    For on_all_sys_* handlers, arguments are not unpacked
    Access via syscall.args array instead
    """
```

SyscallEvent Object

The `syscall` parameter provides access to runtime information:

```
def my_handler(regs, proto, syscall, *args):
    # Basic information
    print(f"Syscall: {syscall.name}")
    print(f"Process: {syscall.task.comm}")
    print(f"PC: {syscall.pc:#x}")

    # Arguments (raw values)
    for i in range(syscall.argc):
        print(f"arg[{i}]: {syscall.args[i]:#x}")

    # Return value (for return handlers)
    print(f"Return value: {syscall.retval}")

    # Modify behavior
    syscall.skip_syscall = True    # Skip syscall execution
    syscall.retval = 42           # Set return value
```

SyscallPrototype Object

The `proto` parameter contains syscall metadata:

```
def my_handler(regs, proto, syscall, *args):
    print(f"Syscall name: {proto.name}")
    print(f"Argument count: {proto.nargs}")
```

(continues on next page)

(continued from previous page)

```
    # Argument metadata
    for i in range(proto.nargs):
        print(f"arg[{i}]: {proto.types[i]} {proto.names[i]}")
```

Practical Examples

Process Monitoring

```
@syscalls.syscall("on_sys_execve_enter")
def track_process_creation(regs, proto, syscall, filename, argv, envp):
    """Monitor process creation"""
    args = []
    try:
        # Read string arguments from guest memory
        filename_str = yield from plugins.mem.read_str(filename)
        args_list = yield from plugins.mem.read_ptrlist(argv)

        print(f"New process: {filename_str}")
        print(f"Arguments: {args_list}")

        # Log to file or database
        with open("process_log.txt", "a") as f:
            f.write(f"{filename_str} {' '.join(args_list)}\n")

    except Exception as e:
        print(f"Error reading process arguments: {e}")
```

File Access Monitoring

```
@syscalls.syscall("on_sys_openat_enter")
def monitor_file_access(regs, proto, syscall, dirfd, pathname, flags, mode):
    """Monitor file access attempts"""
    try:
        path_str = yield from plugins.mem.read_str(pathname)

        # Check for sensitive files
        sensitive_files = ["/etc/passwd", "/etc/shadow", "/etc/ssh/"]
        if any(sens in path_str for sens in sensitive_files):
            print(f"ALERT: Access to sensitive file: {path_str}")

            # Optionally block the access
            if "/etc/shadow" in path_str:
                syscall.skip_syscall = True
                syscall.retval = -13  # EACCES

    except Exception as e:
        print(f"Error reading pathname: {e}")
```

Network Monitoring

```
@syscalls.syscall("on_sys_sendto_enter", comm_filter="target_app")
def monitor_network_send(regs, proto, syscall, sockfd, buf, length, flags, dest_addr,␣
↪addrlen):
    """Monitor network sends from specific application"""
    try:
        # Read the data being sent
        data = yield from plugins.mem.read_bytes(buf, min(length, 100))  # First 100␣
↪bytes

        print(f"Network send: {len(data)} bytes")
        print(f"Data preview: {data[:50]}")

        # Check for suspicious patterns
        if b"password" in data.lower():
            print("WARNING: Potential password transmission detected")

    except Exception as e:
        print(f"Error reading network data: {e}")
```

Return Value Analysis

```
@syscalls.syscall("on_sys_open_return", retval_filter=ValueFilter.error())
def track_failed_opens(regs, proto, syscall):
    """Track failed file open attempts"""
    # Get the original arguments from saved state
    filename = syscall.args[0]  # pathname argument

    try:
        path_str = yield from plugins.mem.read_str(filename)
        error_code = -syscall.retval

        print(f"Failed to open: {path_str} (error: {error_code})")

        # Log failed access attempts
        with open("failed_access.log", "a") as f:
            f.write(f"{path_str} {error_code}\n")

    except Exception as e:
        print(f"Error processing failed open: {e}")
```

Complex Filtering Example

```
@syscalls.syscall(
    "sys_ioctl",
    comm_filter="target_driver",
    arg_filters=[
        None,                                   # fd: any
        ValueFilter.range(0x1000, 0x2000),     # cmd: in range
        ValueFilter.bitmask_set(0x0001)         # arg: has specific bit set
    ],
```

(continues on next page)

(continued from previous page)

```
    retval_filter=ValueFilter.success()
)
def monitor_specific_ioctls(regs, proto, syscall, fd, cmd, arg):
    """Monitor specific ioctl patterns"""
    print(f"Targeted ioctl: fd={fd}, cmd={cmd:#x}, arg={arg:#x}")
```

Performance Considerations

Hook Granularity

- **Specific syscalls**: Lowest overhead, only hook what you need
- **Process filtering**: Reduces events but still processes all syscalls
- **All syscalls**: Highest overhead, use sparingly

```
# Good: Specific and filtered
@syscalls.syscall("sys_write", comm_filter="target_app")

# Moderate: Broader but still filtered
@syscalls.syscall("on_all_sys_enter", comm_filter="target_app")

# Heavy: No filtering
@syscalls.syscall("on_all_sys_enter")
```

Memory Access

Reading guest memory is expensive. Cache results when possible:

```
# Cache frequently accessed strings
_path_cache = {}

def cached_read_string(addr):
    if addr in _path_cache:
        return _path_cache[addr]

    path = yield from plugins.mem.read_str(addr)
    _path_cache[addr] = path
    return path
```

Filtering Efficiency

Use the most restrictive filters first:

```
# Good: Process filter eliminates most events early
@syscalls.syscall("sys_write", comm_filter="target", retval_filter=ValueFilter.success())

# Less efficient: Complex argument filters on all processes
@syscalls.syscall("sys_write", arg_filters=[None, None, ValueFilter.greater(1000)])
```

Error Handling

Memory Access Failures

Always handle memory read failures gracefully:

```
def safe_handler(regs, proto, syscall, pathname):
    try:
        path_str = yield from plugins.mem.read_str(pathname)
        # Process path_str
    except Exception as e:
        # Log error but don't crash
        print(f"Failed to read pathname: {e}")
        return
```

Portal Communication

The portal system handles most communication errors, but be aware of timeouts:

```
def robust_handler(regs, proto, syscall, buf, length):
    if length > 1024 * 1024:  # 1MB
        print("Buffer too large, skipping read")
        return

    try:
        data = yield from plugins.mem.read_bytes(buf, length)
        # Process data
    except Exception as e:
        print(f"Memory read failed: {e}")
```

Hook Management

Dynamic Control

Enable/disable hooks at runtime:

```
# Disable a hook
yield from syscalls.disable_syscall(my_handler_function)

# Re-enable it
yield from syscalls.enable_syscall(my_handler_function)

# Or by function name
yield from syscalls.disable_syscall("my_handler_function")
```

Hook Information

Access hook configuration:

```
# Get syscall prototype information
proto = syscalls.get_syscall_info_by_name("sys_open")
if proto:
    print(f"Syscall: {proto.name}")
```

(continues on next page)

(continued from previous page)

```
    for i in range(proto.nargs):
        print(f"  {proto.types[i]} {proto.names[i]}")
```

Integration with Other Plugins

With Portal Plugin

The syscalls plugin integrates seamlessly with the portal system:

```
def combined_handler(pt_regs, proto, syscall, fd):
    # Use portal for additional context
    proc = yield from plugins.portal.get_proc()
    mappings = yield from plugins.portal.get_mappings()

    print(f"Syscall by PID {proc.pid}: {proto.name}")
```

With OSI Plugin

Combine with OS introspection:

```
def osi_enhanced_handler(pt_regs, proto, syscall, *args):
    # Get process information
    current_proc = yield from plugins.OSI.get_proc()
    if current_proc:
        print(f"Process: {current_proc.name} (PID: {current_proc.pid})")
```

Testing

The system includes comprehensive tests demonstrating usage patterns:

```
# From syscall_test.py
@syscalls.syscall("on_sys_ioctl_enter",
                  comm_filter="send_syscall",
                  arg_filters=[None, 0xabcd])
def test_skip_retval(pt_regs, proto, syscall, fd, op, arg):
    """Test syscall skipping and return value modification"""
    assert fd == 9, f"Expected fd 9, got {fd:#x}"
    assert op == 0xabcd, f"Expected op 0xabcd, got {op:#x}"

    # Skip the syscall and set custom return value
    syscall.skip_syscall = True
    syscall.retval = 43
```

Troubleshooting

Common Issues

1. **Hook not triggering**: Check syscall name normalization

   ```
   # These are equivalent:
   "sys_openat"
   ```

(continues on next page)

(continued from previous page)

```
"openat"
"_sys_openat"
```

2. **Argument mismatch**: Verify syscall prototype

```
proto = syscalls.get_syscall_info_by_name("openat")
print(f"Expected args: {proto.nargs}")
```

3. **Memory read failures**: Handle invalid pointers

```
if addr and addr != 0:
    try:
        data = yield from plugins.mem.read_str(addr)
    except:
        print("Invalid memory address")
```

Debugging

Enable debug logging:

```
import logging
logging.getLogger("syscalls").setLevel(logging.DEBUG)
```

Use the syscall event information for debugging:

```
def debug_handler(regs, proto, syscall, *args):
    print(f"Syscall: {proto.name}")
    print(f"Args: {[hex(arg) for arg in syscall.args[:syscall.argc]]}")
    print(f"Process: {syscall.task.comm}")
    print(f"PC: {syscall.pc:#x}")
```

Best Practices

1. **Filter early and often**: Use the most specific filters possible
2. **Handle errors gracefully**: Always expect memory access to fail
3. **Cache expensive operations**: Don't re-read the same memory
4. **Use appropriate hook points**: Entry for argument analysis, return for result analysis
5. **Test thoroughly**: Use the provided test framework to verify behavior
6. **Monitor performance**: Syscall hooks can impact guest performance significantly

The syscalls system provides a powerful foundation for dynamic analysis, security monitoring, and program understanding in virtualized environments.

class pyplugins.apis.syscalls.**Syscalls**

Bases: *Plugin*

Syscalls Plugin

Provides an interface to monitor and intercept system calls. Uses the portal's interrupt mechanism to handle registration and event delivery.

syscall(*name_or_pattern=None*, *on_enter=None*, *on_return=None*, *comm_filter=None*, *arg_filters=None*, *pid_filter=None*, *retval_filter=None*, *enabled=True*, *read_only=False*, *scope_filter=None*)

Decorator for registering syscall callbacks.

Parameters

name_or_pattern
[str, optional] Syscall name or pattern.

on_enter
[bool, optional] Register for entry events.

on_return
[bool, optional] Register for return events.

comm_filter
[str, optional] Process name filter.

arg_filters
[list, optional] Argument filters.

pid_filter
[int, optional] PID filter.

retval_filter
[any, optional] Return value filter.

enabled
[bool] Whether the hook is enabled.

read_only
[bool] Whether the hook modifies arguments (set to True to optimize).

scope_filter
[bool, optional] Whether to restrict this hook to the firmware-under-analysis process subtree when analysis scoping is enabled. Defaults to True (firmware only). Set False for hooks that must also fire for Penguin's own infrastructure (e.g. the portal transport).

Returns

Callable
Decorator function.

Parameters

- **name_or_pattern** (*str | None*)
- **on_enter** (*bool | None*)
- **on_return** (*bool | None*)
- **comm_filter** (*str | None*)
- **arg_filters** (*List[Any] | None*)
- **pid_filter** (*int | None*)

- **retval_filter** (ValueFilter | *str* | *int* | *None*)
- **enabled** (*bool*)
- **read_only** (*bool*)
- **scope_filter** (*bool* | *None*)

Return type
Callable

unregister(*target*)

Unregister syscall hook(s).

Can accept: 1. A specific handle returned by syscall() (unregisters only that hook). 2. The original function (unregisters ALL hooks for that function). 3. A string name (unregisters ALL hooks with that name).

Parameters

target
[callable or str] The handle, function, or name to unregister.

Parameters
target (*Any*)

Return type
None

class pyplugins.apis.syscalls.**ValueFilter**

Bases: object

Attributes

filter_type
[int] The type of filter.

value
[int] The value for the filter.

min_value
[int] The minimum value for range filters.

max_value
[int] The maximum value for range filters.

bitmask
[int] The bitmask for bitmask filters.

bitmask

classmethod bitmask_clear(*bitmask*)

Create a bitmask clear filter.

Parameters

bitmask
[int] The bitmask to clear.

Returns

ValueFilter
A bitmask clear filter.

Parameters
bitmask (*int*)

Return type
ValueFilter

`classmethod bitmask_set(bitmask)`

Create a bitmask set filter.

Parameters

bitmask
[int] The bitmask to set.

Returns

ValueFilter
A bitmask set filter.

Parameters
bitmask (*int*)

Return type
ValueFilter

`classmethod error()`

Create an error filter (< 0).

Returns

ValueFilter
An error filter.

Return type
ValueFilter

`classmethod exact(value)`

Create an exact match filter.

Parameters

value
[int] The value to match.

Returns

ValueFilter
An exact match filter.

Parameters
value (*int*)

Return type
ValueFilter

`filter_type`

`classmethod greater(value)`
Create a greater than filter.

Parameters

value
[int] The value to compare.

Returns

ValueFilter
A greater than filter.

Parameters
value (*int*)

Return type
ValueFilter

`classmethod greater_equal(value)`
Create a greater than or equal filter.

Parameters

value
[int] The value to compare.

Returns

ValueFilter
A greater than or equal filter.

Parameters
value (*int*)

Return type
ValueFilter

`classmethod less(value)`
Create a less than filter.

Parameters

value
[int] The value to compare.

Returns

ValueFilter
A less than filter.

Parameters
value (*int*)

Return type
ValueFilter

classmethod less_equal(*value*)
Create a less than or equal filter.

Parameters

value
[int] The value to compare.

Returns

ValueFilter
A less than or equal filter.

Parameters
value (*int*)

Return type
ValueFilter

max_value

min_value

classmethod not_equal(*value*)
Create a not equal filter.

Parameters

value
[int] The value to compare.

Returns

ValueFilter
A not equal filter.

Parameters
value (*int*)

Return type
ValueFilter

`pattern`

`classmethod range`(*min_value*, *max_value*)

Create a range filter.

Parameters

min_value
[int] The minimum value.

max_value
[int] The maximum value.

Returns

ValueFilter
A range filter.

Parameters

- **min_value** (*int*)
- **max_value** (*int*)

Return type
ValueFilter

`classmethod string_contains`(*pattern*)

Create a string contains filter.

Parameters
pattern (*str*)

Return type
ValueFilter

`classmethod string_endswith`(*pattern*)

Create a string ends-with filter.

Parameters
pattern (*str*)

Return type
ValueFilter

`classmethod string_exact`(*pattern*)

Create an exact string match filter.

Parameters
pattern (*str*)

Return type
ValueFilter

classmethod string_startswith(*pattern*)

Create a string starts-with filter.

Parameters
pattern (*str*)

Return type
ValueFilter

classmethod success()

Create a success filter (>= 0).

Returns

ValueFilter
A success filter.

Return type
ValueFilter

value

Parameters

- **filter_type** (*int*)
- **value** (*int*)
- **min_value** (*int*)
- **max_value** (*int*)
- **bitmask** (*int*)
- **pattern** (*bytes | None*)

pyplugins.apis.unwind module

Unwind Plugin (unwind.py)

Advanced stack unwinding using DWARF CFI and Heuristics with Capstone-powered instruction validation for multi-architecture support.

Dependencies: - capstone (pip install capstone) - penguin.plugins.* - wrappers.ptregs_wrap

class pyplugins.apis.unwind.**ArchInfo**(*name*, *ptr_size*, *endian*, *dwarf_map*, *cs_arch*, *cs_mode*, *call_offset*)

Bases: object

class pyplugins.apis.unwind.**StackUnwinder**

Bases: *Plugin*

unwind(*regs*)

Parameters
regs (PtRegsWrapper)

Return type
Generator[*Any*, None, *List*[*Dict*[str, *Any*]]]

pyplugins.apis.uprobes module

Uprobes Plugin System

The Uprobes plugin provides a flexible interface for monitoring, filtering, and intercepting user-space function calls in guest processes via the IGLOO hypervisor and portal interface. This enables dynamic tracing, security monitoring, and program analysis at the function level in user applications and libraries.

Overview

The uprobes system operates through a multi-layer architecture:

1. **Kernel Layer**: IGLOO kernel module registers uprobes at specified file offsets or symbols, with process and PID filtering.
2. **Hypervisor Layer**: Portal system manages communication between guest and host, relaying probe events.
3. **Plugin Layer**: Python plugins register uprobes and process events via callback functions.

Quick Start

```
from penguin import plugins, Plugin

class MyUprobeMonitor(Plugin):
    def __init__(self):
        # Register a uprobe on 'main' in /usr/bin/myapp for all processes named 'target'
        @plugins.uprobes.uprobe("/usr/bin/myapp", "main", process_filter="target")
        def on_main_entry(pt_regs):
            print(f"main() called! PC={pt_regs.pc:#x}")

        # Register a return probe on a library function
        @plugins.uprobes.uretprobe("/usr/lib/libfoo.so", "foo_func", process_filter=
↪"target")
        def on_foo_func_return(pt_regs):
            print(f"foo_func returned! Return value: {pt_regs.retval}")
```

Core Concepts

Uprobe Events

A uprobe event is generated when a monitored function is entered or returned from. Each event provides access to the CPU register state (`pt_regs`) at the probe point.

Probe Registration

Probes are registered using the `@uprobes.uprobe()` or `@uprobes.uretprobe()` decorators, supporting flexible filtering and symbol/offset selection.

```
@uprobes.uprobe(
    path="/usr/lib/libfoo.so",
    symbol="foo_func",
    process_filter="target_app",
    on_enter=True,
    on_return=False,
    pid_filter=1234
```

(continues on next page)

(continued from previous page)

```
)
def my_handler(pt_regs):
    # Handler code here
    pass
```

API Reference

Decorator Parameters

path (str or None)

Path to the executable or library file. Supports wildcards (*). If None, will search all known libraries for the symbol.

symbol (str or int)

Function name (string) or file offset (integer) at which to place the probe.

process_filter (Optional[str])

Filter by process name (comm):

```
@uprobes.uprobe("/usr/bin/myapp", "main", process_filter="target")
def monitor_main(pt_regs):
    pass
```

pid_filter (Optional[int])

Filter by specific process ID:

```
@uprobes.uprobe("/usr/bin/myapp", "main", pid_filter=1234)
def monitor_pid(pt_regs):
    pass
```

on_enter / on_return (bool)

Control when the probe triggers:

```
# Entry only (default)
@uprobes.uprobe("/usr/bin/myapp", "main", on_enter=True)

# Return only
@uprobes.uretprobe("/usr/bin/myapp", "main", on_return=True)

# Both entry and return
@uprobes.uprobe("/usr/bin/myapp", "main", on_enter=True, on_return=True)
```

fail_register_ok (bool)

If True, do not raise an error if the symbol is not found.

Handler Function Signature

```
def handler(pt_regs):
    """
    pt_regs: Register state at probe point (see below)
    """
```

pt_regs Object

The `pt_regs` parameter provides access to the CPU register state at the probe point:

```
def my_handler(pt_regs):
    print(f"PC: {pt_regs.pc:#x}")
    print(f"SP: {pt_regs.sp:#x}")
    print(f"Return value: {pt_regs.retval}")  # On return probes
    # Access general-purpose registers as needed
```

Practical Examples

Monitor Function Entry

```
@uprobes.uprobe("/usr/bin/myapp", "main")
def on_main_entry(pt_regs):
    print(f"main() called at PC={pt_regs.pc:#x}")
```

Monitor Function Return

```
@uprobes.uretprobe("/usr/lib/libfoo.so", "foo_func")
def on_foo_func_return(pt_regs):
    print(f"foo_func returned, retval={pt_regs.retval}")
```

Wildcard Library Matching

```
@uprobes.uprobe("libssl*", "SSL_read")
def monitor_ssl_read(pt_regs):
    print("SSL_read called!")
```

Register on All Libraries Containing a Symbol

```
@uprobes.uprobe(None, "malloc")
def monitor_malloc(pt_regs):
    print("malloc called in some library!")
```

Offset-based Probes

```
@uprobes.uprobe("/usr/bin/myapp", 0x1234)
def on_offset(pt_regs):
    print("Hit offset 0x1234 in myapp")
```

Process and PID Filtering

```
@uprobes.uprobe("/usr/bin/myapp", "main", process_filter="target", pid_filter=4321)
def filtered_handler(pt_regs):
    print("main() called by target process with PID 4321")
```

Performance Considerations

- **Specific probes**: Register only the functions you need to minimize overhead.
- **Process/PID filtering**: Reduces the number of events delivered to the host.
- **Wildcard/pathless probes**: May register probes in many libraries; use sparingly.

Error Handling

- Always check for symbol resolution errors; use `fail_register_ok=True` to ignore missing symbols.
- Handle missing or invalid register state gracefully in handlers.

Hook Management

Dynamic Control

Enable/disable probes at runtime:

```
# Unregister a probe by ID
yield from uprobes.unregister(probe_id)
```

Probe Information

Access probe configuration:

```
probe_id = plugins.uprobes._func_to_probe_id[my_handler]
info = plugins.uprobes.probe_info[probe_id]
print(f"Probe at {info['path']}:{info['offset']}")
```

Integration with Other Plugins

With Portal Plugin

Uprobes are managed via the portal system and can be combined with other portal-based plugins for richer context.

With OSI Plugin

Combine with OS introspection for process metadata:

```
def handler(pt_regs):
    proc = yield from plugins.OSI.get_proc()
    print(f"Probe hit in process {proc.name} (PID {proc.pid})")
```

Testing

The system supports automated tests for probe registration, event delivery, and filtering.

```
@uprobes.uprobe("/usr/bin/testprog", "main", process_filter="testproc")
def test_probe(pt_regs):
    assert pt_regs.pc != 0
```

Troubleshooting

Common Issues

1. **Probe not triggering**: Check path and symbol normalization, process filters, and PID filters.
2. **Symbol not found**: Ensure the symbols database is present and up to date.
3. **Multiple matches**: Wildcard or pathless probes may register in multiple libraries.

Debugging

Enable debug logging:

```
import logging
logging.getLogger("uprobes").setLevel(logging.DEBUG)
```

Print probe event details:

```
def debug_handler(pt_regs):
    print(f"PC: {pt_regs.pc:#x}, Registers: {pt_regs.regs}")
```

Best Practices

1. **Be specific**: Register probes only where needed.
2. **Filter early**: Use process and PID filters to reduce event volume.
3. **Handle errors**: Always expect symbol resolution or memory access to fail.
4. **Cache expensive lookups**: If reading memory or symbols, cache results.
5. **Test thoroughly**: Use the test framework to verify probe behavior.
6. **Monitor performance**: Excessive probes can impact guest performance.

The uprobes system provides a powerful foundation for dynamic user-space analysis, security monitoring, and program understanding in virtualized environments.

class pyplugins.apis.uprobes.**Uprobes**

Bases: *Plugin*

Uprobes Plugin

Provides an interface for registering and handling user-space probes (uprobes).

on_fs_init()

unregister(*target*)

Unregister a uprobe by handle, function, or name.

Args:
target: The handle (returned by decorator), function, or name of the uprobe to unregister.

Parameters
target (*Callable | str*)

uprobe(*path=None*, *symbol=None*, *address=None*, *base_addr=None*, *process_filter=None*, *on_enter=True*, *on_return=False*, *pid_filter=None*, *read_only=False*, *fail_register_ok=False*)

Decorator to register a uprobe.

Can register by symbol name, file offset, or virtual address.

Parameters

path
[Optional[str]] Path to the executable or library file (can include wildcards). Required if 'address' is used. If None (and using 'symbol'), matches all libraries containing the symbol.

symbol
[Union[str, int]] Symbol name (string) or file offset (integer).

address
[Optional[int]] Virtual address to hook. If provided, overrides 'symbol'.

base_addr
[Optional[int]] If provided with 'address', offset is calculated as address - base_addr. If None, attempts to resolve 'address' using ELF headers.

process_filter
[Optional[str]] Process name to filter events.

on_enter
[bool] Trigger on function entry (default: True).

on_return
[bool] Trigger on function return (default: False).

pid_filter
[Optional[int]] PID to filter events for a specific process.

read_only: bool fail_register_ok : bool

If True, silently return if symbol not found.

Returns

Callable[[Callable], Callable]
Decorator function that registers the uprobe.

Parameters

- **path** (*str | None*)
- **symbol** (*str | int*)
- **address** (*int | None*)

- **base_addr** (*int | None*)
- **process_filter** (*str | None*)
- **on_enter** (*bool*)
- **on_return** (*bool*)
- **pid_filter** (*int | None*)
- **read_only** (*bool*)
- **fail_register_ok** (*bool*)

Return type
Callable[[*Callable*], *Callable*]

uretprobe(*path=None*, *symbol=None*, *address=None*, *base_addr=None*, ***kwargs*)

Parameters

- **path** (*str | None*)
- **symbol** (*str | int*)
- **address** (*int | None*)
- **base_addr** (*int | None*)

Return type
Callable

pyplugins.compat package

Compatibility Plugins

Older required plugins can be added here to maintain compatibility with previous configurations. These plugins are not intended to be used in current versions of Penguin, but rather to ensure that older configurations can still function without errors.

This module is primarily for backward compatibility and should not be used in new configurations. If you are creating a new plugin, please refer to the current plugin architecture and guidelines.

Submodules

pyplugins.compat.coverage module

class pyplugins.compat.coverage.**Coverage**
Bases: *Plugin*

Coverage Plugin (compat)

Previous versions of penguin had a coverage plugin that was referenced, but not often used by users. This plugin exists to make compatibility with older configs easier, but it does not provide any functionality.

pyplugins.compat.qemu_compat module

pyplugins.compat.qemu_compat.**KVMArch**
alias of *QemuArch*

class pyplugins.compat.qemu_compat.**KVMLibPandaMock**(*lib*, *ffi*, *qemu=None*)

Bases: object

pyplugins.compat.qemu_compat.**KVMQemu**

alias of *QemuCompat*

class pyplugins.compat.qemu_compat.**QemuArch**(*panda*)

Bases: object

get_arg(*cpu*, *index*, *convention='syscall'*)

get_reg(*cpu*, *reg_name*)

set_arg(*cpu*, *index*, *value*, *convention='syscall'*)

set_reg(*cpu*, *reg_name*, *value*)

set_retval(*cpu*, *value*, *convention='default'*, *failure=False*)

class pyplugins.compat.qemu_compat.**QemuCompat**(*lib_path*, *arch*, *mode='kvm'*, *header_path=None*)

Bases: object

Parameters

- **lib_path** (*str | None*)
- **arch** (*str*)
- **mode** (*str*)
- **header_path** (*str | None*)

classmethod active_instances()

bind_hypercall_plugin(*plugin*)

cb_pre_shutdown(*f*)

cpu_env(*cpu=None*, *sync=None*)

Return a typed *CPUArchState* * for the given CPU (default: the current hypercall CPU), giving access to the full per-target state: coprocessor registers, timers, FPU, etc. Requires the *_env.h header generated alongside the QEMU library. Under KVM the CPU state is synchronized first so reads are fresh and writes stick; pass sync=False to skip that.

property direct_syscall_event_writeback: bool

disable_tb_chaining()

emit_qmp_event(*name*, *data=None*)

Emit an arbitrary QMP event to every connected QMP monitor.

The outbound counterpart to _dispatch_qmp(). `name` becomes the QMP `event` member verbatim. `data` is an optional payload; the QMP spec defines the event `data` member as a JSON *object*, so it must be a dict (or None for no payload).

Safe to call from a vCPU thread: the fork defers the work to a main-loop bottom half, so this neither needs the BQL nor blocks on serialization or the monitor write.

Parameters

- **name** (*str*)
- **data** (*Dict[str, Any] | None*)

Return type
None

end_analysis()

property env_supported: bool

classmethod from_installation(*mode*, *arch*)

Parameters

- **mode** (*str*)
- **arch** (*str*)

from_unsigned_guest(*value*)

get_cpu()

get_process_name(*cpu*)

hypercall(*nr*)

property hypercall_handlers

property hypercall_plugin

install_qmp_dispatch(*plugin*)

Bind the Qmp pyplugin and install the single C-level QMP trampoline.

The plugin owns the command name -> handler registry; the compat layer only forwards unrecognized QMP commands to it (mirroring how the hypercall trampoline forwards to the Hypercall plugin). Idempotent: the C callback is installed once and re-binding just updates the plugin.

load_plugin(*name*, *args=None*)

load_snapshot(*name*)

Restore an internal VM snapshot named name (loadvm).

Wraps the fork's penguin_load_snapshot(), which stops the VM, restores device + RAM state, and restores the prior runstate. Same calling-context rules as *save_snapshot()*. Returns True on success.

Parameters
name (*str*)

Return type
bool

property qmp_plugin

register_guest_hypercall(*nr*)

Parameters
nr (*int*)

Return type
bool

run()

save_snapshot(*name*)

Save an internal VM snapshot named `name` (savevm).

Wraps the fork's penguin_save_snapshot(). The underlying save_snapshot() stops the VM, serialises device + RAM state into the active block device's internal-snapshot store, and resumes. Must be invoked from the main loop context (e.g. a readiness/event callback, not a vCPU-thread hypercall handler) to avoid contending with the snapshot's own main-loop pumping; the BQL is taken here if needed. Returns True on success.

Parameters
name (*str*)

Return type
bool

schedule_snapshot(*name*, *load=False*)

Schedule a savevm (load=False) or loadvm (load=True) on the main loop.

Fire-and-forget and safe to call from a vCPU-thread callback (e.g. a guest hypercall / readiness handler): the snapshot runs in the main loop context where it can stop the vCPUs without deadlocking. Prefer this over *save_snapshot()*/*load_snapshot()* from such callbacks. Returns True if the request was successfully scheduled (not whether the snapshot itself succeeded — that is logged to QEMU stderr).

Parameters

- **name** (*str*)
- **load** (*bool*)

Return type
bool

set_after_guest_init_callback(*cb*)

Parameters
cb (*Callable*)

set_hypercall_callback(*cb*)

Parameters
cb (*Callable*)

set_os_name(*name*)

sync_cpu_state(*cpu*)

Synchronize register state out of the accelerator into env (and mark the vCPU dirty so env writes are pushed back). No-op under TCG or when the QEMU library lacks the export.

to_unsigned_guest(*value*, *failure=False*)

unload_plugins()

virtual_memory_read(*cpu*, *addr*, *size*, *fmt=None*)

virtual_memory_write(*cpu*, *addr*, *data*)

pyplugins.compat.qemu_compat.**resolve_qemu_paths**(*mode*, *arch*, *lib_path=None*, *header_path=None*)

Parameters

- **mode** (*str*)
- **arch** (*str*)

- **lib_path** (*str | None*)
- **header_path** (*str | None*)

Return type
tuple[*Path*, *Path* | None]

pyplugins.core package

Submodules

pyplugins.core.core module

core.py - Core plugin for Penguin

This module provides the Core plugin, which performs basic sanity checks, configuration management, and core logic for the penguin emulation environment. It is responsible for:

- Validating required arguments at initialization.
- Writing configuration and loaded plugin information to the output directory.
- Handling SIGUSR1 for graceful shutdown of emulation.
- Creating a *.ran* file in the output directory after a non-crash shutdown.
- Optionally enforcing a timeout for emulation and shutting down after the specified period.
- Setting up environment variables for features like root shell, graphics, shared directory, strace, ltrace, and forced WWW.
- Logging information about available services (e.g., root shell, VNC) based on configuration and environment.

Arguments

- timeout (int, optional): Timeout in seconds for automatic shutdown.

Plugin Interface

This plugin does not provide a direct interface for other plugins, but it writes configuration and plugin information to files in the output directory, which other plugins or tools may read. It also sets environment variables in the configuration dictionary that may be used by other components or plugins.

Overall Purpose

The Core plugin ensures the emulation environment is correctly set up, manages shutdown procedures, and records essential information for reproducibility and debugging.

class pyplugins.core.core.**Core**

Bases: *Plugin*

Core plugin for PyPlugins.

Performs sanity checks, manages configuration, handles shutdown signals, and enforces optional timeouts for the emulation environment.

The shutdown timeout is read from the top-level `core.timeout` config option (overridable by the `--timeout` CLI flag); it is not a plugin arg.

graceful_shutdown(*sig*, *frame*)

Handle SIGUSR1 for graceful shutdown.

Args:
sig (int): Signal number. frame: Current stack frame.

Parameters
sig (*int*)

Return type
None

shutdown_after_timeout(*timeout*, *shutdown_event*)

Shutdown the emulation after a specified timeout.

Args:
panda (Panda): The Panda emulation object. timeout (int): Timeout in seconds before shutdown. shutdown_event (threading.Event): Event to signal early shutdown.

Parameters

- **timeout** (*int*)
- **shutdown_event** (*Event*)

Return type
None

uninit()

Perform cleanup and signal shutdown event if running.

Return type
None

pyplugins.core.core.**shared_dir_and_core_dump_env**(*shared_dir*, *core_dumps*)

Map the (normalized) core.shared_dir / core.core_dumps config to guest env vars consumed by source.d/40_mount_shared_dir.sh.

Both features ride the single /igloo/shared 9p mount but are independent. Returns `(env_updates, deprecated_implicit)` where `deprecated_implicit` is True when core dumps were enabled only because shared_dir was set (the pre-split legacy behavior), so the caller can emit a deprecation warning.

`shared_dir`/`core_dumps` are the config values after schema normalization: a dict when enabled, or falsy when disabled.

pyplugins.core.igloodriver module

IGLOODriver Plugin

Plugin to handle interaction from IGLOO driver to report its module base address.

class pyplugins.core.igloodriver.**IGLOODriver**

Bases: *Plugin*

Plugin to handle the IGLOO driver.

hyp_report_igloo_module_baseaddr(*cpu*)

pyplugins.core.live_image module

LiveImage Plugin

Generates a guest filesystem live by creating a setup script that runs on boot. This version uses a tarball for file deployment and a single, batched hypercall for patching.

Overview

- Stages files and directories for deployment using a tarball.
- Supports various file actions: delete, move, shim, dir, host_file, inline_file, binary_patch, dev, symlink.
- Efficient setup script generation using hyp_file_op for file transfer.
- Batch processing of binary patches via a single hypercall.
- Optional pre-patch verification of expected bytes (`expect` / `on_mismatch`) and per-patch provenance (`why` / `tag`), recorded in `binary_patches.yaml` in the run output directory.
- Handles permissions, symlinks, device nodes, and error reporting.

Usage

The plugin is loaded by the Penguin framework and manages guest filesystem setup and patching.

Arguments

- `proj_dir`: Project directory for resolving host files.
- `conf`: Configuration dictionary specifying static files and actions.

Classes

- LiveImage: Main plugin class for guest filesystem setup and patching.

class pyplugins.core.live_image.**LiveImage**

Bases: *Plugin*

Generates a guest filesystem live by creating a setup script that runs on boot. This version uses a tarball for file deployment and a single, batched hypercall for patching.

fs_init(*func=None*)

Decorator for registering fs init callbacks. Can be used as @plugins.live_image.fs_init

Parameters
func (*Callable | None*)

portalcall_get(*path_ptr*, *offset*, *chunk_size*, *buffer_ptr*)

portalcall_get_perm(*path_ptr*)

portalcall_getsize(*path_ptr*)

portalcall_put(*path_ptr*, *offset*, *chunk_size*, *buffer_ptr*)

pyplugins.core.readiness module

class pyplugins.core.readiness.**Readiness**

Bases: *Plugin*

on_netbind(*sock_type*, *ipvn*, *ip*, *port*, *procname*)

Parameters

- **sock_type** (*str*)
- **ipvn** (*int*)
- **ip** (*str*)
- **port** (*int*)
- **procname** (*str*)

Return type
None

on_readiness(*kind*, *value=''*)

Parameters

- **kind** (*str*)
- **value** (*str*)

pyplugins.core.scope module

Analysis scoping (scope.py)

Restricts per-process **syscall/exec** analysis to the firmware-under-analysis process subtree.

At handoff, `init.sh` runs the real guest init in a fresh UTS namespace, so the firmware subtree (which inherits the namespace across fork/exec) lives outside the kernel's initial namespace, while Penguin's own infrastructure stays in it: the boot machinery, the backgrounded `vpnguin`/`console`/`guesthopper` helpers, and anything spawned via `call_usermodehelper`.

The igloo_driver gates syscall/exec hypercall emission on that distinction (`igloo_in_scope`), which automatically scopes every logger fed by those hypercalls (syscalls, exec, read/write, ...). This plugin's only job is to enable that driver-side gating, early enough to also exclude Penguin's pre-handoff boot machinery.

Two things are deliberately **not** scoped this way:

- **netbinds** – bind events always report, for every process, so Penguin's own service binds remain visible.
- **busybox shell coverage** – scoped in the guest instead: busybox suppresses its coverage hypercalls when `IGLOO_NO_SHELL_COV` is set, which Penguin exports for its infrastructure and clears at the firmware handoff. That needs no host-side per-process tracking, so this plugin no longer maintains one.

class pyplugins.core.scope.**Scope**

Bases: *Plugin*

Enables kernel-side scoping of syscall/exec analysis to the firmware-under-analysis subtree.

Arguments: none (reads `core.analysis_scope` from the global config).

pyplugins.core.snapshot module

Snapshot Plugin (snapshot.py) for Penguin

Saves and restores VM snapshots (QEMU savevm/loadvm). Driven by the `core.snapshot` config section (snapshotting is active whenever `save_at` or `boot_from` is set — there is no separate enable flag):

- `save_at: readiness`: save once the guest reaches steady state.
- `save_at: manual`: save/restore on request — from inside the guest via the `snapshot_save` / `snapshot_load` hypercalls, or from the host via the RemoteCtrl socket (see `cli_snapshot`).
- `boot_from`: the run was already restored at boot via `-loadvm`; this plugin announces the restore so host-side plugins can rehydrate.

All save requests funnel into `request_save()`, which *arms* the capture at a chosen execution boundary ("find a safe spot") rather than snapping at the request instant:

- `when="next_syscall"` (default for host-driven requests): capture at the next syscall return (optionally filtered to a process) — a clean userspace/kernel boundary.
- `when="symbol"`: capture when execution reaches a symbol (one-shot uprobe).
- `when="now"`: capture immediately (used by the in-guest triggers, which are already at a hypercall boundary).

The actual save/restore is always handed to `KVMQemu.schedule_snapshot` (a main-loop bottom half), so it is safe to request from a vCPU-thread callback.

Other plugins that hold host-side state (e.g. the VPN bridge) reconcile by subscribing to this plugin's `on_snapshot` / `on_restore` events.

class pyplugins.core.snapshot.**Snapshot**

Bases: *Plugin*

dispatch_restore()

Rehydrate host-side state for a boot_from restore.

Called by penguin_run after all plugins are loaded and before the guest resumes. Loads the snapshot's host sidecar into each plugin's load_state, then fires on_restore so plugins can re-establish bridges / replay bindings.

Return type
None

on_guest_load(*tag=''*)

Parameters
tag (*str*)

on_guest_save(*tag=''*, *when='now'*, *proc=''*)

Parameters

- **tag** (*str*)
- **when** (*str*)
- **proc** (*str*)

on_ready(*kind='igloo_init'*)

Parameters
kind (*str*)

request_restore(*tag=None*)

Schedule a loadvm of `tag` into the running guest.

Return type
bool

request_save(*tag=None*, *when='next_syscall'*, *proc=None*, *symbol=None*)

Arm a savevm at a safe execution boundary.

Parameters

- **tag** – snapshot name (defaults to the configured tag).
- **when** – `next_syscall` | `symbol` | `now`.
- **proc** – optional process-name filter for the boundary.
- **symbol** – required when `when='symbol'`.

Returns
True if the capture was scheduled/armed.

Return type
bool

pyplugins.hyper package

hyper Plugin Package

The `hyper` package provides a collection of plugins for the Penguin hypervisor environment. These plugins enable advanced monitoring, instrumentation, and communication between the guest system and the hypervisor, supporting a variety of use cases such as coverage analysis, environment variable tracking, command logging, and inter-plugin messaging.

Overview

Purpose: The `hyper` package is designed to extend the capabilities of the Penguin hypervisor by offering modular plugins that respond to guest hypercalls, manage shared memory regions, and facilitate plugin-to-plugin communication.

Key Features: - Logging and auditing of shell and Bash command execution. - Tracking and manipulation of U-Boot environment variables. - Canary value monitoring for integrity checks. - Portal mechanism for efficient command and data transfer between plugins and the hypervisor. - Utilities for handling hypercall constants and plugin configuration.

Typical Usage: Plugins in this package are loaded by the Penguin framework and operate transparently, responding to specific events or hypercalls from the guest. Output is typically written to files in a specified output directory for later analysis.

Included Plugins

- `bash_command`: Logs Bash command executions.
- `canary`: Monitors canary values for security/integrity.
- `consts`: Provides access to hypercall and portal constants.
- `portal`: Implements the portal communication mechanism.
- `shell`: Tracks shell script coverage and environment usage.
- `uboot`: Simulates U-Boot environment variable management.

See individual plugin modules for detailed documentation and usage examples.

Submodules

pyplugins.hyper.bash_command module

pyplugins.hyper.canary module

pyplugins.hyper.consts module

Hypervisor Constants Loader (consts.py)

This module provides a convenient interface for accessing hypervisor-related constants and enumerations used by plugins in the Penguin hypervisor environment. It dynamically loads enums from the kernel FFI and exposes them as Python objects for easy and type-safe access.

Overview

- Loads and exposes enums such as HYPER_OP, portal_type, igloo_hypercall_constants, and others.
- Wraps each enum in a Wrapper object for attribute-style access.
- Ensures that all required enums are present and raises an assertion error if any are missing.

Example Usage

```
from hyper import consts

# Access a constant from HYPER_OP
op_code = consts.HYPER_OP.READ

# Access a constant from igloo_hypercall_constants
hypercall_num = consts.igloo_hypercall_constants.IGLOO_HYPER_REGISTER_MEM_REGION
```

Exposed Enums

- HYPER_OP
- portal_type
- igloo_hypercall_constants
- hyperfs_ops
- hyperfs_file_ops
- value_filter_type

Implementation Details

- Uses plugins.kffi.get_enum_dict to fetch enum values from the kernel FFI.
- Wraps each enum dictionary in a Wrapper for attribute-style access.
- All enums are loaded at module import time.

pyplugins.hyper.portal module

Portal Plugin

This module implements the Portal plugin for the Penguin hypervisor environment. It provides a mechanism for plugins to communicate with the hypervisor and each other using memory-mapped regions and hypercalls. The Portal plugin manages command and data transfer, interrupt handling, and memory region state for efficient and flexible plugin communication.

Usage

The Portal plugin is loaded by the Penguin framework and is not intended for direct invocation. It provides an API for other plugins to register interrupt handlers, queue interrupts, and send/receive commands via the portal mechanism.

Example

```
# Register an interrupt handler
portal.register_interrupt_handler("my_plugin", my_handler_fn)

# Queue an interrupt for a plugin
portal.queue_interrupt("my_plugin")
```

Classes

- PortalCmd: Encapsulates a command to be sent through the portal mechanism.
- Portal: Main plugin class for handling portal communication and interrupts.

Key Features

- Memory-mapped command and data transfer
- Plugin interrupt registration and handling
- Command construction and parsing utilities

class pyplugins.hyper.portal.**Portal**

Bases: *Plugin*

Portal is a plugin that manages communication and interrupts between plugins and the hypervisor.

It provides methods for registering interrupt handlers, queuing interrupts, and reading/writing commands and data to memory-mapped regions.

Attributes:

outdir (str): Output directory for plugin data. endian_format (str): Endianness format character for struct operations. portal_interrupt (Optional[int]): Address of the portal interrupt. _interrupt_handlers (dict): Mapping of plugin names to their interrupt handler functions. _pending_interrupts (set): Set of plugin names with pending interrupts. regions_size (int): Size of the memory region.

queue_interrupt(*plugin_name*)

Queue an interrupt for a plugin.

Parameters: - *plugin_name* (str): Name of the plugin.

Returns: - *bool*: True if queued successfully, False otherwise.

Parameters
plugin_name (*str*)

Return type
bool

register_interrupt_handler(*plugin_name*, *handler_fn*)

Register a plugin to handle portal interrupts.

Parameters: - plugin_name (str): Name of the plugin. - handler_fn (Callable[[], Iterator]): Function to handle interrupts for this plugin.

Must be a generator function that can be used with yield from.

Returns: None

Parameters

- **plugin_name** (*str*)
- **handler_fn** (*Callable[[], Iterator]*)

Return type
None

take_install_budget()

Consume one unit of the per-interrupt-window pseudofile-install budget.

Returns True if budget remained (and one unit was consumed), False if the window's budget is exhausted. Install interrupt handlers should call this before each install and, when it returns False, stop and re-queue their interrupt so the remainder is processed in a later window.

Return type
bool

uninit()

Clean up all interrupt handlers and pending interrupts on plugin unload.

Returns: None

Return type
None

wrap(*f*)

Wrap a function to manage portal command iteration and state.

Parameters: - *f* (Callable): Function to wrap.

Returns: - *Callable*: Wrapped function.

Parameters
f (*Callable*)

Return type
Callable

class pyplugins.hyper.portal.**PortalCmd**(*op*, *addr=0*, *size=0*, *pid=None*, *data=None*)

Bases: object

Encapsulates a command to be sent through the portal mechanism.

This class centralizes the logic for constructing portal commands and reduces complexity in the _handle_output_cmd method.

Attributes:

op (int): Operation code from HYPER_OP constants. addr (int): Address field value. size (int): Size field value. pid (int): Process ID or CURRENT_PID_NUM for current process. data (Optional[bytes]): Optional data payload for the command.

Parameters

- **op** (*int | str*)
- **addr** (*int*)
- **size** (*int*)
- **pid** (*int | None*)
- **data** (*bytes | None*)

addr

data

classmethod none()

Create a command representing no operation.

Returns: - *PortalCmd*: A command with HYPER_OP_NONE operation.

Return type
PortalCmd

op

pid

size

pyplugins.hyper.shell module

Shell Coverage Plugin

This module implements the Shell Coverage (BBCov) plugin for the Penguin hypervisor environment. It tracks shell script execution coverage, traces, and environment variable usage by listening to hypercall events from the guest. The plugin writes coverage, trace, and environment information to CSV files for later analysis.

Usage

The plugin is loaded by the Penguin framework and responds to "igloo_shell" events.

Output Files

- `shell_cov.csv`: Coverage data (filename, line number, pid)
- `shell_cov_trace.csv`: Trace data (filename:lineno, contents)
- `shell_env.csv`: Environment variable data (filename, lineno, pid, envs)

Arguments

- `outdir`: Output directory for generated CSV files.
- `fs`: Path to the tar archive containing the filesystem.
- `verbose`: If set, enables debug logging.

Classes

- BBCov: Main plugin class for handling shell coverage and environment logging.

class pyplugins.hyper.shell.**BBCov**(*panda*)

Bases: *Plugin*

BBCov is a plugin that logs shell script coverage, traces, and environment variable usage.

Arguments: - *outdir* (str): Output directory for generated CSV files. - *fs* (str): Path to the tar archive containing the filesystem. - *verbose* (bool): Enables debug logging if True.

Parameters
panda (*Any*)

igloo_shell_cb(*cpu*, *hc_type*, *argptr*, *length*)

Callback for handling igloo_shell hypercall events.

Parameters: - *cpu* (Any): The CPU object. - *hc_type* (int): Hypercall type. - *argptr* (int): Pointer to arguments in guest memory. - *length* (int): Number of arguments.

Returns: None

Parameters

- **cpu** (*Any*)
- **hc_type** (*int*)
- **argptr** (*int*)
- **length** (*int*)

Return type
None

log_env_args(*cpu*, *argv*)

Log environment variable information for a shell script line.

Parameters: - *cpu* (Any): The CPU object. - *argv* (list): List of argument pointers.

Returns: None

Parameters

- **cpu** (*Any*)
- **argv** (*list*)

Return type
None

log_line_no(*cpu*, *argv*)

Log coverage information for a shell script line.

Parameters: - *cpu* (Any): The CPU object. - *argv* (list): List of argument pointers.

Returns: None

Parameters

- **cpu** (*Any*)
- **argv** (*list*)

Return type
None

try_read_int(*cpu*, *ptr*)

Attempt to read an integer from guest memory.

Parameters: - *cpu* (Any): The CPU object. - *ptr* (int): Pointer to the integer in guest memory.

Returns: - *Optional[int]*: The integer read, or None if not available.

Parameters

- **cpu** (*Any*)
- **ptr** (*int*)

Return type
int | None

try_read_string(*cpu*, *ptr*)

Attempt to read a string from guest memory.

Parameters: - *cpu* (Any): The CPU object. - *ptr* (int): Pointer to the string in guest memory.

Returns: - *Optional[str]*: The string read, or None if not available.

Parameters

- **cpu** (*Any*)
- **ptr** (*int*)

Return type
str | None

pyplugins.hyper.uboot module

pyplugins.hyperfile package

Submodules

pyplugins.hyperfile.anonfs module

class pyplugins.hyperfile.anonfs.**AnonFS**

Bases: *Plugin*

register_anon_file(*vfs_file*, *name='[igloo_anon]'*)

Injects a generic VFS anonymous inode into the guest process table. Returns the raw integer File Descriptor.

Parameters

- **vfs_file** (*VFSFile*)
- **name** (*str*)

Return type
Generator[int, None, int]

register_dynamic_file(*file_model*)

Stores the model to keep FFI pointers alive in memory.

Return type
int

register_socket(*sock_file*)

Injects a true kernel socket object into the guest process table. Returns the raw integer File Descriptor.

Parameters
sock_file (*SocketFile*)

Return type
Generator[int, None, int]

pyplugins.hyperfile.devfs module

class pyplugins.hyperfile.devfs.**Devfs**

Bases: *Plugin*

register(*devfs_file*, *path=None*, *major=None*, *minor=None*)

Parameters

- **devfs_file** (*DevFile*)
- **path** (*str | None*)
- **major** (*int | None*)
- **minor** (*int | None*)

register_devfs(*devfs_file*, *path=None*, *major=None*, *minor=None*)

Register a DevFile for later portal registration.

Parameters

- **devfs_file** (*DevFile*)
- **path** (*str | None*)
- **major** (*int | None*)
- **minor** (*int | None*)

pyplugins.hyperfile.mtd module

class pyplugins.hyperfile.mtd.**MTD**

Bases: *Plugin*

class Args(*, *devices={}*, *pseudofiles={}*)

Bases: *PluginArgs*

Parameters

- **devices** (*Dict*)
- **pseudofiles** (*Dict*)

devices: Dict

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

pseudofiles: Dict

PERSONALITIES = {'nand': {'erase_size': 131072, 'oob_size': 64, 'type': 'nand', 'write_size': 2048}, 'nor': {'erase_size': 65536, 'oob_size': 0, 'type': 'nor', 'write_size': 1}}

Geometry defaults per flash technology. *nand* is the fallback when a device declares no personality.

register(*dev*)

Parameters
dev (*MtdDevice*)

register_mtd(*dev*)

Dynamically queue a new object-oriented MTD device for registration.

Parameters
dev (*MtdDevice*)

pyplugins.hyperfile.procfs module

class pyplugins.hyperfile.procfs.**Proc**

Bases: *Plugin*

register(*proc_file*, *path=None*)

Parameters

- **proc_file** (*ProcFile*)
- **path** (*str | None*)

register_proc(*proc_file*, *path=None*)

Register a ProcFile for later portal registration.

Parameters

- **proc_file** (*ProcFile*)
- **path** (*str | None*)

pyplugins.hyperfile.pseudofile_tracker module

Pseudofile Tracker Plugin

This plugin tracks guest attempts to access missing pseudofiles and monitors the configured pseudofile models.

Purpose

- Monitors syscalls for -ENOENT and -ENOTTY to track missing pseudofile accesses.
- Logs access attempts to missing files in *pseudofiles_failures.yaml*, ranked by impact and annotated with a suggested starting model per path.
- Exports the currently configured pseudofile models to *pseudofiles_modeled.yaml*.

Output format

Entries in *pseudofiles_failures.yaml* are ranked by impact (crashing_callers >> distinct_callers > hits):

```yaml /dev/watchdog:
    impact: {hits: 243, distinct_callers: 2, crashing_callers: 1, crashing_callers_by_name: 0} callers: [init:1,
    watchdogd:412] events:
        ioctl:
            2147768064: {count: 240}
        open: {count: 3}
    suggest:
        read: {model: zero} ioctl: {'*': {model: return_const, val: 0}}
```

suggest is a valid *pseudofiles:* value: paste it into your config as *pseudofiles: {<path>: <suggest block>}* and refine from there.

Usage

Simply add this plugin to your configuration: ```yaml core:

    plugins:
        - pseudofile_tracker

class pyplugins.hyperfile.pseudofile_tracker.PseudofileTracker

Bases: *Plugin*

Passively tracks missing file access and outputs telemetry logs.

class Args(*, *logging=None*)

Bases: *PluginArgs*

Parameters
logging (*str | List[str] | None*)

logging: str | List[str] | None

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

centralized_log(*path*, *event*, *caller=None*, *intents=None*, *event_details=None*)

dump_results()

Flushes the impact-ranked failures telemetry to disk.

log_ioctl_failure(*path*, *cmd*, *caller=None*)

record_default_hit(*path*, *op*, *details=None*)

Record that a synthesized default model actively served an access.

Folds the hit into the same failures view as genuine -ENOENT/-ENOTTY failures (events default_read/default_write/default_ioctl), so an active default surfaces as "needs a real model". Called by the Pseudofiles plugin, already deduplicated per (op, cmd) at the model.

uninit()

Plugin teardown logic ensures final metrics are written.

pyplugins.hyperfile.pseudofile_tracker.**count_crashing_callers**(*callers*, *crash_pairs*, *crash_names*)

Distinct callers of a path that later crashed, as (exact, by_name): callers whose (proc, pid) matches a crash record, and callers matching a crashed proc name only (pid recycled between the access and the fault). Reported and weighted separately – a name-only match is a much weaker signal.

Parameters

- **callers** (*set*)
- **crash_pairs** (*set*)
- **crash_names** (*set*)

pyplugins.hyperfile.pseudofile_tracker.**get_total_counts**(*d*)

Get the sum of all "count" values in a nested dictionary.

Return type
int

pyplugins.hyperfile.pseudofile_tracker.**ignore_cmd**(*ioctl*)

Ignore TTY ioctls, see ioctls.h for T*, TC*, and TIO* ioctls.

Parameters
ioctl (*int*)

Return type
bool

pyplugins.hyperfile.pseudofile_tracker.**ignore_ioctl_path**(*path*)

Filter out ioctl paths that are irrelevant to rehosting.

Parameters
path (*str*)

Return type
bool

pyplugins.hyperfile.pseudofile_tracker.**impact_score**(*impact*)

Parameters
impact (*dict*)

Return type
int

pyplugins.hyperfile.pseudofile_tracker.**load_crashes**(*path*)

Parse the crashes plugin's crashes.yaml into join keys.

Returns (pairs, names): the set of (proc, pid) tuples and the set of proc names that crashed. Tolerates a missing/empty file (this plugin may run without crash tracking) and both the *{crashes: [...]}* dict and a bare list of records.

Parameters
path (*str*)

pyplugins.hyperfile.pseudofile_tracker.**open_access_intents**(*flags*)

Map open(2) flags to the access intents implied by O_ACCMODE.

Parameters
flags (*int*)

Return type
set

pyplugins.hyperfile.pseudofile_tracker.**path_interesting**(*path*)

Determines if a path is likely to be a good candidate for pseudofiles.

Parameters
path (*str*)

Return type
bool

pyplugins.hyperfile.pseudofile_tracker.**sort_file_failures**(*d*)

Recursively sorts the file failures dictionary by total event count.

pyplugins.hyperfile.pseudofile_tracker.**suggest_models**(*events*, *intents*)

Heuristic starting model for a failing path, shaped as a valid *pseudofiles:* value so it pastes straight into a config.

- open/stat ENOENT with read intent (or unknown intent) -> read: zero. The failure happens at open time, before any read size is observable, so small vs. large reads can't be distinguished here; *zero* (a one-byte "0" then EOF) is the safe default and the file header points users at const_buf when real contents matter.
- open ENOENT with write intent -> write: discard.
- ioctl ENOTTY -> catch-all return_const 0.

Parameters
- **events** (*dict*)
- **intents** (*set*)

Return type
dict

pyplugins.hyperfile.pseudofiles module

class pyplugins.hyperfile.pseudofiles.**Pseudofiles**

Bases: *Plugin*

class Args(*, *disable_tracking=False*)

Bases: *PluginArgs*

Parameters
disable_tracking (*bool*)

disable_tracking: bool

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

```
EXTRA_OPS = {'fasync': ({}, <class 'hyperfile.models.seek.FasyncExternalVFS'>),
'flush': ({}, <class 'hyperfile.models.seek.FlushExternalVFS'>), 'fsync': ({},
<class 'hyperfile.models.seek.FsyncExternalVFS'>), 'get_unmapped_area': ({}, <class
'hyperfile.models.seek.GetUnmappedAreaExternalVFS'>), 'lock': ({}, <class
'hyperfile.models.seek.LockExternalVFS'>), 'lseek': ({'default': <class
'hyperfile.models.seek.SeekDefault'>, 'unsupported': <class
'hyperfile.models.seek.SeekUnsupported'>}, <class
'hyperfile.models.seek.SeekExternalVFS'>), 'mmap': ({}, <class
'hyperfile.models.seek.MmapExternalVFS'>), 'open': ({}, <class
'hyperfile.models.seek.OpenExternalVFS'>), 'read_iter': ({}, <class
'hyperfile.models.seek.ReadIterExternalVFS'>), 'release': ({}, <class
'hyperfile.models.seek.ReleaseExternalVFS'>), 'write_iter': ({}, <class
'hyperfile.models.seek.WriteIterExternalVFS'>)}

backing_models = {}

ioctl_models = {'return_const': <class 'hyperfile.models.ioctl.IoctlReturnConst'>,
'zero': <class 'hyperfile.models.ioctl.IoctlZero'>}

mmap_models = {}

open_models = {}

poll_models = {'always_ready': <class 'hyperfile.models.poll.PollAlwaysReady'>,
'blocking': <class 'hyperfile.models.poll.PollNeverReady'>, 'periodic': <class
'hyperfile.models.poll.PollPeriodic'>}

read_models = {'const_buf': <class 'hyperfile.models.read.ReadConstBuf'>,
'const_map': <class 'hyperfile.models.read.ReadConstMap'>, 'const_map_file':
<class 'hyperfile.models.read.ReadConstMapFile'>, 'cycle': <class
'hyperfile.models.read.ReadCycle'>, 'default': <class
'hyperfile.models.read.ReadDefault'>, 'empty': <class
'hyperfile.models.read.ReadEmpty'>, 'from_file': <class
'hyperfile.models.read.ReadFromFile'>, 'one': <class
'hyperfile.models.read.ReadOne'>, 'return_const': <class
'hyperfile.models.read.ReadConstBuf'>, 'sequence': <class
'hyperfile.models.read.ReadSequence'>, 'stateful': <class
'hyperfile.models.read.ReadStateful'>, 'zero': <class
'hyperfile.models.read.ReadZero'>}

release_models = {}

seek_models = {'default': <class 'hyperfile.models.seek.SeekDefault'>,
'unsupported': <class 'hyperfile.models.seek.SeekUnsupported'>}

write_models = {'default': <class 'hyperfile.models.write.WriteDefault'>,
'discard': <class 'hyperfile.models.write.WriteDefault'>, 'return_const': <class
'hyperfile.models.write.WriteReturnConst'>, 'to_file': <class
'hyperfile.models.write.WriteToFile'>, 'unhandled': <class
'hyperfile.models.write.WriteUnhandled'>}
```

pyplugins.hyperfile.qemu_mem module

class pyplugins.hyperfile.qemu_mem.**Allocation**(*name: str*, *offset: int*, *size: int*, *storage: bytearray*, *read_cb: Callable | None*, *write_cb: Callable | None*)

Bases: object

Parameters

- **name** (*str*)
- **offset** (*int*)
- **size** (*int*)
- **storage** (*bytearray*)
- **read_cb** (*Callable | None*)
- **write_cb** (*Callable | None*)

name: str

offset: int

read_cb: Callable | None

size: int

storage: bytearray

write_cb: Callable | None

class pyplugins.hyperfile.qemu_mem.**QemuMem**

Bases: *Plugin*

Native QEMU MMIO aperture used as a backing store for guest mmap() users.

class Args(*, *mmap_alignment=None*, *mmap_base=None*, *mmap_size=None*)

Bases: *PluginArgs*

Parameters

- **mmap_alignment** (*int | str | None*)
- **mmap_base** (*int | str | None*)
- **mmap_size** (*int | str | None*)

mmap_alignment: int | str | None

mmap_base: int | str | None

mmap_size: int | str | None

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

allocate_file(*file_model*)

Return type

int

allocate_region(*name*, *size*, *read_cb=None*, *write_cb=None*, *initial=None*)

Parameters

- **name** (*str*)
- **size** (*int*)

Return type
int

pyplugins.hyperfile.sysctl module

class pyplugins.hyperfile.sysctl.**Sysctl**

Bases: *Plugin*

register(*sysctl_file*, *path=None*)

Parameters

- **sysctl_file** (*SysctlFile*)
- **path** (*str | None*)

register_sysctl(*sysctl_file*, *path=None*)

Register a SysctlFile for later portal registration.

Parameters

- **sysctl_file** (*SysctlFile*)
- **path** (*str | None*)

pyplugins.hyperfile.sysfs module

class pyplugins.hyperfile.sysfs.**Sysfs**

Bases: *Plugin*

register(*sysfs_file*, *path=None*, *mode=420*)

Parameters

- **sysfs_file** (*SysFile*)
- **path** (*str | None*)
- **mode** (*int*)

register_sysfs(*sysfs_file*, *path=None*, *mode=420*)

Parameters

- **sysfs_file** (*SysFile*)
- **path** (*str | None*)
- **mode** (*int*)

pyplugins.interventions package

Penguin Interventions Plugins

This package contains a collection of **Penguin interventions** plugins for dynamic analysis, emulation, and modeling of embedded Linux systems. These plugins provide mechanisms to intercept, log, and model various guest OS behaviors, such as pseudo-file accesses, NVRAM operations, and more.

Overview

Purpose: Enhance the observability, control, and modeling of guest system interactions during emulation or analysis. Useful for firmware analysis, device modeling, and dynamic introspection.

Key Plugins: - `pseudofiles`: Model and log accesses to pseudo-files (e.g., `/dev`, `/proc`, `/sys`). - `nvram2`: Track and log NVRAM get/set/clear operations. - `hyperfile`: Provide a hypercall-based interface for guest/host file operations.

Usage: Plugins are loaded via the Penguin framework and can be configured for specific analysis tasks.

See individual plugin modules for detailed documentation and configuration options.

Submodules

pyplugins.interventions.core_pattern_guard module

Lock /proc/sys/kernel/core_pattern after the guest's init script sets it.

source.d/40_mount_shared_dir.sh writes our pattern via the kernel's real proc_dostring_coredump handler (populating the core_pattern[] global that format_corename reads), then fires the *core_pattern_lock* hypercall. We respond by registering a SysctlFile that hijacks the proc_handler – reads return our pattern, writes are logged at info and dropped, so guest software (systemd-coredump, apport, container runtimes) can't redirect dumps away from /igloo/shared/core_dumps.

The kernel's core_pattern[] global is set BEFORE we hijack (via the init script's echo), and our handler eats subsequent writes without touching the global. The driver's sysctl mutation repoints ctl_table.data to our buffer but never writes through to the original .data target, so the global keeps the value the init script put there.

Dormant when core dumps are disabled or core.core_dumps.lock is false: the init script only fires the hypercall when both CORE_DUMPS and CORE_DUMPS_LOCK are in the guest env (set by core.py from core.core_dumps). We subscribe unconditionally anyway so a stray core_pattern_lock is honored instead of tripping send_hypercall's "Unregistered send_hypercall command" error.

class pyplugins.interventions.core_pattern_guard.**CorePatternGuard**

Bases: *Plugin*

class pyplugins.interventions.core_pattern_guard.**CorePatternSysctl**(*pattern*, *owner*)

Bases: `SysctlFile`

Parameters
pattern (*str*)

MAXLEN: int = 256

MODE: int = 420

PATH: str = 'kernel/core_pattern'

proc_handler(*ptregs*, *ctl*, *write*, *buffer*, *lenp*, *ppos_ptr*)

Unified sysctl entry point. Extracts arguments from ptregs and routes to read() or write().

pyplugins.interventions.hyperfile module

HyperFile Plugin

This module implements the HyperFile plugin for the Penguin framework, enabling hypercall-based file operations between a guest and the host. It provides a model for virtual files that can be read, written, or controlled via ioctl/getattr operations from the guest OS. The plugin is designed to be flexible and extensible, allowing users to specify custom file behaviors via models.

Features

- Handles hypercalls for file operations (read, write, ioctl, getattr)
- Supports dynamic file models for custom device/file behaviors
- Logs and tracks file operation results for analysis
- Provides default behaviors for unhandled operations

Example Usage

```
from pyplugins.interventions.hyperfile import HyperFile

# Register the plugin with Penguin, specifying file models and log file
plugin = HyperFile()
```

File Model Example

```
files = {
    "/dev/zero": {
        fops.HYP_READ: HyperFile.read_zero,
        fops.HYP_WRITE: HyperFile.write_discard,
        "size": 0,
    }
}
```

Classes

- HyperFile: Main plugin class implementing the hypercall interface.

Functions

- hyper(name: str) -> int: Map operation name to hyperfile operation constant.
- hyper2name(num: int) -> str: Map hyperfile operation constant to operation name.

class pyplugins.interventions.hyperfile.**HyperFile**

Bases: *Plugin*

The HyperFile plugin implements a virtual file interface for the guest OS, allowing the guest to perform file operations via hypercalls.

Attributes - *arch_bytes* (*int*): Number of bytes per architecture word. - *log_file* (*Optional[str]*): Path to the log file for operation results. - *files* (*Optional[Dict[str, Dict]]*): File models for virtual devices. - *logger* (*Any*): Logger instance. - *endian* (*str*): Endianness format for struct packing. - *s_word*, *u_word* (*str*): Signed/unsigned

word format for struct packing. - *results* (*Dict*): Stores results of file operations for logging. - *default_model* (*Dict*): Default model for unhandled file operations.

class Args(*, *log_file=None*, *logger=None*, *models=None*)

Bases: *PluginArgs*

Parameters

- **log_file** (*str | None*)
- **logger** (*Any | None*)
- **models** (*Dict | None*)

log_file: str | None

logger: Any | None

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

models: Dict | None

static getattr(*device_name*, *model*)

Handle a getattr operation, returning the file size.

Parameters - *device_name* (*str*): Device name. - *model* (*Dict[str, Any]*): File model dictionary.

Returns - *Tuple[int, int]*: (Return value, file size)

Parameters

- **device_name** (*str*)
- **model** (*Dict[str, Any]*)

Return type
Tuple[int, int]

handle_file_op(*cpu*)

Handle a file operation hypercall (read, write, ioctl, getattr).

Parameters - *cpu* (*Any*): The CPU context from Panda.

Returns - *None*

Parameters
cpu (*Any*)

Return type
None

handle_get_hyperfile_paths(*cpu*)

Handle the hypercall to get the paths of all hyperfiles.

Parameters - *cpu* (*Any*): The CPU context from Panda.

Returns - *None*

Parameters
cpu (*Any*)

Return type
None

`handle_get_num_hyperfiles`(*cpu*)

Handle the hypercall to get the number of hyperfiles.

Parameters - *cpu* (*Any*): The CPU context from Panda.

Returns - *None*

Parameters
cpu (*Any*)

Return type
None

`handle_result`(*device_name*, *event*, *retval*, **data*)

Record the result of a file operation for logging and analysis.

Parameters - *device_name* (*str*): The name of the device/file. - *event* (*str*): The event type ("read", "write", "ioctl", "getattr"). - *retval* (*int*): The return value of the operation. - **data* (*Any*): Additional data relevant to the event.

Returns - *None*

Parameters

- **device_name** (*str*)
- **event** (*str*)
- **retval** (*int*)
- **data** (*Any*)

Return type
None

`static ioctl`(*devname*, *cmd*, *arg*)

Handle an ioctl operation (default: always succeeds).

Parameters - *devname* (*str*): Device name. - *cmd* (*int*): IOCTL command. - *arg* (*int*): IOCTL argument.

Returns - *int*: Return value (0 for success).

Parameters

- **devname** (*str*)
- **cmd** (*int*)
- **arg** (*int*)

Return type
int

`static ioctl_unhandled`(*devname*, *cmd*, *arg*)

Handle an unhandled ioctl operation.

Parameters - *devname* (*str*): Device name. - *cmd* (*int*): IOCTL command. - *arg* (*int*): IOCTL argument.

Returns - *int*: Return value (-25 for ENOTTY).

Parameters

- **devname** (*str*)

- **cmd** (*int*)
- **arg** (*int*)

Return type
int

static read_unhandled(*filename*, *buffer*, *length*, *offset*)

Handle an unhandled read operation.

Parameters - *filename* (*str*): File name. - *buffer* (*int*): Guest buffer address. - *length* (*int*): Number of bytes to read. - *offset* (*int*): Offset into the file.

Returns - *Tuple[bytes, int]*: (Empty bytes, -22 for EINVAL)

Parameters

- **filename** (*str*)
- **buffer** (*int*)
- **length** (*int*)
- **offset** (*int*)

Return type
Tuple[bytes, int]

static read_zero(*devname*, *buffer*, *length*, *offset*)

Return a buffer of zero bytes for read operations.

Parameters - *devname* (*str*): Device name. - *buffer* (*int*): Guest buffer address. - *length* (*int*): Number of bytes to read. - *offset* (*int*): Offset into the file.

Returns - *Tuple[bytes, int]*: (Data read, number of bytes read)

Parameters

- **devname** (*str*)
- **buffer** (*int*)
- **length** (*int*)
- **offset** (*int*)

Return type
Tuple[bytes, int]

uninit()

Dump the results to the log file on plugin unload.

Returns - *None*

Return type
None

static write_discard(*devname*, *buffer*, *length*, *offset*, *contents*)

Discard written data and return the number of bytes written.

Parameters - *devname* (*str*): Device name. - *buffer* (*int*): Guest buffer address. - *length* (*int*): Number of bytes to write. - *offset* (*int*): Offset into the file. - *contents* (*bytes*): Data to write.

Returns - *int*: Number of bytes written.

Parameters

- **devname** (*str*)
- **buffer** (*int*)
- **length** (*int*)
- **offset** (*int*)
- **contents** (*bytes*)

Return type
int

static write_unhandled(*filename*, *buffer*, *length*, *offset*, *contents*)

Handle an unhandled write operation.

Parameters - *filename* (*str*): File name. - *buffer* (*int*): Guest buffer address. - *length* (*int*): Number of bytes to write. - *offset* (*int*): Offset into the file. - *contents* (*bytes*): Data to write.

Returns - *int*: Return value (-22 for EINVAL).

Parameters

- **filename** (*str*)
- **buffer** (*int*)
- **length** (*int*)
- **offset** (*int*)
- **contents** (*bytes*)

Return type
int

pyplugins.interventions.hyperfile.**hyper**(*name*)

Map a string operation name to its corresponding hyperfile operation constant.

Parameters - *name* (*str*): The operation name (“read”, “write”, “ioctl”, “getattr”).

Returns - *int*: The corresponding hyperfile operation constant.

Raises - *ValueError*: If the operation name is unknown.

Parameters
name (*str*)

Return type
int

pyplugins.interventions.hyperfile.**hyper2name**(*num*)

Map a hyperfile operation constant to its string operation name.

Parameters - *num* (*int*): The hyperfile operation constant.

Returns - *str*: The operation name.

Raises - *ValueError*: If the operation constant is unknown.

Parameters
num (*int*)

Return type
str

pyplugins.interventions.kernelversion module

Kernel Version Utilities

This module provides utilities for parsing, comparing, and representing Linux kernel version strings. It is intended to help plugins and scripts that need to reason about kernel versions in the Penguin framework or similar environments.

Features

- Parse kernel version strings into structured objects
- Compare kernel versions for ordering and equality
- Convert kernel version objects back to string representation

Classes

- KernelVersion: Represents a parsed kernel version and supports comparison.

class pyplugins.interventions.kernelversion.**KernelVersion**

Bases: *Plugin*

Represents a Linux kernel version and supports comparison operations.

Attributes - *outdir* (*Optional[str]*): Output directory. - *sysname* (*Optional[str]*): System name. - *nodename* (*Optional[str]*): Node name. - *release* (*Optional[str]*): Kernel release string. - *version* (*Optional[str]*): Kernel version string. - *machine* (*Optional[str]*): Machine architecture. - *domainname* (*Optional[str]*): Domain name.

Example `python v = KernelVersion(5, 10, 0, "-rc1") `

class Args(*, *sysname=None*, *nodename=None*, *release=None*, *kversion=None*, *machine=None*, *domainname=None*)

Bases: *PluginArgs*

Parameters

- **sysname** (*str | None*)
- **nodename** (*str | None*)
- **release** (*str | None*)
- **kversion** (*str | None*)
- **machine** (*str | None*)
- **domainname** (*str | None*)

domainname: str | None

kversion: str | None

machine: str | None

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

nodename: str | None

release: str | None

sysname: str | None

change_uname(*cpu*, *buf_ptr*, *filler*)

Event handler to change the uname string in the guest.

Args:
cpu (Any): The CPU context. buf_ptr (int): Pointer to the buffer where uname is written. filler (Any): Unused filler argument.

Returns:
None

Parameters

- **cpu** (*Any*)
- **buf_ptr** (*int*)
- **filler** (*Any*)

Return type
None

create_string()

Construct a comma-separated string of uname fields.

Returns:
str: The constructed uname string, with 'none' for missing fields.

Return type
str

domainname: str | None

machine: str | None

nodename: str | None

outdir: str | None

release: str | None

sysname: str | None

version: str | None

pyplugins.interventions.kmods module

Kernel Module Tracker

This plugin tracks and controls kernel module loading attempts in the guest system. By default, it blocks all kernel module loading except for igloo.ko (the internal framework module). Optionally, an allowlist can be configured to allow specific kernel modules to load, or a denylist to explicitly block specific modules.

Features

- Intercepts init_module and finit_module syscalls
- Tracks all kernel module loading attempts to modules.log
- Blocks module loading (except igloo.ko) by default
- Supports allowlist for specific modules to allow them to load
- Supports denylist for explicit blocking of specific modules

Configuration

To enable the plugin with default behavior (block all except igloo.ko):

```
plugins:
  kmods: {}
```

To allow specific modules to load, provide an allowlist:

```
plugins:
  kmods:
    allowlist:
      - wireguard
      - nf_conntrack
      - xt_TCPMSS
```

To explicitly block specific modules, provide a denylist:

```
plugins:
  kmods:
    denylist:
      - suspicious_module
      - untrusted_driver
```

To reduce logging verbosity, enable quiet mode:

```
plugins:
  kmods:
    quiet: true
```

Both lists can be used together. Denylist takes precedence over allowlist. Module names should not include the .ko extension or path. When quiet is set to true, only errors are logged; otherwise, info-level logs are shown (default).

Outputs

- modules.log: List of all kernel modules that were attempted to be loaded

class pyplugins.interventions.kmods.**KmodTracker**

Bases: *Plugin*

Tracks and controls kernel module loading in the guest system.

This plugin intercepts kernel module loading syscalls and can either block them (default behavior), allow specific modules via allowlist, or explicitly block specific modules via denylist.

Attributes:
allowlist (list): List of kernel module names allowed to load denylist (list): List of kernel module names to explicitly block quiet (bool): If True, set log level to error; if False, use info level

class Args(*, *allowlist=[]*, *denylist=[]*, *quiet=False*)

Bases: *PluginArgs*

Parameters

- **allowlist** (*List[str]*)
- **denylist** (*List[str]*)
- **quiet** (*bool*)

allowlist: List[str]

denylist: List[str]

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

quiet: bool

finit_module(*regs*, *proto*, *syscall*, *fd*, *param_values*, *flags*)

Handle the finit_module syscall to track and optionally block module loading.

This method intercepts attempts to load kernel modules via the finit_module syscall (which loads modules from a file descriptor). It tracks all module loading attempts and blocks modules unless they are allow-listed.

Args:
regs: CPU register state proto: Syscall prototype syscall: Syscall object with retval and skip_syscall attributes fd: File descriptor of the kernel module file param_values: Module parameters flags: Module loading flags

Yields:
Results from plugins.osi.get_fd_name to retrieve the module path

init_module(*regs*, *proto*, *syscall*, *module_image*, *size*, *param_values*)

Handle the init_module syscall to track and optionally block module loading.

This method intercepts attempts to load kernel modules via the init_module syscall. It always allows igloo.ko to load, tracks all other module loading attempts, and blocks modules unless they are allow-listed.

Args:
regs: CPU register state proto: Syscall prototype syscall: Syscall object with retval and skip_syscall attributes module_image: Pointer to module image in memory size: Size of the module image param_values: Module parameters

Yields:
Results from plugins.osi calls for process and file descriptor information

is_allowed(*kmod_path*)

Check if a kernel module is in the allowlist. Extracts the module name from the path and checks against allowlist.

Args:
kmod_path: Path to the kernel module (e.g., "/lib/modules/foo.ko")

Returns:
True if the module is in the allowlist, False otherwise

Parameters
kmod_path (*str*)

Return type
bool

`is_denied`(*kmod_path*)

Check if a kernel module is in the denylist. Extracts the module name from the path and checks against denylist.

Args:
kmod_path: Path to the kernel module (e.g., "/lib/modules/foo.ko")

Returns:
True if the module is in the denylist, False otherwise

Parameters
kmod_path (*str*)

Return type
bool

`track_kmod`(*kmod_path*)

Track a kernel module loading attempt by recording it to modules.log.

Args:
kmod_path (str): Path to the kernel module being loaded

Parameters
kmod_path (*str*)

pyplugins.interventions.lifeguard module

Lifeguard: Signal Blocking Plugin

This module provides a plugin for the Penguin framework to block specified Linux signals during analysis or emulation. Signals sent through supported signal-send syscalls are blocked before the kernel sends them. Lifeguard also consumes the signal_monitor API for configured signals so delivery paths can be observed and dropped when the driver hook is effective.

Features

- Block user-specified signals (e.g., SIGKILL, SIGTERM) for target processes.
- Log signal delivery and syscall interception events to a CSV file.
- Optionally enable verbose logging for debugging.

Usage

To use this plugin, specify the signals to block in the configuration:

```
{
    "blocked_signals": [9, 15]  # Block SIGKILL and SIGTERM
}
```

The plugin will log blocked signal events to lifeguard.csv in the specified output directory.

Limitations

Delivery-time drops are not equivalent to preventing a signal from being sent. The driver hooks the kernel dequeue path, so it can drop catchable signal deliveries after the kernel has selected a target. Default-fatal signals without a userspace handler may start process or thread-group exit before that delivery hook is reached. For process-preserving behavior, Lifeguard still suppresses signal-send syscalls before the kernel applies signal semantics. SIGKILL and SIGSTOP cannot be caught or ignored by Linux processes, so Lifeguard only suppresses instances generated by supported signal-send syscalls. Synchronous fault signals such as SIGILL or SIGSEGV may need guest state repair when dropped; otherwise execution can return to the same faulting instruction.

class pyplugins.interventions.lifeguard.**Lifeguard**

Bases: *Plugin*

Plugin to block specified signals.

Supported signal-send syscalls are suppressed before send time. For configured signals other than SIGKILL and SIGSTOP, Lifeguard also subscribes to signal_monitor so delivery paths outside those syscalls can be observed and dropped when the driver hook can do so safely.

Attributes - *outdir* (*str*): Output directory for logs. - *blocked_signals* (*list[int]*): List of blocked signal numbers.

blocked_signals: list[int]

delivery_blocked_signals: set[int]

log_signal(*sig*, *pid*, *blocked*, *mechanism*)

Parameters

- **sig** (*int*)
- **pid** (*int*)
- **blocked** (*bool*)
- **mechanism** (*str*)

Return type
None

on_signal_deliver(*cpu*, *event*)

Block configured signals after the kernel has selected a delivery target.

Return type
None

on_signal_send_syscall_enter(*pt_regs*, *proto*, *syscall*, **args*)

Handler for signal-send syscalls. Blocks signals if configured.

Args - *pt_regs* (*object*): The CPU registers at syscall entry. - *proto* (*object*): The syscall prototype. - *syscall* (*object*): The syscall event object. - *args* (*tuple*): The arguments passed to the syscall.

Returns - *None*

outdir: str

syscall_blocked_signals: set[int]

pyplugins.interventions.mount module

Mount Tracker Plugin

This module provides a passive tracker for mount attempts within a guest environment. It is intended for use with the Penguin analysis framework and is implemented as a plugin.

Purpose

- Tracks when the guest tries to mount filesystems.
- Records unsupported filesystem types (e.g., when mount returns EINVAL) to inform kernel support decisions.
- Logs attempts to mount missing devices.
- Can optionally fake mount successes for specific targets or all mounts, aiding in analysis and mitigation.

Usage

The plugin can be configured with the following arguments: - *outdir*: Output directory for logs. - *fake_mounts*: List of mount targets to fake as successful. - *all_succeed*: If set, all mount attempts are faked as successful. - *verbose*: Enables debug logging.

Example

All mount attempts are logged to *mounts.csv* in the specified output directory.

class pyplugins.interventions.mount.**MountTracker**

Bases: *Plugin*

MountTracker Plugin

Tracks and logs mount attempts in the guest.

Attributes

outdir
[str] Output directory for logs.

mounts
[set of tuple] Set of (source, target, fs_type) tuples already logged.

fake_mounts
[list of str] List of mount targets to fake as successful.

all_succeed
[bool] If True, all mount attempts are faked as successful.

Behavior

- Subscribes to exec events to detect */bin/mount* invocations.
- Hooks the mount syscall return to log and optionally fake mount results.

class Args(*, *fake_mounts=[]*, *all_succeed=False*, *verbose=False*)

Bases: *PluginArgs*

Parameters

- **fake_mounts** (*List[str]*)
- **all_succeed** (*bool*)
- **verbose** (*bool*)

all_succeed: bool

fake_mounts: List[str]

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

verbose: bool

find_mount(*event*)

Detects */bin/mount* invocations from exec events and logs them.

Parameters

event
[dict] Exec event dictionary, expected to have 'procname' and 'argv' keys.

Returns

None

Parameters
event (*dict*)

Return type
None

log_mount(*retval*, *results*)

Logs a mount attempt to the output CSV file if not already logged.

Parameters

retval
[int] Return value of the mount syscall or -1 for exec events.

results
[dict] Dictionary with keys 'source', 'target', 'fs_type'.

Returns

None

Parameters

- **retval** (*int*)
- **results** (*dict*)

Return type
None

`post_mount`(*regs*, *proto*, *syscall*, *source*, *target*, *fs_type*, *flags*, *data*)

Coroutine callback for the mount syscall return.

Reads the mount arguments from memory, logs the attempt, and optionally fakes the result.

Parameters

regs
[object] Register state

proto
[object] Protocol context (opaque, framework-specific)

syscall
[object] Syscall context, with *.retval* for return value

source
[int] Pointer to source device string

target
[int] Pointer to mount target string

fs_type
[int] Pointer to filesystem type string

flags
[int] Mount flags

data
[int] Pointer to mount data

Returns

None

pyplugins.interventions.nvram2 module

NVRAM Tracker Plugin

This module provides a plugin for tracking NVRAM (non-volatile RAM) operations in the guest environment. It is intended for use with the Penguin analysis framework and is implemented as a plugin.

Purpose

- Tracks NVRAM get (hit/miss), set, and clear operations.
- Logs all operations to a CSV file for later analysis.
- Optionally enables debug logging for set operations.

Usage

The plugin can be configured with the following arguments: - *outdir*: Output directory for logs. - *verbose*: Enables debug logging for set operations. - *logging*: Enables or disables logging of NVRAM operations (default: True).

Example

```
from penguin import plugins
plugins.load("interventions.nvram2", outdir="/tmp", verbose=True)
```

If logging is enabled, NVRAM operations are logged to *nvram.csv* in the specified output directory.

class pyplugins.interventions.nvram2.**Nvram2**

Bases: *Plugin*

Nvram2 Plugin

Tracks and logs NVRAM operations in the guest.

Attributes

outdir
: [str] Output directory for logs.

Behavior

- Subscribes to NVRAM get (hit/miss), set, and clear events.
- Logs each operation to a CSV file.

class Args(*, *logging=True*, *persist=False*)

Bases: *PluginArgs*

Parameters

- **logging** (*bool*)
- **persist** (*bool*)

logging: bool

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

persist: bool

load_state(*data*)

Restore NVRAM set-value state captured at snapshot time.

Return type
None

log_write(*entry*)

Write a log entry to the CSV file.

Parameters

entry
: [str] Log entry to write

Returns

None

Parameters
entry (*str*)

Return type
None

`on_nvram_clear`(*regs*, *key*)

Handles and logs an NVRAM clear operation.

Parameters

regs
[object] CPU register/context (opaque, framework-specific)

key
[str] NVRAM key being cleared

Returns

None

Parameters
key (*str*)

Return type
None

`on_nvram_get`(*regs*, *key*, *hit*)

Logs an NVRAM get operation (hit or miss).

Parameters

regs
[object] CPU register/context (opaque, framework-specific)

key
[str] NVRAM key accessed

hit
[bool] True if get was a hit, False if miss

Returns

None

Parameters

- **key** (*str*)
- **hit** (*bool*)

Return type
None

`on_nvram_get_hit`(*regs*, *key*)

Handles an NVRAM get hit event.

Parameters

regs
[object] CPU register/context (opaque, framework-specific)

key
[str] NVRAM key accessed

Returns

None

Parameters
key (*str*)

Return type
None

`on_nvram_get_miss`(*regs*, *key*)

Handles an NVRAM get miss event.

Parameters

regs
[object] CPU register/context (opaque, framework-specific)

key
[str] NVRAM key accessed

Returns

None

Parameters
key (*str*)

Return type
None

`on_nvram_logging_enabled`(*regs*)

Returns the NVRAM_LOG_* bitmask telling the guest which nvram ops to report.

Parameters

regs
[object] CPU register/context (opaque, framework-specific)

Returns

None

Return type
None

`on_nvram_set`(*regs*, *key*, *newval*)

Handles and logs an NVRAM set operation.

Parameters

regs
[object] CPU register/context (opaque, framework-specific)

key
[str] NVRAM key being set

newval
[str] New value being set

Returns

None

Parameters

- **key** (*str*)
- **newval** (*str*)

Return type
None

`save_state()`

Snapshot the in-memory NVRAM set-value state for restore continuity.

`uninit()`

Return type
None

`pyplugins.interventions.nvram2.add_lib_inject_all_abis(conf, cache_dir, proj_dir=None, build_log_path=None)`

Add lib_inject for all supported ABIs to /igloo

`pyplugins.interventions.nvram2.add_lib_inject_for_abi(config, abi, cache_dir, proj_dir=None, build_log_path=None)`

Compile lib_inject for the ABI and put it in /igloo, using cache_dir for caching

`pyplugins.interventions.nvram2.prep_config(conf, cache_dir, proj_dir=None, build_log_path=None)`

pyplugins.interventions.remotectrl module

RemoteCtrl Plugin

The RemoteCtrl plugin serves as the external control interface for the Penguin analysis environment. It binds to a Unix Domain Socket and accepts JSON-formatted commands to orchestrate dynamic instrumentation and plugin management during runtime without pausing or restarting the emulation.

This plugin acts as the backend server for command-line tools (like `cli_breakpoint.py`).

Socket Protocol

- **Socket Path**: Defaults to `<outdir>/remotectrl.sock` (e.g., `/tmp/remotectrl.sock`).
- **Input**: JSON-formatted string representing a command.
- **Output**: JSON-formatted string containing a `status` ("success" or "error") and relevant data.

Supported Commands

1. Instrumentation (Delegated to HookLogger)

These commands interface directly with the `HookLogger` plugin to register dynamic probes.

- ``**uprobe**``
  - `path` *(str)*: Target binary or library path (e.g., `/bin/ls`).
  - `symbol` *(str)*: Function name or address (e.g., `malloc`, `0x400500`).
  - `action` *(str)*: The format string defining capture logic (e.g., `print(%s)`).
  - `logfile` *(str, optional)*: Filename in results dir to append output to.
  - `pid_filter` / `process_filter`: Optional scope constraints.
- ``**syscall**``
  - `name` *(str)*: Syscall name (e.g., `sys_read`).
  - `action` *(str)*: The format string defining capture logic.
- ``**list**``
  - Returns a list of all active hooks, their IDs, targets, and output destinations.
- ``**disable**``
  - `id` *(int, optional)*: The specific hook ID to unregister. If omitted, **ALL** hooks are disabled.

2. Plugin Management

Allows for dynamic loading and toggling of other Penguin analysis plugins.

- ``**load_plugin**``**: Import and initialize a new plugin from disk.**
  - `name`: Plugin filename or module name.
  - `args`: Dictionary of arguments for the plugin.
- ``**enable_plugin**``: Call the `enable()` method of a loaded plugin.
- ``**disable_plugin**``: Call the `disable()` method of a loaded plugin.

Example Payload

To register a hook via the socket, send a JSON payload like this:

```
{
    "type": "uprobe",
    "path": "/lib/libc.so.6",
    "symbol": "open",
    "action": "print(%s) = %d",
    "process_filter": "nginx",
    "logfile": "nginx_opens.log"
}
```

Dependencies

This plugin requires the **HookLogger** plugin to be loaded to perform actual instrumentation. It attempts to load HookLogger automatically if it is missing.

class pyplugins.interventions.remotectrl.**RemoteCtrl**

Bases: *Plugin*

RemoteCtrl Plugin

The remote control plane for Penguin instrumentation. Listens on a Unix socket to register probes and manipulate execution via the HookLogger plugin.

uninit()

pyplugins.loggers package

Loggers

This package contains plugins for logging various system and process events in the framework.

Overview

The loggers in this module provide persistent recording of system calls, process executions, file I/O, and other runtime events from the guest system. Each logger is implemented as a plugin and is responsible for capturing specific types of events and storing them in the database for later analysis.

Included Loggers

- **Syscalls Logger**: Records all system call events, including arguments, return values, and error codes.
- **Read/Write Logger**: Captures file descriptor read and write operations, including buffer contents.
- **Exec Logger**: Logs process execution (exec) events, including argument vectors and environment.
- **DB Logger**: Provides a database-backed event buffer and writer for persistent storage.

These loggers enable comprehensive tracing and auditing of guest activity, supporting security analysis, debugging, and research.

Usage

Loggers are loaded automatically as part of the plugin system. See individual logger modules for details.

Submodules

pyplugins.loggers.db module

DB Logger Plugin

This module implements a database-backed event logger plugin for the framework. It uses SQLAlchemy to persist events to a SQLite database in a buffered, asynchronous manner.

Features

- Buffers events in memory and flushes them to disk in batches for performance.
- Uses a background thread to periodically flush events or when the buffer is full.
- Thread-safe event queueing.
- Schema is auto-created on first flush.
- Configurable buffer size and output directory.

Usage

```
from pyplugins.loggers.db import DB

db_logger = DB()
db_logger.add_event(Syscall, row_dict)
db_logger.uninit()
```

Arguments

- outdir: Output directory for the SQLite database file.
- bufsize: Buffer size before flushing to disk (default: 100000).
- verbose: Enable debug logging.

class pyplugins.loggers.db.**DB**

Bases: *Plugin*

Optimized Database-backed event logger. Uses SQLAlchemy Core for bulk inserts and minimizes locking contention.

class Args(*, *bufsize=100000*)

Bases: *PluginArgs*

Parameters
bufsize (*int*)

bufsize: int

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

add_event(*table_cls*, *data*)

Add an event to the buffer. Arguments:

table_cls: The SQLAlchemy class (e.g., Syscall) data: A dictionary representing the row

Parameters
data (*dict*)

Return type
None

buffer_size: int

flush()

Synchronously drain the queued events to the database.

Unlike setting `flush_event` (which only *signals* the worker), this returns once the currently-queued events are written. Consumers that read the DB back during teardown (e.g. the process-tree derived view) must call this first, because plugins unload in reverse load order – so a downstream plugin's `uninit` runs before this plugin's own final flush. Safe to call from any thread; serialized via `flush_lock`.

Return type
None

query(*table_cls*)

Return all rows of `table_cls` as ORM objects.

Ensures the schema exists first (so querying a table that never received a row yields `[]` instead of raising). Call *flush()* beforehand if you need events buffered during the run to be visible.

Return type
list

queued_events: list

uninit()

Clean up the plugin and flush any remaining events.

- Triggers a final flush.
- Stops the background worker thread.
- Disposes of the SQLAlchemy engine.

Returns: None

Return type
None

pyplugins.loggers.exec_logger module

Exec Logger Plugin

This plugin records process execution (exec) events to the penguin database. It subscribes to `exec_event` events published by the `execs` plugin, extracts relevant execution details, and stores them as `Exec` events in the database.

Purpose

- Monitors process execution events in the guest.
- Records argument vectors, environment, and process credentials for each exec event.
- Enables later analysis of process launches and their context.

Usage

Simply add this plugin by name to your config.

The plugin extracts relevant fields and stores them in the database using the `Exec` event type.

class pyplugins.loggers.exec_logger.**ExecLog**

Bases: *Plugin*

Plugin for logging process execution (exec) events to the database.

Subscribes to *exec_event* events from the *execs* plugin and records them as *Exec* events.

on_exec_event(*event*)

Callback for handling *exec_event* events.

Parameters: - *event* (*dict* or *Wrapper*): The exec event data, either as a dictionary or a Wrapper object.

Extracts argument count, argument vector, environment, and process credentials, then records the event in the database as an *Exec* event.

Returns: None

Return type
None

pyplugins.loggers.rw_logger module

Read/Write Logger Plugin

This plugin records read and write system call events to the penguin database. It hooks into the system call return events for `read` and `write`, extracts relevant details such as file descriptor, buffer content, and process name, and stores them as `Read` and `Write` events in the database.

Purpose

- Monitors file descriptor read and write operations in the guest.
- Records buffer contents, file descriptor names, and process names for each event.
- Enables later analysis of file I/O activity and data flow.

Usage

Simply add this plugin by name to your config.

The plugin extracts relevant fields and stores them in the database using the `Read` and `Write` event types.

class pyplugins.loggers.rw_logger.**RWLog**

Bases: *Plugin*

Plugin for logging read and write system call events to the database.

Hooks into system call return events and records them as *Read* and *Write* events.

disable()

enable()

read(*regs*, *proto*, *syscall*, *fd*, *buf*, *count*)

Callback for handling read syscall return events.

Parameters: - *proto*: Protocol or plugin-specific context. - *syscall*: Syscall number or identifier. - *fd*: File descriptor being read from. - *buf*: Buffer address containing data read. - *count*: Number of bytes read.

Reads the buffer content, resolves the file descriptor name and process name, and records the event in the database as a *Read* event.

Returns: None

Return type
None

write(*regs*, *proto*, *syscall*, *fd*, *buf*, *count*)

Callback for handling write syscall return events.

Parameters: - *regs*: CPU registers at the time of the syscall. - *proto*: Protocol or plugin-specific context. - *syscall*: Syscall number or identifier. - *fd*: File descriptor being written to. - *buf*: Buffer address containing data written. - *count*: Number of bytes written.

Reads the buffer content, resolves the file descriptor name and process name, and records the event in the database as a *Write* event.

Returns: None

Return type
None

pyplugins.loggers.syscalls_logger module

Syscalls Logger Plugin

This plugin records system call events to the penguin database. It parses Linux error codes from header files, maps error numbers to names and explanations, and logs detailed syscall information including arguments, return values, and process context.

Purpose

- Monitors all system call return events and execve/execveat entries in the guest.
- Records syscall arguments, return values, and error codes with explanations.
- Enables later analysis of system call activity and process behavior.

Usage

```
from pyplugins.loggers.syscalls_logger import PyPandaSysLog

syscalls_logger = PyPandaSysLog(panda)
# Syscall events will be logged automatically.
```

This plugin is loaded automatically as part of the penguin plugin system. It requires the syscalls, mem, portal, and osi plugins to be active.

The plugin extracts relevant fields and stores them in the database using the Syscall event type.

Arguments

- outdir: Output directory for the SQLite database file.
- procs: Optional list of process names to filter syscall logging. If not provided, all processes are logged.

class pyplugins.loggers.syscalls_logger.**PyPandaSysLog**(*panda*)

Bases: *Plugin*

Plugin for logging system call events to the database.

Hooks into system call return and execve/execveat entry events and records them as *Syscall* events.

class Args(*, *procs=None*)

Bases: *PluginArgs*

Parameters
procs (*List[str] | None*)

model_config = {'extra': 'forbid'}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

procs: List[str] | None

all_sys_ret(*regs*, *proto*, *syscall*)

Callback for handling all syscall return events.

Parameters: - *regs*: Register/context object. - *proto*: Syscall prototype. - *syscall*: Syscall object.

Yields from *handle_syscall* to log the syscall event, except for execve.

Returns: None

Return type
None

cstr(*x*)

Return type
str

disable()

enable(*args={}*)

get_syscall_processors(*proto*)

Returns cached list of: (arg_name, handler_func, is_gen, extra_data)

handle_syscall(*regs*, *proto*, *syscall*)

Handle and log a syscall event.

Parameters: - *regs*: Register/context object. - *proto*: Syscall prototype. - *syscall*: Syscall object.

Extracts arguments, formats them, determines return value and error code, and logs the event to the database.

Returns: None

Return type
None

sys_record_enter(*regs*, *proto*, *syscall*, **args*)

Return type
None

pyplugins.wrappers package

wrappers - Pythonic wrappers for plugin and emulation data structures

This package provides a set of Python wrapper classes for working with data structures returned by plugins, emulators, or analysis frameworks (such as PANDA). The wrappers offer a uniform, Pythonic interface for accessing fields, iterating over arrays, and performing architecture-agnostic analysis of process state, memory mappings, and more.

Purpose

- Abstract away raw C structs, ctypes, or dicts returned by plugins.
- Provide convenient attribute access, conversion to dict/list, and pretty-printing.
- Enable architecture-independent analysis of process state (registers, memory maps, etc).
- Facilitate building analysis tools, plugins, and scripts that work across architectures and plugins.

Example Usage

Importing wrappers for use in your analysis code:

```
from wrappers.generic import Wrapper, ArrayWrapper
from wrappers.ptregs_wrap import get_pt_regs_wrapper, PtRegsWrapper
from wrappers.osi_wrap import MappingWrapper, MappingsWrapper
```

Wrapping plugin data:

```
# Wrap a struct or dict
obj = Wrapper(plugin.get_struct())
print(obj.field1)

# Wrap an array of objects
objs = ArrayWrapper([Wrapper(x) for x in plugin.get_array()])
for o in objs:
    print(o)

# Architecture-agnostic register access
regs = plugin.get_pt_regs()
wrapper = get_pt_regs_wrapper(panda, regs, arch_name=panda.arch_name)
print(wrapper.get_pc())

# Memory mapping analysis
raw_maps = plugin.get_mappings()
maps = MappingsWrapper([MappingWrapper(m) for m in raw_maps])
print(maps)
```

See the individual modules for more details and advanced usage.

Submodules

pyplugins.wrappers.generic module

generic.py - Base wrappers for plugin data structures

This module provides generic Python wrapper classes for plugin data structures, such as those returned by PANDA or other emulation/analysis frameworks. These wrappers provide a uniform interface for accessing fields, converting to dicts, and working with arrays of wrapped objects.

Overview

The main classes are: - Wrapper: A base class for wrapping a single object, providing attribute access, dict conversion, and pretty-printing. - ArrayWrapper: A base class for wrapping a list/array of objects, providing list-like access and iteration.

These classes are intended to be subclassed for specific data structures, but can also be used directly for simple cases.

Typical Usage

Suppose you have a plugin that returns a C struct or dict-like object:

```
from wrappers.generic import Wrapper, ArrayWrapper

```

(continues on next page)

(continued from previous page)

```
# Wrap a single object
data = plugin.get_struct()
obj = Wrapper(data)
print(obj.field1)
print(obj.to_dict())

# Wrap an array of objects
array_data = plugin.get_array()
objs = ArrayWrapper([Wrapper(x) for x in array_data])
for o in objs:
    print(o)
```

Classes

- Wrapper: Base class for single-object wrappers.
- ArrayWrapper: Base class for array/list wrappers.

class pyplugins.wrappers.generic.**ArrayWrapper**(*data*, *wrapper_cls=<class 'pyplugins.wrappers.generic.Wrapper'>*)

Bases: Generic[T]

Optimized base class for wrapping a list/array of objects. Performs LAZY wrapping: objects are only wrapped when accessed.

Parameters

- **data** (*Sequence[Any]*)
- **wrapper_cls** (*Type[T]*)

to_list()

Return type
List[*T*]

class pyplugins.wrappers.generic.**ConstDictWrapper**(*data*)

Bases: object

A read-only wrapper for dictionaries optimized for maximum read speed.

Attributes are injected directly into the instance's __dict__ at initialization time, allowing for native C-speed attribute access (obj.field) without the overhead of __getattr__.

Parameters
data (*Dict[str, Any]*)

items()

Pass-through for dict iteration.

keys()

Pass-through for keys.

to_dict()

Return the underlying data as a dict.

Return type
Dict[str, *Any*]

values()

Pass-through for values.

class pyplugins.wrappers.generic.**Wrapper**(*obj*)

Bases: object

Optimized base class for wrapping a single object. Uses __slots__ and cached type flags to minimize __getattr__ overhead.

Parameters
obj (*Any*)

to_dict()

Return type
Dict[str, *Any*]

pyplugins.wrappers.osi_wrap module

osi_wrap.py - Wrappers for process memory mappings (Operating System Introspection)

This module provides Pythonic wrappers for handling process memory mappings, typically as returned by a plugin or API that exposes Linux process memory map information. The wrappers are designed to make it easy to inspect, filter, and display memory mappings, such as those found in /proc/<pid>/maps.

Overview

This module defines two main classes: - MappingWrapper: Wraps a single memory mapping, providing properties for permissions, addresses, and device info. - MappingsWrapper: Wraps a list of MappingWrapper objects, providing search and display utilities.

These wrappers are useful for analyzing process memory layouts, debugging, or building tools that need to inspect memory regions of Linux processes. They abstract away the raw dictionary or struct data and provide convenient Pythonic accessors and search methods.

Typical Usage

Suppose you have a plugin or API that returns a list of memory mapping dictionaries for a process (e.g., from /proc/<pid>/maps or a similar source):

```
from wrappers.osi_wrap import MappingWrapper, MappingsWrapper

# Example: plugin.get_mappings() returns a list of dicts, one per mapping
raw_mappings = plugin.get_mappings()  # Each dict should have keys: flags, base, size,␣
↪dev, pgoff, inode, name
mappings = [MappingWrapper(m) for m in raw_mappings]
all_mappings = MappingsWrapper(mappings)
```

You can then perform various queries and inspections:

```
# Find a mapping by address:
mapping = all_mappings.get_mapping_by_addr(0x7f1234567000)
if mapping:
    print(f"Permissions: {mapping.perms}, Name: {mapping.name}")

# List all mappings for a library:
```

(continues on next page)

(continued from previous page)

```
libc_maps = all_mappings.get_mappings_by_name('libc')
for m in libc_maps:
    print(m)

# Print all mappings in a format similar to /proc/<pid>/maps:
print(all_mappings)
```

The MappingWrapper exposes properties such as .perms, .start, .end, .dev_major, .dev_minor, and .name, making it easy to work with mapping attributes.

Classes

- MappingWrapper: Wraps a single memory mapping, providing properties for permissions, addresses, and device info.
- MappingsWrapper: Wraps a list of MappingWrapper objects, providing search and display utilities.

class pyplugins.wrappers.osi_wrap.**MappingWrapper**(*obj*)

Bases: `Wrapper`

Wraps a single process memory mapping, providing convenient properties for permissions, addresses, and device information.

Attributes:

flags (int): Bitmask of mapping permissions and flags. base (int): Start address of the mapping. size (int): Size of the mapping in bytes. dev (int): Device number (major:minor) of the mapping. pgoff (int): Offset into the mapped file. inode (int): Inode number of the mapped file. name (str): Name or path of the mapped file.

Parameters

obj (*Any*)

property dev_major: int

Return the major number of the mapping. In Linux, the major number is in the high 12 bits of dev.

property dev_minor: int

Return the minor number of the mapping. In Linux, the minor number is in the low 20 bits of dev.

property end: int

Return the end address of the mapping.

property exec: bool

Check if the mapping is executable.

property executable: bool

Check if the mapping is executable.

get_addr_offset(*addr*)

Return the offset of the address within the mapping.

Parameters

addr (*int*)

Return type

int | None

property perms: str

Return the permissions of the mapping.

property read: bool

Check if the mapping is readable.

property share: bool

Check if the mapping is shareable.

property start: int

Return the start address of the mapping.

property write: bool

Check if the mapping is writable.

class pyplugins.wrappers.osi_wrap.MappingsWrapper(*data*, *wrapper_cls=<class 'wrappers.generic.Wrapper'>*)

Bases: ArrayWrapper

Wraps a list of MappingWrapper objects, providing utilities to search and display memory mappings.

Methods:

get_mapping_by_addr(addr): Return the mapping containing the given address. get_mappings_by_name(name): Return all mappings whose name contains the given string.

Parameters

- **data** (*Sequence[Any]*)
- **wrapper_cls** (*Type[T]*)

get_first_mapping_by_name(*name*)

Find the first (lowest in memory) mapping whose name contains the given string.

Parameters
name (*str*)

Return type
MappingWrapper | None

get_mapping_by_addr(*addr*)

Find the mapping for a given address.

Parameters
addr (*int*)

Return type
MappingWrapper | None

get_mappings_by_name(*name*)

Find all mappings whose name contains the given string.

Parameters
name (*str*)

Return type
List[MappingWrapper]

pyplugins.wrappers.ptregs_wrap module

ptregs_wrap.py - Architecture-agnostic wrappers for Linux pt_regs structures

This module provides Pythonic, type-annotated wrappers for Linux kernel pt_regs structures across multiple CPU architectures. It enables convenient, architecture-independent access to process register state, such as that captured at system call entry/exit, exceptions, or context switches. The wrappers abstract away the raw struct layout and provide a unified interface for reading/writing registers, extracting syscall arguments, and handling calling conventions.

Overview

The module defines a base PtRegsWrapper class and a set of subclasses for each supported architecture (x86, x86_64, ARM, AArch64, MIPS, PowerPC, LoongArch64, RISC-V, etc). Each subclass knows how to access registers and arguments according to its architecture's ABI and pt_regs layout. The wrappers can be used with PANDA or other emulation/analysis frameworks that expose pt_regs-like objects.

The module also provides a get_pt_regs_wrapper() factory function to select the correct wrapper for a given architecture.

Typical Usage

Suppose you have a PANDA plugin or other tool that provides a pt_regs struct (e.g., at a syscall, exception, or context switch):

```
from wrappers.ptregs_wrap import get_pt_regs_wrapper
# Assume 'regs' is a pt_regs struct and 'panda' is a PANDA object
wrapper = get_pt_regs_wrapper(panda, regs, arch_name=panda.arch_name)

# Access registers in an architecture-agnostic way
pc = wrapper.get_pc()
sp = wrapper.get_sp()
retval = wrapper.get_retval()

# Get syscall arguments (handles calling convention automatically)
arg0 = wrapper.get_syscall_arg(0)
arg1 = wrapper.get_syscall_arg(1)
# Or get userland function arguments
user_arg0 = wrapper.get_userland_arg(0)

# Dump all registers as a dictionary
reg_dict = wrapper.dump()

# Coroutine-style argument access (portal):
# get_args_portal and get_arg_portal are generator-based and can yield if a memory read␣
↪is required (e.g., stack argument).
# Use 'yield from' to drive these coroutines in a portal/coroutine context.
args = yield from wrapper.get_args_portal(3)
```

The wrappers also support advanced features such as handling 32-bit compatibility mode on x86_64/AArch64, stack argument extraction, and portal-style coroutine memory reads. The get_args_portal and get_arg_portal methods are generator-based and will yield if a memory read is required (such as when reading stack arguments may fail and need to be retried or handled asynchronously).

Classes

- PtRegsWrapper: Base class for all pt_regs wrappers, provides generic register access and argument extraction.
- X86PtRegsWrapper, X86_64PtRegsWrapper, ArmPtRegsWrapper, ...: Architecture-specific subclasses.
- PandaMemReadFail: Exception for failed memory reads (for portal/coroutine use).

Functions

- get_pt_regs_wrapper(panda: Optional[Any], regs: Any, arch_name: Optional[str] = None) -> PtRegsWrapper: Factory to select the correct wrapper for a given architecture.

These wrappers are useful for dynamic analysis, syscall tracing, emulation, and any tool that needs to reason about process register state in a cross-architecture way.

class pyplugins.wrappers.ptregs_wrap.**AArch64PtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for AArch64 pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_register(*reg_name*)

Get register value by name, handling AArch32 compatibility mode if needed

Parameters
reg_name (*str*)

Return type
int | None

get_syscall_arg(*num*)

Get AArch64 syscall argument, considering compatibility mode

Parameters
num (*int*)

Return type
int | None

get_syscall_number()

Get syscall number, considering compatibility mode. In AArch64, the syscall number is in syscallno. In AArch32 mode, use ARM implementation.

Return type
int | None

get_userland_arg(*num*)

Get AArch64 userland argument, considering compatibility mode

Parameters
num (*int*)

Return type
int | None

i = 30

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

set_register(*reg_name*, *value*)

Set register value by name, handling AArch32 compatibility mode if needed

Parameters

- **reg_name** (*str*)
- **value** (*int*)

Return type
bool

class pyplugins.wrappers.ptregs_wrap.**ArmPtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for ARM pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_syscall_arg(*num*)

Get ARM syscall argument

Parameters
num (*int*)

Return type
int | None

get_syscall_number()

Get syscall number from r7 register

Return type
int | None

get_userland_arg(*num*)

Get ARM userland argument

Parameters
num (*int*)

Return type
int | None

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

class pyplugins.wrappers.ptregs_wrap.**LoongArch64PtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for LoongArch64 pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

field = 'csr_estat'

get_syscall_arg(*num*)

Get LoongArch64 syscall argument

Parameters
num (*int*)

Return type
int | None

get_syscall_number()

Get syscall number from a7 register

Return type
int | None

get_userland_arg(*num*)

Get LoongArch64 userland argument

Parameters
num (*int*)

Return type
int | None

idx = 31

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

name = 's8'

class pyplugins.wrappers.ptregs_wrap.**Mips64PtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *MipsPtRegsWrapper*

Wrapper for MIPS64 pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_syscall_arg(*num*)

Get MIPS64 syscall argument

Parameters
num (*int*)

Return type
int | None

get_userland_arg(*num*)

Get MIPS64 userland argument

Parameters
num (*int*)

Return type
int | None

class pyplugins.wrappers.ptregs_wrap.**MipsPtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for MIPS pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_syscall_arg(*num*)

Get MIPS syscall argument

Parameters
num (*int*)

Return type
int | None

get_syscall_number()

Get syscall number from v0 register

Return type
int | None

get_userland_arg(*num*)

Get MIPS userland argument

Parameters
num (*int*)

Return type
int | None

idx = 31

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

name = 'ra'

exception pyplugins.wrappers.ptregs_wrap.**PandaMemReadFail**(*addr*, *size*)

Bases: Exception

Exception for failed memory reads, used for portal/coroutine use-cases.

Attributes:

addr (int): The address that failed to read. size (int): The size of the attempted read.

Parameters

- **addr** (*int*)
- **size** (*int*)

Return type

None

addr: int

size: int

class pyplugins.wrappers.ptregs_wrap.**PowerPC64PtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *PowerPCPtRegsWrapper*

Wrapper for PowerPC64 pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

class pyplugins.wrappers.ptregs_wrap.**PowerPCPtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for PowerPC pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_syscall_arg(*num*)

Get PowerPC syscall argument

Parameters

num (*int*)

Return type

int | None

get_syscall_number()

Get syscall number from r0 register

Return type

int | None

get_userland_arg(*num*)

Get PowerPC userland argument

Parameters
num (*int*)

Return type
int | None

i = 31

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

reg = 'result'

class pyplugins.wrappers.ptregs_wrap.**PtRegsWrapper**(*obj*, *panda=None*, *extra_context=None*)

Bases: Wrapper

Base class for pt_regs wrappers across different architectures.

Args:
obj: The pt_regs structure to wrap. panda: Optional PANDA object for memory reading. extra_context: Optional dictionary to pass state to register operations.

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

property REG_NAMES: List[str]

Returns a list of all valid register names for this architecture.

dump()

Dump all registers to a dictionary.

Return type
Dict[str, int | None]

get_arg(*num*, *convention=None*)

Get function argument based on calling convention.

Args:
num: Argument number (0-based) convention: Calling convention ('syscall' or 'userland')

Returns:
The value of the requested argument

Parameters

- **num** (*int*)
- **convention** (*str | None*)

Return type
int | None

`get_arg_portal`(*num*, *convention=None*)

Coroutine/generator version of get_arg for portal/coroutine use.

Args:
num: Argument number (0-based) convention: Calling convention ('syscall' or 'userland')

Returns:
The value of the requested argument (or None if unavailable).

Parameters

- **num** (*int*)
- **convention** (*str | None*)

Return type
Generator[int | None, *Any*, int | None]

`get_args`(*count*, *convention=None*, *func_type=None*)

Get a list of function arguments according to the calling convention.

Args:
count: Number of arguments to retrieve. convention: Calling convention ('syscall' or 'userland'). func_type: Optional explicit signature definition.

Returns:
List of argument values (may include None if unavailable).

Parameters

- **count** (*int*)
- **convention** (*str | None*)
- **func_type** (*Dict | None*)

Return type
List[int | None]

`get_args_portal`(*count*, *convention=None*, *func_type=None*)

Coroutine/generator version of get_args for portal/coroutine use.

Args:
count: Number of arguments to retrieve. convention: Calling convention ('syscall' or 'userland'). func_type: Optional explicit signature definition.

Returns:
List of argument values (may include None if unavailable).

Parameters

- **count** (*int*)
- **convention** (*str | None*)
- **func_type** (*Dict | None*)

Return type
Generator[int | None, *Any*, *List*[int | None]]

`get_pc()`

Get program counter.

Return type
int | None

`get_register(reg_name)`

Get register value by name (Optimized).

Parameters
reg_name (*str*)

Return type
int | None

`get_retaddr()`

Get the return address (best guess for this architecture). Supports both generator and non-generator implementations in subclasses.

Return type
int | None

`get_retaddr_portal()`

Coroutine/generator version of get_retaddr for portal/coroutine use.

Return type
Generator[int | None, *Any*, int | None]

`get_return_address()`

Alias for get_retaddr.

Return type
int | None

`get_return_value()`

Get return value (typically in a0/r0/rax).

Return type
int | None

`get_retval()`

Get return value (alias for get_return_value).

Return type
int | None

`get_sp()`

Get stack pointer.

Return type
int | None

`get_syscall_arg(num)`

Get syscall argument. Subclasses must implement.

Parameters
num (*int*)

Return type
int | None

get_syscall_number()

Get syscall number. Subclasses must implement.

Return type
int | None

get_typed_args(*count*, *convention=None*)

Get a list of function arguments, automatically cast to their dwarffi equivalents (Ptr or BoundTypeInstance) using ffi.new().

Parameters

- **count** (*int*)
- **convention** (*str | None*)

Return type
List[*Any*]

get_typed_args_portal(*count*, *convention=None*)

Coroutine/generator version of get_typed_args for portal use.

Parameters

- **count** (*int*)
- **convention** (*str | None*)

Return type
Generator[*Any*, *Any*, *List*[*Any*]]

get_userland_arg(*num*)

Get userland argument. Subclasses must implement.

Parameters
num (*int*)

Return type
int | None

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

read_stack_arg(*arg_num*, *word_size=None*)

Read a function argument from the stack.

Args:
arg_num: Argument number (0-based). word_size: Word size override (default: based on architecture).

Returns:
The argument value read from the stack.

Parameters

- **arg_num** (*int*)
- **word_size** (*int | None*)

Return type
int | None

`set_pc`(*value*)

Set program counter.

Parameters
value (*int*)

Return type
None

`set_register`(*reg_name*, *value*)

Set register value by name (Optimized).

Parameters

- **reg_name** (*str*)
- **value** (*int*)

Return type
bool

`set_retval`(*value*)

Set the return value (typically in a0/r0/rax), accounting for 64-bit splits on 32-bit architectures if the function type indicates it.

Parameters
value (*Any*)

Return type
None

`to_bytes`()

Pass-through to underlying bound object for serialization.

`class pyplugins.wrappers.ptregs_wrap.Riscv32PtRegsWrapper`(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for RISC-V 32-bit pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

`abi_name = 't6'`

`get_syscall_arg`(*num*)

Get RISC-V 32-bit syscall argument

Parameters
num (*int*)

Return type
int | None

get_syscall_number()

Get syscall number from a7 register

Return type
int | None

get_userland_arg(*num*)

Get RISC-V 32-bit userland argument

Parameters
num (*int*)

Return type
int | None

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

x_reg = 'x31'

class pyplugins.wrappers.ptregs_wrap.Riscv64PtRegsWrapper(*obj*, *panda=None*, *extra_context=None*)

Bases: *Riscv32PtRegsWrapper*

Wrapper for RISC-V 64-bit pt_regs - same structure as 32-bit but with 64-bit registers

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_userland_arg(*num*)

Get RISC-V 64-bit userland argument

Parameters
num (*int*)

Return type
int | None

class pyplugins.wrappers.ptregs_wrap.X86PtRegsWrapper(*obj*, *panda=None*, *extra_context=None*)

Bases: *PtRegsWrapper*

Wrapper for x86 (32-bit) pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

get_syscall_arg(*num*)

Get x86 syscall argument

Parameters
num (*int*)

Return type
int | None

`get_syscall_number()`

Get syscall number from EAX register

Return type
int | None

`get_userland_arg(num)`

Get x86 userland argument from stack

Parameters
num (*int*)

Return type
int | None

`in_kernel()`

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

`class pyplugins.wrappers.ptregs_wrap.X86_64PtRegsWrapper(obj, panda=None, extra_context=None)`

Bases: *PtRegsWrapper*

Wrapper for x86_64 pt_regs

Parameters

- **obj** (*Any*)
- **panda** (*Any | None*)
- **extra_context** (*Dict | None*)

`get_register(reg_name)`

Get register value by name (Optimized).

Parameters
reg_name (*str*)

Return type
int | None

`get_syscall_arg(num)`

Get x86_64 syscall argument, considering compatibility mode

Parameters
num (*int*)

Return type
int | None

`get_syscall_number()`

Get syscall number, considering compatibility mode. In x86_64, the syscall number is in orig_rax. In compatibility mode, use x86 implementation.

Return type
int | None

get_userland_arg(*num*)

Get x86_64 userland argument, considering compatibility mode

Parameters
num (*int*)

Return type
int | None

in_kernel()

Returns True if the pt_regs represents kernel mode, False otherwise. Subclasses should override for architecture-specific logic.

Return type
bool

k = 'rsp'

set_register(*reg_name*, *value*)

Set register value by name (Optimized).

Parameters

- **reg_name** (*str*)
- **value** (*int*)

Return type
bool

v = 'sp'

pyplugins.wrappers.ptregs_wrap.**get_pt_regs_wrapper**(*panda*, *regs*, *arch_name=None*, *extra_context=None*)

Factory function to create the appropriate pt_regs wrapper based on architecture.

Args:
panda: PANDA object (may be used to determine architecture if arch_name not provided) regs: The pt_regs structure to wrap arch_name: Architecture name (optional, will be determined from PANDA if not provided)

Returns:
An appropriate PtRegsWrapper subclass instance.

Parameters

- **panda** (*Any | None*)
- **regs** (*Any*)
- **arch_name** (*str | None*)
- **extra_context** (*Dict[str, Any] | None*)

Return type
PtRegsWrapper

PYTHON MODULE INDEX

p

penguin, ??
penguin.abi_info, ??
penguin.analyses, ??
penguin.arch, ??
penguin.arch_registry, ??
penguin.boot_env, ??
penguin.common, ??
penguin.compose, ??
penguin.config_patchers, ??
penguin.defaults, ??
penguin.dropin_compile, ??
penguin.gen_config, ??
penguin.gen_image, ??
penguin.graphs, ??
penguin.init_plugin, ??
penguin.init_runner, ??
penguin.manager, ??
penguin.penguin_config, ??
penguin.penguin_config.errors, ??
penguin.penguin_config.gen_docs, ??
penguin.penguin_config.structure, ??
penguin.penguin_config.templating, ??
penguin.penguin_config.versions, ??
penguin.penguin_config.versions.v2, ??
penguin.penguin_prep, ??
penguin.penguin_run, ??
penguin.plugin_manager, ??
penguin.q_config, ??
penguin.run_summary, ??
penguin.static_analyses, ??
penguin.stubs, ??
penguin.testing, ??
penguin.testing.harness, ??
penguin.utils, ??
penguin.utils_cli, ??
pengutils, ??
pengutils.events, ??
pengutils.events.base, ??
pengutils.events.types, ??
pengutils.utils, ??
pengutils.utils.cli_breakpoint, ??
pengutils.utils.cli_db, ??
pengutils.utils.cli_guest_cmd, ??
pengutils.utils.cli_pengutils, ??
pengutils.utils.cli_plugins, ??
pengutils.utils.cli_snapshot, ??
pengutils.utils.util_base, ??
pengutils.utils.util_events, ??
pyplugins, ??
pyplugins.actuation, ??
pyplugins.actuation.fetch_web, ??
pyplugins.actuation.nmap, ??
pyplugins.actuation.run_on_bind, ??
pyplugins.actuation.vpn, ??
pyplugins.analysis, ??
pyplugins.analysis.crashes, ??
pyplugins.analysis.env, ??
pyplugins.analysis.ficd, ??
pyplugins.analysis.health, ??
pyplugins.analysis.hooklogger, ??
pyplugins.analysis.indiv_debug, ??
pyplugins.analysis.interfaces, ??
pyplugins.analysis.netbinds, ??
pyplugins.analysis.processes, ??
pyplugins.analysis.repl, ??
pyplugins.apis, ??
pyplugins.apis.events, ??
pyplugins.apis.execs, ??
pyplugins.apis.exit_monitor, ??
pyplugins.apis.fs, ??
pyplugins.apis.hypercall, ??
pyplugins.apis.kffi, ??
pyplugins.apis.kprobes, ??
pyplugins.apis.mem, ??
pyplugins.apis.net, ??
pyplugins.apis.osi, ??
pyplugins.apis.portalcall, ??
pyplugins.apis.qmp, ??
pyplugins.apis.send_hypercall, ??
pyplugins.apis.signal_monitor, ??
pyplugins.apis.signals, ??
pyplugins.apis.static_fs, ??
pyplugins.apis.symbols, ??

pyplugins.apis.syscalls, ??
pyplugins.apis.unwind, ??
pyplugins.apis.uprobes, ??
pyplugins.compat, ??
pyplugins.compat.coverage, ??
pyplugins.compat.qemu_compat, ??
pyplugins.core, ??
pyplugins.core.core, ??
pyplugins.core.igloodriver, ??
pyplugins.core.live_image, ??
pyplugins.core.readiness, ??
pyplugins.core.scope, ??
pyplugins.core.snapshot, ??
pyplugins.hyper, ??
pyplugins.hyper.consts, ??
pyplugins.hyper.portal, ??
pyplugins.hyper.shell, ??
pyplugins.hyperfile, ??
pyplugins.hyperfile.anonfs, ??
pyplugins.hyperfile.devfs, ??
pyplugins.hyperfile.mtd, ??
pyplugins.hyperfile.procfs, ??
pyplugins.hyperfile.pseudofile_tracker, ??
pyplugins.hyperfile.pseudofiles, ??
pyplugins.hyperfile.qemu_mem, ??
pyplugins.hyperfile.sysctl, ??
pyplugins.hyperfile.sysfs, ??
pyplugins.interventions, ??
pyplugins.interventions.core_pattern_guard,
    ??
pyplugins.interventions.hyperfile, ??
pyplugins.interventions.kernelversion, ??
pyplugins.interventions.kmods, ??
pyplugins.interventions.lifeguard, ??
pyplugins.interventions.mount, ??
pyplugins.interventions.nvram2, ??
pyplugins.interventions.remotectrl, ??
pyplugins.loggers, ??
pyplugins.loggers.db, ??
pyplugins.loggers.exec_logger, ??
pyplugins.loggers.rw_logger, ??
pyplugins.loggers.syscalls_logger, ??
pyplugins.wrappers, ??
pyplugins.wrappers.generic, ??
pyplugins.wrappers.osi_wrap, ??
pyplugins.wrappers.ptregs_wrap, ??